Deep Learning and Reinforcement Learning: Lecture notes

Lectures 1–5

Compiled on November 2, 2026

Contents

1 Introduction **1**
- 1.1 A motivating regression example . . . 1
- 1.2 Population risk, empirical risk, and the empirical measure . . . 2
- 1.3 Exact ERM and the three-error decomposition . . . 3
- 1.4 Capacity, underfitting, and overfitting . . . 3
- 1.5 Double descent . . . 8

2 First-order optimization **13**
- 2.1 Gradient descent: least squares and basic geometry . . . 13
- 2.2 Gradient descent under strong convexity . . . 16
- 2.3 Gradient descent under convexity . . . 17
- 2.4 Stochastic gradient descent . . . 19
- 2.5 Uncertainty and asymptotic normality of SGD . . . 25
- 2.6 Momentum . . . 30
- 2.7 Adaptive methods: AdaGrad, RMSprop, and Adam . . . 30
- 2.8 AdamW and Muon . . . 31

3 Concentration inequalities and uniform convergence **34**
- 3.1 From the error decomposition to concentration . . . 34
- 3.2 Chebyshev's inequality: a finite-variance baseline . . . 34
- 3.3 Hoeffding's inequality: bounded variables . . . 35
- 3.4 Bernstein's inequality: using the variance . . . 37
- 3.5 Comparing the three inequalities . . . 38
- 3.6 Sub-Gaussian and sub-exponential random variables . . . 40
- 3.7 From one predictor to a selected predictor . . . 42
- 3.8 Covering numbers: a finite approximation . . . 43
- 3.9 VC dimension for binary classification . . . 44
- 3.10 Rademacher complexity for a loss class . . . 48

4 Neural networks: representation, training, and architectures **53**
- 4.1 Feedforward networks: learned compositions . . . 53
  - 4.1.1 Why nonlinear features are needed . . . 53
  - 4.1.2 Layer notation and parameter count . . . 54
  - 4.1.3 Activation functions . . . 54
- 4.2 Approximation theory: what expressivity guarantees . . . 54
  - 4.2.1 A constructive approximation in one dimension . . . 55
- 4.3 Output rules and losses . . . 55
- 4.4 Backpropagation is a chain-rule computation . . . 56
  - 4.4.1 A scalar computation graph . . . 56
  - 4.4.2 The layerwise recursion . . . 56
  - 4.4.3 From gradients to a training step . . . 57
- 4.5 Depth, gradient flow, and initialization . . . 58
- 4.6 Generalization: choosing among fitted predictors . . . 58
- 4.7 Explicit regularization . . . 59
- 4.8 Implicit regularization and optimizer selection . . . 60
  - 4.8.1 Gradient descent from zero . . . 60

4.8.2 Early stopping and ridge as spectral filters . . . 60
4.8.3 Other sources of implicit bias . . . 61
4.9 Convolutional networks: locality and equivariance . . . 62
4.9.1 From a dense image layer to a sliding filter . . . 62
4.9.2 Two-dimensional filters and feature maps . . . 62
4.9.3 Input channels and output channels . . . 63
4.9.4 Padding, stride, and output size . . . 63
4.9.5 Shared parameters and their gradients . . . 64
4.9.6 Pooling and a complete CNN classifier . . . 64
4.9.7 Receptive fields and dilation . . . 65
4.9.8 Translation equivariance and invariant readout . . . 65
4.10 Modeling ordered context: recurrent networks . . . 66
4.10.1 Sequential prediction and a shared recurrent cell . . . 66
4.10.2 A linear RNN as a lag filter . . . 67
4.10.3 Memory decay under the same inputs . . . 67
4.10.4 Backpropagation through time . . . 68
4.10.5 From exponential smoothing to a GRU . . . 68
4.10.6 LSTM: a separate stored memory . . . 69
4.11 Transformers and self-attention . . . 70
4.11.1 From one context vector to weighted retrieval . . . 70
4.11.2 Queries, keys, values, and masks . . . 70
4.11.3 Position information and Transformer blocks . . . 71
4.11.4 The product photo as a sequence of patches . . . 72
4.11.5 Which architecture fits the needed relation? . . . 72
4.12 Optional mathematical arguments . . . 73
4.12.1 ReLU parameter symmetries . . . 73
4.12.2 Depth separation via the tent map . . . 73
4.12.3 Why logistic loss keeps increasing the margin . . . 74
4.12.4 Proof of minimum-norm selection by gradient descent . . . 74
4.12.5 The exact linear dropout calculation . . . 74
4.12.6 Permutation equivariance of attention . . . 75

5 Reinforcement learning: MDPs, planning, and learning 76
5.1 The big picture: planning, learning, and policy optimization . . . 76
5.2 Sequential decisions and discounted Markov decision processes . . . 78
5.2.1 States, actions, transitions, and rewards . . . 78
5.2.2 Decision rules and policies . . . 79
5.2.3 Why discount future rewards? . . . 79
5.3 Discounted policy evaluation . . . 79
5.3.1 Expected state values and action values . . . 79
5.3.2 The Bellman equation for a fixed policy . . . 80
5.3.3 A linear system and discounted state occupancy . . . 80
5.3.4 Iterative evaluation of a fixed policy . . . 81
5.3.5 Bellman equations and iteration for action values . . . 82
5.3.6 Discounted state–action occupancy . . . 83
5.4 Bellman optimality and stationary optimal policies . . . 83
5.4.1 Monotonicity and contraction . . . 84
5.4.2 Why one stationary policy attains the optimum . . . 84
5.4.3 Optimal action values . . . 85
5.5 Planning algorithms . . . 86
5.5.1 Value iteration and a stopping certificate . . . 86
5.5.2 Policy improvement and exact policy iteration . . . 87
5.5.3 A route example: value iteration versus policy iteration . . . 88
5.6 Learning from sampled experience . . . 88

5.6.1 A generative-model oracle . . . 89
5.6.2 The empirical MDP and its optimal policy . . . 89
5.6.3 Coordinatewise estimation of rewards and transitions . . . 89
5.6.4 Simulation lemma: model error and value error . . . 90
5.6.5 Evaluation error and the learned policy's loss . . . 91
5.6.6 The naive sample-complexity bound . . . 91
5.6.7 Data access, prediction, and control . . . 92
5.7 Monte Carlo policy evaluation . . . 93
5.7.1 First-visit evaluation . . . 93
5.7.2 The running-average update . . . 93
5.7.3 Truncated rollouts for the same discounted objective . . . 94
5.8 Temporal-difference policy evaluation . . . 94
5.8.1 The conditional mean of a TD update . . . 94
5.8.2 A stochastic-approximation convergence tool . . . 95
5.8.3 Exercises . . . 96
5.9 Function approximation for policy evaluation . . . 96
5.9.1 Linear features and Monte Carlo regression . . . 96
5.9.2 TD targets and semi-gradients . . . 97
5.9.3 Linear TD and the projected Bellman equation . . . 97
5.9.4 A fitted iteration that diverges . . . 99
5.10 Control from sampled transitions . . . 99
5.10.1 SARSA: the next action actually taken . . . 99
5.10.2 Expected SARSA . . . 100
5.10.3 Q-learning: sampled optimality backups . . . 100
5.10.4 Exploration and coverage . . . 101
5.10.5 Three targets for the same transition . . . 101
5.10.6 Control with an approximate action-value function . . . 101
5.10.7 Fitted Q-iteration . . . 102
5.10.8 Deep Q-networks: replay and target parameters . . . 102
5.10.9 Exercises . . . 102
5.11 Off-policy learning and evaluation . . . 103
5.11.1 Action coverage and a one-step change of measure . . . 103
5.11.2 An importance-corrected TD update . . . 103
5.11.3 Complete trajectories and reward prefixes . . . 104
5.11.4 Function approximation, offline data, and replay . . . 104
5.12 Policy gradients . . . 105
5.12.1 Policy scores . . . 105
5.12.2 A trajectory likelihood and the likelihood-ratio identity . . . 105
5.12.3 Causality and reward-to-go . . . 106
5.12.4 The infinite discounted limit and occupancy form . . . 106
5.12.5 REINFORCE . . . 107
5.12.6 Baselines and advantages . . . 108
5.12.7 A critic and an actor–critic update . . . 108

A Mathematical background **110**
A.1 Probability review . . . 110

Bibliographical notes **115**

Lecture 1

Introduction

1.1 A motivating regression example

Suppose we observe a sample

$$S = \{(X_i, Y_i)\}_{i=1}^n. \tag{1.1.1}$$

A useful motivational model is

$$Y = f^\dagger(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0. \tag{1.1.2}$$

Here $f^\dagger$ is an unknown signal and ε represents intrinsic randomness or unobserved variation. Gaussian noise is one example, not a universal assumption. Most of the theory below does not require the additive model (1.1.2).

For a predictor $f : \mathcal{X} \to \mathbb{R}$, the empirical squared loss is

$$\widehat{L}_n(f) := \frac{1}{2n} \sum_{i=1}^n \big(Y_i - f(X_i)\big)^2. \tag{1.1.3}$$

The linear class is

$$\mathcal{F}_{\text{lin}} := \{f_\theta : x \mapsto \theta^\top x,\ \theta \in \mathbb{R}^d\}. \tag{1.1.4}$$

Choosing this class makes a representation assumption: the signal is being approximated by a linear function. An affine predictor $x \mapsto \theta^\top x + b$ includes an intercept b. This conventional *bias parameter* is unrelated to the statistical bias of an estimator defined later in this chapter.

Generalized linear classes. For a fixed response map $\psi : \mathbb{R} \to \mathbb{R}$, consider

$$\mathcal{F}_{\text{glm}} := \{f_\theta : x \mapsto \psi(\theta^\top x),\ \theta \in \mathbb{R}^d\}. \tag{1.1.5}$$

The logistic map $\psi(u) = (1 + e^{-u})^{-1}$ produces probabilities for a binary response, while $\psi(u) = e^u$ produces a positive conditional mean. A full generalized linear model pairs the response map with an observation model and a corresponding loss. The predictor is nonlinear in x after ψ, but it still depends on x only through the one-dimensional score $\theta^\top x$.

Neural-network classes. With the ReLU activation $\rho(u) := \max\{u, 0\}$, a one-hidden-layer class is

$$\mathcal{F}_{\text{NN}}(m) := \left\{ x \mapsto a_0 + \sum_{j=1}^m a_j \rho(w_j^\top x + b_j) : {\scriptstyle a_0, a_j, b_j \in \mathbb{R},\ w_j \in \mathbb{R}^d} \right\}. \tag{1.1.6}$$

Unlike the linear and generalized linear classes above, which use the single score $\theta^\top x$ with a fixed response map, a neural network learns the hidden features $x \mapsto \rho(w_j^\top x + b_j)$ together with their output weights. This flexibility usually makes the training objective nonconvex.

These examples separate several kinds of modeling flexibility. A linear predictor uses the identity response map; a generalized linear class adds a fixed nonlinear response map; and a neural network learns multiple hidden features. Moving to a richer class or increasing m can reduce approximation error, but it also changes the statistical and optimization problems. In every case the observations are the labeled sample S, and performance is judged by prediction loss on a fresh draw rather than only by training loss.

1.2 Population risk, empirical risk, and the empirical measure

Let $Z = (X, Y)$ have an unknown distribution P on $\mathcal{X} \times \mathcal{Y}$. A loss $\ell : \mathcal{U} \times \mathcal{Y} \to [0, \infty)$ evaluates a prediction.

Common choices of loss. The loss must match the prediction space and the response encoding. Common examples include

$$\begin{aligned} \ell_{\mathrm{sq}}(a, y) &:= \frac{1}{2}(a - y)^2, && a, y \in \mathbb{R}, \\ \ell_{0\text{-}1}(a, y) &:= \mathbf{1}\{ya \le 0\}, && a \in \mathbb{R}, \quad y \in \{-1, +1\}, \\ \ell_{\mathrm{bce}}(p, y) &:= -y \log p - (1 - y) \log(1 - p), && p \in (0, 1), \quad y \in \{0, 1\}. \end{aligned}$$

The squared loss is the choice used in (1.1.3); its unrestricted population target is the conditional mean. The 0-1 loss directly counts classification errors but is discontinuous. Binary cross-entropy assesses a predicted probability. For a real-valued score and a signed label, its logistic form is

$$\ell_{\log}(a, y) := \log(1 + e^{-ya}), \qquad a \in \mathbb{R}, \quad y \in \{-1, +1\}.$$

For regression with possible outliers, the absolute loss $\ell_{\mathrm{abs}}(a, y) := |a - y|$ is a robust choice whose unrestricted population target is a conditional median. A differentiable compromise is the Huber loss

$$\ell_{\mathrm{H},\delta}(a, y) := \begin{cases} \frac{1}{2}(a - y)^2, & |a - y| \le \delta, \\ \delta|a - y| - \frac{1}{2}\delta^2, & |a - y| > \delta, \end{cases} \qquad \delta > 0.$$

It is quadratic for small residuals and linear in the tails, so a large residual has less influence than under squared loss. Only the 0-1 loss above is automatically bounded by 1; the other losses generally require additional assumptions or different concentration arguments.

Definition 1.1 (Population and empirical risk)**.** The population risk of $f : \mathcal{X} \to \mathcal{U}$ is

$$L(f) := \mathbb{E}_{Z \sim P}[\ell(f(X), Y)]. \tag{1.2.1}$$

For independent observations $Z_1, \ldots, Z_n \sim P$, the empirical risk is

$$\widehat{L}_n(f) := \frac{1}{n} \sum_{i=1}^{n} \ell(f(X_i), Y_i). \tag{1.2.2}$$

Define the empirical measure

$$P_n := \frac{1}{n} \sum_{i=1}^{n} \delta_{Z_i}. \tag{1.2.3}$$

Then $L(f) = P\ell_f$ and $\widehat{L}_n(f) = P_n \ell_f$, where $\ell_f(x, y) := \ell(f(x), y)$. Learning replaces an expectation under the unknown P by an expectation under the random P_n.

Quantity	Data role	Interpretation
Training error $\widehat{L}_{\mathrm{tr}}(f)$	Fits ordinary model parameters	An empirical objective, usually optimistically small after fitting
Population or generalization error $L(f)$	Defined by the unknown distribution	Expected loss on a fresh draw; the target quantity
Validation error $\widehat{L}_{\mathrm{val}}(f)$	Selects hyperparameters or stopping time	A selection criterion, not a final unbiased report after repeated use
Test error $\widehat{L}_{\mathrm{test}}(f)$	Used after all choices are frozen	A finite-sample estimate of $L(f)$, with its own uncertainty

1.3 Exact ERM and the three-error decomposition

Given a model class $\mathcal{F}$, empirical risk minimization is the prescription

$$\widehat{f}_n \in \arg\min_{f\in\mathcal{F}} \widehat{L}_n(f). \tag{1.3.1}$$

The operator arg min hides computation. A practical optimizer returns an iterate $\widetilde{f}_n$, not an abstract exact minimizer.

Definition 1.2 (Empirical optimization gap)**.** The empirical optimization gap of $\widetilde{f}_n \in \mathcal{F}$ is

$$\varepsilon_{\mathrm{opt}} := \widehat{L}_n(\widetilde{f}_n) - \inf_{f\in\mathcal{F}} \widehat{L}_n(f). \tag{1.3.2}$$

Equivalently,

$$\widehat{L}_n(\widetilde{f}_n) \le \inf_{f\in\mathcal{F}} \widehat{L}_n(f) + \varepsilon_{\mathrm{opt}}. \tag{1.3.3}$$

The gap measures objective value, not distance between parameters. In an overparameterized problem many parameters may have the same empirical loss, so a small gap alone does not identify which solution the algorithm selects.

For the linear regression example, let $\mathbf{X} \in \mathbb{R}^{n\times d}$ have rows $X_i^\top$ and let $\mathbf{y} = (Y_1, \dots, Y_n)^\top$. Exact ERM satisfies the normal equation

$$\mathbf{X}^\top \mathbf{X}\widehat{\theta} = \mathbf{X}^\top \mathbf{y}. \tag{1.3.4}$$

If $\mathbf{X}^\top\mathbf{X}$ is invertible, this gives a unique closed form; otherwise minimizers may not be unique. Large neural-network objectives do not offer such a formula, which is why the algorithmic output must be kept separate from the abstract ERM prescription.

Let $\mathcal{F}_{\mathrm{all}}$ be a broad benchmark class and let $\mathcal{F} \subseteq \mathcal{F}_{\mathrm{all}}$ be the working model class. Set

$$L^\star := \inf_{f\in\mathcal{F}_{\mathrm{all}}} L(f), \qquad L^\star_{\mathcal{F}} := \inf_{f\in\mathcal{F}} L(f), \qquad \Delta_n(\mathcal{F}) := \sup_{f\in\mathcal{F}} |L(f) - \widehat{L}_n(f)|. \tag{1.3.5}$$

The exact split

$$L(\widetilde{f}_n) - L^\star = \underbrace{L^\star_{\mathcal{F}} - L^\star}_{\text{approximation}} + \underbrace{L(\widetilde{f}_n) - L^\star_{\mathcal{F}}}_{\text{within-class excess risk}} \tag{1.3.6}$$

separates the price of restricting the representation from everything that happens within the class. Uniform comparison of L and $\widehat{L}_n$ then separates sampling from computation.

Theorem 1.3 (Master excess-risk bound)**.** *If $\widetilde{f}_n$ satisfies (1.3.3), then*

$$\boxed{L(\widetilde{f}_n) - L^\star \le \underbrace{L^\star_{\mathcal{F}} - L^\star}_{\textit{approximation error}} + \underbrace{2\Delta_n(\mathcal{F})}_{\textit{statistical control}} + \underbrace{\varepsilon_{\mathrm{opt}}}_{\textit{optimization error}}.} \tag{1.3.7}$$

Proof. For every comparator $f \in \mathcal{F}$,

$$\begin{aligned} L(\widetilde{f}_n) &\le \widehat{L}_n(\widetilde{f}_n) + \Delta_n(\mathcal{F}) \\ &\le \inf_{g\in\mathcal{F}} \widehat{L}_n(g) + \varepsilon_{\mathrm{opt}} + \Delta_n(\mathcal{F}) \\ &\le \widehat{L}_n(f) + \varepsilon_{\mathrm{opt}} + \Delta_n(\mathcal{F}) \\ &\le L(f) + \varepsilon_{\mathrm{opt}} + 2\Delta_n(\mathcal{F}). \end{aligned}$$

Subtract $L^\star$ and take the infimum over $f \in \mathcal{F}$. □

1.4 Capacity, underfitting, and overfitting

Capacity describes the variety of functions that a learning procedure can effectively choose among [9, Section 5.2]. It is therefore not synonymous with parameter count: two parameterizations may describe the same functions, while a norm constraint or a training rule may make only a small part of a large parameterized family practically accessible.

Representational capacity and three different risks

Suppose that a complexity index q produces nested classes

$$\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \cdots . \tag{1.4.1}$$

Depending on the application, q might count selected features, polynomial degree, network width or depth, or the radius of a norm constraint.

Concrete example: polynomial degree. For a scalar input, consider

$$\mathcal{F}_q^{\mathrm{poly}} := \left\{ x \mapsto \sum_{j=0}^{q} \theta_j x^j : \theta \in \mathbb{R}^{q+1} \right\}. \tag{1.4.2}$$

If the signal is approximately quadratic, a line may miss systematic curvature, while a quadratic can capture the main pattern. At the other extreme, with n distinct training inputs, a polynomial of degree at most $n-1$ can interpolate arbitrary observed responses.

The construction in Figure 1.1 follows the book's example: six noiseless observations lie exactly on a quadratic signal. A line cannot express its curvature. The quadratic class contains the signal. A degree-nine class contains the signal too, but its ten coefficients are underdetermined by six interpolation constraints, so many functions achieve zero training loss. A tie-breaking rule such as a Moore–Penrose solution can select an interpolant whose behavior between observations is very different. Thus sparse sampling alone can permit overfitting; label noise would make the problem still more visible.

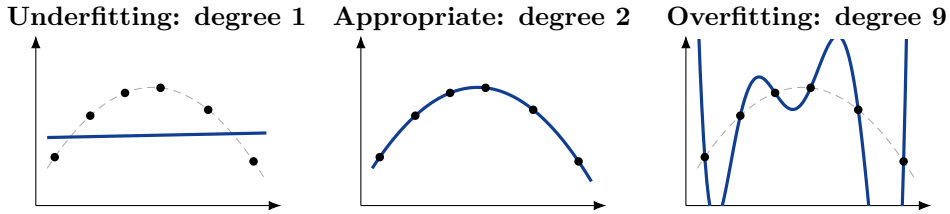


Figure 1.1: Three fits to the same sparse, noiseless quadratic sample. The gray dashed curve is the signal and the blue curve is the fitted predictor. The right panel shows one possible degree-nine interpolant: every observed point is fitted exactly, yet the behavior between points is poorly constrained. Adapted from [9, Figure 5.2].

The example makes capacity visible, but it does not make degree or parameter count a universal measure. Architecture, parameter sharing, norms, and the training algorithm all change which fitted functions are easy to obtain.

Returning to the general nested-class setup, three quantities must be kept separate:

$$\widehat{L}^{\star}_{n,q} := \inf_{f \in \mathcal{F}_q} \widehat{L}_n(f), \qquad L^{\star}_{\mathcal{F}_q} := \inf_{f \in \mathcal{F}_q} L(f), \qquad \widehat{f}_{q,S} := \mathsf{A}_q(S). \tag{1.4.3}$$

The first is the best training risk available in the class, the second is the best population risk available in the class, and the third is what a specified algorithm actually returns from sample S. Nesting gives the deterministic monotonicity relations

$$\widehat{L}^{\star}_{n,q+1} \le \widehat{L}^{\star}_{n,q}, \qquad L^{\star}_{\mathcal{F}_{q+1}} \le L^{\star}_{\mathcal{F}_q}. \tag{1.4.4}$$

In contrast, $L(\widehat{f}_{q,S})$ need not decrease with q. The fitted predictor depends on a finite sample, initialization, optimization, and any regularization or stopping rule. A larger class creates more opportunities to represent useful structure, but also more opportunities for the procedure to respond to sample-specific variation.

In particular, nesting does not produce a U-shaped population-risk curve: both class-oracle quantities in (1.4.4) decrease.

Bias–variance trade-off under squared loss

Let $S \sim P^n$ be the training sample and let $\widehat{f}_{q,S} := \mathsf{A}_q(S)$ be the predictor returned at capacity q. We will use the repeated-sample averages

$$T(q) := \mathbb{E}_S\left[\widehat{L}_n(\widehat{f}_{q,S})\right], \qquad R(q) := \mathbb{E}_S\left[L(\widehat{f}_{q,S})\right]. \tag{1.4.5}$$

For the exact additive explanation under squared loss, let $m(x) := \mathbb{E}[Y \mid X = x]$, let the evaluation pair (X, Y) be independent of S, and set $\overline{f}_q(x) := \mathbb{E}_S[\widehat{f}_{q,S}(x)]$. At a fixed input x, the estimator bias and variance are

$$\mathrm{Bias}_S\{\widehat{f}_{q,S}(x)\} := \overline{f}_q(x) - m(x), \qquad \mathrm{Var}_S\{\widehat{f}_{q,S}(x)\} := \mathbb{E}_S\left[\{\widehat{f}_{q,S}(x) - \overline{f}_q(x)\}^2\right].$$

They are properties of the learning procedure across hypothetical repetitions of the complete training sample, not quantities that can be read from one fitted model. Because this chapter uses $\ell(a, y) = \frac{1}{2}(a - y)^2$,

$$R(q) = \underbrace{\frac{1}{2}\mathbb{E}_X[\mathrm{Var}(Y \mid X)]}_{\text{irreducible noise}} + \underbrace{\frac{1}{2}\left\|\overline{f}_q - m\right\|^2_{L^2(P_X)}}_{\text{squared estimator bias}} + \underbrace{\frac{1}{2}\mathbb{E}_X\left[\mathrm{Var}_S\{\widehat{f}_{q,S}(X)\}\right]}_{\text{estimator variance}}. \tag{1.4.6}$$

To verify the identity, write

$$Y - \widehat{f}_{q,S}(X) = \{Y - m(X)\} + \{m(X) - \overline{f}_q(X)\} + \{\overline{f}_q(X) - \widehat{f}_{q,S}(X)\}.$$

The cross terms vanish after conditioning on X and averaging over the independent training sample. In the motivating additive model, $m = f^\dagger$.

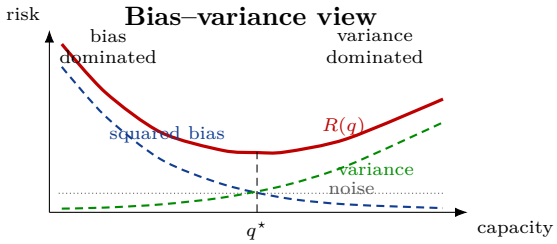


Figure 1.2: Bias–variance view of the finite-sample capacity trade-off under squared loss, adapted from [9, Figure 5.6]. The red curve is the repeated-sample population risk $R(q)$ in (1.4.5), decomposed by (1.4.6). The U shape is a typical schematic, not a universal law.

Diagnosing underfitting and overfitting

Underfitting means that the fitted predictor fails to capture useful, transferable structure. High training and independent-validation losses are a common symptom. The cause may be a restrictive class, incomplete optimization, excessive regularization, inappropriate features, or a poorly matched loss. *Overfitting* means that the fitted predictor uses sample-specific variation in a way that does not transfer. Low training loss together with a substantially larger independent-validation loss is a common symptom. Low training loss by itself is not evidence of overfitting. Likewise, the mnemonics *underfitting means high bias* and *overfitting means high variance* describe common mechanisms, not definitions. Bias and variance average over hypothetical repetitions of the whole training sample; one train–validation split can reveal symptoms but cannot identify either quantity.

Observed pattern	Plausible diagnosis	Useful next comparison
Training and validation losses are both high	The representation or optimization may be too restrictive	Increase class size, train longer, or weaken regularization, changing one factor at a time.
Training loss is low, but validation loss is much higher	The procedure may be sensitive to the sample; leakage or excessive tuning is also possible	Add data or regularization, reduce effective capacity, and audit the split and preprocessing pipeline.
Training and validation losses are both low and close	No obvious underfitting or overfitting signal	Freeze modeling decisions and use the untouched test set once for final estimation.

Learning curves add an important axis to this diagnosis. If the training– validation gap shrinks as the sample size grows, finite-sample variability is a plausible limitation. If both curves plateau at an unsatisfactory level, representation, optimization, regularization, or the chosen loss may be the bottleneck. These patterns are clues, not logical equivalences: distribution shift, label noise, leakage, and a mismatched evaluation metric can imitate the same symptoms.

Hyperparameters, validation, and regularization

Ordinary parameters are fitted inside a chosen training procedure. *Hyperparameters* specify that procedure: examples include class size q, network architecture, a regularization level, learning rate, batch size, and stopping time. Capacity-controlling hyperparameters cannot be chosen by minimizing training error alone, because that criterion systematically favors more fit and less regularization.

A clean protocol is nested:

1. use the training set to produce a candidate $\widehat{f}_\lambda$ for each $\lambda \in \Lambda$;
2. use independent validation data to choose $\widehat{\lambda}$;
3. freeze every choice and use an untouched test set once to estimate $L(\widehat{f}_{\widehat{\lambda}})$.

Validation is itself a model-selection problem, so its reported minimum is optimistic after search. If the test set is inspected repeatedly, it has quietly become another validation set. There is no universal 80/20 split: the allocation balances fitting accuracy against selection and evaluation uncertainty. The finite-class uniform-convergence argument in Lecture 3 quantifies this selection effect when the candidate set is finite and the validation sample is independent.

Regularization expresses a preference among solutions. A constrained version uses

$$\mathcal{F}_R := \{f \in \mathcal{F} : \Omega(f) \le R\},$$

whereas a penalized procedure minimizes

$$\widehat{L}_n(f) + \lambda\Omega(f).$$

The second display defines a different empirical objective for every λ. Its optimization gap must be measured relative to that penalized objective, not silently substituted for ε_{opt} in the unregularized master bound. Regularization can change approximation bias, sampling variability, and optimization geometry at the same time; it is not a fourth independent error term.

Ridge regression as a capacity-control example. For clarity, omit the intercept; in practice it is usually left unpenalized, and all centering constants must be learned from the training set only. For $\mathbf{X} \in \mathbb{R}^{n\times d}$, $\mathbf{y} \in \mathbb{R}^n$, and $\lambda \ge 0$, ridge regression solves

$$\widehat{\theta}_\lambda \in \operatorname*{arg\,min}_{\theta\in\mathbb{R}^d} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\theta\|^2 + \frac{\lambda}{2} \|\theta\|^2 \right\}.$$

For $\lambda > 0$, the first-order condition gives

$$\widehat{\theta}_\lambda = (\mathbf{X}^\top\mathbf{X} + n\lambda I_d)^{-1}\mathbf{X}^\top\mathbf{y}.$$

The factor $n\lambda$ appears because the empirical squared loss is divided by n. At $\lambda = 0$, take the minimum-norm least-squares solution $\widehat{\theta}_0 = \mathbf{X}^\dagger\mathbf{y}$; it is the limit of $\widehat{\theta}_\lambda$ as $\lambda \downarrow 0$.

Let $\mathbf{X} = \mathbf{U}_r \operatorname{diag}(s_1, \dots, s_r)\mathbf{V}_r^\top$ be a thin singular-value decomposition and define

$$a_j(\lambda) := \frac{s_j^2}{s_j^2 + n\lambda}.$$

Then

$$\widehat{\theta}_\lambda = \sum_{j=1}^{r} \frac{s_j}{s_j^2 + n\lambda}(\mathbf{u}_j^\top \mathbf{y})\mathbf{v}_j,$$

$$\widehat{\mathbf{y}}_\lambda := \mathbf{X}\widehat{\theta}_\lambda = \sum_{j=1}^{r} a_j(\lambda)(\mathbf{u}_j^\top \mathbf{y})\mathbf{u}_j.$$

Thus ridge shrinks the unregularized fit in direction j by $a_j(\lambda)$. Small-singular-value directions are damped most strongly, and the effective degrees of freedom

$$\operatorname{tr}\big[\mathbf{X}(\mathbf{X}^\top\mathbf{X} + n\lambda I_d)^{-1}\mathbf{X}^\top\big] = \sum_{j=1}^{r} a_j(\lambda)$$

decrease with λ.

The training residual makes clear why training loss cannot choose the amount of regularization:

$$\|\mathbf{y} - \widehat{\mathbf{y}}_\lambda\|^2 = \big\|(I_n - \mathbf{U}_r\mathbf{U}_r^\top)\mathbf{y}\big\|^2 + \sum_{j=1}^{r}\left(\frac{n\lambda}{s_j^2 + n\lambda}\right)^2 (\mathbf{u}_j^\top \mathbf{y})^2.$$

Every λ-dependent factor on the right is nondecreasing, so training squared error favors $\lambda = 0$, up to ties. Minimized penalized-objective values should not be used to select λ: changing λ changes the criterion, so their ordering does not estimate a common prediction risk. Instead, fit each candidate on the training set and choose

$$\widehat{\lambda} \in \arg\min_{\lambda \in \Lambda} \frac{1}{2n_{\text{val}}} \left\|\mathbf{y}_{\text{val}} - \mathbf{X}_{\text{val}}\widehat{\theta}_\lambda^{\text{tr}}\right\|^2.$$

Ridge also makes the coefficient bias–variance mechanism explicit. Conditional on the design, suppose $\mathbf{y} = \mathbf{X}\theta^\star + \boldsymbol{\varepsilon}$, with $\mathbb{E}[\boldsymbol{\varepsilon} \mid \mathbf{X}] = 0$ and $\operatorname{Cov}(\boldsymbol{\varepsilon} \mid \mathbf{X}) = \sigma^2 I_n$. For $\alpha_j := \mathbf{v}_j^\top \theta^\star$,

$$\mathbf{v}_j^\top \widehat{\theta}_\lambda = a_j(\lambda)\alpha_j + \frac{s_j}{s_j^2 + n\lambda}\mathbf{u}_j^\top \boldsymbol{\varepsilon},$$

so the coefficient in direction j has

$$\operatorname{Bias}_j^2(\lambda) = \{1 - a_j(\lambda)\}^2\alpha_j^2, \qquad \operatorname{Var}_j(\lambda) = \frac{\sigma^2 s_j^2}{(s_j^2 + n\lambda)^2}.$$

As λ increases, squared shrinkage bias is nondecreasing and sampling variance is nonincreasing in each observable direction. For isotropic future covariates, summing these terms, together with any unidentifiable null-space bias, gives the prediction bias and variance; a general test covariance weights and couples directions differently. The best trade-off therefore depends on the unknown signal, noise, and future covariate distribution, which is precisely why λ is selected by validation. It also previews why ridge can smooth the interpolation peak discussed next.

Remark 1.4 (k-fold cross-validation)**.** When data are scarce, split the sample into k folds, train k times, and evaluate each observation using a model that did not train on its fold. This uses data efficiently but costs k fits. Because the fitted training sets overlap, fold errors are dependent, so a naive independent-fold standard error is not justified. If cross-validation both selects hyperparameters and reports final performance, an outer evaluation layer or untouched test set is needed.

1.5 Double descent

Figure 1.2 shows the classical picture obtained when capacity is increased through an underfitting regime and into an overfitting regime. If the horizontal axis is continued until the learning procedure first achieves zero training loss, and then continued farther, a second pattern can appear. Write q_{int} for this *interpolation threshold*. The expected population risk may rise sharply near q_{int} and then decrease again in a more strongly overparameterized regime. The resulting curve is called *double descent* [1].

Throughout this subsection, $n := |S|$ is the number of training examples. We hold n fixed while increasing the general capacity index q. In the linear model below we set $q = d$, where d is the number of input coordinates retained by the learner and, because no intercept is fitted, also the number of unknown regression coefficients. The aspect ratio $\gamma := d/n$ distinguishes the underparameterized regime $d < n$, the critical regime $d \approx n$, and the overparameterized regime $d > n$.

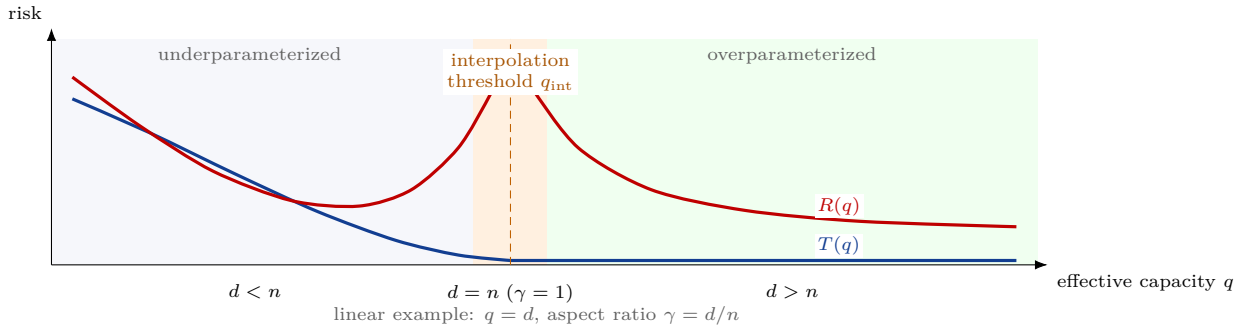


Figure 1.3: Schematic model-wise double descent, adapted from [1]. The quantities $T(q)$ and $R(q)$ are the repeated-sample averages defined in (1.4.5). Training risk first reaches zero at the interpolation threshold. Population risk can peak there and then fall as the procedure moves farther into an overparameterized regime. The shape is qualitative, not a theorem for every nested class; the labels involving d specialize the generic capacity axis to the linear model below.

A Gaussian linear model where the peak is explicit. Let $X = (X_1, X_2, \ldots)$ have independent standard Gaussian coordinates and consider the fixed regression function

$$m(X) = \sum_{j \geq 1} \beta_j X_j, \qquad Y = m(X) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2), \qquad \varepsilon \perp\!\!\!\perp X,$$

with $\sum_{j \geq 1} \beta_j^2 < \infty$. At capacity d, the learner uses only $X_1, \ldots, X_d$ and minimizes squared loss. For a training sample $S = \{(X_i, Y_i)\}_{i=1}^n$, let $\mathbf{X}_d \in \mathbb{R}^{n \times d}$ have row $X_{i,1:d}^\top$, and let $\mathbf{y} = (Y_1, \ldots, Y_n)^\top$. Thus $\mathbf{X}_d$ has one row per training example and one column per retained feature.

Solving the three least-squares regimes. For a coefficient vector $b \in \mathbb{R}^d$, the empirical objective, its gradient, and the normal equation are

$$\widehat{L}_{n,d}(b) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}_d b\|_2^2, \qquad \nabla \widehat{L}_{n,d}(b) = \frac{1}{n} \mathbf{X}_d^\top (\mathbf{X}_d b - \mathbf{y}), \qquad \mathbf{X}_d^\top \mathbf{X}_d b = \mathbf{X}_d^\top \mathbf{y}. \tag{1.5.1}$$

A Gaussian n-by-d design has rank $\min\{n, d\}$ almost surely. Consequently, the Moore–Penrose solution has the three explicit forms

$$\widehat{\beta}_d := \mathbf{X}_d^\dagger \mathbf{y} = \begin{cases} (\mathbf{X}_d^\top \mathbf{X}_d)^{-1} \mathbf{X}_d^\top \mathbf{y}, & d < n, \\ \mathbf{X}_d^{-1} \mathbf{y}, & d = n, \\ \mathbf{X}_d^\top (\mathbf{X}_d \mathbf{X}_d^\top)^{-1} \mathbf{y}, & d > n, \end{cases} \qquad \widehat{f}_d(x) = x_{1:d}^\top \widehat{\beta}_d. \tag{1.5.2}$$

For $d < n$, $\mathbf{X}_d^\top \mathbf{X}_d$ is invertible, so the normal equation has the unique ordinary-least-squares solution in the first line. At $d = n$, the square matrix $\mathbf{X}_d$ is invertible and the unique solution fits $\mathbf{X}_d \widehat{\beta}_d = \mathbf{y}$. For $d > n$, the matrix has full row rank. Set

$$\beta_0 := \mathbf{X}_d^\top (\mathbf{X}_d \mathbf{X}_d^\top)^{-1} \mathbf{y} = \mathbf{X}_d^\dagger \mathbf{y}.$$

This is one interpolator because

$$\mathbf{X}_d \beta_0 = \mathbf{X}_d \mathbf{X}_d^\top (\mathbf{X}_d \mathbf{X}_d^\top)^{-1} \mathbf{y} = \mathbf{y}.$$

Consequently, if $v \in \ker(\mathbf{X}_d)$, then $\mathbf{X}_d(\beta_0 + v) = \mathbf{y}$, so every $\beta_0 + v$ is an interpolator. Conversely, if b is any interpolator, then $\mathbf{X}_d(b - \beta_0) = \mathbf{y} - \mathbf{y} = 0$, so $b - \beta_0 \in \ker(\mathbf{X}_d)$. The two inclusions prove that the complete solution set is

$$\{b \in \mathbb{R}^d : \mathbf{X}_d b = \mathbf{y}\} = \{\beta_0 + v : v \in \ker(\mathbf{X}_d)\} = \left\{\mathbf{X}_d^\top (\mathbf{X}_d \mathbf{X}_d^\top)^{-1} \mathbf{y} + v : v \in \ker(\mathbf{X}_d)\right\}. \tag{1.5.3}$$

Moreover, $\beta_0 \in \text{range}(\mathbf{X}_d^\top)$, while the fundamental theorem of linear algebra gives $\text{range}(\mathbf{X}_d^\top) = \ker(\mathbf{X}_d)^\perp$. Hence $\beta_0 \perp v$ and

$$\|\beta_0 + v\|_2^2 = \|\beta_0\|_2^2 + \|v\|_2^2.$$

Thus the norm is minimized uniquely at $v = 0$, proving that $\mathbf{X}_d^\dagger \mathbf{y}$ is the minimum-norm interpolator. It is also the solution selected by gradient descent from zero for a suitable fixed step size; the zero initialization is essential for this implicit selection.

The distinction is visible directly in the fitted values and training loss. For $d < n$, define the hat matrix $\mathbf{H}_d := \mathbf{X}_d(\mathbf{X}_d^\top \mathbf{X}_d)^{-1} \mathbf{X}_d^\top$. Then

$$\mathbf{X}_d \widehat{\beta}_d = \begin{cases} \mathbf{H}_d \mathbf{y}, & d < n, \\ \mathbf{y}, & d \geq n, \end{cases} \qquad \widehat{L}_{n,d}(\widehat{\beta}_d) = \begin{cases} \dfrac{1}{2n} \|(\mathbf{I}_n - \mathbf{H}_d)\mathbf{y}\|_2^2, & d < n, \\ 0, & d \geq n. \end{cases} \tag{1.5.4}$$

With nonzero continuous observation or omitted-coordinate noise, the residual in the first line is nonzero almost surely. Thus $d = n$ is the first dimension at which arbitrary training responses can be interpolated. For this specialization $q = d$, the interpolation threshold is $q_{\text{int}} = n$, equivalently $\gamma = 1$.

Define the omitted-signal energy, included-signal energy, and effective noise by

$$a_d := \sum_{j>d} \beta_j^2, \qquad B_d^2 := \sum_{j \leq d} \beta_j^2, \qquad \tau_d^2 := \sigma^2 + a_d.$$

The omitted coordinates are independent of the fitted coordinates and act like additional Gaussian training noise. More precisely,

$$\mathbf{y} = \mathbf{X}_d \beta_{1:d} + \boldsymbol{\eta}_d, \qquad \boldsymbol{\eta}_d \sim \mathcal{N}(0, \tau_d^2 \mathbf{I}_n), \qquad \boldsymbol{\eta}_d \perp\!\!\!\perp \mathbf{X}_d.$$

For $d < n$, equation (1.5.4) and $(\mathbf{I}_n - \mathbf{H}_d)\mathbf{X}_d = 0$ give the following calculation:

$$\mathbb{E}\left[\|(\mathbf{I}_n - \mathbf{H}_d)\mathbf{y}\|_2^2 \mid \mathbf{X}_d\right] = \tau_d^2 (n - d).$$

This follows because $\mathbf{H}_d$ is a rank-d orthogonal projector; the residual projector has trace $n - d$. Therefore the repeated-sample training curve is exactly

$$T(d) = \mathbb{E}_S\left[\widehat{L}_{n,d}(\widehat{\beta}_d)\right] = \begin{cases} \dfrac{\tau_d^2}{2}\left(1 - \dfrac{d}{n}\right), & d < n, \\ 0, & d \geq n. \end{cases} \tag{1.5.5}$$

Deriving the test curve. Set $\theta_d := \beta_{1:d} \in \mathbb{R}^d$, so $\|\theta_d\|_2^2 = B_d^2$. For an independent test observation, write

$$Y' = (X'_{1:d})^\top \theta_d + \eta'_d, \qquad \eta'_d := \sum_{j>d} \beta_j X'_j + \varepsilon'.$$

Then $X'_{1:d} \sim \mathcal{N}(0, \mathbf{I}_d)$, $\eta'_d \sim \mathcal{N}(0, \tau_d^2)$, and these variables are independent of each other and of the training sample. With $\Delta_d := \widehat{\beta}_d - \theta_d$, conditioning on the realized training sample gives

$$\mathbb{E}\left[(Y' - \widehat{f}_d(X'))^2 \mid S\right] = \mathbb{E}\left[\left(\eta'_d - (X'_{1:d})^\top \Delta_d\right)^2 \mid S\right]$$

$$= \tau_d^2 + \Delta_d^\top \mathbb{E}[X'_{1:d}(X'_{1:d})^\top]\Delta_d$$
$$= \tau_d^2 + \|\Delta_d\|_2^2.$$

The cross term in the second line is zero because η'_d and $X'_{1:d}$ are independent and centered. Thus it remains to calculate the mean squared coefficient error.

For $d < n$, ordinary least squares gives

$$\Delta_d = (\mathbf{X}_d^\top \mathbf{X}_d)^{-1}\mathbf{X}_d^\top \boldsymbol{\eta}_d.$$

Consequently,

$$\begin{aligned}\mathbb{E}[\|\Delta_d\|_2^2 \mid \mathbf{X}_d] &= \operatorname{tr}\big(\mathbb{E}[\Delta_d\Delta_d^\top \mid \mathbf{X}_d]\big)\\ &= \tau_d^2 \operatorname{tr}\big((\mathbf{X}_d^\top \mathbf{X}_d)^{-1}\mathbf{X}_d^\top \mathbf{X}_d(\mathbf{X}_d^\top \mathbf{X}_d)^{-1}\big)\\ &= \tau_d^2 \operatorname{tr}\big((\mathbf{X}_d^\top \mathbf{X}_d)^{-1}\big).\end{aligned}$$

Here $\mathbf{X}_d^\top \mathbf{X}_d$ is a d-dimensional Wishart matrix with n degrees of freedom. Its inverse first moment is

$$\mathbb{E}[(\mathbf{X}_d^\top \mathbf{X}_d)^{-1}] = \frac{\mathbf{I}_d}{n-d-1}, \qquad n > d+1.$$

It follows that

$$\mathbb{E}[\|\Delta_d\|_2^2] = \tau_d^2 \frac{d}{n-d-1}, \qquad d < n-1.$$

For $d > n$, introduce the d-by-d row-space projector

$$\boldsymbol{\Pi}_d := \mathbf{X}_d^\top (\mathbf{X}_d\mathbf{X}_d^\top)^{-1}\mathbf{X}_d$$

and $\mathbf{A}_d := \mathbf{X}_d^\top (\mathbf{X}_d\mathbf{X}_d^\top)^{-1}$. The minimum-norm solution satisfies

$$\widehat{\beta}_d = \boldsymbol{\Pi}_d\theta_d + \mathbf{A}_d\boldsymbol{\eta}_d, \qquad \Delta_d = -(\mathbf{I}_d - \boldsymbol{\Pi}_d)\theta_d + \mathbf{A}_d\boldsymbol{\eta}_d.$$

The first term lies in $\ker(\mathbf{X}_d)$, whereas the second lies in $\operatorname{range}(\mathbf{X}_d^\top) = \ker(\mathbf{X}_d)^\perp$. They are therefore orthogonal for every realization. Moreover, $\mathbf{A}_d^\top \mathbf{A}_d = (\mathbf{X}_d\mathbf{X}_d^\top)^{-1}$, so

$$\mathbb{E}[\|\Delta_d\|_2^2 \mid \mathbf{X}_d] = \|(\mathbf{I}_d - \boldsymbol{\Pi}_d)\theta_d\|_2^2 + \tau_d^2 \operatorname{tr}\big((\mathbf{X}_d\mathbf{X}_d^\top)^{-1}\big).$$

The Gaussian row space is a uniformly random n-dimensional subspace of $\mathbb{R}^d$. Rotational invariance therefore gives $\mathbb{E}[\boldsymbol{\Pi}_d] = c\mathbf{I}_d$; taking traces and using $\operatorname{tr}(\boldsymbol{\Pi}_d) = n$ gives $c = n/d$. Since $\mathbf{I}_d - \boldsymbol{\Pi}_d$ is itself an orthogonal projector,

$$\mathbb{E}\big[\|(\mathbf{I}_d - \boldsymbol{\Pi}_d)\theta_d\|_2^2\big] = \theta_d^\top\big(\mathbf{I}_d - \mathbb{E}[\boldsymbol{\Pi}_d]\big)\theta_d = \Big(1 - \frac{n}{d}\Big)B_d^2.$$

Also $\mathbf{X}_d\mathbf{X}_d^\top$ is an n-dimensional Wishart matrix with d degrees of freedom, and hence

$$\mathbb{E}[(\mathbf{X}_d\mathbf{X}_d^\top)^{-1}] = \frac{\mathbf{I}_n}{d-n-1}, \qquad d > n+1.$$

Therefore

$$\mathbb{E}[\|\Delta_d\|_2^2] = \Big(1 - \frac{n}{d}\Big)B_d^2 + \tau_d^2 \frac{n}{d-n-1}, \qquad d > n+1.$$

Both Wishart identities follow from one inverse-Gram calculation. If $\mathbf{Z} \in \mathbb{R}^{m\times p}$ has independent standard Gaussian entries and $m > p+1$, then

$$\mathbb{E}[(\mathbf{Z}^\top\mathbf{Z})^{-1}] = \frac{\mathbf{I}_p}{m-p-1}.$$

Applying it to $\mathbf{Z} = \mathbf{X}_d$ and $\mathbf{Z} = \mathbf{X}_d^\top$ yields the two moments used above and explains the strict dimension conditions.

Combining the coefficient errors with the independent test-noise term τ_d^2, the expected test mean-squared error is, away from the critical dimensions,

$$\mathcal{E}_d := \mathbb{E}\Big[(Y' - \widehat{f}_d(X'))^2\Big] = \begin{cases} \tau_d^2\left(1 + \dfrac{d}{n-d-1}\right), & d < n-1, \\ B_d^2\left(1 - \dfrac{n}{d}\right) + \tau_d^2\left(1 + \dfrac{n}{d-n-1}\right), & d > n+1. \end{cases} \tag{1.5.6}$$

Under the loss convention of this chapter, $R(d) = \frac{1}{2}\mathcal{E}_d$. The expectation in (1.5.6) is not finite at $d \in \{n-1, n, n+1\}$ when $\tau_d^2 > 0$: individual fitted risks are finite almost surely, but nearly singular designs give the average a heavy tail [10].

When does this formula actually give double descent? Equation (1.5.6) permits double descent, but does not force it. On the underparameterized branch,

$$\mathcal{E}_d = \tau_d^2 \frac{n-1}{n-d-1}, \qquad \tau_{d+1}^2 = \tau_d^2 - \beta_{d+1}^2.$$

For $d < n-2$, so that both adjacent expectations are finite, direct subtraction gives

$$\begin{aligned} \mathcal{E}_{d+1} - \mathcal{E}_d &= \frac{n-1}{n-d-2}(\tau_d^2 - \beta_{d+1}^2) - \frac{n-1}{n-d-1}\tau_d^2 \\ &= \frac{n-1}{(n-d-2)(n-d-1)}\left[\tau_d^2 - (n-d-1)\beta_{d+1}^2\right]. \end{aligned}$$

Consequently,

$$\mathcal{E}_{d+1} < \mathcal{E}_d \quad \Longleftrightarrow \quad \beta_{d+1}^2 > \frac{\tau_d^2}{n-d-1}.$$

The reverse inequality gives an ascent, and equality gives a flat step. Thus the decrease in τ_d^2 and the increase in $(n-1)/(n-d-1)$ compete; monotonicity of the two factors does not determine their product. For example, if $\beta_{d+1} = 0$ and $\tau_d^2 > 0$, that step strictly increases risk. If all added coordinates carry no signal, the entire finite underparameterized branch increases, so there is no first descent. Irregularly ordered coefficient magnitudes can even create several local turns.

The comparison from $d = n-2$ to $d = n-1$ is excluded from the finite difference above. When $\tau_{n-1}^2 > 0$, the expected risk at $d = n-1$ is infinite. This produces the singular peak around interpolation, but does not imply that the finite branch $d < n-1$ contains both a descent and an ascent.

On the overparameterized branch, write

$$\mathcal{E}_d = B_d^2\left(1 - \frac{n}{d}\right) + \tau_d^2 v_d, \qquad v_d := \frac{d-1}{d-n-1}, \qquad d > n+1.$$

The noise multiplier decreases because

$$v_{d+1} - v_d = \frac{d}{d-n} - \frac{d-1}{d-n-1} = -\frac{n}{(d-n)(d-n-1)} < 0.$$

Since τ_d^2 is also nonincreasing, the effective-noise contribution $\tau_d^2 v_d$ decreases beyond the threshold. In contrast, $B_d^2(1 - n/d)$ is nondecreasing and can offset that reduction. The familiar double-descent shape therefore requires three comparisons to go the right way: informative early coordinates create a first descent, positive effective noise creates the interpolation peak, and post-threshold noise reduction dominates the growing null-space signal term. The formula displays this mechanism; it is not a universal shape theorem.

The three parts of the curve now have distinct causes.

1. As informative coordinates are added, a_d decreases. If that approximation gain initially dominates estimation variability, the first descent appears.
2. Near $d = n$, the smallest singular values of $\mathbf{X}_d$ can be tiny. Least squares divides by those singular values, strongly amplifying label noise and omitted-feature noise. This produces the interpolation peak.
3. For $d > n$, training loss is zero and there are infinitely many interpolators. The minimum-norm selector has noise-amplification term $\tau_d^2 n/(d-n-1)$, which decreases beyond the threshold and can produce the second descent.

Connection to bias and variance. Equation (1.4.6) remains exact. For example, in the overparameterized Gaussian model above, the squared estimator bias and variance contributions to excess prediction MSE are

$$\mathrm{Bias}_d^2 = a_d + \left(1 - \frac{n}{d}\right)^2 B_d^2,$$
$$\mathrm{Var}_d = \frac{n}{d}\left(1 - \frac{n}{d}\right) B_d^2 + \tau_d^2 \frac{n}{d - n - 1}.$$

Their sum is $\mathcal{E}_d - \sigma^2$, up to the factor $1/2$ in our loss. Thus double descent is not a new term outside bias–variance; it is a possible nonmonotone shape of their sum. In particular, variance can peak near interpolation and then decrease.

Related literatures. The mechanism is analytically visible in ridgeless or minimum-norm linear regression with random design [10]. Random-feature regression, equivalently a two-layer network whose random first-layer weights are frozen and whose output layer is fit by least squares, provides a nonlinear model with a precise double-descent analysis [17]. Model-wise and epoch-wise double descent have also been observed empirically for CNNs, ResNets, and Transformers trained close to interpolation [20]. These examples are not a sufficient condition: the peak depends on the capacity path, noise, regularization, early stopping, and the optimizer's implicit choice among interpolators. Ridge regularization or early stopping can substantially smooth or remove it.

Lecture 2

First-order optimization

The master decomposition in Theorem 1.3 shows that good population performance requires control of three distinct quantities:

$$L(\widetilde{f}_n) - L^\star \le \underbrace{L^\star_{\mathcal{F}} - L^\star}_{\text{approximation}} + \underbrace{2\Delta_n(\mathcal{F})}_{\text{statistical control}} + \underbrace{\varepsilon_{\text{opt}}}_{\text{optimization}} .$$

The model class determines the approximation term, and Lecture 3 will control the empirical-to-population deviation. This lecture addresses the remaining question: with finite computation, how close does an algorithm get to the best empirical objective value available inside the chosen class?

More concretely, suppose $\mathcal{F} = \{f_\theta : \theta \in \mathcal{K}\}$, define the empirical objective $\Phi_S(\theta) := \widehat{L}_n(f_\theta)$, and let an algorithm return θ_T after T updates. Its contribution to the master bound is

$$\varepsilon_{\text{opt,T}} := \Phi_S(\theta_T) - \inf_{\theta \in \mathcal{K}} \Phi_S(\theta) \ge 0.$$

Conditional on the training sample S, this is an ordinary optimization problem: full-batch methods are deterministic once initialized, whereas SGD adds mini-batch randomness. The goal below is to relate $\varepsilon_{\text{opt,T}}$ to curvature, step sizes, gradient noise, and the iteration budget. A small parameter error is not the definition of success; the quantity required by the learning decomposition is the objective gap displayed above.

2.1 Gradient descent: least squares and basic geometry

To avoid confusing the population risk L with a smoothness constant, write the optimization objective as $\Phi : \mathbb{R}^d \to \mathbb{R}$ and use β for smoothness.

Gradient descent for least squares

Begin with the concrete objective

$$\Phi(\theta) = \frac{1}{2} \|A\theta - b\|^2 . \tag{2.1.1}$$

Its gradient is

$$\nabla\Phi(\theta) = A^\top A\theta - A^\top b, \tag{2.1.2}$$

so gradient descent with a constant step size η becomes

$$\theta_{t+1} = (I - \eta A^\top A)\theta_t + \eta A^\top b. \tag{2.1.3}$$

If $\theta^\star$ satisfies the normal equation $A^\top A\theta^\star = A^\top b$, subtraction gives the exact error recursion

$$\theta_{t+1} - \theta^\star = (I - \eta A^\top A)(\theta_t - \theta^\star). \tag{2.1.4}$$

We now turn this recursion into an exact convergence rate. Assume first that A has full column rank, so

$$H := A^\top A \succ 0, \qquad \mu := \lambda_{\min}(H) > 0, \qquad \beta := \lambda_{\max}(H). \tag{2.1.5}$$

In this case least squares is μ-strongly convex and β-smooth.

Proposition 2.1 (Exact convergence of least-squares gradient descent)**.** *Let A have full column rank, let $\theta^\star$ be the unique least-squares minimizer, and use a constant step size $0 < \eta < 2/\beta$. Define*

$$q(\eta) := \max_{1\le j\le d} |1 - \eta\lambda_j(H)| < 1. \tag{2.1.6}$$

Then, for every integer $T \ge 0$,

$$\|\theta_T - \theta^\star\| \le q(\eta)^T \|\theta_0 - \theta^\star\|, \tag{2.1.7}$$

$$\Phi(\theta_T) - \Phi(\theta^\star) \le q(\eta)^{2T}\{\Phi(\theta_0) - \Phi(\theta^\star)\}. \tag{2.1.8}$$

The spectral proof is assigned in Homework 1, where the same calculation is carried out in the rank-deficient, overparameterized setting and also shows which interpolating solution gradient descent selects.

The upper bound $2/\beta$ is the exact stability boundary for this quadratic problem; the frequently used value $1/\beta$ is not the boundary. Indeed, since the spectrum of H lies in $[\mu, \beta]$, the worst spectral factor occurs at one of the two endpoints:

$$q(\eta) = \max\{|1 - \eta\mu|, |1 - \eta\beta|\}. \tag{2.1.9}$$

The different step-size ranges have visibly different dynamics.

(i) If $0 < \eta < 1/\beta$, then every multiplier $1 - \eta\lambda_j$ lies in $(0, 1)$. Every eigencoordinate contracts without changing sign, and
$$q(\eta) = 1 - \eta\mu. \tag{2.1.10}$$

(ii) If $\eta = 1/\beta$, the largest-curvature eigencoordinate is eliminated in one step, while all other eigencoordinates contract without changing sign. Here $q(\eta) = 1 - \mu/\beta$.

(iii) If $1/\beta < \eta < 2/\beta$, eigencoordinates with $\lambda_j > 1/\eta$ have negative multipliers. They alternate signs, so the iterates oscillate in those directions, but their magnitudes still shrink because $|1 - \eta\lambda_j| < 1$. In this range,
$$q(\eta) = \max\{1 - \eta\mu, \eta\beta - 1\} < 1, \tag{2.1.11}$$
and both the parameter error and objective gap still converge geometrically.

(iv) At $\eta = 2/\beta$, the multiplier in a β-eigendirection is -1, so that component generally does not decay. For $\eta > 2/\beta$, its magnitude exceeds one and that component diverges. Thus strict inequality in Proposition 2.1 is necessary for convergence from every initialization.

Thus $1/\beta$ separates nonoscillatory behavior from possible oscillation in the highest-curvature directions, whereas $2/\beta$ is the stability limit.

We can now choose the optimal constant step size. At $\eta = 1/\beta$,

$$\Phi(\theta_T) - \Phi(\theta^\star) \le \left(1 - \frac{1}{\kappa}\right)^{2T} \{\Phi(\theta_0) - \Phi(\theta^\star)\}, \qquad \kappa := \frac{\beta}{\mu}. \tag{2.1.12}$$

As η increases from zero, the slow low-curvature factor $1 - \eta\mu$ decreases, while beyond $1/\beta$ the oscillatory high-curvature factor $\eta\beta - 1$ increases. The maximum of the two is minimized when they are equal. Therefore

$$1 - \eta_{\text{opt}}\mu = \eta_{\text{opt}}\beta - 1, \qquad \eta_{\text{opt}} = \frac{2}{\mu + \beta}, \qquad q(\eta_{\text{opt}}) = \frac{\beta - \mu}{\beta + \mu} = \frac{\kappa - 1}{\kappa + 1}. \tag{2.1.13}$$

Consequently the objective gap contracts by $((\kappa - 1)/(\kappa + 1))^{2T}$. Both choices require $T = O(\kappa \log(1/\varepsilon))$ iterations to reduce the initial objective gap by a factor ε, but the optimized constant step has the sharper spectral factor. Unless $\mu = \beta$, this optimal step lies strictly between $1/\beta$ and $2/\beta$: it deliberately introduces controlled oscillation in the high-curvature directions to speed up the slow low-curvature directions.

This quadratic example motivates the geometric assumptions used for a general differentiable objective.

Basic geometric definitions

Definition 2.2 (Convexity)**.** A differentiable function $\Phi : \mathbb{R}^d \to \mathbb{R}$ is convex if

$$\Phi(v) \ge \Phi(u) + \langle \nabla\Phi(u), v - u\rangle \qquad \text{for all } u, v \in \mathbb{R}^d. \tag{2.1.14}$$

Definition 2.3 (Strong convexity)**.** For $\mu > 0$, the function Φ is μ-strongly convex if

$$\Phi(v) \geq \Phi(u) + \langle \nabla\Phi(u), v - u\rangle + \frac{\mu}{2}\|v - u\|^2 \qquad \text{for all } u, v \in \mathbb{R}^d. \tag{2.1.15}$$

Definition 2.4 (Smoothness)**.** For $\beta > 0$, the function Φ is β-smooth if

$$\|\nabla\Phi(u) - \nabla\Phi(v)\| \leq \beta\|u - v\| \qquad \text{for all } u, v \in \mathbb{R}^d. \tag{2.1.16}$$

Convexity gives a global affine lower bound. Strong convexity adds positive quadratic curvature and yields a unique minimizer. Smoothness gives a quadratic upper bound and controls how aggressive a gradient step can be.

For a general differentiable objective, gradient descent has the form

$$\theta_{t+1} = \theta_t - \eta_t \nabla\Phi(\theta_t). \tag{2.1.17}$$

The least-squares Hessian is $A^\top A$. Its gradient has Lipschitz modulus $\|A\|_{\text{op}}^2$, so Φ is convex and β-smooth for any positive $\beta \geq \lambda_{\max}(A^\top A) = \|A\|_{\text{op}}^2$. It is μ-strongly convex exactly when $A^\top A$ is positive definite, in which case $\mu = \lambda_{\min}(A^\top A)$. These are the same spectral quantities that appeared directly in (2.1.4).

A common smoothness tool

Lemma 2.5 (Descent lemma)**.** *If Φ is β-smooth, then*

$$\Phi(v) \leq \Phi(u) + \langle \nabla\Phi(u), v - u\rangle + \frac{\beta}{2}\|v - u\|^2. \tag{2.1.18}$$

Proof. Let $d = v - u$. By the fundamental theorem of calculus,

$$\Phi(v) - \Phi(u) - \langle \nabla\Phi(u), d\rangle = \int_0^1 \langle \nabla\Phi(u + sd) - \nabla\Phi(u), d\rangle \, \mathrm{d}s.$$

Smoothness and Cauchy–Schwarz bound the integrand by $\beta s\|d\|^2$. Integrating over $s \in [0, 1]$ proves the claim. □

Corollary 2.6 (One gradient step decreases the objective)**.** *If Φ is β-smooth and $\theta^+ = \theta - \eta\nabla\Phi(\theta)$, then*

$$\Phi(\theta^+) \leq \Phi(\theta) - \eta\left(1 - \frac{\beta\eta}{2}\right)\|\nabla\Phi(\theta)\|^2. \tag{2.1.19}$$

In particular, if $0 < \eta \leq 1/\beta$, then

$$\Phi(\theta^+) \leq \Phi(\theta) - \frac{\eta}{2}\|\nabla\Phi(\theta)\|^2. \tag{2.1.20}$$

Proof. Insert $v = \theta - \eta\nabla\Phi(\theta)$ and $u = \theta$ into Lemma 2.5. □

The geometry behind the two assumptions is summarized in Figure 2.1. At a base point u, both models start with the tangent plane $\Phi(u) + \langle \nabla\Phi(u), v - u\rangle$. Smoothness adds an upward quadratic term to obtain an *upper* model, whereas strong convexity adds an upward quadratic term that remains a *lower* model.

Figure 2.2 isolates the distinction between convexity and strong convexity. Convexity requires a supporting affine lower bound, but it permits flat directions. Strong convexity requires a uniform positive quadratic term above that affine support.

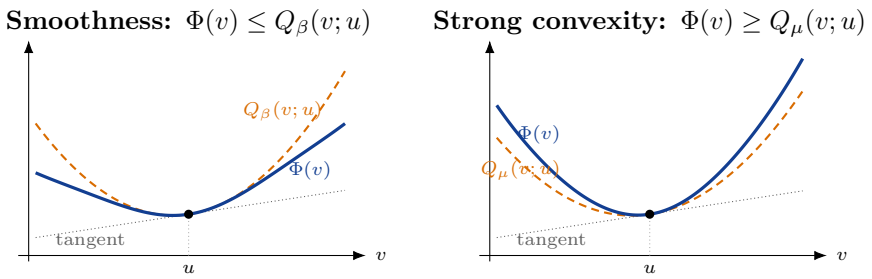


Figure 2.1: Quadratic models around a base point u. The blue curve is the objective, the gray dotted line is its tangent at u, and the orange dashed curve is the quadratic model. A β-smooth objective lies below its quadratic upper model; a μ-strongly convex objective lies above its quadratic lower model.

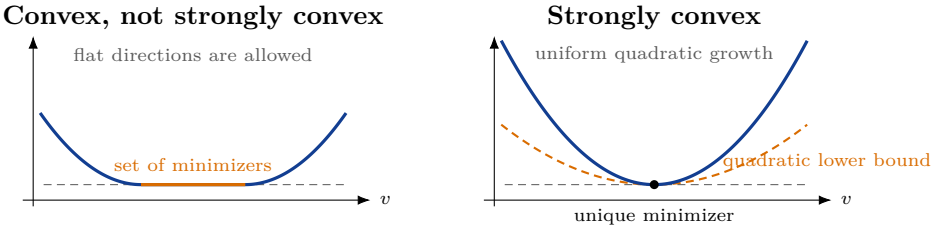


Figure 2.2: Convexity versus strong convexity. Convexity alone can allow a flat set of minimizers, as on the left. Strong convexity supplies uniform quadratic growth above the affine support and therefore forces a unique minimizer, as on the right. A unique minimizer by itself does not imply strong convexity.

2.2 Gradient descent under strong convexity

Lemma 2.7 (Strong convexity implies gradient domination)**.** *If Φ is differentiable and μ-strongly convex, then*

$$\|\nabla\Phi(\theta)\|^2 \ge 2\mu\{\Phi(\theta) - \Phi(\theta^\star)\}. \tag{2.2.1}$$

Proof. Strong convexity gives, for every v,

$$\Phi(v) \ge \Phi(\theta) + \langle\nabla\Phi(\theta), v-\theta\rangle + \frac{\mu}{2}\|v-\theta\|^2.$$

Taking the infimum of the right-hand side over v gives

$$\Phi(\theta^\star) \ge \Phi(\theta) - \frac{1}{2\mu}\|\nabla\Phi(\theta)\|^2.$$

Rearrange. □

Theorem 2.8 (Linear convergence of GD)**.** *Suppose Φ is μ-strongly convex and β-smooth. For a constant step size $0 < \eta \le 1/\beta$, gradient descent satisfies*

$$\Phi(\theta_T) - \Phi(\theta^\star) \le (1-\mu\eta)^T\{\Phi(\theta_0) - \Phi(\theta^\star)\}. \tag{2.2.2}$$

In particular, $\eta = 1/\beta$ gives

$$\Phi(\theta_T) - \Phi(\theta^\star) \le \left(1 - \frac{\mu}{\beta}\right)^T\{\Phi(\theta_0) - \Phi(\theta^\star)\}. \tag{2.2.3}$$

Proof. Corollary 2.6 and $\eta \le 1/\beta$ give

$$\Phi(\theta_{t+1}) \le \Phi(\theta_t) - \frac{\eta}{2}\|\nabla\Phi(\theta_t)\|^2.$$

Apply Lemma 2.7:

$$\Phi(\theta_{t+1}) - \Phi(\theta^\star) \le (1 - \mu\eta)\{\Phi(\theta_t) - \Phi(\theta^\star)\}.$$

Iterating proves (2.2.2); substituting $\eta = 1/\beta$ gives (2.2.3). □

Theorem 2.9 (Parameter convergence of gradient descent)**.** *Under the assumptions of Theorem 2.8, for every integer $T \ge 0$,*

$$\begin{aligned}\|\theta_T - \theta^\star\|^2 &\le \frac{2}{\mu}\{\Phi(\theta_T) - \Phi(\theta^\star)\} \\ &\le \frac{2}{\mu}(1 - \mu\eta)^T\{\Phi(\theta_0) - \Phi(\theta^\star)\}. \end{aligned} \tag{2.2.4}$$

Consequently,

$$\|\theta_T - \theta^\star\| \le \sqrt{\frac{2\{\Phi(\theta_0) - \Phi(\theta^\star)\}}{\mu}}(1 - \mu\eta)^{T/2}. \tag{2.2.5}$$

Moreover,

$$\|\theta_T - \theta^\star\| \le \sqrt{\frac{\beta}{\mu}}(1 - \mu\eta)^{T/2}\,\|\theta_0 - \theta^\star\|\,. \tag{2.2.6}$$

Thus the iterates themselves converge geometrically to the unique minimizer.

Proof. Since $\nabla\Phi(\theta^\star) = 0$, strong convexity gives

$$\Phi(\theta) - \Phi(\theta^\star) \ge \frac{\mu}{2}\,\|\theta - \theta^\star\|^2\,. \tag{2.2.7}$$

Combining this inequality with Theorem 2.8 proves (2.2.4); taking square roots proves (2.2.5). Finally, smoothness and $\nabla\Phi(\theta^\star) = 0$ imply

$$\Phi(\theta_0) - \Phi(\theta^\star) \le \frac{\beta}{2}\,\|\theta_0 - \theta^\star\|^2\,.$$

Substitution proves (2.2.6). □

The square-root exponent in Theorem 2.9 appears because strong convexity controls the *squared* parameter error by the objective gap. For least squares, the direct spectral rate in Proposition 2.1 is sharper.

The condition number

$$\kappa := \frac{\beta}{\mu} \tag{2.2.8}$$

controls the complexity. Since $(1 - 1/\kappa)^T \le e^{-T/\kappa}$, reducing the initial objective gap by a factor ε takes $T = O(\kappa \log(1/\varepsilon))$ iterations. For full-column-rank least squares, $\kappa = \lambda_{\max}(A^\top A)/\lambda_{\min}(A^\top A)$. The exact quadratic analysis in Proposition 2.1 is sharper: it tracks each eigendirection and gives a squared contraction factor for the objective gap.

2.3 Gradient descent under convexity

Without strong convexity, the objective gap no longer controls $\|\theta - \theta^\star\|^2$: a convex objective may have an entire flat set of minimizers. Consequently, convergence of the objective alone does not generally imply convergence of θ_t to a particular minimizer, nor does it provide a uniform rate in parameter distance. Convexity and smoothness nevertheless still guarantee convergence of the objective value, now at the sublinear rate established below.

The next figure previews a second distinction that will matter when we turn to stochastic optimization. On the same objective, full-gradient descent follows a deterministic trajectory, whereas SGD follows a random, visibly noisier path because every update uses a gradient estimate.

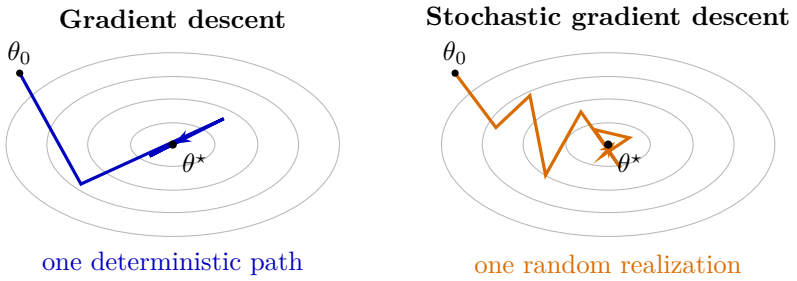


Figure 2.3: Two optimization paths on the same strongly convex objective. The contours are level sets. GD uses the exact gradient and hence has a deterministic path; SGD is perturbed by gradient noise and therefore follows a random path even when its average direction is correct.

Theorem 2.10 (GD on a convex smooth objective)**.** *Suppose* Φ *is convex and* β*-smooth and attains its minimum at* $\theta^\star$*. Let*

$$\theta_{t+1} = \theta_t - \eta \nabla \Phi(\theta_t), \qquad 0 < \eta \le 1/\beta. \tag{2.3.1}$$

Then for every integer $T \ge 1$*,*

$$\Phi(\theta_T) - \Phi(\theta^\star) \le \frac{\|\theta_0 - \theta^\star\|^2}{2\eta T}. \tag{2.3.2}$$

With $\eta = 1/\beta$*, the right-hand side is* $\beta \|\theta_0 - \theta^\star\|^2 / (2T)$*.*

Proof. Write $g_t = \nabla \Phi(\theta_t)$. Convexity gives

$$\Phi(\theta_t) - \Phi(\theta^\star) \le \langle g_t, \theta_t - \theta^\star \rangle.$$

To see the update identity explicitly, subtract $\theta^\star$ from both sides of (2.3.1) and expand the squared norm:

$$\begin{aligned} \|\theta_{t+1} - \theta^\star\|^2 &= \|\theta_t - \theta^\star - \eta g_t\|^2 \\ &= \|\theta_t - \theta^\star\|^2 - 2\eta \langle g_t, \theta_t - \theta^\star \rangle + \eta^2 \|g_t\|^2 . \end{aligned}$$

Move the inner-product term to the left and divide by 2η. This gives

$$\langle g_t, \theta_t - \theta^\star \rangle = \frac{\|\theta_t - \theta^\star\|^2 - \|\theta_{t+1} - \theta^\star\|^2}{2\eta} + \frac{\eta}{2} \|g_t\|^2 . \tag{2.3.3}$$

By Corollary 2.6,

$$\frac{\eta}{2} \|g_t\|^2 \le \Phi(\theta_t) - \Phi(\theta_{t+1}).$$

Combining the last three displays and canceling $\Phi(\theta_t)$ yields

$$\Phi(\theta_{t+1}) - \Phi(\theta^\star) \le \frac{\|\theta_t - \theta^\star\|^2 - \|\theta_{t+1} - \theta^\star\|^2}{2\eta}.$$

Sum from $t = 0$ to $T - 1$. The squared distances telescope. Moreover, Corollary 2.6 shows that the objective values are nonincreasing, so each of the T objective gaps in the sum is at least the final gap. Therefore

$$T\{\Phi(\theta_T) - \Phi(\theta^\star)\} \le \frac{\|\theta_0 - \theta^\star\|^2}{2\eta}.$$

□

Why use a constant learning rate here? For deterministic full-gradient descent there is no persistent gradient noise to average away. Once the step size is small enough to guarantee descent, shrinking it only reduces the cumulative progress. To make this precise, allow

$$\theta_{t+1} = \theta_t - \eta_t \nabla \Phi(\theta_t), \qquad 0 < \eta_t \le 1/\beta. \tag{2.3.4}$$

The same calculation as above, now before dividing by η_t, gives

$$2\eta_t\{\Phi(\theta_{t+1}) - \Phi(\theta^\star)\} \le \|\theta_t - \theta^\star\|^2 - \|\theta_{t+1} - \theta^\star\|^2 .$$

Summing and using the fact that the objective gaps are nonincreasing yields the variable-step bound

$$\Phi(\theta_T) - \Phi(\theta^\star) \le \frac{\|\theta_0 - \theta^\star\|^2}{2\sum_{t=0}^{T-1} \eta_t}. \tag{2.3.5}$$

Thus the rate is governed by the cumulative learning rate. The largest constant choice allowed by this proof, $\eta_t = 1/\beta$, maximizes that quantity and recovers the $O(1/T)$ bound. For example, if $\eta_t = c(t+1)^{-\alpha}$, where $0 < c \le 1/\beta$, then

$$0 < \alpha < 1: \quad \Phi(\theta_T) - \Phi(\theta^\star) \le \frac{(1-\alpha)\|\theta_0 - \theta^\star\|^2}{2c\{(T+1)^{1-\alpha} - 1\}} = O\big(T^{-(1-\alpha)}\big), \tag{2.3.6}$$

$$\alpha = 1: \quad \Phi(\theta_T) - \Phi(\theta^\star) \le \frac{\|\theta_0 - \theta^\star\|^2}{2c\log(T+1)} = O\big(1/\log T\big). \tag{2.3.7}$$

For comparison, under μ-strong convexity the one-step contraction from Theorem 2.8 applies separately at every iteration:

$$\Phi(\theta_T) - \Phi(\theta^\star) \le \{\Phi(\theta_0) - \Phi(\theta^\star)\} \prod_{t=0}^{T-1} (1 - \mu\eta_t) \le \{\Phi(\theta_0) - \Phi(\theta^\star)\} \exp\left(-\mu \sum_{t=0}^{T-1} \eta_t\right). \tag{2.3.8}$$

Writing $\Delta_t := \Phi(\theta_t) - \Phi(\theta^\star)$ and $R := \|\theta_0 - \theta^\star\|$, the comparison is summarized below.

Table 2.1: Objective-gap bounds for deterministic GD. Every displayed step size satisfies $0 < \eta_t \le 1/\beta$.

Learning-rate schedule	μ-strongly convex and β-smooth	Convex and β-smooth
Constant, $\eta_t \equiv \eta$	$\Delta_T \le \Delta_0(1-\mu\eta)^T = O(e^{-\mu\eta T})$	$\Delta_T \le R^2/(2\eta T) = O(T^{-1})$

The table compares objective gaps. Strong convexity additionally converts these bounds into parameter-distance bounds through $\|\theta_T - \theta^\star\|^2 \le 2\Delta_T/\mu$, with the corresponding rates square-rooted for the norm itself. In the merely convex case there is no analogous uniform parameter-distance rate. Decaying learning rates become essential later for SGD because stochastic-gradient noise persists; that noise–progress tradeoff is absent in deterministic GD.

2.4 Stochastic gradient descent

A full gradient of a finite-sum objective

$$\Phi(\theta) = \frac{1}{n}\sum_{i=1}^{n} \phi_i(\theta) \tag{2.4.1}$$

requires n per-example gradients. SGD replaces it by a random gradient estimate. The randomness must be described conditionally because θ_t depends on all previous random draws.

Throughout the SGD discussion, we consider the unconstrained update on $\mathbb{R}^d$:

$$\theta_{t+1} = \theta_t - \eta_t g_t, \qquad t = 1, \dots, T. \tag{2.4.2}$$

Let $\mathcal{H}_t$ contain the data, initialization, and all algorithmic randomness observed before g_t is drawn. We assume θ_t and the positive step size η_t are $\mathcal{H}_t$-measurable; the schedules used below are deterministic special cases.

A standard example is a mini-batch gradient. For the finite sum (2.4.1), conditionally sample $I_{t,1}, \dots, I_{t,b}$ independently and uniformly from $\{1, \dots, n\}$, and set

$$g_t := \frac{1}{b} \sum_{j=1}^{b} \nabla \phi_{I_{t,j}}(\theta_t). \tag{2.4.3}$$

Then

$$\mathbb{E}[g_t \mid \mathcal{H}_t] = \nabla \Phi(\theta_t), \tag{2.4.4}$$

$$\mathrm{Cov}(g_t \mid \mathcal{H}_t) = \frac{1}{b} \mathrm{Cov}(\nabla \phi_I(\theta_t) \mid \mathcal{H}_t), \tag{2.4.5}$$

where I is one conditional uniform draw. Thus independent mini-batching reduces the conditional covariance by a factor b, although it does not remove the randomness of the training sample itself. This construction motivates the following abstract oracle assumption.

Definition 2.11 (Stochastic gradient oracle)**.** The random vector g_t is a conditionally unbiased stochastic gradient with bounded conditional second moment if

$$\mathbb{E}[g_t \mid \mathcal{H}_t] = \nabla \Phi(\theta_t), \qquad \mathbb{E}[\|g_t\|^2 \mid \mathcal{H}_t] \le G^2 \quad \text{almost surely.} \tag{2.4.6}$$

Conditional unbiasedness concerns the gradient estimate. It does not say that the iterate, the objective value, or the test error is unbiased.

Strongly convex objectives

We begin with strong convexity. Its quadratic lower bound turns the expected alignment of the stochastic gradient with the parameter error into a contraction term. We first hold the learning rate constant. This separates a geometrically decaying optimization transient from the nonvanishing effect of gradient noise. We then decrease the learning rate to remove that noise floor and obtain an objective rate faster than the general convex rate considered afterward.

For the constant-step analysis, separate the stochastic gradient into signal and noise:

$$g_t = \nabla \Phi(\theta_t) + \xi_t, \qquad \mathbb{E}[\xi_t \mid \mathcal{H}_t] = 0, \qquad \mathbb{E}[\|\xi_t\|^2 \mid \mathcal{H}_t] \le \sigma^2. \tag{2.4.7}$$

The oracle assumption (2.4.6) implies the last bound with $\sigma^2 = G^2$, but the separate symbol σ^2 makes the role of gradient noise explicit.

Theorem 2.12 (Constant-step SGD under strong convexity)**.** *Suppose Φ is μ-strongly convex and β-smooth, attains its minimum at $\theta^\star$, and (2.4.7) holds. Use*

$$\theta_{t+1} = \theta_t - \eta g_t, \qquad 0 < \eta \le 1/\beta. \tag{2.4.8}$$

Then, for every integer $T \ge 1$,

$$\begin{aligned} \mathbb{E}[\Phi(\theta_T)] - \Phi(\theta^\star) &\le (1 - \mu\eta)^{T-1} \{\mathbb{E}[\Phi(\theta_1)] - \Phi(\theta^\star)\} \\ &\quad + \frac{\beta \eta \sigma^2}{2\mu} \{1 - (1 - \mu\eta)^{T-1}\} \qquad (2.4.9) \\ &\le (1 - \mu\eta)^{T-1} \{\mathbb{E}[\Phi(\theta_1)] - \Phi(\theta^\star)\} + \frac{\beta \eta \sigma^2}{2\mu}. \qquad (2.4.10) \end{aligned}$$

Thus constant-step SGD approaches an $O(\eta)$ objective neighborhood of the optimum at a geometric rate.

Proof. First fix a realization of the stochastic gradient g_t. By β-smoothness and (2.4.8), the following pathwise inequality holds before taking any expectation:

$$\Phi(\theta_{t+1}) \le \Phi(\theta_t) + \langle \nabla \Phi(\theta_t), \theta_{t+1} - \theta_t \rangle + \frac{\beta}{2} \|\theta_{t+1} - \theta_t\|^2$$

$$= \Phi(\theta_t) - \eta\langle \nabla\Phi(\theta_t), g_t\rangle + \frac{\beta\eta^2}{2}\|g_t\|^2 .$$

Now condition on $\mathcal{H}_t$. Because θ_t and hence $\nabla\Phi(\theta_t)$ are $\mathcal{H}_t$-measurable, the preceding display gives

$$\mathbb{E}[\Phi(\theta_{t+1}) \mid \mathcal{H}_t] \le \Phi(\theta_t) - \eta\langle \nabla\Phi(\theta_t), \mathbb{E}[g_t \mid \mathcal{H}_t]\rangle + \frac{\beta\eta^2}{2}\mathbb{E}[\|g_t\|^2 \mid \mathcal{H}_t].$$

The conditional mean-zero property of ξ_t makes the cross term vanish, so

$$\mathbb{E}[\|g_t\|^2 \mid \mathcal{H}_t] = \|\nabla\Phi(\theta_t)\|^2 + \mathbb{E}[\|\xi_t\|^2 \mid \mathcal{H}_t].$$

Using (2.4.7) and $\eta \le 1/\beta$ therefore gives

$$\begin{aligned}\mathbb{E}[\Phi(\theta_{t+1}) \mid \mathcal{H}_t] &\le \Phi(\theta_t) - \eta\left(1 - \frac{\beta\eta}{2}\right)\|\nabla\Phi(\theta_t)\|^2 + \frac{\beta\eta^2\sigma^2}{2} \\ &\le \Phi(\theta_t) - \frac{\eta}{2}\|\nabla\Phi(\theta_t)\|^2 + \frac{\beta\eta^2\sigma^2}{2}.\end{aligned}$$

Strong convexity implies the Polyak–Łojasiewicz inequality

$$\|\nabla\Phi(\theta_t)\|^2 \ge 2\mu\{\Phi(\theta_t) - \Phi(\theta^\star)\}.$$

Taking total expectations and writing $\delta_t := \mathbb{E}[\Phi(\theta_t)] - \Phi(\theta^\star)$ yields

$$\delta_{t+1} \le (1 - \mu\eta)\delta_t + \frac{\beta\eta^2\sigma^2}{2}. \tag{2.4.11}$$

Iterating this affine recursion and summing the geometric series proves (2.4.9); dropping the two factors bounded by one gives (2.4.10). □

The two terms in (2.4.10) expose the basic tradeoff. A larger η makes the transient $(1-\mu\eta)^{T-1}$ disappear faster but raises the asymptotic noise floor; a smaller η lowers that floor but slows the transient. When $\sigma^2 = 0$, the noise floor disappears and the deterministic geometric rate is recovered. With persistent noise, exact convergence requires the learning rate to decrease.

Decreasing learning rates. We now let the step size shrink with t. With smoothness, the same objective-gap recursion used for constant-step SGD closes at every time and therefore gives a guarantee for the final iterate itself.

Proposition 2.13 (Final-iterate SGD with a decreasing step size)**.** *Suppose Φ is μ-strongly convex and β-smooth, attains its minimum at $\theta^\star$, and (2.4.7) holds. Let*

$$\eta_t = \frac{2}{\mu(t+\tau)}, \qquad \tau \ge \frac{2\beta}{\mu}. \tag{2.4.12}$$

Then, for every integer $T \ge 1$,

$$\mathbb{E}[\Phi(\theta_T)] - \Phi(\theta^\star) \le \frac{\tau\{\mathbb{E}[\Phi(\theta_1)] - \Phi(\theta^\star)\} + 2\beta\sigma^2/\mu^2}{T + \tau - 1} = O\left(\frac{1}{T}\right). \tag{2.4.13}$$

Thus the decreasing step size removes the constant-step noise floor without averaging the iterates.

Proof. Write $\delta_t := \mathbb{E}[\Phi(\theta_t)] - \Phi(\theta^\star)$. Because $\eta_t \le 1/\beta$, the proof of Theorem 2.12, applied with the current value of η_t, gives

$$\delta_{t+1} \le (1 - \mu\eta_t)\delta_t + \frac{\beta\eta_t^2\sigma^2}{2} = \left(1 - \frac{2}{t+\tau}\right)\delta_t + \frac{C}{(t+\tau)^2}, \qquad C := \frac{2\beta\sigma^2}{\mu^2}. \tag{2.4.14}$$

Set $A := \tau\delta_1 + C$. We prove by induction that $\delta_t \le A/(t+\tau-1)$. The claim holds at $t = 1$. If it holds at time t, set $s = t + \tau$ in (2.4.14) to obtain

$$\delta_{t+1} \le \frac{A(s-2)}{s(s-1)} + \frac{C}{s^2} \le \frac{A}{s}.$$

The last inequality follows from $A \ge C$, since the difference between the first and last fractions involving A is $A/[s(s-1)] \ge C/s^2$. This proves (2.4.13). □

For compactness, write

$$\mathcal{A} := \tau\{\mathbb{E}[\Phi(\theta_1)] - \Phi(\theta^\star)\} + \frac{2\beta\sigma^2}{\mu^2}. \tag{2.4.15}$$

Corollary 2.14 (Parameter convergence of the final iterate)**.** *Under the assumptions and step-size schedule of Proposition 2.13,*

$$\mathbb{E}\,\|\theta_T - \theta^\star\|^2 \le \frac{2\mathcal{A}}{\mu(T+\tau-1)} = O\left(\frac{1}{T}\right), \tag{2.4.16}$$

$$\|\mathbb{E}[\theta_T] - \theta^\star\| \le \mathbb{E}\,\|\theta_T - \theta^\star\| \le \sqrt{\frac{2\mathcal{A}}{\mu(T+\tau-1)}} = O\left(\frac{1}{\sqrt{T}}\right). \tag{2.4.17}$$

Thus $\theta_T \to \theta^\star$ in mean square and in mean; the squared parameter error has the same $O(1/T)$ order as the objective gap, while the parameter norm itself has the square-root rate.

Proof. Strong convexity at the minimizer gives

$$\Phi(\theta) - \Phi(\theta^\star) \ge \frac{\mu}{2}\,\|\theta - \theta^\star\|^2.$$

Combine this inequality with (2.4.13). Jensen's inequality gives the first inequality in (2.4.17), and Cauchy–Schwarz gives the second. □

Proposition 2.15 (Polyak averaging in expectation)**.** *Under the assumptions and step-size schedule of Proposition 2.13, define the arithmetic Polyak average*

$$\overline{\theta}_T^{\mathrm{PR}} := \frac{1}{T}\sum_{t=1}^{T}\theta_t. \tag{2.4.18}$$

Then

$$\mathbb{E}\left\|\overline{\theta}_T^{\mathrm{PR}} - \theta^\star\right\|^2 \le \frac{8\mathcal{A}}{\mu T}, \tag{2.4.19}$$

$$\mathbb{E}[\Phi(\overline{\theta}_T^{\mathrm{PR}})] - \Phi(\theta^\star) \le \frac{4\beta\mathcal{A}}{\mu T} = O\left(\frac{1}{T}\right). \tag{2.4.20}$$

Consequently, the arithmetic average converges to $\theta^\star$ in mean square, and its objective converges to the optimum in expectation. The mean-square parameter error and the expected objective gap are both $O(1/T)$.

Proof. For $e_t := \theta_t - \theta^\star$, strong convexity and (2.4.13) imply

$$\mathbb{E}\,\|e_t\|^2 \le \frac{2\mathcal{A}}{\mu(t+\tau-1)} \le \frac{2\mathcal{A}}{\mu t}.$$

Minkowski's inequality in L^2, followed by $\sum_{t=1}^{T} t^{-1/2} \le 2\sqrt{T}$, gives

$$\begin{aligned}\left(\mathbb{E}\left\|\overline{\theta}_T^{\mathrm{PR}} - \theta^\star\right\|^2\right)^{1/2} &\le \frac{1}{T}\sum_{t=1}^{T}\left(\mathbb{E}\,\|e_t\|^2\right)^{1/2} \\ &\le \frac{1}{T}\sqrt{\frac{2\mathcal{A}}{\mu}}\sum_{t=1}^{T}\frac{1}{\sqrt{t}} \le \sqrt{\frac{8\mathcal{A}}{\mu T}}.\end{aligned}$$

This proves (2.4.19). Smoothness at the minimizer, where $\nabla\Phi(\theta^\star) = 0$, gives the quadratic upper bound

$$\Phi(\theta) - \Phi(\theta^\star) \le \frac{\beta}{2}\,\|\theta - \theta^\star\|^2.$$

Apply it at $\overline{\theta}_T^{\mathrm{PR}}$, take expectations, and use (2.4.19). □

Remark 2.16 (What changes without a smoothness assumption)**.** The arithmetic-average objective rates with and without smoothness are not the same. Smoothness gives the $O(1/T)$ rate in Proposition 2.15: after averaging the parameter errors, its quadratic upper model converts their mean-square rate into an objective-gap rate. If we assume only differentiability, μ-strong convexity, and the bounded conditional second moment in (2.4.6), expanding the squared parameter distance instead gives

$$2\eta_t\delta_t \le (1-\mu\eta_t)d_t - d_{t+1} + \eta_t^2 G^2, \qquad d_t := \mathbb{E}\left\|\theta_t - \theta^\star\right\|^2. \tag{2.4.21}$$

Without a quadratic upper model, this inequality naturally controls a sum of objective gaps rather than converting an averaged parameter error into an objective gap. The stochastic objective values need not decrease monotonically, so the same calculation also cannot isolate δ_T. In particular, $\eta_t = 1/(\mu t)$ and the uniform average give

$$\mathbb{E}\left[\Phi\left(\frac{1}{T}\sum_{t=1}^{T}\theta_t\right)\right] - \Phi(\theta^\star) \le \frac{G^2 H_T}{2\mu T} = O\left(\frac{\log T}{T}\right),$$

whereas $\eta_t = 2/[\mu(t+1)]$ together with weights proportional to t gives $2G^2/[\mu(T+1)] = O(1/T)$. Thus the standard arithmetic Polyak average has an $O(1/T)$ expected objective rate with smoothness but only the general $O(\log T/T)$ guarantee without it. A different, linearly weighted average recovers $O(1/T)$ without smoothness.

Convex objectives

Without strong convexity, the contraction term disappears. The same squared-distance calculation still controls an average objective gap, but at the slower $O(T^{-1/2})$ rate.

Theorem 2.17 (Convex SGD with step-size-weighted averaging)**.** *Suppose Φ is convex and differentiable, attains its minimum at $\theta^\star$, and (2.4.6) holds. Let $\eta_1, \dots, \eta_T > 0$ be deterministic, and define*

$$S_T := \sum_{t=1}^{T}\eta_t, \qquad Q_T := \sum_{t=1}^{T}\eta_t^2, \qquad \overline{\theta}_T^{(\eta)} := \frac{1}{S_T}\sum_{t=1}^{T}\eta_t\theta_t. \tag{2.4.22}$$

Suppose the possibly random initialization satisfies $\mathbb{E}\left\|\theta_1 - \theta^\star\right\|^2 \le R^2$. Then

$$\mathbb{E}[\Phi(\overline{\theta}_T^{(\eta)})] - \Phi(\theta^\star) \le \frac{R^2 + G^2 Q_T}{2S_T}. \tag{2.4.23}$$

For the constant schedule $\eta_t \equiv \eta$, the weighted average is the uniform average

$$\overline{\theta}_T := \frac{1}{T}\sum_{t=1}^{T}\theta_t, \tag{2.4.24}$$

and (2.4.23) becomes

$$\mathbb{E}[\Phi(\overline{\theta}_T)] - \Phi(\theta^\star) \le \frac{R^2}{2\eta T} + \frac{\eta G^2}{2}. \tag{2.4.25}$$

For $R > 0$ and $G > 0$, choosing $\eta = R/(G\sqrt{T})$ gives

$$\mathbb{E}[\Phi(\overline{\theta}_T)] - \Phi(\theta^\star) \le \frac{RG}{\sqrt{T}}. \tag{2.4.26}$$

Proposition 2.18 (Smooth convex SGD with bounded variance)**.** *Suppose Φ is convex and β-smooth, attains its minimum at $\theta^\star$, and the noise decomposition (2.4.7) holds. Suppose also that $\mathbb{E}\left\|\theta_1 - \theta^\star\right\|^2 \le R^2$. For deterministic step sizes $0 < \eta_t \le 1/\beta$, define S_T, Q_T, and $\overline{\theta}_T^{(\eta)}$ as in (2.4.22). Then*

$$\mathbb{E}[\Phi(\overline{\theta}_T^{(\eta)})] - \Phi(\theta^\star) \le \frac{R^2 + \sigma^2 Q_T}{2S_T}. \tag{2.4.27}$$

In particular, for a constant step $0 < \eta \le 1/\beta$,

$$\mathbb{E}[\Phi(\overline{\theta}_T)] - \Phi(\theta^\star) \le \frac{R^2}{2\eta T} + \frac{\eta\sigma^2}{2}. \tag{2.4.28}$$

If $R > 0$, *the horizon-tuned choice*

$$\eta = \frac{1}{\beta + \sigma\sqrt{T}/R} \tag{2.4.29}$$

satisfies $\eta \le 1/\beta$ *and gives*

$$\mathbb{E}[\Phi(\overline{\theta}_T)] - \Phi(\theta^\star) \le \frac{\beta R^2}{2T} + \frac{R\sigma}{\sqrt{T}}. \tag{2.4.30}$$

Thus smoothness separates an $O(T^{-1})$ *deterministic term from the* $O(T^{-1/2})$ *term caused by persistent gradient noise. When* $\sigma = 0$, *the latter disappears and the deterministic smooth-convex rate is recovered.*

Proof. Smooth convexity gives the strengthened first-order inequality

$$\langle \nabla\Phi(\theta), \theta - \theta^\star \rangle \ge \Phi(\theta) - \Phi(\theta^\star) + \frac{1}{2\beta} \|\nabla\Phi(\theta)\|^2 . \tag{2.4.31}$$

Write $\xi_t := g_t - \nabla\Phi(\theta_t)$. Conditional unbiasedness makes the cross term between $\nabla\Phi(\theta_t)$ and ξ_t vanish. Expanding the next squared distance and applying (2.4.31) therefore gives

$$\begin{aligned}\mathbb{E}[\|\theta_{t+1} - \theta^\star\|^2 \mid \mathcal{H}_t] \le\ & \|\theta_t - \theta^\star\|^2 - 2\eta_t\{\Phi(\theta_t) - \Phi(\theta^\star)\} \\ & - \eta_t(1/\beta - \eta_t)\|\nabla\Phi(\theta_t)\|^2 + \eta_t^2\sigma^2.\end{aligned} \tag{2.4.32}$$

Because $\eta_t \le 1/\beta$, the gradient-norm term is nonpositive and may be discarded. Taking total expectations and summing yields

$$2\sum_{t=1}^{T} \eta_t\{\mathbb{E}[\Phi(\theta_t)] - \Phi(\theta^\star)\} \le R^2 + \sigma^2 Q_T.$$

Jensen's inequality proves (2.4.27); the constant-step formula follows immediately. Substituting (2.4.29) into (2.4.28) and using $\eta\sigma^2/2 \le R\sigma/(2\sqrt{T})$ gives (2.4.30). □

For a mini-batch of size b, if one example has conditional noise variance at most σ_1^2, then (2.4.5) permits $\sigma^2 = \sigma_1^2/b$. The stochastic term in (2.4.30) is then $R\sigma_1/\sqrt{bT}$.

For a common notation in the comparison below, write $\Delta(\theta) := \Phi(\theta) - \Phi(\theta^\star)$, $\Delta_T := \Delta(\theta_T)$, and $e_T := \theta_T - \theta^\star$. Expectations are included for stochastic methods.

Table 2.2: Convergence rates obtained through Section 2.4. The reported iterate and assumptions are part of each guarantee.

Method	Geometry and reported iterate	Objective-gap rate	Parameter conclusion
GD	μ-strongly convex, β-smooth; final	$\Delta_T = O(e^{-T/\kappa})$	$\|e_T\| = O(e^{-T/(2\kappa)})$ from the general bound
GD	Convex, β-smooth; final	$\Delta_T = O(T^{-1})$	No uniform rate to a particular minimizer
SGD, constant η	μ-strongly convex, β-smooth; final	$\mathbb{E}\Delta_T = O(e^{-\mu\eta T}) + O(\eta\sigma^2)$	$\mathbb{E}\,\|e_T\|^2$ has the same transient–floor form
SGD, $\eta_t \asymp 1/t$	μ-strongly convex, β-smooth; final or arithmetic average	$\mathbb{E}\Delta_T = O(T^{-1})$	$\mathbb{E}\,\|e_T\|^2 = O(T^{-1})$, also for the average
SGD, horizon-tuned η	Convex with bounded second moment; uniform average	$\mathbb{E}\Delta(\overline{\theta}_T) = O(T^{-1/2})$	No uniform parameter rate
SGD, tuned constant η	Convex, β-smooth, bounded variance; uniform average	$\mathbb{E}\Delta(\overline{\theta}_T) = O(\beta R^2/T + R\sigma/\sqrt{T})$	No uniform parameter rate

The two deterministic rows have no gradient-noise floor. Strong convexity is what turns an objective rate into a parameter rate; averaging turns the cumulative convex-SGD bounds into guarantees for one reported point. Smoothness replaces the global stochastic-gradient second-moment bound by a noise-variance bound and exposes the deterministic $O(1/T)$ term, but persistent noise still leaves an $O(1/\sqrt{T})$ term.

2.5 Uncertainty and asymptotic normality of SGD

The preceding results measure optimization error through expected objective gaps. Under randomness, however, how should we measure the uncertainty of the estimated parameter itself? A central limit theorem answers this by giving an asymptotic distribution, and hence an approximate covariance and confidence region. We first study the final SGD iterate and then show how Polyak–Ruppert averaging changes its limiting covariance.

A local stochastic-approximation model

Consider the unconstrained recursion

$$\theta_{t+1} = \theta_t - \eta_t \widehat{g}_{t+1}(\theta_t), \qquad \xi_{t+1} := \widehat{g}_{t+1}(\theta_t) - \nabla\Phi(\theta_t). \tag{2.5.1}$$

Here $\mathcal{H}_t$ contains the history before the new stochastic gradient is drawn. Write $e_t := \theta_t - \theta^\star$. To make the two central limit theorems transparent, assume:

1. $\theta_t \to \theta^\star$ in probability, where $\nabla\Phi(\theta^\star) = 0$. For each step-size schedule below, the local mean-square rate satisfies $\mathbb{E}\,\|e_t\|^2 = O(\eta_t)$.
2. $H := \nabla^2\Phi(\theta^\star)$ is positive definite and the Hessian is locally Lipschitz. Thus, near $\theta^\star$,
$$\nabla\Phi(\theta_t) = He_t + r_t, \qquad \|r_t\| \le C\,\|e_t\|^2. \tag{2.5.2}$$
3. $\mathbb{E}[\xi_{t+1} \mid \mathcal{H}_t] = 0$, and for some positive-semidefinite matrix S,
$$\mathbb{E}[\xi_{t+1}\xi_{t+1}^\top \mid \mathcal{H}_t] \longrightarrow S \quad \text{in probability;} \tag{2.5.3}$$
4. the noise has a locally uniform conditional $(2+\delta)$-moment for some $\delta > 0$. In particular, the conditional Lindeberg condition needed by the martingale central limit theorem holds.

The first condition is a stability and rate assumption, not a consequence of the local calculation. Global regularity or a separate convergence theorem is needed to show that the recursion enters this neighborhood and has the stated mean-square rate.

Asymptotic normality of the final SGD iterate

Theorem 2.19 (Last-iterate SGD central limit theorem)**.** *Under the local assumptions above, let $\eta_t = a/t$ with $a > 0$. If $2a\lambda_{\min}(H) > 1$, then*

$$\sqrt{T}(\theta_T - \theta^\star) \xrightarrow{d} \mathcal{N}(0, V_a), \tag{2.5.4}$$

where V_a is the unique symmetric solution of the Lyapunov equation

$$\left(aH - \frac{1}{2}I\right)V_a + V_a\left(aH - \frac{1}{2}I\right)^\top = a^2 S. \tag{2.5.5}$$

Proof. By (2.5.2), the error recursion is

$$e_{t+1} = \left(I - \frac{aH}{t}\right)e_t - \frac{a}{t}\xi_{t+1} - \frac{a}{t}r_t. \tag{2.5.6}$$

For $1 \le t < T$, define the transition matrix

$$\Gamma_{T,t} := \prod_{j=t+1}^{T-1}\left(I - \frac{aH}{j}\right), \qquad \Gamma_{T,0} := \prod_{j=1}^{T-1}\left(I - \frac{aH}{j}\right),$$

where an empty product is the identity. Iterating (2.5.6) gives

$$\sqrt{T}e_T = \sqrt{T}\Gamma_{T,0}e_1 - \sum_{t=1}^{T-1} W_{T,t}\xi_{t+1} - \sum_{t=1}^{T-1} W_{T,t}r_t, \qquad W_{T,t} := \sqrt{T}\frac{a}{t}\Gamma_{T,t}. \tag{2.5.7}$$

Because H is positive definite, $\|\Gamma_{T,t}\| \le C(t/T)^{a\lambda_{\min}(H)}$. Hence $2a\lambda_{\min}(H) > 1$ makes the initialization term in (2.5.7) vanish. The quadratic remainder bound in (2.5.2), together with $\mathbb{E}\,\|e_t\|^2 = O(t^{-1})$, similarly gives

$$\sum_{t=1}^{T-1} W_{T,t}r_t = o_{\mathbb{P}}(1).$$

It remains to analyze the weighted martingale differences. For $x \in (0,1]$, write $x^{aH} := \exp(aH\log x)$. The product approximation $\Gamma_{T,t} = (t/T)^{aH} + o(1)$, the covariance assumption, and a Riemann-sum argument give

$$\sum_{t=1}^{T-1} W_{T,t}\mathbb{E}[\xi_{t+1}\xi_{t+1}^\top \mid \mathcal{H}_t]W_{T,t}^\top \xrightarrow{\mathbb{P}} V_a := a^2\int_0^1 x^{aH-I}Sx^{aH^\top - I}\,dx. \tag{2.5.8}$$

The integral is finite under the displayed stability condition. The conditional moment assumption gives the corresponding Lindeberg condition, so the martingale triangular-array central limit theorem yields $\sum_t W_{T,t}\xi_{t+1} \Rightarrow \mathcal{N}(0, V_a)$.

Finally, with $A = aH - I/2$ and the change of variables $x = e^{-s}$,

$$V_a = a^2\int_0^\infty e^{-As}Se^{-A^\top s}\,ds.$$

Differentiating the integrand and integrating from zero to infinity gives $AV_a + V_aA^\top = a^2S$. Slutsky's theorem applied to (2.5.7) proves (2.5.4). □

In one dimension, if $H = h > 0$ and $S = \sigma_g^2$, then

$$V_a = \frac{a^2\sigma_g^2}{2ah-1}, \qquad ah > \frac{1}{2}. \tag{2.5.9}$$

This variance is minimized at $a = 1/h$, where it equals σ_g^2/h^2. Thus the last iterate is asymptotically normal, but its covariance can be sensitive to the learning-rate constant and the curvature.

Polyak–Ruppert averaging

Proposition 2.15 gave a finite-time $O(1/T)$ expectation bound for the arithmetic average under global smooth strong convexity and a $1/t$-scale step size. We now ask a finer question: after normalization, what is the limiting distribution of that average? For this covariance-efficient limit, we use a slightly slower decay $t^{-\alpha}$ with $1/2 < \alpha < 1$ and the local assumptions above.

Theorem 2.20 (Polyak–Ruppert central limit theorem)**.** *Under the same local assumptions, use*

$$\eta_t = at^{-\alpha}, \qquad a > 0, \qquad \frac{1}{2} < \alpha < 1, \tag{2.5.10}$$

and report the arithmetic average

$$\bar{\theta}_T^{\mathrm{PR}} := \frac{1}{T}\sum_{t=1}^{T}\theta_t. \tag{2.5.11}$$

Then

$$\sqrt{T}(\bar{\theta}_T^{\mathrm{PR}} - \theta^\star) \xrightarrow{d} \mathcal{N}\big(0, H^{-1}S(H^{-1})^\top\big). \tag{2.5.12}$$

In particular, the leading covariance does not depend on a or α within this range.

Proof. Substitute (2.5.2) into the SGD recursion:

$$e_{t+1} - e_t = -\eta_t(He_t + \xi_{t+1} + r_t).$$

Rearrange and average from $t = 1$ to T:

$$H(\bar{\theta}_T^{\mathrm{PR}} - \theta^\star) = -\frac{1}{T}\sum_{t=1}^{T}\xi_{t+1} - \frac{1}{T}\sum_{t=1}^{T}\frac{\theta_{t+1}-\theta_t}{\eta_t} - \frac{1}{T}\sum_{t=1}^{T} r_t. \tag{2.5.13}$$

For the increment term, summation by parts gives

$$\sum_{t=1}^{T}\frac{e_{t+1}-e_t}{\eta_t} = \frac{e_{T+1}}{\eta_T} - \frac{e_1}{\eta_1} + \sum_{t=2}^{T} e_t\left(\frac{1}{\eta_{t-1}} - \frac{1}{\eta_t}\right). \tag{2.5.14}$$

Here $\eta_t^{-1} = a^{-1}t^\alpha$ and the local mean-square bound implies $e_t = O_{\mathbb{P}}(t^{-\alpha/2})$. Consequently, the right-hand side of (2.5.14) is $O_{\mathbb{P}}(T^{\alpha/2})$, so after division by $\sqrt{T}$ it is $o_{\mathbb{P}}(1)$ because $\alpha < 1$.

The quadratic Taylor remainder and $\mathbb{E}\left\|e_t\right\|^2 = O(t^{-\alpha})$ give

$$\frac{1}{\sqrt{T}}\sum_{t=1}^{T} r_t = O_{\mathbb{P}}(T^{1/2-\alpha}) = o_{\mathbb{P}}(1),$$

where the last step uses $\alpha > 1/2$. Multiplying (2.5.13) by $\sqrt{T}$ therefore yields the asymptotic linear representation

$$\sqrt{T}(\bar{\theta}_T^{\mathrm{PR}} - \theta^\star) = -H^{-1}\frac{1}{\sqrt{T}}\sum_{t=1}^{T}\xi_{t+1} + o_{\mathbb{P}}(1). \tag{2.5.15}$$

By (2.5.3), the Cesàro average of the conditional covariances converges in probability to S. The conditional Lindeberg condition then gives

$$\frac{1}{\sqrt{T}}\sum_{t=1}^{T}\xi_{t+1} \xrightarrow{d} \mathcal{N}(0, S)$$

by the martingale central limit theorem. Apply Slutsky's theorem to (2.5.15) to obtain (2.5.12). □

The limiting covariance coincides with the covariance obtained by using the ideal matrix gain H^{-1} in the linearized recursion; this is the sense in which Polyak–Ruppert averaging is asymptotically covariance-efficient. The learning-rate parameters still affect transient behavior and finite-time error. The restriction $1/2 < \alpha < 1$ balances $\sum_t \eta_t = \infty$ with $\sum_t \eta_t^2 < \infty$; the endpoint $\alpha = 1$ requires additional conditions and can retain tuning effects.

Example 2.21 (Online least squares)**:** Let

$$\Phi(\theta) = \frac{1}{2}\mathbb{E}[(Y - X^\top\theta)^2], \qquad \widehat{g}(\theta; X, Y) = (X^\top\theta - Y)X.$$

Write $\varepsilon = Y - X^\top\theta^\star$, and assume $H = \mathbb{E}[XX^\top]$ is positive definite. At the optimum,

$$S = \mathbb{E}[\varepsilon^2 XX^\top], \qquad \sqrt{T}(\bar{\theta}_T^{\mathrm{PR}} - \theta^\star) \xrightarrow{d} \mathcal{N}\left(0, H^{-1}SH^{-1}\right). \tag{2.5.16}$$

Under homoskedastic noise, $\mathbb{E}[\varepsilon^2 \mid X] = \sigma^2$, so $S = \sigma^2 H$ and the limiting covariance simplifies to $\sigma^2 H^{-1}$. This is the familiar least-squares covariance.

Output and step size	$\sqrt{T}$-scale limit	Main point
Final iterate, $\eta_t = a/t$	$\mathcal{N}(0, V_a)$, with V_a from (2.5.5)	Covariance depends on the scalar gain and curvature.
PR arithmetic average, $\eta_t = at^{-\alpha}$	$\mathcal{N}(0, H^{-1}S(H^{-1})^\top)$	Efficient leading covariance without tuning a to H.

These limits concern the number T of stochastic-approximation iterations for a fixed population objective and noise mechanism. Conditional finite-sum SGD has an analogous algorithmic limit around its empirical minimizer, but this is distinct from an $n \to \infty$ generalization or statistical sampling limit.

Inference after Polyak–Ruppert averaging

Write the asymptotic covariance in Theorem 2.20 as

$$\Sigma_{\mathrm{PR}} := H^{-1} S (H^{-1})^\top. \tag{2.5.17}$$

The central limit theorem identifies the limiting distribution, but a usable confidence interval must also handle the unknown matrix Σ_{PR}. For a fixed contrast $c \in \mathbb{R}^d$, with $c^\top \Sigma_{\mathrm{PR}} c > 0$,

$$\sqrt{T}\,\frac{c^\top(\overline{\theta}_T^{\mathrm{PR}} - \theta^\star)}{\sqrt{c^\top \Sigma_{\mathrm{PR}} c}} \xrightarrow{d} \mathcal{N}(0,1).$$

Taking $c = e_j$ gives inference for the j-th coordinate. Whenever $\widehat{\Sigma}_T \xrightarrow{\mathbb{P}} \Sigma_{\mathrm{PR}}$, an asymptotic $100(1-\delta)\%$ Wald interval is

$$c^\top \overline{\theta}_T^{\mathrm{PR}} \;\pm\; z_{1-\delta/2}\sqrt{\frac{c^\top \widehat{\Sigma}_T c}{T}}, \tag{2.5.18}$$

where z_q is the q-quantile of the standard normal distribution. The following methods differ mainly in how they estimate or avoid Σ_{PR}.

Plug-in covariance estimator. Suppose the stochastic gradient is generated by a twice differentiable sample loss,

$$\widehat{g}_{t+1}(\theta) = \nabla_\theta \ell(\theta; Z_{t+1}).$$

Along the SGD path, define

$$\begin{aligned}
\widehat{H}_T &:= \frac{1}{T}\sum_{t=0}^{T-1} \nabla_\theta^2 \ell(\theta_t; Z_{t+1}),\\
\widehat{S}_T &:= \frac{1}{T}\sum_{t=0}^{T-1} \widehat{g}_{t+1}(\theta_t)\widehat{g}_{t+1}(\theta_t)^\top,\\
\widehat{\Sigma}_T^{\mathrm{plug}} &:= \widehat{H}_T^{-1} \widehat{S}_T (\widehat{H}_T^{-1})^\top.
\end{aligned} \tag{2.5.19}$$

Since $\theta_t \xrightarrow{\mathbb{P}} \theta^\star$, suitable laws of large numbers give $\widehat{H}_T \xrightarrow{\mathbb{P}} H$ and $\widehat{S}_T \xrightarrow{\mathbb{P}} S$, hence $\widehat{\Sigma}_T^{\mathrm{plug}} \xrightarrow{\mathbb{P}} \Sigma_{\mathrm{PR}}$. In finite samples one may floor the eigenvalues of $\widehat{H}_T$ before inversion.

The intuition is the same as for a sandwich covariance estimator: $\widehat{S}_T$ measures stochastic-gradient noise, while $\widehat{H}_T^{-1}$ converts gradient perturbations into parameter perturbations. The matrices can be updated online, but computing and storing the Hessian and its inverse can be prohibitive for a large model. For one contrast, an explicit inverse is unnecessary: solve $\widehat{H}_T^\top u = c$, then $c^\top \widehat{\Sigma}_T^{\mathrm{plug}} c = u^\top \widehat{S}_T u$ [5].

Online multiplier bootstrap. Let $U_t^{(b)}$, $b = 1, \dots, B$, be nonnegative weights that are mutually independent across both t and b, independent of the data, and satisfy

$$\mathbb{E}[U_t^{(b)}] = 1, \qquad \mathrm{Var}(U_t^{(b)}) = 1.$$

Exponential or Poisson weights are common choices. Initialize every replica from the original starting point, $\theta_0^{*(b)} = \theta_0$, and run a perturbed SGD trajectory on the same observation stream:

$$\begin{aligned}
\theta_{t+1}^{*(b)} &= \theta_t^{*(b)} - \eta_t U_{t+1}^{(b)} \widehat{g}_{t+1}(\theta_t^{*(b)}),\\
\overline{\theta}_T^{*(b)} &:= \frac{1}{T}\sum_{t=1}^{T} \theta_t^{*(b)}.
\end{aligned} \tag{2.5.20}$$

Conditionally on the observed stream, the empirical distribution of

$$\sqrt{T}\,(\overline{\theta}_T^{*(b)} - \overline{\theta}_T^{\mathrm{PR}})$$

approximates the sampling distribution of $\sqrt{T}(\overline{\theta}_T^{\mathrm{PR}} - \theta^\star)$.

Let $q^*_{\tau,T}$ be the empirical τ-quantile of

$$\left\{\sqrt{T}\, c^\top(\overline{\theta}_T^{*(b)} - \overline{\theta}_T^{\mathrm{PR}}) : b = 1, \dots, B\right\}.$$

A basic bootstrap interval is

$$\left[c^\top \overline{\theta}_T^{\mathrm{PR}} - \frac{q^*_{1-\delta/2,T}}{\sqrt{T}}, \quad c^\top \overline{\theta}_T^{\mathrm{PR}} - \frac{q^*_{\delta/2,T}}{\sqrt{T}}\right]. \tag{2.5.21}$$

This avoids an analytic covariance formula and still uses only one pass over the data. Its main cost is maintaining B extra trajectories and evaluating a gradient at every replica's parameter. Naively resampling the dependent SGD iterates themselves is not equivalent to this procedure [6].

Batch-means covariance estimator. Here a "batch" is a block of consecutive SGD iterates used for variance estimation; it is not the mini-batch used to compute a stochastic gradient. After an initial burn-in block, divide the trajectory into blocks $\mathcal{B}_1, \dots, \mathcal{B}_M$, with sizes $m_1, \dots, m_M$, and set

$$\widetilde{\theta}_k := \frac{1}{m_k} \sum_{t \in \mathcal{B}_k} \theta_t, \qquad \widetilde{\theta} := \frac{\sum_{k=1}^M m_k \widetilde{\theta}_k}{\sum_{k=1}^M m_k}.$$

The weighted batch-means estimator is

$$\widehat{\Sigma}_T^{\mathrm{BM}} := \frac{1}{M} \sum_{k=1}^M m_k (\widetilde{\theta}_k - \widetilde{\theta})(\widetilde{\theta}_k - \widetilde{\theta})^\top. \tag{2.5.22}$$

A long block average has covariance approximately Σ_{PR}/m_k; multiplication by m_k therefore puts every block on the same covariance scale.

Because decreasing-step SGD is time-inhomogeneous, the block lengths should also increase. For $\eta_t = at^{-\alpha}$, $1/2 < \alpha < 1$, one schedule-aware choice is

$$e_k = \left\lfloor \{(k+1)D\}^{1/(1-\alpha)} \right\rfloor, \quad k = 0, \dots, M, \qquad \mathcal{B}_k = \{e_{k-1} + 1, \dots, e_k\}, \tag{2.5.23}$$

where e_0 ends the discarded burn-in block. Then $\sum_{t \in \mathcal{B}_k} \eta_t \asymp D$, so successive batches span comparable amounts of algorithmic time. For a fixed horizon T, one may choose $D = T^{1-\alpha}/(M+1)$, up to integer rounding, so that $e_M \approx T$; equivalently, retain every complete block with $e_M \le T$ and regard e_M as the effective horizon. With M and D growing at suitable rates, $\widehat{\Sigma}_T^{\mathrm{BM}}$ consistently estimates Σ_{PR} and can be inserted into (2.5.18). It uses only the iterates, but short batches retain strong dependence, whereas very long batches leave too few covariance replicates [5].

Random scaling and self-normalization. Random scaling avoids consistently estimating Σ_{PR}. Define the prefix averages $\overline{\theta}_s = s^{-1} \sum_{t=1}^s \theta_t$ and the random normalizer

$$\widehat{R}_T := \frac{1}{T^2} \sum_{s=1}^T s^2 (\overline{\theta}_s - \overline{\theta}_T^{\mathrm{PR}})(\overline{\theta}_s - \overline{\theta}_T^{\mathrm{PR}})^\top. \tag{2.5.24}$$

This construction deliberately uses the dependence along the entire SGD trajectory. Its pathwise limit typically requires uniform high-moment and tightness conditions beyond those used for the pointwise central limit theorem. Under the resulting functional central limit theorem,

$$\frac{1}{\sqrt{T}} \sum_{t=1}^{\lfloor Tr \rfloor} (\theta_t - \theta^\star) \Longrightarrow \Sigma_{\mathrm{PR}}^{1/2} W(r), \qquad 0 \le r \le 1,$$

the scalar self-normalized statistic satisfies

$$\frac{\sqrt{T}\, c^\top(\overline{\theta}_T^{\mathrm{PR}} - \theta^\star)}{\sqrt{c^\top \widehat{R}_T c}} \xrightarrow{d} \frac{W(1)}{\left[\int_0^1 \{W(r) - rW(1)\}^2 \, dr\right]^{1/2}} =: \mathcal{T}_{\mathrm{RS}}, \tag{2.5.25}$$

where W is a scalar standard Brownian motion. The unknown scale $c^\top \Sigma_{\mathrm{PR}} c$ cancels, so the limit is pivotal, but it is not standard normal.

If κ_q is the q-quantile of $\mathcal{T}_{\mathrm{RS}}$, the corresponding interval is

$$c^\top \overline{\theta}_T^{\mathrm{PR}} \ \pm \ \kappa_{1-\delta/2} \sqrt{\frac{c^\top \widehat{R}_T c}{T}}. \tag{2.5.26}$$

For example, the two-sided 95% interval uses $\kappa_{0.975} \approx 6.747$, not 1.96. The critical values can be simulated once from Brownian motion and reused. The method needs neither Hessians nor bootstrap replicas and $\widehat{R}_T$ admits online updates. However, $\widehat{R}_T$ is a random normalizer, not a consistent estimator of Σ_{PR}, and validity requires the functional central limit theorem [14].

Method	Additional input	Main trade-off
Plug-in	Gradients, Hessians, and linear solves	Direct normal inference, but intrusive and expensive in high dimension.
Bootstrap	B perturbed SGD paths	Avoids a covariance formula, but multiplies computation.
Batch means	One path, burn-in, and a batch schedule	Derivative-free, but sensitive to dependence and batch length.
Random scaling	One path and nonstandard critical values	No covariance estimate or resampling, but uses a non-Gaussian limit.

These are fixed-dimensional local results around an isolated minimizer with a nonsingular Hessian. They do not automatically justify coordinatewise inference for an overparameterized neural network, where symmetries and flat directions often make H singular. The source of randomness must also match the inferential target: an i.i.d. online data stream can support inference for a population minimizer, whereas repeated finite-sum epochs, sampling without replacement, and adaptive mini-batches introduce a different dependence structure. None of the four procedures repairs a failure of the underlying Polyak–Ruppert central limit theorem.

2.6 Momentum

Let g_t denote either the full gradient or a mini-batch gradient evaluated at θ_{t-1}. Momentum keeps an exponential moving average of recent gradients. Initialize $m_0 = 0$, choose $0 \le \rho < 1$ and a fixed base learning rate $\eta > 0$, and update

$$\begin{aligned} m_t &= \rho m_{t-1} + (1-\rho) g_t, \\ \theta_t &= \theta_{t-1} - \eta m_t. \end{aligned} \tag{2.6.1}$$

Persistent gradient directions reinforce one another in m_t, while directions that alternate rapidly tend to cancel. This often reduces zig-zagging across a narrow valley and speeds motion along the valley. A larger ρ gives longer memory but can also increase overshoot, so ρ and the learning rate η should be tuned together. Some references omit the factor $1-\rho$; for fixed ρ, that convention can be absorbed into the learning rate [21, 9].

2.7 Adaptive methods: AdaGrad, RMSprop, and Adam

The methods in this section rescale each coordinate using its recent gradient history. In all displayed formulas, products, square roots, and divisions of vectors are coordinatewise, $\varepsilon > 0$ is a numerical stabilizer, and g_t is evaluated at θ_{t-1}. The displayed updates use a fixed base learning rate $\eta > 0$.

AdaGrad

Initialize $r_0 = 0$. AdaGrad accumulates all past squared gradients:

$$r_t = r_{t-1} + g_t \odot g_t,$$

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{r_t} + \varepsilon}. \tag{2.7.1}$$

Coordinate i has effective learning rate $\eta/(\sqrt{r_{t,i}}+\varepsilon)$. Coordinates with frequent or large gradients are damped more strongly, while rare features retain relatively larger steps. Because r_t never forgets, the effective learning rates can become very small late in training.

RMSprop

Initialize $r_0 = 0$ and choose $0 \le \rho < 1$. RMSprop replaces AdaGrad's cumulative sum by an exponential moving average:

$$\begin{aligned} r_t &= \rho r_{t-1} + (1-\rho) g_t \odot g_t, \\ \theta_t &= \theta_{t-1} - \eta \frac{g_t}{\sqrt{r_t} + \varepsilon}. \end{aligned} \tag{2.7.2}$$

The denominator estimates the recent root-mean-square gradient in each coordinate. Forgetting old gradients prevents the accumulator from growing forever and lets the effective step size respond to a changing optimization landscape. Implementations may place ε inside the square root, so the convention should be stated when exact reproducibility matters [9].

Adam

Adam combines a momentum-like first moment with RMSprop-style second-moment scaling. Initialize $m_0 = v_0 = 0$, choose $0 \le \beta_1, \beta_2 < 1$, and update

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)(g_t \odot g_t), \tag{2.7.3}$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{2.7.4}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}. \tag{2.7.5}$$

The first moment supplies a smoothed direction, while the second moment sets a coordinatewise scale. Bias correction compensates for initializing both moving averages at zero and matters most during the first iterations. Adam is deterministic when g_t is the full-batch gradient and stochastic when g_t is a mini-batch estimate; coordinatewise adaptation does not remove the need to tune the global learning rate η. A decay schedule is a separate design choice and is not part of the displayed update [12].

2.8 AdamW and Muon

AdamW: decoupled weight decay

Compute m_t, v_t, $\widehat{m}_t$, and $\widehat{v}_t$ as in Adam, using the gradient of the data-fitting loss. With the same fixed base learning rate $\eta > 0$, AdamW then applies weight decay as a separate parameter shrinkage:

$$\theta_t = (1 - \eta\lambda)\theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}, \qquad \lambda \ge 0. \tag{2.8.1}$$

This differs from adding $\lambda\theta_{t-1}$ to g_t: in that case the regularization term passes through Adam's moment estimates and adaptive coordinate scaling. AdamW keeps the adaptive loss update and direct multiplicative shrinkage separate. In practice, weight decay is often applied to weight matrices but omitted for biases and normalization parameters [16].

Muon: orthogonalized momentum for matrix parameters

Muon is designed for a matrix-valued hidden-layer parameter $W \in \mathbb{R}^{p\times q}$. Let G_t be its full-batch or mini-batch gradient, initialize $B_0 = 0$, and first form a momentum matrix

$$B_t = \rho B_{t-1} + (1-\rho)G_t, \qquad 0 \le \rho < 1. \tag{2.8.2}$$

Let $r = \operatorname{rank}(B_t)$ and write a compact singular-value decomposition $B_t = U_t\Sigma_t V_t^\top$, where $U_t \in \mathbb{R}^{p\times r}$ and $V_t \in \mathbb{R}^{q\times r}$. Define the polar partial isometry

$$O_t := \operatorname{Ortho}(B_t) = U_t V_t^\top. \tag{2.8.3}$$

A one-dimensional derivation and proof. For a nonzero scalar b, its polar factor is $\operatorname{sign}(b) = b/\sqrt{b^2}$. The core numerical problem is therefore an inverse square root. Given $a > 0$, solve $h(y) = y^{-2} - a = 0$. Newton's method gives

$$y_{k+1} = y_k - \frac{h(y_k)}{h'(y_k)} = \frac{1}{2}y_k\big(3 - a y_k^2\big). \tag{2.8.4}$$

After rescaling by $s_k = \sqrt{a}\, y_k$, this becomes the universal scalar iteration

$$s_{k+1} = \phi_{\mathrm{NS}}(s_k), \qquad \phi_{\mathrm{NS}}(s) = \frac{1}{2}s(3-s^2) = \frac{3}{2}s - \frac{1}{2}s^3. \tag{2.8.5}$$

Suppose $0 < s_0 \le 1$, and define the squared error $e_k = 1 - s_k^2$. Then $0 \le e_0 < 1$, and direct substitution gives

$$e_{k+1} = 1 - \left\{\frac{1}{2}s_k(3-s_k^2)\right\}^2 = \frac{1}{4}e_k^2(3+e_k) \le e_k^2. \tag{2.8.6}$$

The identity preserves $0 \le e_k < 1$, and consequently $0 \le e_k \le e_0^{2^k} \to 0$. Thus $s_k \to 1$ with local quadratic convergence; a zero starting value remains zero. If $a = b^2$ and $x_k := by_k$, then $|x_k| = s_k$, the sign of x_k is preserved, and $x_k \to \operatorname{sign}(b)$. Notice that when s_0 is extremely small, e_0 is very close to one: quadratic convergence near the limit does not guarantee fast progress in the first few steps.

From the scalar iteration to the matrix polar factor. For a full-row-rank matrix C, the polar factor can be written as $(CC^\top)^{-1/2}C$; the compact-SVD formulation also covers rank-deficient matrices. The scalar argument extends one singular value at a time.

Proposition 2.22 (Classical Newton–Schulz polar iteration)**.** *Let $C \in \mathbb{R}^{m\times n}$, with $m \le n$, be nonzero and have compact SVD $C = P\operatorname{diag}(\sigma_1,\dots,\sigma_r)Q^\top$, where $\sigma_1 \ge \cdots \ge \sigma_r > 0$. Choose $\alpha > 0$ such that $\alpha\sigma_1 \le 1$, set $X_0 = \alpha C$, and iterate*

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}\big(X_kX_k^\top\big)X_k. \tag{2.8.7}$$

Then $X_k \to PQ^\top$, the polar partial isometry of C. More precisely, if $s_i^{(k)}$ are the positive singular values of X_k and

$$E_k := \max_{1\le i\le r}\big\{1 - (s_i^{(k)})^2\big\},$$

then $0 \le E_{k+1} \le E_k^2$ and $E_k \le E_0^{2^k} \to 0$.

Proof. The initialization has $X_0 = P\operatorname{diag}(s_i^{(0)})Q^\top$, where $s_i^{(0)} = \alpha\sigma_i \in (0,1]$. If $X_k = P\operatorname{diag}(s_i^{(k)})Q^\top$, orthonormality of the compact SVD factors gives

$$(X_kX_k^\top)X_k = P\operatorname{diag}\big((s_i^{(k)})^3\big)Q^\top.$$

Hence the singular vectors stay fixed and each singular value obeys exactly the scalar recurrence (2.8.5). Applying (2.8.6) to every i yields $0 \le E_{k+1} \le E_k^2$, and therefore $E_k \le E_0^{2^k} \to 0$. Thus every positive singular value tends to one, while the null space remains unchanged, so $X_k \to PI_rQ^\top = PQ^\top$. □

Application to Muon. Let $C_t = B_t$ when $p \le q$, and let $C_t = B_t^\top$ otherwise, so that C_t has no more rows than columns. It uses the Frobenius-normalized initialization

$$X_{t,0} = \frac{C_t}{\|C_t\|_{\mathrm{F}} + \varepsilon_{\mathrm{NS}}}, \qquad \varepsilon_{\mathrm{NS}} > 0.$$

Because $\|C_t\|_2 \le \|C_t\|_{\mathrm{F}}$, every positive initial singular value lies in $(0, 1)$, so the proposition applies to the classical iteration without computing an SVD.

The five-step Muon approximation. Muon uses a more aggressive, tuned quintic Newton–Schulz-type polynomial. For $k = 0, \dots, 4$, it computes

$$A_{t,k} = X_{t,k} X_{t,k}^\top, \qquad X_{t,k+1} = aX_{t,k} + \big(bA_{t,k} + cA_{t,k}^2\big)X_{t,k}, \tag{2.8.8}$$

Here $(a, b, c) = (3.4445, -4.7750, 2.0315)$. On singular values, one step applies $\phi_{\mathrm{M}}(s) = as + bs^3 + cs^5$. Its large slope at the origin, $\phi'_{\mathrm{M}}(0) = 3.4445$, lifts very small singular values much faster than the classical cubic map: in the local linear approximation, five compositions amplify a tiny value by about $3.4445^5 \approx 485$, compared with $(3/2)^5 \approx 7.6$ for the classical iteration. The nonlinear terms then keep larger values in a comparatively narrow band.

There are two distinct "fives" here: each step uses a degree-five polynomial, and the standard Muon implementation repeats that step five times. The latter is an empirical accuracy–cost choice, not a theorem that the polar factor is reached after exactly five steps. Indeed, $\phi_{\mathrm{M}}(1) = 0.701 \neq 1$, so these tuned coefficients deliberately favor rapid finite-step spectral flattening over asymptotically exact orthogonalization. The original experiments found five steps sufficient; subsequent scaling experiments found that ten steps produced a more accurate polar approximation but no better training performance, while requiring more dense matrix multiplications [13, 15].

Transpose the result back when necessary: set $\widetilde{O}_t = X_{t,5}$ if $p \le q$, and $\widetilde{O}_t = X_{t,5}^\top$ if $p > q$. With a shape-dependent scale $s_{p,q} > 0$ and fixed learning rate $\eta > 0$, a schematic decoupled-decay update is

$$W_t = (1 - \eta\lambda)W_{t-1} - \eta s_{p,q}\widetilde{O}_t, \qquad \widetilde{O}_t \approx O_t. \tag{2.8.9}$$

The momentum step aggregates gradient information; finite-step spectral flattening then balances the update across singular directions. Muon is normally used for hidden two-dimensional weight matrices (and reshaped convolution kernels), while embeddings, output heads, biases, gains, and other scalar or vector parameters are handled by an auxiliary optimizer such as AdamW [13, 15].

The proposition proves convergence of the classical Newton–Schulz iteration. Muon's tuned quintic is instead a deliberately finite-step spectral transformation, so no analogous asymptotic convergence claim is intended.

Lecture 3

Concentration inequalities and uniform convergence

3.1 From the error decomposition to concentration

The master bound from Lecture 1 isolates the statistical quantity that this lecture must control:

$$L(\widetilde{f}_n) - L^\star \le \{L^\star_{\mathcal{F}} - L^\star\} + 2\Delta_n(\mathcal{F}) + \varepsilon_{\mathrm{opt}}, \qquad \Delta_n(\mathcal{F}) = \sup_{f\in\mathcal{F}} |\widehat{L}_n(f) - L(f)|. \tag{3.1.1}$$

Thus the statistical goal is a high-probability bound on the difference between empirical and population loss, simultaneously over the function class $\mathcal{F}$. The supremum matters because the learned predictor is selected from $\mathcal{F}$ using the same sample.

The i.i.d. assumption reduces the fixed-function part of this problem to a standard empirical-mean problem. Let $S = (Z_1, \dots, Z_n) \sim P^n$, with $Z_i = (X_i, Y_i)$, and fix a function f independently of the sample. Define

$$W_i^{(f)} := \ell(f(X_i), Y_i). \tag{3.1.2}$$

Then $W_1^{(f)}, \dots, W_n^{(f)}$ are i.i.d., and

$$\widehat{L}_n(f) = \frac{1}{n}\sum_{i=1}^n W_i^{(f)}, \qquad L(f) = \mathbb{E}[W_i^{(f)}]. \tag{3.1.3}$$

Consequently, for each fixed f, controlling $|\widehat{L}_n(f) - L(f)|$ is exactly the problem of controlling the deviation of an i.i.d. empirical mean from its expectation. The available bound depends on the assumptions on the loss random variable:

- finite variance gives Chebyshev's polynomial tail;
- a bounded range gives Hoeffding's sub-Gaussian tail; and
- bounded centered variables plus variance information give Bernstein's variance-sensitive tail.

These inequalities first solve the fixed-f problem. We then turn that pointwise control into a single event that holds for every $f \in \mathcal{F}$, so it continues to hold for a predictor selected after seeing the data.

3.2 Chebyshev's inequality: a finite-variance baseline

Theorem 3.1 (Chebyshev for an empirical mean)**.** *Let $X_1, \dots, X_n$ be i.i.d. with mean μ and variance $\sigma^2 < \infty$. Then for every $\varepsilon > 0$,*

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^n X_i - \mu\right| \ge \varepsilon\right) \le \frac{\sigma^2}{n\varepsilon^2}. \tag{3.2.1}$$

Proof. The sample mean has expectation μ and variance σ^2/n. Apply Markov's inequality to the nonnegative random variable $\left(\frac{1}{n}\sum_i X_i - \mu\right)^2$. □

Chebyshev needs no boundedness or exponential moment. Its price is a polynomial tail bound: its guarantee of failure probability at most δ uses ε of order $1/\sqrt{n\delta}$, rather than $\sqrt{\log(1/\delta)/n}$.

Corollary 3.2 (Expected error of the empirical mean)**.** *Under the assumptions of Theorem 3.1, write* $\overline{X}_n = n^{-1}\sum_{i=1}^n X_i$*. Its mean squared error (MSE) and mean absolute error (MAE) satisfy*

$$\mathbb{E}(\overline{X}_n - \mu)^2 = \frac{\sigma^2}{n}, \qquad \mathbb{E}|\overline{X}_n - \mu| \le \frac{\sigma}{\sqrt{n}}. \tag{3.2.2}$$

Proof. Unbiasedness gives $\mathbb{E}(\overline{X}_n - \mu)^2 = \mathrm{Var}(\overline{X}_n) = \sigma^2/n$. Cauchy–Schwarz then gives $\mathbb{E}|\overline{X}_n - \mu| \le \sqrt{\mathbb{E}(\overline{X}_n - \mu)^2}$. □

The *signed* expected error $\mathbb{E}(\overline{X}_n - \mu)$ is zero and does not measure estimation accuracy. The finite-variance consequence requested in [29, Exercise 1.3.3] is the MAE bound, while the exact MSE identity also identifies the standard error $\sigma/\sqrt{n}$.

3.3 Hoeffding's inequality: bounded variables

Lemma 3.3 (Hoeffding's lemma)**.** *If* $X \in [a,b]$ *almost surely, where* $a < b$*, then for every* $\lambda \in \mathbb{R}$*,*

$$\log \mathbb{E}\exp\{\lambda(X - \mathbb{E}X)\} \le \frac{\lambda^2(b-a)^2}{8}. \tag{3.3.1}$$

Proof. Let $\mu = \mathbb{E}X$ and $p = (\mu - a)/(b-a)$. Convexity of $x \mapsto e^{\lambda x}$ on $[a,b]$ gives

$$e^{\lambda x} \le \frac{b-x}{b-a}e^{\lambda a} + \frac{x-a}{b-a}e^{\lambda b}.$$

Taking expectations, centering by μ, and setting $s = \lambda(b-a)$ yields

$$\mathbb{E}e^{\lambda(X-\mu)} \le (1-p)e^{-ps} + pe^{(1-p)s}.$$

Let

$$h(s) := \log\{(1-p)e^{-ps} + pe^{(1-p)s}\}.$$

Direct differentiation gives $h(0) = h'(0) = 0$ and $h''(s) = q_s(1-q_s) \le 1/4$ for some $q_s \in [0,1]$. Integrating this second-derivative bound twice gives $h(s) \le s^2/8$. □

Theorem 3.4 (Hoeffding's inequality)**.** *Let* $X_1, \dots, X_n$ *be independent, with* $X_i \in [a,b]$ *almost surely, where* $a < b$*, and common mean* μ*. Then for every* $\varepsilon > 0$*,*

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^n X_i - \mu\right| \ge \varepsilon\right) \le 2\exp\left\{-\frac{2n\varepsilon^2}{(b-a)^2}\right\}. \tag{3.3.2}$$

Consequently, for every $\delta \in (0,1)$*, with probability at least* $1-\delta$*,*

$$\left|\frac{1}{n}\sum_{i=1}^n X_i - \mu\right| \le (b-a)\sqrt{\frac{\log(2/\delta)}{2n}}. \tag{3.3.3}$$

Proof. For $\lambda > 0$, Markov's inequality, independence, and Lemma 3.3 give

$$\begin{aligned}\mathbb{P}\left(\frac{1}{n}\sum_{i=1}^n X_i - \mu \ge \varepsilon\right) &\le e^{-\lambda n\varepsilon}\prod_{i=1}^n \mathbb{E}e^{\lambda(X_i-\mu)} \\ &\le \exp\left\{-\lambda n\varepsilon + \frac{n\lambda^2(b-a)^2}{8}\right\}.\end{aligned}$$

The exponent is minimized at $\lambda = 4\varepsilon/(b-a)^2$, which gives the upper-tail bound $\exp\{-2n\varepsilon^2/(b-a)^2\}$. Apply the same argument to $-X_i$ and add the two tail probabilities. Taking $\varepsilon = (b-a)\sqrt{\log(2/\delta)/(2n)}$ makes the right-hand side of (3.3.2) equal to δ. □

The exponential-moment method extends to functions of independent observations whose values change only a little when one observation is replaced.

Lemma 3.5 (McDiarmid's bounded-difference inequality)**.** *Let $Z_1, \dots, Z_n$ be independent random variables taking values in $\mathcal{Z}_1, \dots, \mathcal{Z}_n$, respectively, and let $U = u(Z_1, \dots, Z_n)$, where $u : \prod_{i=1}^n \mathcal{Z}_i \to \mathbb{R}$ is measurable. Suppose constants $c_1, \dots, c_n \geq 0$ satisfy, for every i and every pair of inputs differing only in coordinate i,*

$$\left|u(z_1, \dots, z_i, \dots, z_n) - u(z_1, \dots, \widetilde{z}_i, \dots, z_n)\right| \leq c_i.$$

Write $C = \sum_{i=1}^n c_i^2$. If $C > 0$, then for every $t > 0$,

$$\mathbb{P}(U - \mathbb{E}U \geq t) \leq \exp\left\{-\frac{2t^2}{C}\right\}, \qquad \mathbb{P}(\mathbb{E}U - U \geq t) \leq \exp\left\{-\frac{2t^2}{C}\right\}.$$

Consequently,

$$\mathbb{P}(|U - \mathbb{E}U| \geq t) \leq 2\exp\left\{-\frac{2t^2}{C}\right\}.$$

If $C = 0$, then U is constant and all these tail probabilities are zero.

Proof. Changing all coordinates one at a time changes u by at most $\sum_i c_i$, so u has bounded range and U is integrable. If $C = 0$, every coordinate change leaves u unchanged, hence u is constant. Assume from now on that $C > 0$.

Step 1: reveal the observations and bound each conditional range. Set

$$M_k := \mathbb{E}[U \mid Z_1, \dots, Z_k], \qquad D_k := M_k - M_{k-1}, \quad k = 1, \dots, n,$$

with $M_0 = \mathbb{E}U$ and $M_n = U$. These conditional expectations form the Doob martingale obtained by revealing one observation at a time, and $U - \mathbb{E}U = \sum_{k=1}^n D_k$.

Fix a realization $z_1, \dots, z_{k-1}$ of the first $k-1$ observations, and define

$$q_k(z) := \mathbb{E}_{Z_{k+1}, \dots, Z_n} u(z_1, \dots, z_{k-1}, z, Z_{k+1}, \dots, Z_n).$$

Thus q_k averages over the future observations while keeping the first $k-1$ observations and the proposed value z fixed. Independence ensures that, conditional on this fixed prefix, $M_k = q_k(Z_k)$ and $M_{k-1} = \mathbb{E}_{Z_k} q_k(Z_k)$. The coordinate-change assumption gives

$$\begin{aligned} |q_k(z) - q_k(\widetilde{z})| &\leq \mathbb{E}_{Z_{k+1}, \dots, Z_n} \big|u(z_1, \dots, z_{k-1}, z, Z_{k+1}, \dots, Z_n) \\ &\qquad - u(z_1, \dots, z_{k-1}, \widetilde{z}, Z_{k+1}, \dots, Z_n)\big| \\ &\leq c_k. \end{aligned}$$

Hence $q_k(Z_k)$ has conditional range of length at most c_k. Subtracting its conditional mean gives D_k, preserving the range length and making the conditional mean zero. Applying Lemma 3.3 conditionally yields, for every $\lambda \in \mathbb{R}$,

$$\mathbb{E}\left[e^{\lambda D_k} \mid Z_1, \dots, Z_{k-1}\right] \leq \exp\left\{\frac{\lambda^2 c_k^2}{8}\right\}.$$

When $c_k = 0$, $D_k = 0$, so the same bound holds directly.

Step 2: combine the conditional exponential bounds. Since $D_1, \dots, D_{k-1}$ are determined by the first $k-1$ observations, the tower property gives

$$\begin{aligned} \mathbb{E}e^{\lambda \sum_{j=1}^k D_j} &= \mathbb{E}\left[e^{\lambda \sum_{j=1}^{k-1} D_j} \mathbb{E}\left[e^{\lambda D_k} \mid Z_1, \dots, Z_{k-1}\right]\right] \\ &\leq \exp\left\{\frac{\lambda^2 c_k^2}{8}\right\} \mathbb{E}e^{\lambda \sum_{j=1}^{k-1} D_j}. \end{aligned}$$

Iterating over $k = n, n-1, \dots, 1$ gives

$$\mathbb{E}e^{\lambda(U - \mathbb{E}U)} \leq \exp\left\{\frac{\lambda^2}{8} \sum_{k=1}^n c_k^2\right\} = \exp\left\{\frac{\lambda^2 C}{8}\right\}.$$

Step 3: optimize the exponential tail bound. For $\lambda > 0$, Markov's inequality implies

$$\mathbb{P}(U - \mathbb{E}U \geq t) \leq e^{-\lambda t}\mathbb{E}e^{\lambda(U-\mathbb{E}U)} \leq \exp\left\{-\lambda t + \frac{\lambda^2 C}{8}\right\}.$$

The exponent is minimized at $\lambda = 4t/C$, giving $\mathbb{P}(U - \mathbb{E}U \geq t) \leq e^{-2t^2/C}$. The function $-u$ has the same coordinate-change bounds, so applying the upper-tail result to $-U$ proves the lower-tail result. Adding the two tail bounds proves the two-sided inequality. □

In particular, if each coordinate-change bound is $c_i = 1/n$, then $C = 1/n$ and $\mathbb{P}(U - \mathbb{E}U \geq t) \leq e^{-2nt^2}$. This is the form used for Rademacher uniform convergence below.

The same exponential-moment method also controls a maximum of unbounded Gaussian variables.

Corollary 3.6 (Expected maximum of i.i.d. Gaussians)**.** *Let $G_1, \ldots, G_m$ be i.i.d. $N(\mu, \tau^2)$, where $\tau > 0$ and $m \geq 2$. Then*

$$\mathbb{E}\max_{j\leq m} G_j - \mu \leq \tau\sqrt{2\log m}, \qquad \mathbb{E}\max_{j\leq m} |G_j - \mu| \leq \tau\sqrt{2\log(2m)}. \tag{3.3.4}$$

The first bound has the sharp leading order: $\mathbb{E}\max_{j\leq m} G_j - \mu \sim \tau\sqrt{2\log m}$ as $m \to \infty$.

Proof. For $Z_j = (G_j - \mu)/\tau$, the Gaussian MGF is $\mathbb{E}e^{\lambda Z_j} = e^{\lambda^2/2}$. Hence log-sum-exp and Jensen's inequality give

$$\mathbb{E}\max_j Z_j \leq \frac{1}{\lambda}\log\sum_{j=1}^m \mathbb{E}e^{\lambda Z_j} = \frac{\log m}{\lambda} + \frac{\lambda}{2}.$$

Minimize over $\lambda > 0$. Including both signs gives the absolute maximum bound with m replaced by $2m$. For sharpness, let $\Phi(t) = \mathbb{P}(Z_1 \leq t)$. The Gaussian tail lower bound [29, Proposition 2.1.2] implies that, for every $\eta \in (0,1)$ and $t_m = (1-\eta)\sqrt{2\log m}$, $m\{1 - \Phi(t_m)\} \to \infty$. Thus $\mathbb{P}(\max_j Z_j \geq t_m) = 1 - \Phi(t_m)^m \to 1$. The negative part of $\max_j Z_j$ has expectation at most $\mathbb{E}(-Z_1)_+ 2^{-(m-1)}$, so $\liminf_m \mathbb{E}\max_j Z_j/\sqrt{2\log m} \geq 1 - \eta$. Let $\eta \downarrow 0$ and combine with the upper bound. □

Gaussian variables are unbounded, but their exact quadratic MGF lets the same method used in Hoeffding's proof control their maximum. The $\sqrt{\log m}$ scale anticipates the cost of handling multiple candidate predictors in Section 3.7.

3.4 Bernstein's inequality: using the variance

The boundedness condition here is imposed on the centered variable. This distinction prevents the mean from being hidden inside the range parameter.

Theorem 3.7 (Bernstein's inequality)**.** *Let $X_1, \ldots, X_n$ be independent with common mean μ. Suppose $M > 0$ and $\sigma^2 \geq 0$ satisfy*

$$|X_i - \mu| \leq M \quad \text{almost surely}, \qquad \frac{1}{n}\sum_{i=1}^n \mathrm{Var}(X_i) \leq \sigma^2. \tag{3.4.1}$$

Then for every $\varepsilon > 0$,

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^n X_i - \mu\right| \geq \varepsilon\right) \leq 2\exp\left\{-\frac{n\varepsilon^2}{2(\sigma^2 + M\varepsilon/3)}\right\}. \tag{3.4.2}$$

For $\delta \in (0,1)$, let $t = \log(2/\delta)$. Then, with probability at least $1 - \delta$,

$$\left|\frac{1}{n}\sum_{i=1}^n X_i - \mu\right| \leq \frac{Mt}{3n} + \sqrt{\frac{2\sigma^2 t}{n} + \left(\frac{Mt}{3n}\right)^2}. \tag{3.4.3}$$

Proof. Write $Z_i = X_i - \mu$. For $0 \le \lambda < 3/M$, the power series of the exponential, the inequalities $\mathbb{E}[Z_i^k] \le \mathbb{E}|Z_i|^k \le M^{k-2}\mathbb{E}[Z_i^2]$ for $k \ge 2$, and $k! \ge 2 \cdot 3^{k-2}$ imply

$$\begin{aligned}\mathbb{E}e^{\lambda Z_i} &= 1 + \sum_{k=2}^{\infty} \frac{\lambda^k \mathbb{E}[Z_i^k]}{k!} \\ &\le 1 + \frac{\lambda^2 \operatorname{Var}(X_i)}{2(1 - \lambda M/3)} \\ &\le \exp\left\{ \frac{\lambda^2 \operatorname{Var}(X_i)}{2(1 - \lambda M/3)} \right\}.\end{aligned}$$

Markov's inequality and independence therefore give

$$\mathbb{P}\left(\frac{1}{n} \sum_i Z_i \ge \varepsilon \right) \le \exp\left\{ -\lambda n \varepsilon + \frac{n\lambda^2 \sigma^2}{2(1 - \lambda M/3)} \right\}.$$

If $\sigma^2 = 0$, every $X_i = \mu$ almost surely and the result is immediate. Otherwise choose $\lambda = \varepsilon/(\sigma^2 + M\varepsilon/3)$. The exponent becomes $-n\varepsilon^2/[2(\sigma^2 + M\varepsilon/3)]$. Apply the same argument to $-Z_i$ for the lower tail. The confidence radius in (3.4.3) is the nonnegative root of $n\varepsilon^2 = 2t(\sigma^2 + M\varepsilon/3)$; substituting it into (3.4.2) makes the upper bound δ. □

3.5 Comparing the three inequalities

Inequality	Main assumption	Typical two-sided tail for the mean	What it remembers
Chebyshev	$\operatorname{Var}(X) < \infty$	$\sigma^2/(n\varepsilon^2)$	Only the variance; polynomial confidence dependence
Hoeffding	$X \in [a, b]$	$2e^{-2n\varepsilon^2/(b-a)^2}$	Only the range; sub-Gaussian tail
Bernstein	$\lvert X - \mathbb{E}X\rvert \le M$, variance at most σ^2	$2e^{-n\varepsilon^2/\{2(\sigma^2 + M\varepsilon/3)\}}$	Variance for small deviations and range for large deviations

Figure 3.1 puts the three guarantees on the same scale. The curves use $n = 200$, range width $b - a = 1$, variance bound $\sigma^2 = 0.01$, and centered bound $M = 1$; every upper bound is truncated at 1.

Bernstein behaves approximately like $\exp\{-n\varepsilon^2/(2\sigma^2)\}$ when $\varepsilon \ll \sigma^2/M$, and like $\exp\{-3n\varepsilon/(2M)\}$ for large deviations. Hoeffding can be cleaner when only a range is known; Bernstein can be much sharper when the variance is small.

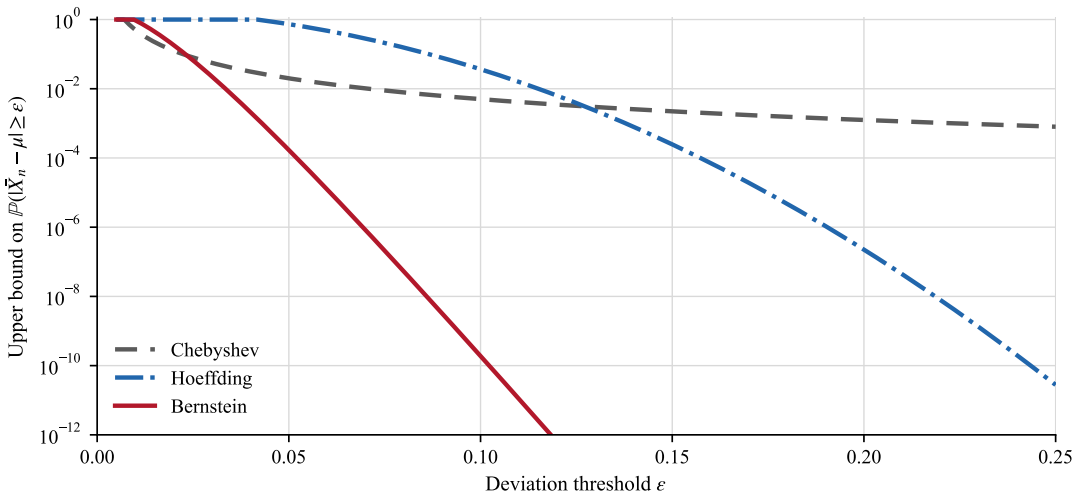


Figure 3.1: Two-sided tail upper bounds for the same empirical-mean problem. The vertical axis is logarithmic. The low variance in this illustration allows Bernstein's inequality to improve substantially on the range-only Hoeffding bound, while Chebyshev decays only polynomially. These curves compare guarantees, not the exact tail probability of a specified distribution.

Rosenthal's inequality for even moments. For independent, zero-mean random variables $Y_1, \dots, Y_n$ with finite $2m$-th moments, where $m \geq 1$ is an integer, there is a constant R_m depending only on m such that

$$\mathbb{E}\left|\sum_{i=1}^{n} Y_i\right|^{2m} \leq R_m \left\{ \sum_{i=1}^{n} \mathbb{E}|Y_i|^{2m} + \left(\sum_{i=1}^{n} \mathbb{E}Y_i^2 \right)^m \right\}. \tag{3.5.1}$$

Proof of the even-moment inequality. For $m = 1$, independence and centering give $\mathbb{E}(\sum_i Y_i)^2 = \sum_i \mathbb{E}Y_i^2$, so the claim holds with $R_1 = 1$. Assume $m \geq 2$, and write

$$A := \sum_i \mathbb{E}Y_i^2, \qquad B := \sum_i \mathbb{E}|Y_i|^{2m}, \qquad M_r := \sum_i \mathbb{E}|Y_i|^r.$$

For $2 \leq r \leq 2m$, set $\theta_r = (r-2)/(2m-2)$. Hölder's inequality, first within each expectation and then over the index i, gives

$$M_r \leq \sum_i (\mathbb{E}Y_i^2)^{1-\theta_r} (\mathbb{E}|Y_i|^{2m})^{\theta_r} \leq A^{1-\theta_r} B^{\theta_r}. \tag{3.5.2}$$

Expand $\mathbb{E}(\sum_i Y_i)^{2m}$ and group the terms by which of the $2m$ positions carry the same index. Independence and $\mathbb{E}Y_i = 0$ make every term with an index occurring just once vanish. Let $\mathcal{P}_{2m}$ be the set of partitions of the $2m$ positions whose blocks all have size at least two. A partition π with blocks $D_1, \dots, D_k$ has $k \leq m$. Independence bounds its contribution, after taking absolute values and dropping the requirement that distinct blocks carry distinct indices, by

$$\sum_{i_1, \dots, i_k \text{ distinct}} \prod_{j=1}^{k} \mathbb{E}|Y_{i_j}|^{|D_j|} \leq \prod_{j=1}^{k} M_{|D_j|}.$$

Since $\sum_j |D_j| = 2m$, we have $\sum_j \theta_{|D_j|} = (m-k)/(m-1)$. Thus (3.5.2) and the weighted arithmetic–geometric mean inequality yield

$$\prod_{j=1}^{k} M_{|D_j|} \leq (A^m)^{(k-1)/(m-1)} B^{(m-k)/(m-1)} \leq A^m + B.$$

There are only $R_m := |\mathcal{P}_{2m}|$ such partitions, a number depending on m alone. Summing their contributions proves (3.5.1). □

A finite even moment gives a polynomial tail bound for the sample mean. The following result is adapted from [30, Exercise 2.20].

Theorem 3.10 (Tail bound under a finite even moment)**.** *Let $m \geq 1$ be a fixed integer, and let $X_1, \dots, X_n$ be independent real random variables with $\mathbb{E}X_i = 0$ and*

$$\|X_i\|_{2m} := \big(\mathbb{E}|X_i|^{2m}\big)^{1/(2m)} \leq C_m \qquad \text{for every } i,$$

where $C_m > 0$. Write $\overline{X}_n = n^{-1}\sum_{i=1}^n X_i$, and set $B_m := 2R_m C_m^{2m}$, using the constant from (3.5.1). Then, for every $\varepsilon > 0$,

$$\mathbb{P}\big(|\overline{X}_n| \geq \varepsilon\big) \leq \frac{B_m}{n^m \varepsilon^{2m}} = B_m \left(\frac{1}{\sqrt{n}\,\varepsilon}\right)^{2m}. \tag{3.5.3}$$

Equivalently, for any $\delta \in (0,1)$, with probability at least $1-\delta$,

$$|\overline{X}_n| \leq \frac{B_m^{1/(2m)}}{\sqrt{n}\,\delta^{1/(2m)}} = \frac{C_m}{\sqrt{n}} \left(\frac{2R_m}{\delta}\right)^{1/(2m)}. \tag{3.5.4}$$

Proof. Put $S_n = \sum_{i=1}^n X_i$. The assumed moment bound gives $\mathbb{E}|X_i|^{2m} \leq C_m^{2m}$, while monotonicity of L^p norms gives

$$\mathbb{E}X_i^2 \leq \big(\mathbb{E}|X_i|^{2m}\big)^{1/m} \leq C_m^2.$$

Substituting both bounds into the proved Rosenthal inequality (3.5.1), and using $n \leq n^m$ for $n, m \geq 1$, we obtain

$$\mathbb{E}|S_n|^{2m} \leq R_m\big(nC_m^{2m} + n^m C_m^{2m}\big) \leq 2R_m C_m^{2m} n^m = B_m n^m.$$

Finally, Markov's inequality applied to the nonnegative variable $|S_n|^{2m}$ yields

$$\mathbb{P}\big(|\overline{X}_n| \geq \varepsilon\big) = \mathbb{P}\big(|S_n|^{2m} \geq n^{2m}\varepsilon^{2m}\big) \leq \frac{\mathbb{E}|S_n|^{2m}}{n^{2m}\varepsilon^{2m}} \leq \frac{B_m}{n^m \varepsilon^{2m}}.$$

Substituting $\varepsilon = B_m^{1/(2m)}/(\sqrt{n}\,\delta^{1/(2m)})$ gives (3.5.4). □

For $m = 1$, independence gives the sharper identity $\mathbb{E}S_n^2 = \sum_i \mathbb{E}X_i^2 \leq nC_1^2$. Thus the tail bound sharpens to $\mathbb{P}(|\overline{X}_n| \geq \varepsilon) \leq C_1^2/(n\varepsilon^2)$, recovering Chebyshev's inequality. A fixed higher even moment improves the tail bound at fixed ε to order n^{-m}; at fixed confidence level, the error radius retains the $n^{-1/2}$ scale and depends on δ through $\delta^{-1/(2m)}$.

3.6 Sub-Gaussian and sub-exponential random variables

Hoeffding and Bernstein reflect two common tail classes. Let $Z := X - \mathbb{E}X$. Centering matters: the definitions describe fluctuations around the mean, not the size of the mean itself.

Definitions. The variable X is *sub-Gaussian* with variance proxy $\sigma^2 > 0$ if

$$\log \mathbb{E}e^{\lambda Z} \leq \frac{\sigma^2\lambda^2}{2} \qquad \text{for every } \lambda \in \mathbb{R}. \tag{3.6.1}$$

It is *sub-exponential* with parameters (ν^2, b), where $\nu > 0$ and $b > 0$, if

$$\log \mathbb{E}e^{\lambda Z} \leq \frac{\nu^2\lambda^2}{2} \qquad \text{whenever } |\lambda| < \frac{1}{b}. \tag{3.6.2}$$

The quantities σ^2 and ν^2 are variance proxies; they need not equal $\mathrm{Var}(X)$. Constants and parameter names vary across texts, but the distinction between global and local moment-generating-function control is invariant. Here *sub-exponential* means exponential-tail, or ψ_1, control. In parts of heavy-tail theory the same name denotes a different convolution-tail class that includes Pareto distributions; that is not the meaning used here.

For $t \geq 0$, Chernoff's method converts these conditions into

$$\mathbb{P}(|Z| \geq t) \leq 2\exp\left\{-\frac{t^2}{2\sigma^2}\right\} \qquad \text{for a sub-Gaussian variable,} \tag{3.6.3}$$

$$\mathbb{P}(|Z| \geq t) \leq 2\exp\left\{-\frac{1}{2}\min\left(\frac{t^2}{\nu^2}, \frac{t}{b}\right)\right\} \qquad \text{for a sub-exponential variable.} \tag{3.6.4}$$

Thus sub-exponential tails look Gaussian for moderate deviations $t \lesssim \nu^2/b$, while the generic bound becomes exponential for larger deviations. A sub-Gaussian variable retains the quadratic exponent at every scale.

Equivalent ways to recognize the two classes. For a random variable Z, define the Orlicz norms

$$\|Z\|_{\psi_2} := \inf\{K > 0 : \mathbb{E}e^{Z^2/K^2} \leq 2\}, \qquad \|Z\|_{\psi_1} := \inf\{K > 0 : \mathbb{E}e^{|Z|/K} \leq 2\}. \tag{3.6.5}$$

For centered Z, $\|Z\|_{\psi_2} < \infty$ is equivalent, up to universal changes of constants, to the global MGF bound (3.6.1); $\|Z\|_{\psi_1} < \infty$ is similarly equivalent to the local MGF bound (3.6.2) for some finite parameters. The norm definitions directly imply

$$\mathbb{P}(|Z| \geq t) \leq 2e^{-t^2/\|Z\|_{\psi_2}^2}, \qquad (\mathbb{E}|Z|^p)^{1/p} \leq C\|Z\|_{\psi_2}\sqrt{p}, \tag{3.6.6}$$

$$\mathbb{P}(|Z| \geq t) \leq 2e^{-t/\|Z\|_{\psi_1}}, \qquad (\mathbb{E}|Z|^p)^{1/p} \leq C\|Z\|_{\psi_1}p, \tag{3.6.7}$$

for $t \geq 0$ and $p \geq 1$, with an absolute constant C. Conversely, either tail or moment-growth property, with a suitable finite scale, implies the corresponding Orlicz and centered MGF bounds. For example, the moment estimates follow by integrating $\mathbb{E}|Z|^p = p\int_0^\infty t^{p-1}\mathbb{P}(|Z| > t)\,dt$. These equivalences are developed in [29, Propositions 2.5.2 and 2.7.1].

Property	Sub-Gaussian	Sub-exponential
MGF control	Quadratic for every $\lambda \in \mathbb{R}$	Quadratic only for $\lvert\lambda\rvert < 1/b$
Large-deviation decay	$\exp(-ct^2)$	$\exp(-ct)$
Moment growth	$(\mathbb{E}\lvert Z\rvert^p)^{1/p} = O(\sigma\sqrt{p})$	$(\mathbb{E}\lvert Z\rvert^p)^{1/p} = O(\nu\sqrt{p} + bp)$
Typical examples	Gaussian, Rademacher, and centered bounded variables	Centered exponential and Laplace variables; centered products and squares of sub-Gaussians

Every sub-Gaussian variable is sub-exponential because a global MGF bound also holds on any smaller interval around zero. The converse is false. If $E \sim \mathrm{Exp}(1)$, then $E-1$ is sub-exponential but not sub-Gaussian: its MGF diverges at $\lambda = 1$. Another useful closure fact is that, for sub-Gaussian U and V, even without independence,

$$\|U^2\|_{\psi_1} = \|U\|_{\psi_2}^2, \qquad \|UV\|_{\psi_1} \leq \|U\|_{\psi_2}\|V\|_{\psi_2}. \tag{3.6.8}$$

The second inequality follows from $2|UV|/(K_U K_V) \leq U^2/K_U^2 + V^2/K_V^2$ and Cauchy–Schwarz for the exponential moment. Centering changes the ψ_1 norm by at most an absolute factor, so $UV - \mathbb{E}[UV]$ and $U^2 - \mathbb{E}[U^2]$ are sub-exponential. This is why covariance entries, squared residuals, and squared-gradient terms naturally lead to Bernstein-type control [29, Lemmas 2.7.6–2.7.7 and Exercise 2.7.10].

Sums and empirical means. If independent centered Z_i have sub-Gaussian proxies σ_i^2, MGFs multiply, and any weighted sum satisfies

$$\mathbb{P}\left(\left|\sum_{i=1}^n a_i Z_i\right| \geq t\right) \leq 2\exp\left\{-\frac{t^2}{2\sum_{i=1}^n a_i^2\sigma_i^2}\right\}. \tag{3.6.9}$$

For independent centered sub-exponential Z_i with parameters (ν_i^2, b_i), the weighted sum remains sub-exponential, with

$$\nu_{\mathrm{sum}}^2 = \sum_i a_i^2\nu_i^2, \qquad b_{\mathrm{sum}} = \max_i |a_i|b_i.$$

The quadratic MGF bound holds wherever all individual MGFs are controlled.

For i.i.d. sub-exponential observations with common mean μ and common parameters (ν^2, b), independence gives, for every $\varepsilon > 0$,

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i - \mu\right| \geq \varepsilon\right) \leq 2\exp\left\{-\frac{n}{2}\min\left(\frac{\varepsilon^2}{\nu^2}, \frac{\varepsilon}{b}\right)\right\}. \tag{3.6.10}$$

The corresponding sub-Gaussian mean bound keeps only the quadratic term, $2\exp\{-n\varepsilon^2/(2\sigma^2)\}$, for every deviation size. This one-regime versus two-regime distinction is the conceptual difference between sub-Gaussian and sub-exponential concentration [3]. Equivalently, with probability at least $1-\delta$,

$$|\overline{X}_n - \mu| \leq \nu\sqrt{\frac{2\log(2/\delta)}{n}} + \frac{2b\log(2/\delta)}{n} \tag{3.6.11}$$

for the sub-exponential case; in the sub-Gaussian case the second term disappears and ν is replaced by σ.

Expected maxima reveal the tail speed. For $m \geq 2$ centered variables Z_j with $\|Z_j\|_{\psi_2} \leq K_2$, a union bound and tail integration give

$$\mathbb{E}\max_{j\leq m}|Z_j| \leq CK_2\sqrt{\log m}. \tag{3.6.12}$$

The same argument under $\|Z_j\|_{\psi_1} \leq K_1$ gives $\mathbb{E}\max_{j\leq m}|Z_j| \leq CK_1\log m$. Independence is unnecessary for these upper bounds. Independent standard Gaussians attain the $\sqrt{\log m}$ order, so it cannot generally be improved [29, Exercises 2.5.10–2.5.11]. For i.i.d. Exp(1) variables, the expected maximum is $\sum_{k=1}^{m} 1/k \sim \log m$, so the sub-exponential order is sharp as well. For i.i.d. variables with common mean μ, these bounds also control $\mathbb{E}\max_{j\leq m} X_j$ by adding μ to the corresponding centered bound. For comparison, i.i.d. Pareto variables with $\mathbb{P}(X > t) = (x_0/t)^\alpha$ satisfy $\mathbb{P}(\max_{j\leq m} X_j \leq t) = [1-(x_0/t)^\alpha]^m$ for $t \geq x_0$. Tail integration gives $\mathbb{E}\max_{j\leq m} X_j \asymp x_0 m^{1/\alpha}$ when $\alpha > 1$. Thus Gaussian, exponential, and power-law tails lead to different costs when taking a maximum over many coordinates or candidate models.

3.7 From one predictor to a selected predictor

Corollary 3.11 (Hoeffding for a fixed predictor)**.** *If f is fixed independently of the sample and $0 \leq \ell(f(X), Y) \leq 1$, then*

$$\mathbb{P}\big(|\widehat{L}_n(f) - L(f)| \geq \varepsilon\big) \leq 2e^{-2n\varepsilon^2}. \tag{3.7.1}$$

The independence phrase is essential. An empirical risk minimizer is selected using the sample, so the random summands and the selected function are coupled. For a finite class, a union bound constructs one event on which every candidate is controlled simultaneously.

Theorem 3.12 (Finite-class uniform convergence)**.** *Let $\mathcal{F}$ be a deterministic finite class with $|\mathcal{F}| = N$, and suppose $0 \leq \ell(f, Z) \leq 1$ for all $f \in \mathcal{F}$. For every $\delta \in (0,1)$, with probability at least $1-\delta$,*

$$\Delta_n(\mathcal{F}) = \sup_{f\in\mathcal{F}}|\widehat{L}_n(f) - L(f)| \leq \sqrt{\frac{\log(2N/\delta)}{2n}}. \tag{3.7.2}$$

Proof. For any $\varepsilon > 0$, Corollary 3.11 and the union bound imply

$$\mathbb{P}\left(\sup_{f\in\mathcal{F}}|\widehat{L}_n(f) - L(f)| \geq \varepsilon\right) \leq \sum_{f\in\mathcal{F}}\mathbb{P}\big(|\widehat{L}_n(f) - L(f)| \geq \varepsilon\big) \leq 2Ne^{-2n\varepsilon^2}.$$

Set the right-hand side equal to δ and solve for ε. □

The theorem supplies the statistical term required by the master bound (1.3.7). Its $\log N$ dependence measures the effective cost of selecting among N candidates.

Scope: uniform convergence for an infinite class

For an infinite $\mathcal{F}$, cardinality no longer measures how many functions the sample can distinguish. Introduce the induced loss class

$$\mathcal{G} := \{z = (x, y) \mapsto \ell(f(x), y) : f \in \mathcal{F}\}. \tag{3.7.3}$$

The target remains $\Delta_n(\mathcal{F})$, but $\log|\mathcal{F}|$ is replaced by a notion of the effective size of $\mathcal{G}$.

The three criteria used in this lecture emphasize different structure:

Criterion	Typical setting	What it measures
VC dimension	Binary classifiers	The largest sample on which every binary labeling can be realized
Covering numbers and metric entropy	A class equipped with a metric	The number of distinguishable functions at resolution r
Rademacher complexity	General loss classes	How strongly the class can correlate with random noise, often measured on the observed inputs

Covering numbers extend the finite-class union bound by discretizing at a chosen resolution. VC dimension controls the number of binary prediction patterns a class can realize. Rademacher complexity uses symmetrization and random signs to measure the richness of the loss class on the observed sample. The next three sections make these routes concrete.

3.8 Covering numbers: a finite approximation

Definition 3.13 (Covering number)**.** Let $(\mathcal{G}, d)$ be a metric space. An r-cover is a finite set $\mathcal{G}_r \subseteq \mathcal{G}$ such that every $g \in \mathcal{G}$ has a representative $\pi_r(g) \in \mathcal{G}_r$ with $d(g, \pi_r(g)) \le r$. The smallest possible cardinality is the covering number

$$N(r, \mathcal{G}, d) := \min\{|\mathcal{G}_r| : \mathcal{G}_r \text{ is an } r\text{-cover of } \mathcal{G}\}. \tag{3.8.1}$$

Its logarithm is called the metric entropy.

Write

$$P_n g := \frac{1}{n}\sum_{i=1}^{n} g(Z_i), \qquad Pg := \mathbb{E}[g(Z)].$$

The following proposition shows explicitly how a finite cover enters a uniform-convergence bound.

Proposition 3.14 (Uniform convergence from a sup-norm cover)**.** *Let $\mathcal{G} \subseteq [0,1]^{\mathcal{Z}}$, and suppose $\mathcal{G}_r$ is a finite r-cover under $\|g - h\|_\infty := \sup_z |g(z) - h(z)|$. With probability at least $1 - \delta$,*

$$\sup_{g \in \mathcal{G}} |P_n g - Pg| \le 2r + \sqrt{\frac{\log\{2|\mathcal{G}_r|/\delta\}}{2n}}. \tag{3.8.2}$$

Proof. For $g \in \mathcal{G}$, let $h = \pi_r(g)$. Then

$$|P_n g - Pg| \le |P_n(g - h)| + |P_n h - Ph| + |P(h - g)| \le 2r + |P_n h - Ph|.$$

Apply Hoeffding's inequality and a union bound to the finite set $\mathcal{G}_r$. □

Equation (3.8.2) exposes a bias-complexity trade-off: a smaller r reduces the approximation term but enlarges the cover. For example, suppose Θ lies in a Euclidean ball of radius B in $\mathbb{R}^d$ and $\|g_\theta - g_{\theta'}\|_\infty \le L\|\theta - \theta'\|_2$. A volumetric parameter-space cover gives

$$N(r, \{g_\theta : \theta \in \Theta\}, \|\cdot\|_\infty) \le \left(1 + \frac{2LB}{r}\right)^d. \tag{3.8.3}$$

Substitution yields a rate of order $\sqrt{d \log n / n}$, up to the choice of r. More refined chaining arguments combine covers at many resolutions rather than committing to one.

Example: the operator norm of a sub-Gaussian matrix. Let $A = (A_{ij}) \in \mathbb{R}^{m\times d}$ have independent centered entries, each satisfying the sub-Gaussian MGF bound (3.6.1) with the same proxy σ^2. The operator norm, or largest singular value, is

$$\|A\|_{\mathrm{op}} = \max_{x\in S^{d-1}} \|Ax\|_2 = \max_{\substack{x\in S^{d-1}\\ y\in S^{m-1}}} y^\top Ax, \qquad S^{k-1} := \{v \in \mathbb{R}^k : \|v\|_2 = 1\}.$$

Choose 1/4-nets $\mathcal{N}_d \subset S^{d-1}$ and $\mathcal{N}_m \subset S^{m-1}$. A volume comparison gives $|\mathcal{N}_d| \le 9^d$ and $|\mathcal{N}_m| \le 9^m$: the 1/8-balls around a maximal 1/4-separated set are disjoint and fit inside the ball of radius 9/8. For any unit x, y, choose net points x_0, y_0 within 1/4. Then

$$y^\top Ax \le y_0^\top Ax_0 + \big(\|x - x_0\|_2 + \|y - y_0\|_2\big)\,\|A\|_{\mathrm{op}}\,, \qquad \|A\|_{\mathrm{op}} \le 2 \max_{\substack{x\in\mathcal{N}_d\\ y\in\mathcal{N}_m}} y^\top Ax.$$

For fixed net points, independence and the sub-Gaussian MGF bound give

$$\log \mathbb{E}e^{\lambda y^\top Ax} \le \frac{\sigma^2\lambda^2}{2} \sum_{i=1}^m \sum_{j=1}^d y_i^2 x_j^2 = \frac{\sigma^2\lambda^2}{2}.$$

Thus $\mathbb{P}(y^\top Ax \ge s) \le e^{-s^2/(2\sigma^2)}$ for a fixed pair. A union bound over the two nets yields

$$\mathbb{P}\big(\|A\|_{\mathrm{op}} \ge 2s\big) \le 9^{m+d} e^{-s^2/(2\sigma^2)}.$$

Solving for s, we find that for any $\delta \in (0,1)$, with probability at least $1-\delta$,

$$\|A\|_{\mathrm{op}} \le 2\sigma\sqrt{2\big[(m+d)\log 9 + \log(1/\delta)\big]}. \tag{3.8.4}$$

For example, if $A_{ij} \sim \mathcal{N}(0, 1/m)$, the upper bound has order $1 + \sqrt{d/m} + \sqrt{\log(1/\delta)/m}$. The dimension term comes from the logarithm of the number of net pairs, as in [29, Section 4.4].

3.9 VC dimension for binary classification

Let $\mathcal{H} \subseteq \{0,1\}^{\mathcal{X}}$. Its growth function is

$$\Pi_{\mathcal{H}}(m) := \sup_{x_1,\dots,x_m\in\mathcal{X}} \big|\{(h(x_1),\dots,h(x_m)) : h \in \mathcal{H}\}\big|\,. \tag{3.9.1}$$

Definition 3.15 (VC dimension)**.** The class $\mathcal{H}$ shatters $x_1, \dots, x_m$ if it realizes all 2^m binary labelings on those points. Its VC dimension $d_{\mathrm{VC}}(\mathcal{H})$ is the largest shattered sample size, or infinity if samples of every finite size can be shattered.

To prove an exact VC dimension, establish two inequalities. For the lower bound, exhibit a set that is shattered. For the upper bound, show that every larger set misses at least one labeling. The following examples illustrate this pattern.

Example 3.16 (Thresholds on the line)**:** Consider

$$\mathcal{H}_{\mathrm{thr}} := \{h_a(x) = \mathbf{1}\{x \ge a\} : a \in \mathbb{R}\}.$$

Then $d_{\mathrm{VC}}(\mathcal{H}_{\mathrm{thr}}) = 1$.

Proof. Lower bound. One point x_1 can be labeled either way: choose $a = x_1$ for label 1, and choose $a > x_1$ for label 0. Thus one point is shattered.

Upper bound. Take two distinct points and order them as $x_1 < x_2$. Every threshold is monotone, so the labeling

$$(h_a(x_1), h_a(x_2)) = (1, 0)$$

is impossible. Hence no two-point set is shattered. □

Example 3.17 (Intervals on the line)**:** Let

$$\mathcal{H}_{\text{int}} := \{x \mapsto \mathbf{1}\{a \le x \le b\} : a, b \in \mathbb{R},\ a \le b\}.$$

Then $d_{\text{VC}}(\mathcal{H}_{\text{int}}) = 2$.

Proof. Lower bound. Fix $x_1 < x_2$. An interval disjoint from both points realizes $(0, 0)$, the singleton intervals $[x_1, x_1]$ and $[x_2, x_2]$ realize $(1, 0)$ and $(0, 1)$, and $[x_1, x_2]$ realizes $(1, 1)$. Thus the two points are shattered.

Upper bound. For any three distinct points $x_1 < x_2 < x_3$, the labeling $(1, 0, 1)$ is impossible: every interval containing both endpoints also contains the middle point. Therefore no three-point set is shattered. □

Example 3.18 (Affine halfspaces in $\mathbb{R}^d$)**:** For $d \ge 1$, define

$$\mathcal{H}_{\text{hs}}^{(d)} := \{x \mapsto \mathbf{1}\{w^\top x + b \ge 0\} : w \in \mathbb{R}^d,\ b \in \mathbb{R}\}.$$

Then $d_{\text{VC}}(\mathcal{H}_{\text{hs}}^{(d)}) = d + 1$.

Proof. Lower bound. Consider the $d + 1$ points

$$x_0 = 0, \qquad x_i = e_i, \quad i = 1, \dots, d,$$

where e_i is the i-th coordinate vector. Given arbitrary labels $y_0, \dots, y_d \in \{0, 1\}$, set $s_i = 2y_i - 1$, $b = s_0$, and $w_i = s_i - s_0$. Then

$$w^\top x_0 + b = s_0, \qquad w^\top x_i + b = s_i \quad (i = 1, \dots, d).$$

The signs therefore realize the prescribed labels, so these $d + 1$ points are shattered.

Upper bound. Take any $d+2$ points $x_1, \dots, x_{d+2} \in \mathbb{R}^d$. They are affinely dependent, so there are coefficients α_i, not all zero, such that

$$\sum_{i=1}^{d+2} \alpha_i x_i = 0, \qquad \sum_{i=1}^{d+2} \alpha_i = 0.$$

Label x_i by 1 when $\alpha_i > 0$, by 0 when $\alpha_i < 0$, and arbitrarily when $\alpha_i = 0$. Suppose an affine score $f(x) = w^\top x + b$ realized this labeling. Then $\alpha_i f(x_i) \ge 0$ whenever $\alpha_i > 0$, while $\alpha_i f(x_i) > 0$ whenever $\alpha_i < 0$. Both signs occur among the nonzero α_i, because their sum is zero. Consequently,

$$\sum_{i=1}^{d+2} \alpha_i f(x_i) > 0.$$

On the other hand, affine dependence gives

$$\sum_{i=1}^{d+2} \alpha_i f(x_i) = w^\top \sum_{i=1}^{d+2} \alpha_i x_i + b \sum_{i=1}^{d+2} \alpha_i = 0,$$

a contradiction. Thus no $d + 2$ points can be shattered. □

The definition is combinatorial, but its statistical role comes from bounding the number of distinct prediction vectors on a sample.

Lemma 3.19 (Sauer–Shelah)**.** *If $d_{\text{VC}}(\mathcal{H}) = d < \infty$, then*

$$\Pi_{\mathcal{H}}(m) \le \sum_{j=0}^{d} \binom{m}{j}. \tag{3.9.2}$$

For $m \ge d \ge 1$, the right-hand side is at most $(em/d)^d$.

Proof. Fix inputs $x_1, \dots, x_m$, let S be the set of distinct inputs, and restrict the original class to this finite set:

$$\mathcal{H}_S := \mathcal{H}|_S = \{h|_S : h \in \mathcal{H}\} \subseteq \{0,1\}^S.$$

Here $h|_S$ records only the values of h on S. Functions that agree on S give the same element of $\mathcal{H}_S$, so $|\mathcal{H}_S|$ is the number of distinct labelings of the chosen inputs, even if $\mathcal{H}$ is infinite. Moreover, for every $T \subseteq S$, $\mathcal{H}_S$ shatters T if and only if $\mathcal{H}$ does.

Pajor's counting lemma [29, Lemma 8.3.13] states that

$$|\mathcal{H}_S| \le \big|\{T \subseteq S : T \text{ is shattered by } \mathcal{H}_S\}\big|.$$

We prove this inequality for every class of Boolean functions on a finite set by induction on $|S|$. The case $S = \varnothing$ is immediate. For $x_0 \in S$, set $S_0 = S \setminus \{x_0\}$ and

$$\mathcal{H}_{S,b} := \{g|_{S_0} : g \in \mathcal{H}_S,\ g(x_0) = b\}, \qquad b \in \{0,1\}.$$

Thus $\mathcal{H}_{S,b}$ consists of labelings on S_0 that can be extended to S with label b at x_0. Once b is fixed, restriction to S_0 loses no information, so $|\mathcal{H}_S| = |\mathcal{H}_{S,0}| + |\mathcal{H}_{S,1}|$. Let $\mathcal{C}_b$ be the subsets of S_0 shattered by $\mathcal{H}_{S,b}$. By induction, $|\mathcal{H}_{S,b}| \le |\mathcal{C}_b|$. Every $T \in \mathcal{C}_0 \cup \mathcal{C}_1$ is shattered by $\mathcal{H}_S$. If $T \in \mathcal{C}_0 \cap \mathcal{C}_1$, then $T \cup \{x_0\}$ is also shattered: each labeling of T has an extension with $g(x_0) = 0$ and one with $g(x_0) = 1$. These two groups of shattered subsets are disjoint, since only the second contains x_0. Thus

$$\big|\{T \subseteq S : T \text{ is shattered by } \mathcal{H}_S\}\big| \ge |\mathcal{C}_0 \cup \mathcal{C}_1| + |\mathcal{C}_0 \cap \mathcal{C}_1| = |\mathcal{C}_0| + |\mathcal{C}_1| \ge |\mathcal{H}_{S,0}| + |\mathcal{H}_{S,1}| = |\mathcal{H}_S|.$$

This proves the counting lemma. Since $\mathcal{H}_S$ and $\mathcal{H}$ shatter the same subsets of S, each such subset has at most d points. Consequently,

$$|\mathcal{H}_S| \le \sum_{j=0}^{d} \binom{|S|}{j} \le \sum_{j=0}^{d} \binom{m}{j}.$$

Taking the supremum over the chosen inputs proves (3.9.2).

Finally, suppose $m \ge d \ge 1$ and set $t = d/m \in (0,1]$. Since $t^d \le t^j$ for $0 \le j \le d$, the binomial theorem and $1 + t \le e^t$ give

$$t^d \sum_{j=0}^{d} \binom{m}{j} \le \sum_{j=0}^{d} \binom{m}{j} t^j \le (1+t)^m \le e^{mt} = e^d.$$

Dividing by $t^d = (d/m)^d$ yields $\sum_{j=0}^{d} \binom{m}{j} \le (em/d)^d$. □

Thus an infinite class with finite VC dimension produces only polynomially many distinct labelings on m points, rather than all 2^m labelings. Combining this fact with symmetrization and Hoeffding's inequality yields the classical VC bound.

Theorem 3.20 (VC uniform convergence)**.** *Let $S = ((X_1, Y_1), \dots, (X_n, Y_n)) \sim P^n$, let $\varnothing \ne \mathcal{H} \subseteq \{0,1\}^{\mathcal{X}}$ be a class of measurable classifiers, and use zero-one loss. Assume the suprema below are measurable, as they are for a countable class. For every $\varepsilon > 0$,*

$$\mathbb{P}\left(\sup_{h \in \mathcal{H}} |\widehat{L}_n(h) - L(h)| > \varepsilon\right) \le 4\Pi_{\mathcal{H}}(2n) e^{-n\varepsilon^2/8}. \tag{3.9.3}$$

Consequently, if $d_{\mathrm{VC}}(\mathcal{H}) = d \le n$, then for every $\delta \in (0,1)$, with probability at least $1 - \delta$,

$$\sup_{h \in \mathcal{H}} |\widehat{L}_n(h) - L(h)| \le \sqrt{\frac{8}{n}\left\{d \log \frac{2en}{d} + \log \frac{4}{\delta}\right\}}. \tag{3.9.4}$$

When $d = 0$, interpret $d \log(2en/d)$ as zero.

Proof. Write $Z_i = (X_i, Y_i)$ and $\ell_h(x, y) = \mathbf{1}\{h(x) \neq y\}$, so that

$$L(h) = \mathbb{E}\ell_h(Z), \qquad \widehat{L}_n(h) = \frac{1}{n}\sum_{i=1}^{n} \ell_h(Z_i).$$

Let $S' = (Z'_1, \ldots, Z'_n) \sim P^n$ be independent of S, and write $\widehat{L}'_n(h) = n^{-1}\sum_{i=1}^n \ell_h(Z'_i)$. This independent copy is called a *ghost sample*.

Step 1: replace the population risk by the ghost-sample risk. First suppose $n\varepsilon^2 \geq 2$. Let

$$A := \left\{ \sup_{h\in\mathcal{H}} |\widehat{L}_n(h) - L(h)| > \varepsilon \right\}.$$

For each fixed realization $S \in A$, choose a classifier h_S with $|\widehat{L}_n(h_S) - L(h_S)| > \varepsilon$. Once S is fixed, h_S is fixed relative to the independent sample S'. Its ghost-sample losses are Bernoulli variables, so

$$\mathbb{E}_{S'}\widehat{L}'_n(h_S) = L(h_S), \qquad \mathrm{Var}_{S'}\big(\widehat{L}'_n(h_S)\big) \leq \frac{1}{4n}.$$

Chebyshev's inequality therefore gives

$$\mathbb{P}_{S'}\left(|\widehat{L}'_n(h_S) - L(h_S)| \leq \frac{\varepsilon}{2}\right) \geq 1 - \frac{1}{n\varepsilon^2} \geq \frac{1}{2}.$$

On this event, the triangle inequality implies

$$|\widehat{L}_n(h_S) - \widehat{L}'_n(h_S)| \geq |\widehat{L}_n(h_S) - L(h_S)| - |\widehat{L}'_n(h_S) - L(h_S)| > \frac{\varepsilon}{2}.$$

Define the two-sample event

$$B := \left\{ \sup_{h\in\mathcal{H}} |\widehat{L}_n(h) - \widehat{L}'_n(h)| > \frac{\varepsilon}{2} \right\}.$$

The event A depends only on S, whereas B depends on both S and S'. The preceding argument shows that, for every fixed $S \in A$, drawing the independent sample S' makes B occur with probability at least $1/2$. For $S \notin A$, this conditional probability is still nonnegative. Thus, for every realization of S,

$$\mathbb{P}_{S'}(B \mid S) \geq \frac{1}{2}\mathbf{1}\{S \in A\}.$$

Here $\mathbb{P}_{S'}(B \mid S)$ holds S fixed and takes probability only over the ghost sample S'. The law of total probability gives

$$\begin{aligned}\mathbb{P}_{S,S'}(B) &= \mathbb{E}_S\big[\mathbb{P}_{S'}(B \mid S)\big] \\ &\geq \frac{1}{2}\mathbb{E}_S\big[\mathbf{1}\{S \in A\}\big] = \frac{1}{2}\mathbb{P}_S(A).\end{aligned}$$

In words, the original samples in A contribute at least half of their total probability to B. Rearranging gives

$$\mathbb{P}_S(A) \leq 2\mathbb{P}_{S,S'}(B).$$

Step 2: randomize the difference by swapping sample pairs. Let $\xi_1, \ldots, \xi_n$ be independent signs, each equally likely to be 1 or -1, and independent of S, S'. Swap Z_i with Z'_i whenever $\xi_i = -1$. Each pair consists of two independent observations with law P, so these swaps preserve the joint distribution of the samples. They multiply the i-th loss difference by ξ_i. Hence

$$\begin{aligned}&\mathbb{P}_{S,S'}\left(\sup_{h\in\mathcal{H}} |\widehat{L}_n(h) - \widehat{L}'_n(h)| > \frac{\varepsilon}{2}\right) \\ &\quad = \mathbb{E}_{S,S'}\left[\mathbb{P}_\xi\left(\sup_{h\in\mathcal{H}}\left|\frac{1}{n}\sum_{i=1}^n \xi_i\big(\ell_h(Z_i) - \ell_h(Z'_i)\big)\right| > \frac{\varepsilon}{2} \,\middle|\, S, S'\right)\right].\end{aligned}$$

Step 3: count prediction patterns and apply Hoeffding. Condition on S, S', including their labels. The $2n$ loss values of a classifier are determined by its predictions on $X_1, \dots, X_n, X'_1, \dots, X'_n$. There are at most $\Pi_{\mathcal{H}}(2n)$ distinct prediction vectors, and therefore at most that many loss-difference vectors. For any one such vector, set $a_i = \ell_h(Z_i) - \ell_h(Z'_i) \in \{-1, 0, 1\}$. Conditional on the samples, the variables $\xi_i a_i$ are independent, have mean zero, and lie in $[-1, 1]$. Applying Theorem 3.4 with interval length 2 gives

$$\mathbb{P}_\xi\left(\left|\frac{1}{n}\sum_{i=1}^{n}\xi_i a_i\right| > \frac{\varepsilon}{2}\,\middle|\, S, S'\right) \le 2\exp\left\{-\frac{2n(\varepsilon/2)^2}{2^2}\right\} = 2e^{-n\varepsilon^2/8}.$$

A union bound over the distinct vectors gives

$$\mathbb{P}_\xi\left(\sup_{h\in\mathcal{H}}\left|\frac{1}{n}\sum_{i=1}^{n}\xi_i\big(\ell_h(Z_i) - \ell_h(Z'_i)\big)\right| > \frac{\varepsilon}{2}\,\middle|\, S, S'\right) \le 2\Pi_{\mathcal{H}}(2n)e^{-n\varepsilon^2/8}.$$

This bound holds for every fixed pair of samples. Taking expectations and using Steps 1–2 proves (3.9.3) when $n\varepsilon^2 \ge 2$. If $n\varepsilon^2 < 2$, then $\Pi_{\mathcal{H}}(2n) \ge 1$ because the class is nonempty, and

$$4\Pi_{\mathcal{H}}(2n)e^{-n\varepsilon^2/8} > 4e^{-1/4} > 1.$$

Since a probability is at most one, (3.9.3) holds in this case as well.

Step 4: insert the VC dimension and solve for the confidence radius. For $1 \le d \le n$, Lemma 3.19 gives $\Pi_{\mathcal{H}}(2n) \le (2en/d)^d$. Set

$$\varepsilon_\delta := \sqrt{\frac{8}{n}\left\{d\log\frac{2en}{d} + \log\frac{4}{\delta}\right\}}.$$

Substituting this value into (3.9.3) bounds the failure probability by

$$4\left(\frac{2en}{d}\right)^d \exp\left\{-d\log\frac{2en}{d} - \log\frac{4}{\delta}\right\} = \delta,$$

which proves (3.9.4). If $d = 0$, the same lemma gives $\Pi_{\mathcal{H}}(2n) \le 1$, and choosing $\varepsilon_\delta = \sqrt{8\log(4/\delta)/n}$ proves the stated zero-dimensional case. □

The complexity term $d\log(2en/d)$ comes from the logarithm of the number of prediction patterns on the doubled sample. At fixed confidence, the resulting rate is of order $\sqrt{d\log(2en/d)/n}$ for $d \ge 1$.

3.10 Rademacher complexity for a loss class

Let $\xi_1, \dots, \xi_n$ be independent random signs with $\mathbb{P}(\xi_i = 1) = \mathbb{P}(\xi_i = -1) = 1/2$, independent of the sample.

Definition 3.21 (Rademacher complexity)**.** For a sample $S = (Z_1, \dots, Z_n) \sim P^n$, define

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) := \mathbb{E}_\xi\left[\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^{n}\xi_i g(Z_i)\right|\,\middle|\, S\right], \tag{3.10.1}$$

$$\mathfrak{R}_n(\mathcal{G}) := \mathbb{E}_S[\widehat{\mathfrak{R}}_S(\mathcal{G})]. \tag{3.10.2}$$

A rich class can align one of its functions with many random sign patterns, which makes its Rademacher complexity large. A constrained class cannot.

Theorem 3.22 (Rademacher uniform convergence)**.** *Let $S = (Z_1, \dots, Z_n) \sim P^n$, with $n \ge 1$, and let $\varnothing \ne \mathcal{G} \subseteq [0,1]^{\mathcal{Z}}$ be a class of measurable functions. Assume the suprema below are measurable, as for a countable class. Write $Pg = \mathbb{E}_{Z\sim P}g(Z)$ and $P_n g = n^{-1}\sum_{i=1}^{n} g(Z_i)$. For every $\delta \in (0,1)$, with probability at least $1-\delta$,*

$$\sup_{g\in\mathcal{G}}|P_n g - Pg| \le 2\mathfrak{R}_n(\mathcal{G}) + \sqrt{\frac{\log(1/\delta)}{2n}}. \tag{3.10.3}$$

An empirical version, also valid with probability at least $1-\delta$*, is*

$$\sup_{g\in\mathcal{G}}|P_n g - Pg| \le 2\widehat{\mathfrak{R}}_S(\mathcal{G}) + 3\sqrt{\frac{\log(2/\delta)}{2n}}.$$

Proof. Set $\Delta(S) := \sup_{g\in\mathcal{G}}|P_n g - Pg|$. We first bound its expectation and then its deviation from that expectation. Throughout, $\widehat{\mathfrak{R}}_S$ uses the absolute-value convention in (3.10.1).

Step 1: introduce an independent ghost sample. Let $S' = (Z'_1,\dots,Z'_n) \sim P^n$ be independent of S, and put $P'_n g = n^{-1}\sum_{i=1}^n g(Z'_i)$. Fix S. For every $g\in\mathcal{G}$, $\mathbb{E}_{S'}P'_n g = Pg$, so

$$\begin{aligned}|P_n g - Pg| &= \left|\mathbb{E}_{S'}\left[P_n g - P'_n g\right]\right| \\ &\le \mathbb{E}_{S'}|P_n g - P'_n g| \\ &\le \mathbb{E}_{S'}\left[\sup_{u\in\mathcal{G}}|P_n u - P'_n u|\right].\end{aligned}$$

The first inequality is Jensen's inequality for the absolute value. The last expression is the same upper bound for every g, so taking the supremum on the left gives

$$\Delta(S) \le \mathbb{E}_{S'}\left[\sup_{g\in\mathcal{G}}|P_n g - P'_n g|\right].$$

Taking expectation over S, and using iterated expectation on the right, yields

$$\begin{aligned}\mathbb{E}_S\Delta(S) &\le \mathbb{E}_S\left[\mathbb{E}_{S'}\left[\sup_{g\in\mathcal{G}}|P_n g - P'_n g|\right]\right] \\ &= \mathbb{E}_{S,S'}\left[\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\big(g(Z_i)-g(Z'_i)\big)\right|\right].\end{aligned}$$

Step 2: insert independent random signs. Let $\xi_1,\dots,\xi_n$ be independent uniform signs, independent of both samples. For each fixed sign vector, swap Z_i and Z'_i when $\xi_i=-1$. Since the observations in each pair are independent and have the same law, these swaps preserve the joint distribution. For every g, a swapped pair changes its difference to $\xi_i(g(Z_i)-g(Z'_i))$. Thus, first taking expectation over the samples and then averaging over the signs gives

$$\begin{aligned}&\mathbb{E}_{S,S'}\left[\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\big(g(Z_i)-g(Z'_i)\big)\right|\right] \\ &\quad= \mathbb{E}_{S,S',\xi}\left[\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\xi_i\big(g(Z_i)-g(Z'_i)\big)\right|\right].\end{aligned}$$

This equality uses the distribution of the sample pairs; it does not assert equality for each fixed realization of S, S'.

Step 3: split the two sample terms. For each fixed S, S', ξ, the triangle inequality gives

$$\begin{aligned}&\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\xi_i\big(g(Z_i)-g(Z'_i)\big)\right| \\ &\quad\le \sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\xi_i g(Z_i)\right| + \sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\xi_i g(Z'_i)\right|.\end{aligned}$$

The two suprema may use different functions, which is why this is an inequality. By the definition of empirical Rademacher complexity,

$$\mathbb{E}_{S,\xi}\left[\sup_{g\in\mathcal{G}}\left|\frac{1}{n}\sum_{i=1}^n\xi_i g(Z_i)\right|\right] = \mathbb{E}_S\widehat{\mathfrak{R}}_S(\mathcal{G}) = \mathfrak{R}_n(\mathcal{G}).$$

The same equality holds for S', because it has the same distribution as S. Consequently, Steps 1–3 imply

$$\mathbb{E}_S \Delta(S) \le 2\mathfrak{R}_n(\mathcal{G}).$$

The factor 2 comes from these two identically distributed sample terms.

Step 4: control the fluctuation around the expectation. Let $S^{(j)}$ be obtained by replacing only Z_j with a point $\widetilde{Z}_j$, and let $P_n^{(j)}$ be its empirical measure. For every $g \in \mathcal{G}$, boundedness gives

$$|P_n g - P_n^{(j)} g| = \frac{1}{n} |g(Z_j) - g(\widetilde{Z}_j)| \le \frac{1}{n}.$$

The reverse triangle inequality and $\left|\sup_g a_g - \sup_g b_g\right| \le \sup_g |a_g - b_g|$ yield

$$\begin{aligned} |\Delta(S) - \Delta(S^{(j)})| &\le \sup_{g \in \mathcal{G}} \left| |P_n g - Pg| - |P_n^{(j)} g - Pg| \right| \\ &\le \sup_{g \in \mathcal{G}} |P_n g - P_n^{(j)} g| \le \frac{1}{n}. \end{aligned}$$

Apply Lemma 3.5 to $U = \Delta(S)$, with $c_j = 1/n$ for every j. Since $\sum_{j=1}^n c_j^2 = 1/n$, it gives

$$\mathbb{P}\big(\Delta(S) - \mathbb{E}_S \Delta(S) \ge t\big) \le e^{-2nt^2}.$$

Choose $t_\delta = \sqrt{\log(1/\delta)/(2n)}$. Since $e^{-2nt_\delta^2} = \delta$, with probability at least $1 - \delta$,

$$\Delta(S) \le \mathbb{E}_S \Delta(S) + t_\delta \le 2\mathfrak{R}_n(\mathcal{G}) + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

This proves (3.10.3).

Step 5: obtain the empirical version. For fixed signs, changing one observation changes the corresponding supremum of absolute signed averages by at most $1/n$, by the same supremum and reverse-triangle inequalities used above. Taking expectation over the signs gives

$$\left| \widehat{\mathfrak{R}}_S(\mathcal{G}) - \widehat{\mathfrak{R}}_{S^{(j)}}(\mathcal{G}) \right| \le \mathbb{E}_\xi \left[\sup_{g \in \mathcal{G}} \left| \frac{\xi_j}{n} \big(g(Z_j) - g(\widetilde{Z}_j)\big) \right| \right] \le \frac{1}{n}.$$

Applying Lemma 3.5 to $U = -\widehat{\mathfrak{R}}_S(\mathcal{G})$, whose expectation is $-\mathfrak{R}_n(\mathcal{G})$, yields

$$\mathbb{P}\left(\mathfrak{R}_n(\mathcal{G}) - \widehat{\mathfrak{R}}_S(\mathcal{G}) \ge t \right) \le e^{-2nt^2}.$$

Set $t = \sqrt{\log(2/\delta)/(2n)}$. The preceding bound and the bound for Δ each fail with probability at most $\delta/2$. A union bound therefore gives, with probability at least $1 - \delta$, both

$$\Delta(S) \le 2\mathfrak{R}_n(\mathcal{G}) + t, \qquad \mathfrak{R}_n(\mathcal{G}) \le \widehat{\mathfrak{R}}_S(\mathcal{G}) + t.$$

On their intersection,

$$\Delta(S) \le 2\widehat{\mathfrak{R}}_S(\mathcal{G}) + 3t = 2\widehat{\mathfrak{R}}_S(\mathcal{G}) + 3\sqrt{\frac{\log(2/\delta)}{2n}},$$

as claimed. □

Example 3.23 (Euclidean linear predictors)**:** Let $\mathcal{F}_B = \{x \mapsto w^\top x : \|w\|_2 \le B\}$, and suppose $\|X_i\|_2 \le R$. Symmetry of the Euclidean ball and Cauchy–Schwarz give

$$\begin{aligned}\widehat{\mathfrak{R}}_{S_X}(\mathcal{F}_B) &= \frac{B}{n}\mathbb{E}_\xi \left\| \sum_{i=1}^n \xi_i X_i \right\|_2 \\ &\le \frac{B}{n}\sqrt{\sum_{i=1}^n \|X_i\|_2^2} \le \frac{BR}{\sqrt{n}}. \end{aligned} \tag{3.10.4}$$

To justify the first inequality, condition on $S_X = (X_1, \dots, X_n)$, so the inputs are fixed, and set $V = \sum_{i=1}^n \xi_i X_i$. Cauchy–Schwarz, applied to the random variables $\|V\|_2$ and 1, gives

$$\mathbb{E}_\xi \|V\|_2 \le \sqrt{\mathbb{E}_\xi \|V\|_2^2}\sqrt{\mathbb{E}_\xi 1^2} = \sqrt{\mathbb{E}_\xi \|V\|_2^2}.$$

The independent signs satisfy $\mathbb{E}_\xi \xi_i = 0$ and $\mathbb{E}_\xi \xi_i^2 = 1$, so $\mathbb{E}_\xi(\xi_i \xi_j) = 0$ for $i \ne j$. Expanding the squared norm therefore yields

$$\mathbb{E}_\xi \|V\|_2^2 = \sum_{i=1}^n \sum_{j=1}^n \mathbb{E}_\xi(\xi_i \xi_j) X_i^\top X_j = \sum_{i=1}^n \|X_i\|_2^2 .$$

Substituting this identity gives the first inequality in (3.10.4). The last inequality uses $\sum_{i=1}^n \|X_i\|_2^2 \le nR^2$. ⋄

The next examples illustrate three other ways to calculate the complexity: counting a finite class, using the geometry of an ℓ_1 constraint, and allowing every binary labeling on the sample. All calculations condition on the sample and average only over the independent signs.

Example 3.24 (A finite class)**:** Let $\mathcal{G}$ consist of $M \ge 1$ functions, and suppose $|g(Z_i)| \le A$ for every $g \in \mathcal{G}$ and i, where $A \ge 0$. Then

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \le A\sqrt{\frac{2\log(2M)}{n}}. \tag{3.10.5}$$

For bounded losses in $[0, 1]$, take $A = 1$. This is a finite-class form of Massart's bound [19].
If $A = 0$, every signed sum is zero and the bound is immediate. For $A > 0$, set $T_g = n^{-1}\sum_{i=1}^n \xi_i g(Z_i)$. Independence and Hoeffding's lemma give, for every real λ,

$$\mathbb{E}_\xi e^{\lambda T_g} \le \exp\left\{\frac{\lambda^2}{2n^2}\sum_{i=1}^n g(Z_i)^2\right\} \le \exp\left\{\frac{\lambda^2 A^2}{2n}\right\}.$$

The same bound holds for $-T_g$. For $\lambda > 0$, Jensen's inequality and the fact that a maximum of positive terms is bounded by their sum yield

$$\begin{aligned}\exp\{\lambda \widehat{\mathfrak{R}}_S(\mathcal{G})\} &= \exp\{\lambda \mathbb{E}_\xi \max_{g\in\mathcal{G}} |T_g|\} \\ &\le \mathbb{E}_\xi \exp\{\lambda \max_{g\in\mathcal{G}} |T_g|\} \\ &\le \sum_{g\in\mathcal{G}} \left(\mathbb{E}_\xi e^{\lambda T_g} + \mathbb{E}_\xi e^{-\lambda T_g}\right) \\ &\le 2M \exp\left\{\frac{\lambda^2 A^2}{2n}\right\}. \end{aligned}$$

Taking logarithms and dividing by λ gives

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \le \frac{\log(2M)}{\lambda} + \frac{\lambda A^2}{2n}.$$

Choose $\lambda = \sqrt{2n\log(2M)}/A$ to obtain (3.10.5). The factor $2M$ counts both signs of each function, as required by the absolute-value definition. For fixed M and A, the bound decays as $n^{-1/2}$. ⋄

Example 3.25 (ℓ_1-constrained linear predictors)**:** For inputs in $\mathbb{R}^d$, with $d \geq 1$ and $B, R \geq 0$, let

$$\mathcal{F}_B^{(1)} := \{x \mapsto w^\top x : \|w\|_1 \leq B\}, \qquad \|X_i\|_\infty \leq R.$$

The identity $\sup_{\|w\|_1 \leq B} |w^\top v| = B \|v\|_\infty$ follows from $|w^\top v| \leq \|w\|_1 \|v\|_\infty$, with equality obtained by placing all the weight on a coordinate of largest $|v_j|$. Consequently,

$$\begin{aligned}
\widehat{\mathfrak{R}}_{S_X}(\mathcal{F}_B^{(1)}) &= \frac{B}{n} \mathbb{E}_\xi \left\| \sum_{i=1}^n \xi_i X_i \right\|_\infty \\
&= B \mathbb{E}_\xi \max_{1 \leq j \leq d} \left| \frac{1}{n} \sum_{i=1}^n \xi_i X_{ij} \right| \\
&\leq BR \sqrt{\frac{2 \log(2d)}{n}}. \qquad (3.10.6)
\end{aligned}$$

For the last step, apply Example 3.24 to the d coordinate functions $x \mapsto x_j$, whose absolute values on the sample are bounded by R. The logarithmic dependence on d arises because the dual norm selects the largest of d signed coordinate averages. ◇

Example 3.26 (A class realizing every binary labeling)**:** Suppose $S = (Z_1, \ldots, Z_n)$ consists of distinct points, and a Boolean class $\mathcal{G}_{\text{all}}$ realizes all 2^n labelings on these points:

$$\{(g(Z_1), \ldots, g(Z_n)) : g \in \mathcal{G}_{\text{all}}\} = \{0, 1\}^n.$$

For fixed signs, write $N_+ = |\{i : \xi_i = 1\}|$ and $N_- = |\{i : \xi_i = -1\}|$. Labeling exactly the positive-sign points by 1 gives a signed sum of N_+; labeling exactly the negative-sign points by 1 gives a signed sum of $-N_-$. Every other binary labeling gives a sum between these two values, so

$$\sup_{g \in \mathcal{G}_{\text{all}}} \left| \frac{1}{n} \sum_{i=1}^n \xi_i g(Z_i) \right| = \frac{\max(N_+, N_-)}{n} = \frac{1}{2} + \frac{1}{2n} \left| \sum_{i=1}^n \xi_i \right|.$$

Taking expectation over the signs gives

$$\frac{1}{2} \leq \widehat{\mathfrak{R}}_S(\mathcal{G}_{\text{all}}) = \frac{1}{2} + \frac{1}{2n} \mathbb{E}_\xi \left| \sum_{i=1}^n \xi_i \right| \leq \frac{1}{2} + \frac{1}{2\sqrt{n}}. \qquad (3.10.7)$$

The last inequality follows from Cauchy–Schwarz and $\mathbb{E}_\xi (\sum_i \xi_i)^2 = n$. When the class realizes every binary labeling on distinct samples of every size, its complexity stays at least $1/2$ as n grows. This illustrates how the ability to fit arbitrary random labels keeps Rademacher complexity large. ◇

VC dimension is especially natural for zero-one classification. Covering numbers are natural when a useful metric and continuity estimate are available. Rademacher complexity is often the most direct route for bounded or Lipschitz loss classes. Norm and margin restrictions for neural networks typically enter by upper-bounding one of these complexities; worst-case bounds can still be loose for the solutions reached by training [19, 26].

Lecture 4

Neural networks: representation, training, and architectures

The lecture slides begin with tasks whose inputs and outputs have very different structures: detecting objects in an image, transcribing speech, captioning a photo, generating an image from text, predicting protein structure, and selecting a move in Go. A neural network provides a learned map between representations of these objects. The architecture specifies how the map combines information; the task specifies its output and loss. For example, a captioning model must produce an ordered sequence, whereas an image classifier produces one vector of class scores. In Go, a learned network can be combined with a search algorithm rather than making every decision by itself.

We use the slides' *catalog-photo classifier* as a running example. The input is a product photo and the classes are coat, shirt, and shoe. A displayed probability vector such as $(0.84, 0.11, 0.05)$ illustrates the output format; it is not a measured accuracy. We first ask how hidden layers represent nonlinear class scores, then how a loss and backpropagation train those scores, and finally how regularization and architecture affect predictions on new photos. Product captions and image patches will provide the bridge to recurrent models and attention.

Lecture 1 introduced a one-hidden-layer ReLU class in (1.1.6). The three questions from the earlier lectures remain distinct: which functions can the class represent, which predictors can be learned reliably from a sample, and which parameters does an optimizer find? Backpropagation addresses a computational question inside the third one: given parameters and a loss, it evaluates a gradient. The feedforward, convolutional, and recurrent material follows Chapters 6–10 of [9]; the attention material follows [36]. The main text follows the slides' teaching order. Section 4.12 collects the supplementary mathematical arguments corresponding to their optional proof slides.

4.1 Feedforward networks: learned compositions

4.1.1 Why nonlinear features are needed

For two binary inputs, XOR labels $(0, 0)$ and $(1, 1)$ by zero, and $(1, 0)$ and $(0, 1)$ by one. A single affine threshold cannot separate these pairs, but two nonlinear hidden features can.

Example 4.1 (XOR with two ReLU units)**:** Let $x = (x_1, x_2) \in \{0, 1\}^2$, and label it 1 when exactly one coordinate is 1. With $\rho(u) = \max\{u, 0\}$, the two-unit network

$$f(x) = \rho(x_1 - x_2) + \rho(x_2 - x_1) = |x_1 - x_2| \tag{4.1.1}$$

gives the desired labels exactly. No affine score followed by a single threshold separates the two positive vertices from the two negative vertices: their convex hulls both contain $(1/2, 1/2)$. The hidden layer has changed the representation before the final linear combination. ◇

4.1.2 Layer notation and parameter count

Fix an input dimension $d_0 = d$, output dimension $d_L = K$, and layer widths $d_1, \dots, d_{L-1}$. We count the output affine map as layer L. For one input $x \in \mathbb{R}^{d_0}$, set $h^0 = x$ and define

$$z^\ell = W^\ell h^{\ell-1} + b^\ell, \qquad W^\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \qquad b^\ell \in \mathbb{R}^{d_\ell}, \qquad 1 \le \ell \le L, \tag{4.1.2}$$

$$h^\ell = \phi_\ell(z^\ell), \qquad 1 \le \ell < L, \tag{4.1.3}$$

where each hidden activation acts coordinatewise. The final vector $a_\theta(x) := z^L$ is a vector of *output scores* or *logits*. An output rule turns it into a prediction: it may be the identity for regression, a sigmoid for binary classification, or a softmax for multiclass classification. The parameters $\theta = (W^1, b^1, \dots, W^L, b^L)$ number

$$P = \sum_{\ell=1}^{L} d_\ell(d_{\ell-1} + 1). \tag{4.1.4}$$

Thus *width* describes the d_ℓ, while *depth* describes the number of successive learned maps. One hidden layer means $L = 2$.

Proposition 4.2 (Why a hidden nonlinearity matters)**.** *If every hidden activation in (4.1.3) is the identity, then $x \mapsto a_\theta(x)$ is an affine map, regardless of L.*

Proof. The composition of two affine maps has the form $W^2(W^1x + b^1) + b^2 = (W^2W^1)x + (W^2b^1 + b^2)$. Apply this identity repeatedly. □

4.1.3 Activation functions

An activation acts on every coordinate of a hidden layer. The slides compare five choices. In the table, Φ_{N} and φ_{N} denote the standard normal distribution function and density; they are unrelated to the training objective Φ_n.

Activation	Formula	Derivative away from any kink
Sigmoid	$\sigma(u) = (1 + e^{-u})^{-1}$	$\sigma(u)(1 - \sigma(u))$
Tanh	$\tanh u$	$1 - \tanh^2 u$
ReLU	$\rho(u) = \max\{0, u\}$	$\mathbf{1}_{\{u>0\}}$
Leaky ReLU	$\max\{\alpha u, u\}$, $0 < \alpha < 1$	α for $u < 0$, 1 for $u > 0$
GELU	$u\Phi_{\mathrm{N}}(u)$	$\Phi_{\mathrm{N}}(u) + u\varphi_{\mathrm{N}}(u)$

Sigmoid has values in $(0, 1)$ and tanh in $(-1, 1)$; both saturate when $|u|$ is large, where their derivatives become small. ReLU is piecewise linear and has derivative one on its positive half-line, but an inactive unit has zero local derivative. Leaky ReLU retains a small negative-side slope. GELU smoothly weights an input by $\Phi_{\mathrm{N}}(u)$ [37]. These are hidden-layer choices; a sigmoid or softmax output has a separate role determined by the classification loss.

ReLU and leaky ReLU have no ordinary derivative at zero. Software chooses a local derivative convention there, commonly zero for ReLU. This does not assert differentiability of the whole network loss at that parameter value. Section 4.5 explains why the derivatives in the table matter during training.

4.2 Approximation theory: what expressivity guarantees

A hidden layer can do more than solve XOR. Approximation theory asks whether a network can represent a given target to a prescribed accuracy. It does not provide a training algorithm or a sample-size guarantee.

Theorem 4.3 (Qualitative universal approximation)**.** *Let $\phi : \mathbb{R} \to \mathbb{R}$ be continuous and not a polynomial. For every compact $K \subset \mathbb{R}^d$, every continuous $f : K \to \mathbb{R}$, and every $\varepsilon > 0$, there are a finite integer m, coefficients $c, a_j, b_j \in \mathbb{R}$, and vectors $w_j \in \mathbb{R}^d$ such that*

$$\sup_{x \in K} \left| f(x) - c - \sum_{j=1}^{m} a_j \phi(w_j^\top x + b_j) \right| < \varepsilon.$$

This is the continuous-activation specialization of the density theorem of Leshno et al. [33]. It applies, in particular, to all five activations in the preceding table. The hidden biases and the freedom to increase the width are part of the statement. The theorem gives no rate for the required m, no guarantee that gradient descent finds suitable parameters, and no finite-sample generalization guarantee.

4.2.1 A constructive approximation in one dimension

In one dimension we can prove a quantitative ReLU approximation result directly.

Proposition 4.4 (Constructive ReLU approximation on an interval)**.** *Suppose $f : [0,1] \to \mathbb{R}$ is M-Lipschitz. For an integer $m \geq 1$, put $x_j = j/m$ and $s_j = m\{f(x_{j+1}) - f(x_j)\}$, $0 \leq j < m$. Then the ReLU network*

$$I_m f(x) = f(0) + s_0 \rho(x) + \sum_{j=1}^{m-1} (s_j - s_{j-1}) \rho(x - x_j) \tag{4.2.1}$$

interpolates f at all the knots and satisfies

$$\sup_{x \in [0,1]} |f(x) - I_m f(x)| \leq \frac{M}{2m}. \tag{4.2.2}$$

Proof. On (x_j, x_{j+1}), the derivative of the right side of (4.2.1) is $s_0 + \sum_{r=1}^{j} (s_r - s_{r-1}) = s_j$. Its value at zero is $f(0)$, so it is exactly the linear interpolant. For $x = x_j + t$ with $0 \leq t \leq h := 1/m$, the interpolant is $(1 - t/h) f(x_j) + (t/h) f(x_{j+1})$. Lipschitz continuity gives

$$|f(x) - I_m f(x)| \leq M \{(1 - t/h)t + (t/h)(h - t)\} = \frac{2Mt(h-t)}{h} \leq \frac{Mh}{2}.$$

□

The rate above is for a one-dimensional Lipschitz target. In higher dimension, a rectangular grid with m intervals per coordinate already has m^d cells. Qualitative universality alone therefore does not remove the curse of dimensionality; useful rates need assumptions about the target's structure.

Depth can also reduce the number of units needed for certain structured functions. Proposition 4.14, proved in Section 4.12, uses repeated tent maps to compare a compact deep representation with the exponentially many units needed in one hidden layer. These examples explain what width and depth can buy; neither replaces statistical or optimization analysis.

4.3 Output rules and losses

The hidden representation need not change when we change tasks. What changes is the interpretation of z^L and the loss placed on it. For scalar regression, take $K = 1$, predict $a_\theta(x) = z^L$, and use $\ell_{\mathrm{sq}}(a, y) = \frac{1}{2}(a - y)^2$. For binary classification with $y \in \{0, 1\}$, one logit a gives

$$p_\theta(Y = 1 \mid x) = \sigma(a) := \frac{1}{1 + e^{-a}}, \qquad \ell_{\mathrm{bin}}(a, y) = -y \log \sigma(a) - (1 - y) \log(1 - \sigma(a)).$$

The logit form is useful because $\partial \ell_{\mathrm{bin}} / \partial a = \sigma(a) - y$.

For $K \geq 2$ classes, let $y \in \{1, \ldots, K\}$ and define

$$p_k = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}}, \qquad \ell_{\mathrm{ce}}(a, y) = -\log p_y = -a_y + \log \sum_{j=1}^{K} e^{a_j}. \tag{4.3.1}$$

The softmax probabilities sum to one; prediction may use $\arg\max_k p_k = \arg\max_k a_k$. Subtracting $\max_j a_j$ from every logit leaves p unchanged and avoids unnecessarily large exponentials in a numerical implementation.

Proposition 4.5 (Softmax and cross-entropy gradient)**.** *For ℓ_{ce} in (4.3.1), let e_y be the y-th coordinate vector. Then*

$$\nabla_a \ell_{\mathrm{ce}}(a, y) = p - e_y. \tag{4.3.2}$$

Proof. Differentiating the last expression in (4.3.1) gives

$$\frac{\partial \ell_{\mathrm{ce}}}{\partial a_k} = -\mathbf{1}_{\{k=y\}} + \frac{e^{a_k}}{\sum_j e^{a_j}} = p_k - \mathbf{1}_{\{k=y\}}.$$

□

For example, if $K = 3$, $a = (\log 2, 0, 0)$, and $y = 1$, then $p = (1/2, 1/4, 1/4)$. The loss is $\log 2$, while the logit gradient is $(-1/2, 1/4, 1/4)$. It increases the first score relative to the other two under a gradient step.

For a sample $\{(X_i, Y_i)\}_{i=1}^n$, the training objective can be written as

$$\Phi_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(a_\theta(X_i), Y_i) + R(\theta), \tag{4.3.3}$$

where R is an optional regularizer. The prediction loss used to assess population performance need not be identical to this differentiable training loss. For example, cross-entropy provides useful gradients when the reported classification measure is the zero-one error.

4.4 Backpropagation is a chain-rule computation

4.4.1 A scalar computation graph

The chain rule can be read locally before introducing matrix notation. Follow the slides' graph with $x = 2$, $w = 3$, $b = 1$:

$$m = xw = 6, \qquad z = m + b = 7, \qquad \mathcal{L} = \tfrac{1}{2} z^2 = 24.5.$$

The backward pass starts at $\partial \mathcal{L}/\partial z = z = 7$. Because $\partial z/\partial m = \partial z/\partial b = 1$, it gives $\partial \mathcal{L}/\partial m = 7$ and $\partial \mathcal{L}/\partial b = 7$. Finally,

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial m} \frac{\partial m}{\partial w} = 7x = 14, \qquad \frac{\partial \mathcal{L}}{\partial x} = 7w = 21.$$

The input gradient is useful for tracing sensitivity; the trainable parameters in this graph are w, b. A learning step uses their computed gradients only after this reverse pass is complete.

4.4.2 The layerwise recursion

A direct expansion of every derivative would repeat the same calculations many times. The forward pass stores the z^ℓ and h^ℓ in (4.1.2)–(4.1.3). The backward pass reuses them in reverse order. Consider one example (x, y) and let $\mathcal{L}(\theta) = \ell(z^L, y)$. Define the *error signal* $\delta^\ell := \nabla_{z^\ell} \mathcal{L} \in \mathbb{R}^{d_\ell}$.

Theorem 4.6 (Backpropagation for a dense feedforward network)**.** *Suppose the hidden activations are differentiable at the forward-pass values and the loss is differentiable at z^L. Starting from $\delta^L = \nabla_{z^L} \ell(z^L, y)$, compute*

$$\delta^\ell = \big((W^{\ell+1})^\top \delta^{\ell+1}\big) \odot \phi_\ell'(z^\ell), \qquad \ell = L-1, \dots, 1. \tag{4.4.1}$$

Here $\odot$ is coordinatewise multiplication. The parameter gradients at each layer are

$$\nabla_{W^\ell} \mathcal{L} = \delta^\ell (h^{\ell-1})^\top, \qquad \nabla_{b^\ell} \mathcal{L} = \delta^\ell. \tag{4.4.2}$$

The input gradient, when needed, is $\nabla_x \mathcal{L} = (W^1)^\top \delta^1$.

Proof. For a hidden layer, the Jacobian of $z^{\ell+1} = W^{\ell+1}\phi_\ell(z^\ell) + b^{\ell+1}$ with respect to z^ℓ is $W^{\ell+1}\,\mathrm{diag}(\phi'_\ell(z^\ell))$. Transposing this Jacobian and applying the chain rule to the column gradient $\delta^{\ell+1}$ gives (4.4.1). For each entry,

$$\frac{\partial z_j^\ell}{\partial W_{jk}^\ell} = h_k^{\ell-1}, \qquad \frac{\partial z_j^\ell}{\partial b_j^\ell} = 1.$$

Multiplication by δ_j^ℓ yields the outer product and bias gradient in (4.4.2). The first affine map gives the input-gradient formula in the same way. □

The outer product has shape $d_\ell \times d_{\ell-1}$, exactly the shape of W^ℓ. With dense matrices, evaluating all layer gradients for one example takes arithmetic on the order of $\sum_{\ell=1}^{L} d_\ell d_{\ell-1}$, the same order as a forward pass. The intermediate activations must be retained or recomputed for the backward pass. At a ReLU kink, software uses its chosen local derivative convention in the same recursion; the theorem itself concerns differentiable forward-pass values.

Example 4.7 (One hidden layer with a binary output)**:** Let $z = Wx + b$, $h = \rho(z)$, $a = v^\top h + c$, $p = \sigma(a)$, and use binary cross-entropy. At points where every $z_j \neq 0$,

$$\frac{\partial \ell}{\partial a} = p - y, \qquad \nabla_v \ell = (p-y)h, \qquad \nabla_W \ell = \big[(p-y)v \odot \mathbf{1}_{\{z>0\}}\big]x^\top.$$

The remaining gradients are $\partial\ell/\partial c = p - y$ and $\nabla_b \ell = (p-y)v \odot \mathbf{1}_{\{z>0\}}$. Thus the output error is first passed through v, then through the local ReLU derivatives, before it reaches W. ◇

Branches and shared parameters. The chain rule is equally simple when a computation graph has more than one path. For a residual map $r = x + F_\theta(x)$ and a scalar loss $\ell(r)$, let $g = \nabla_r \ell$. The two paths from x to r contribute

$$\nabla_x \ell = g + (D_x F_\theta(x))^\top g, \qquad \nabla_\theta \ell = (D_\theta F_\theta(x))^\top g. \tag{4.4.3}$$

When one parameter is used at several graph nodes, its gradient is the sum of the contributions from all occurrences. For example, the matrix A in a recurrent network is used at every time step. Unrolling the recurrence and accumulating these contributions gives its exact gradient, including the indirect effects of an early state on later states. The reverse pass is efficient because it accumulates each intermediate derivative once rather than separately expanding every path from the loss.

Backpropagation computes a gradient; it does not choose a step size or update the parameters. An optimizer such as the mini-batch SGD methods of Lecture 2 uses the gradient after the backward pass.

4.4.3 From gradients to a training step

Training repeats four operations: evaluate the network on a batch, evaluate its loss, backpropagate to compute parameter gradients, and apply an optimizer update. The forward and backward passes are computations at the current parameters; SGD specifies how those parameters change.

For a mini-batch B_t of size m, a basic update for the objective in (4.3.3) is

$$g_t = \frac{1}{m}\sum_{i \in B_t} \nabla_\theta \ell(a_{\theta_t}(X_i), Y_i) + \nabla R(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t g_t. \tag{4.4.4}$$

Each example gradient is computed by a forward and a backward pass. For sampling with replacement, B_t is a multiset: repeated indices contribute once per occurrence. Sample the new indices independently and uniformly conditional on the fixed training data and the current iterate. The mini-batch average is then an unbiased gradient of the unregularized empirical loss, conditional on θ_t. Lecture 2 developed mini-batch SGD and its optimizer variants; the new issue here is that the parameterized network objective is generally nonconvex. The convex objective-gap guarantees from that lecture cannot simply be attached to (4.4.4).

At a differentiable point, uniform sampling with replacement gives the more precise conditional identity

$$\mathbb{E}[g_t \mid \theta_t, \{(X_i, Y_i)\}_{i=1}^n] = \nabla \Phi_n(\theta_t).$$

This statement concerns the average over possible mini-batches on the fixed training sample. It does not identify that conditional gradient with the gradient of the unknown population risk.

4.5 Depth, gradient flow, and initialization

An extra layer can reuse features produced by earlier layers. The relevant cost is that the chain rule multiplies more Jacobians. Write $D^\ell = \operatorname{diag}(\phi'_\ell(z^\ell))$. Applying (4.4.1) repeatedly gives the elementary bound

$$\left\|\delta^\ell\right\|_2 \le \left\{\prod_{j=\ell}^{L-1} \left\|D^j\right\|_{\mathrm{op}} \left\|W^{j+1}\right\|_{\mathrm{op}}\right\} \left\|\delta^L\right\|_2, \qquad \ell < L. \tag{4.5.1}$$

If every factor is appreciably below one, an early-layer error signal can become small. A large upper bound does not by itself prove that a gradient will explode, because matrix products can cancel or act differently on different directions. The scalar recursion $h^\ell = \tanh(wh^{\ell-1})$ at $h^0 = 0$ makes the possibility transparent:

$$\left.\frac{dh^L}{dh^0}\right|_{h^0=0} = w^L.$$

For nonzero states, saturation of tanh can further shrink the derivatives. ReLU avoids saturation on its positive half-line but has zero derivative on its negative half-line.

Two choices made before optimization begins affect these products. First, identical hidden units with identical incoming and outgoing parameters receive identical gradients, so deterministic gradient updates do not separate them. Random initialization breaks that symmetry. Second, the scale of the random weights matters. The following calculation explains a common scale for ReLU networks.

Proposition 4.8 (Forward second moment at ReLU initialization)**.** *At one hidden layer, suppose $b^\ell = 0$, the entries of W^ℓ are independent of $h^{\ell-1}$, independent across input coordinates, symmetric about zero, and have common variance v_ℓ. If each input coordinate has second moment $\mathbb{E}[(h_i^{\ell-1})^2] = q_{\ell-1}$, then each ReLU output coordinate has second moment*

$$q_\ell := \mathbb{E}[(h_j^\ell)^2] = \frac{d_{\ell-1} v_\ell}{2}\, q_{\ell-1}. \tag{4.5.2}$$

Consequently $v_\ell = 2/d_{\ell-1}$ preserves this second moment from one such layer to the next.

Proof. Conditioning on $h^{\ell-1}$, centering and independence of the weights give

$$\mathbb{E}[(z_j^\ell)^2 \mid h^{\ell-1}] = v_\ell \sum_{i=1}^{d_{\ell-1}} (h_i^{\ell-1})^2.$$

The conditional distribution of z_j^ℓ is symmetric, and $\rho(z)^2 = z^2 \mathbf{1}_{\{z>0\}}$. Symmetry therefore gives $\mathbb{E}[\rho(z_j^\ell)^2 \mid h^{\ell-1}] = \frac{1}{2}\mathbb{E}[(z_j^\ell)^2 \mid h^{\ell-1}]$. Take expectations and sum the coordinate second moments. □

The proposition concerns the forward signal at random initialization. It does not ensure that every backward gradient is well scaled, that the assumptions persist after training, or that the network generalizes. When widths vary, forward and backward scales can impose different requirements. Initialization is therefore part of the training design, not a substitute for checking actual activations and gradients.

4.6 Generalization: choosing among fitted predictors

Several functions can pass through the same observed training points and behave differently between them. Likewise, several classifiers can fit our labeled product photos and assign different scores to a new photo. A small training loss therefore leaves a model-selection question. Lecture 3 studied this through hypothesis classes and generalization bounds; here we examine choices made in an actual network training run.

An *explicit* regularization mechanism changes the objective or the training data distribution, for example by adding a weight penalty or sampling transformed photos. An *implicit* mechanism is a preference created by the optimization procedure, initialization, or stopping time without an added penalty. Both can affect the final predictor, and they can operate together.

Choose such settings with a validation set, including the checkpoint selected by early stopping. Keep the test set for assessing the chosen procedure. Reusing test performance to tune a model turns that set into additional selection data. A regularizer is a modeling choice, not a guarantee that every new photo will be classified correctly.

4.7 Explicit regularization

Data augmentation modifies the training photos, weight penalties constrain parameter choices, and dropout randomizes hidden features. Each mechanism specifies a training criterion before an optimizer is run; Chapter 7 of [9] discusses their uses.

Data augmentation. If a transformation distribution $\mathcal{T}$ preserves the task label, an explicit augmented criterion is

$$\Phi_{\text{aug}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{T\sim\mathcal{T}}[\ell(a_\theta(TX_i), Y_i)]. \tag{4.7.1}$$

This encourages consistent predictions on transformed inputs. Whether a particular rotation, crop, time shift, or other transformation really preserves the label is a modeling assumption, not a consequence of the formula.

Norm penalties and constraints. The standard example is weight decay:

$$\Phi_{n,\lambda}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(a_\theta(X_i), Y_i) + \frac{\lambda}{2}\sum_{\ell=1}^{L} \left\|W^\ell\right\|_F^2, \qquad \lambda \geq 0. \tag{4.7.2}$$

Bias vectors are often omitted from the penalty. Under plain SGD, the update for W^ℓ includes $-\eta_t\lambda W^\ell$. An ℓ_1 penalty or a parameter-norm constraint makes a different modeling choice. For adaptive methods, adding an ℓ_2 gradient and decoupling weight decay from the gradient step are generally different algorithms, as seen for AdamW in Lecture 2 [16].

The ReLU rescaling in Proposition 4.13 shows why a parameter penalty is not merely a property of the represented function. Consider the version that also penalizes biases. For a single unit with output weight a and incoming vector $v = (w, b)$, all $c > 0$ represent the same function after the replacement $(a, v) \mapsto (a/c, cv)$, but their quadratic penalties are

$$\frac{1}{2}\left(\frac{a^2}{c^2} + c^2\left\|v\right\|_2^2\right) \geq |a|\ \left\|v\right\|_2.$$

The lower bound follows by the arithmetic–geometric mean inequality and is reached when $c^2 = |a|/\left\|v\right\|_2$ for $a \neq 0$ and $v \neq 0$. Thus a quadratic parameter penalty also selects how an equivalent feature is distributed across layers.

One can instead control a property closer to the function itself. If every hidden activation is 1-Lipschitz in Euclidean norm, then

$$\left\|a_\theta(x) - a_\theta(x')\right\|_2 \leq \left(\prod_{\ell=1}^{L}\left\|W^\ell\right\|_{\text{op}}\right)\left\|x - x'\right\|_2. \tag{4.7.3}$$

Indeed, each affine map changes distances by at most its operator norm, each activation changes them by at most one, and the bounds multiply under composition. Constraining spectral norms is therefore one way to control input sensitivity. The product bound can be loose: it is an upper bound, not the exact Lipschitz constant.

Dropout as a modified training objective. Dropout samples a masked network on each training pass [38]. During training, an inverted-dropout mask replaces a hidden coordinate h_j by $\widetilde{h}_j = M_j h_j/q$, where $0 < q \leq 1$ is the retention probability and $M_j \sim \text{Bernoulli}(q)$ is independent of h_j. Then $\mathbb{E}[\widetilde{h}_j \mid h_j] = h_j$. The criterion optimized in expectation is

$$\Phi_{\text{drop}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_M[\ell(a_\theta(X_i; M), Y_i)], \tag{4.7.4}$$

where $a_\theta(\cdot; M)$ denotes the masked network. At inference one usually uses the unmasked network with the inverted scaling convention. For a nonlinear downstream map, its output need not equal the mean of all masked outputs.

Example 4.9 (An exact dropout penalty for linear square loss)**:** Let $a_w(x) = w^\top x$ and mask each input coordinate independently as $\widetilde{x}_j = M_j x_j/q$. For fixed (x, y),

$$\mathbb{E}_M\left[\frac{1}{2}(y - w^\top \widetilde{x})^2\right] = \frac{1}{2}(y - w^\top x)^2 + \frac{1-q}{2q}\sum_{j=1}^{d} w_j^2 x_j^2. \tag{4.7.5}$$

A full expansion is given in Section 4.12.5. Thus this special case is an input-dependent quadratic penalty. There is no identical penalty formula for a general nonlinear network. ◇

4.8 Implicit regularization and optimizer selection

Even when two parameter vectors attain the same training loss, the training procedure need not be equally likely to find them. Initialization, update geometry, step size, stochasticity, and stopping time can all affect which solution is selected. The following linear example makes one such preference exact.

4.8.1 Gradient descent from zero

Theorem 4.10 (Gradient descent selects a minimum-norm interpolant)**.** *Let $X \in \mathbb{R}^{n\times d}$ have full row rank with $d > n$, and let $y \in \mathbb{R}^n$. Apply full-batch gradient descent to*

$$F(\theta) = \frac{1}{2n}\|X\theta - y\|_2^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{n}X^\top(X\theta_t - y), \qquad \theta_0 = 0,$$

with $0 < \eta \le n/\|X\|_{\mathrm{op}}^2$. Then

$$\theta_t \longrightarrow \theta^\dagger = X^\top(XX^\top)^{-1}y, \tag{4.8.1}$$

the unique solution of minimum Euclidean norm among all θ satisfying $X\theta = y$.

The proof in Section 4.12.4 has two steps: every gradient update stays in the row space of X, and each singular coordinate converges to the corresponding interpolation coefficient. The row-space interpolant is orthogonal to $\ker X$, which gives its minimum-norm property.

The initial condition matters. From an arbitrary θ_0, the component in $\ker X$ never changes, and the limit is $\theta^\dagger + \Pi_{\ker X}\theta_0$. Thus the objective alone does not choose a unique interpolant.

4.8.2 Early stopping and ridge as spectral filters

Use the thin singular-value decomposition $X = U\,\mathrm{diag}(s_1, \dots, s_n)V^\top$, with $U \in \mathbb{R}^{n\times n}$, $V \in \mathbb{R}^{d\times n}$, and $s_j > 0$. The recurrence from the minimum-norm calculation gives the exact finite-time iterate

$$\theta_t = \sum_{j=1}^{n} q_t(s_j)\frac{u_j^\top y}{s_j}\,v_j, \qquad q_t(s) = 1 - \left(1 - \frac{\eta s^2}{n}\right)^t. \tag{4.8.2}$$

Indeed, for $\alpha_{t,j} = v_j^\top\theta_t$, subtract the fixed point $u_j^\top y/s_j$ from the scalar update. The resulting error is multiplied by $1 - \eta s_j^2/n$ at every step, starting from the negative of that fixed point. This yields (4.8.2).

For comparison, ridge regression explicitly minimizes $F(\theta) + \lambda\|\theta\|_2^2/2$, with $\lambda > 0$. Setting the gradient to zero gives $(X^\top X + n\lambda I_d)\widehat{\theta}_\lambda = X^\top y$, so

$$\begin{aligned}\widehat{\theta}_\lambda &= (X^\top X + n\lambda I_d)^{-1}X^\top y \\ &= \sum_{j=1}^{n} r_\lambda(s_j)\frac{u_j^\top y}{s_j}\,v_j, \qquad\qquad r_\lambda(s) = \frac{s^2}{s^2 + n\lambda}.\end{aligned} \tag{4.8.3}$$

To obtain the second equality, note that v_j is an eigenvector of $X^\top X$ with eigenvalue s_j^2, and that $v_j^\top X^\top y = s_j u_j^\top y$. The right side has no component in $\ker X$.

With the theorem's step size, $0 \le q_t(s_j) \le 1$, and small singular values are learned more slowly. For small $\eta t s^2/n$, $q_t(s)$ is approximately $\eta t s^2/n$, whereas it approaches one after enough iterations for a fixed positive s. Ridge likewise shrinks directions with small s. This matters because an interpolation coefficient divides $u_j^\top y$ by s_j, which can amplify noise in a weakly observed direction. Stopping early can reduce this effect at the cost of leaving some signal unfitted.

The two filters are different functions of s; generally no single λ makes all their coefficients equal. The slides use $\eta/n = 0.2$, $t = 5$, and $n\lambda = 1$ to compare their shapes. The spectral interpretation is exact for this linear quadratic objective [9, 42]. In a nonlinear network, validation-based early stopping chooses a point on its actual training path, rather than applying this fixed linear filter.

4.8.3 Other sources of implicit bias

The examples on the slides concern different models and different training rules. Their assumptions specify when each conclusion applies.

SGD and averaging. In one-hot least squares, a fresh observation has input equal to a coordinate vector. Consider SGD initialized at zero, using a constant step size on fresh independent samples and averaging its final half of the iterates. The model is well specified, with centered finite-variance noise independent of the input. Under the remaining assumptions in [43], Theorem 4.1 shows that for each ridge solution whose expected excess risk is below the noise scale, a suitably tuned SGD step size gives expected excess risk within a constant factor at the same sample budget. This example shows that sampling and averaging can produce useful regularization without an explicit ridge penalty. It is an instance-wise comparison in a specified model, not a universal claim that every SGD run outperforms ridge.

The loss function. For a finite sample strictly separable by a homogeneous linear predictor, write its labels as $\widetilde{y}_i \in \{-1, +1\}$. Unregularized logistic loss tends to zero as the separating weights are scaled up, so it has no finite minimizing weight vector. With a suitable small fixed step size, full-batch GD has $\|w_t\|_2 \to \infty$, while

$$\frac{w_t}{\|w_t\|_2} \longrightarrow \frac{w_{\mathrm{mm}}}{\|w_{\mathrm{mm}}\|_2}, \qquad w_{\mathrm{mm}} = \arg\min_w \frac{1}{2}\|w\|_2^2 \quad \text{subject to } \widetilde{y}_i w^\top x_i \ge 1 \text{ for all } i.$$

The selected object is a maximum-margin *direction*, rather than a finite minimum-norm minimizer of logistic loss [34]. The optional margin discussion below explains why decreasing a classification loss can keep changing the weights after the training classification error is already zero.

Update geometry. A steepest-descent step with respect to a chosen norm $\|\cdot\|$ uses an increment that solves

$$\Delta_t \in \arg\min_{\Delta} \left\{ \langle \nabla F(w_t), \Delta \rangle + \tfrac{1}{2}\|\Delta\|^2 \right\}, \qquad w_{t+1} = w_t + \eta \Delta_t.$$

Here F denotes the differentiable objective being optimized. For separable linear classification, take $F(w) = n^{-1}\sum_i \exp(-\widetilde{y}_i w^\top x_i)$. With a sufficiently small fixed step size, the normalized margin $\min_i \widetilde{y}_i w_t^\top x_i / \|w_t\|$ approaches

$$\max_{\|w\| \le 1} \min_i \widetilde{y}_i w^\top x_i$$

for that norm [44]. When the maximizing direction is unique, this also specifies the limiting direction; otherwise the margin conclusion need not choose one unique vector. Euclidean GD is one choice of geometry. Changing the geometry can change the implicit preference even when the data and loss are fixed.

Initialization scale. Even a linear predictor can have a nonlinear parameterization. Consider $\beta = u \odot u - v \odot v$, with prediction $\beta^\top x$, and balanced initialization $u(0) = v(0) = \alpha \mathbf{1}$, $\alpha > 0$. For squared-loss gradient flow, if the limiting predictor interpolates, its implicit criterion depends on α. In the uniform balanced case analyzed in [45], the selected interpolant approaches the minimum Euclidean-norm solution as α grows; as α vanishes its ℓ_1 norm approaches the smallest interpolating ℓ_1 norm. If the latter minimizer is not unique, that statement alone does not identify a unique limiting predictor. Thus changing initialization

can change the preference while keeping the represented linear function class fixed. The small-scale statement takes positive α tending to zero; initializing both parameter vectors exactly at zero would instead make every gradient zero.

Explicit and implicit mechanisms can coexist. Dropout changes the objective and the noise in its stochastic gradients. Weight decay changes the objective while an optimizer can still favor particular minima. Representation, statistical control, and optimization remain separate parts of the population-risk problem from Lecture 1.

4.9 Convolutional networks: locality and equivariance

An image has two spatial coordinates and a collection of channels. Nearby pixels often describe related parts of an object, and the same visual pattern may occur at many locations. A convolutional neural network (CNN) uses these two properties through *local connections* and *shared parameters* [9, Chapter 9]. We first build the operation from small numerical examples, then combine it with nonlinearities and pooling to obtain a classifier.

4.9.1 From a dense image layer to a sliding filter

Flattening a 640×480 RGB image produces $640 \cdot 480 \cdot 3 = 921{,}600$ input coordinates. A dense layer with 1000 output units consequently has

$$(640 \cdot 480 \cdot 3) \cdot 1000 + 1000 = 921{,}601{,}000$$

parameters, including one bias per unit. Each pixel-to-unit connection has its own weight. This representation does not express the fact that two neighboring pixels are neighbors, or that a useful local pattern could be recognized with the same weights elsewhere in the image.

A sliding filter makes this reuse explicit. For the one-dimensional signal $x = (0, 0, 4, 0, 0)$, consider

$$K = \tfrac{1}{4}(1, 2, 1), \qquad y_t = \tfrac{1}{4}x_t + \tfrac{1}{2}x_{t+1} + \tfrac{1}{4}x_{t+2}.$$

Without padding, the length-three window can start at only three positions:

Position t	Input window	Output y_t
1	$(0, 0, 4)$	$\frac{1}{4}(0 + 2 \cdot 0 + 4) = 1$
2	$(0, 4, 0)$	$\frac{1}{4}(0 + 2 \cdot 4 + 0) = 2$
3	$(4, 0, 0)$	$\frac{1}{4}(4 + 2 \cdot 0 + 0) = 1$

Thus the output is $(1, 2, 1)$. The windows overlap, but the three weights are reused at every position. This particular filter is a fixed smoothing rule; a CNN learns its filter weights from data.

4.9.2 Two-dimensional filters and feature maps

For one input channel $X \in \mathbb{R}^{H \times W}$, a filter $K \in \mathbb{R}^{k_h \times k_w}$, and a scalar bias b, define

$$z_{u,v} = b + \sum_{r=0}^{k_h - 1} \sum_{t=0}^{k_w - 1} K_{r+1,t+1} X_{u+r,v+t}.$$

Here we use stride 1 and no padding. The valid positions are $u = 1, \ldots, H - k_h + 1$ and $v = 1, \ldots, W - k_w + 1$: the whole filter must fit inside the input. Each output is an inner product between the filter and one local input patch, plus the bias. The resulting array is a *feature map*.

Example 4.11 (A 3×3 filter on a 4×4 image)**:** Take bias 0 and

$$X = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 0 & 0 & 3 \\ 2 & 0 & 1 & 0 \\ 0 & 2 & 1 & 0 \end{pmatrix}, \qquad K = \begin{pmatrix} 0 & -1 & 1 \\ 0 & 1 & 0 \\ 1 & -1 & 0 \end{pmatrix}.$$

At the upper-left position, multiplication and summation give

$$z_{1,1} = (1 \cdot 0 + 0 \cdot (-1) + 0 \cdot 1) + (0 \cdot 0 + 0 \cdot 1 + 0 \cdot 0) + (2 \cdot 1 + 0 \cdot (-1) + 1 \cdot 0) = 2.$$

Sliding the same filter one column to the right, one row down, or both gives $z_{1,2} = 2 - 1 = 1$, $z_{2,1} = -2$, and $z_{2,2} = 3 + 1 + 2 - 1 = 5$. Therefore

$$Z = \begin{pmatrix} 2 & 1 \\ -2 & 5 \end{pmatrix}.$$

A hidden layer applies its activation separately to each entry. With $\rho(a) = \max\{a, 0\}$, its output becomes

$$H = \rho(Z) = \begin{pmatrix} 2 & 1 \\ 0 & 5 \end{pmatrix}.$$

The convolution supplies shared affine features, while the activation makes their composition nonlinear. ◇

The formulas above do not reverse the filter indices. In classical terminology this operation is *cross-correlation*; CNN literature and software conventionally call it convolution. We use that convention throughout.

4.9.3 Input channels and output channels

An RGB image has $C_{\text{in}} = 3$ input channels. One filter has a spatial slice for each input channel and adds their contributions:

$$z = K_R * X_R + K_G * X_G + K_B * X_B + b.$$

Here $*$ denotes the sliding operation just defined. The sum is one feature map, so one filter produces *one output channel*; it does not produce a separate output channel for each color.

A bank of C_{out} filters has the parameter arrays

$$K \in \mathbb{R}^{k_h \times k_w \times C_{\text{in}} \times C_{\text{out}}}, \qquad b \in \mathbb{R}^{C_{\text{out}}}.$$

For stride 1 and no padding, its preactivations are

$$z_{u,v,j} = b_j + \sum_{r=0}^{k_h-1} \sum_{t=0}^{k_w-1} \sum_{c=1}^{C_{\text{in}}} K_{r+1,t+1,c,j} X_{u+r,v+t,c}, \qquad j = 1, \ldots, C_{\text{out}}. \tag{4.9.1}$$

The activation is $h_{u,v,j} = \rho(z_{u,v,j})$, or another coordinatewise nonlinearity. Thus 16 filters on an RGB input produce 16 feature maps, each combining information from all three colors. The output shape is $H' \times W' \times C_{\text{out}}$.

4.9.4 Padding, stride, and output size

Padding specifies what the filter sees beyond the original image boundary. Zero padding of width p places p rows or columns of zeros on every side. For example, padding a 4×4 image with $p = 1$ gives a 6×6 array. A 3×3 filter with stride 1 then produces a 4×4 output, because $6 - 3 + 1 = 4$. For an odd square filter width k, padding $p = (k-1)/2$ preserves the spatial size when the stride is 1. Reflection and periodic padding use different boundary conventions.

Stride s is the distance between successive filter positions. In one dimension, an input of width 7, a filter of width 3, no padding, and stride 2 give windows starting at positions $1, 3, 5$. To derive the output length, index output positions by $v = 1, \ldots, W'$. The last index sampled by window v is $1 + s(v-1) + (k_w - 1)$. It must be at most the padded width $W + 2p$, so $s(v-1) \le W + 2p - k_w$. Consequently,

$$H' = \left\lfloor \frac{H + 2p - k_h}{s} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W + 2p - k_w}{s} \right\rfloor + 1. \tag{4.9.2}$$

This formula assumes no dilation, the same stride and padding along both axes, and a filter that fits in the padded input.

For a 5×5 input and 3×3 filters, the resulting spatial sizes are

Padding p	Stride s	Output spatial shape
0	1	3×3
1	1	5×5
0	2	2×2
1	2	3×3

Stride and padding determine spatial size; the number of filters determines the number of output channels.

4.9.5 Shared parameters and their gradients

Each output depends only on one $k_h \times k_w$ patch across the C_{in} input channels. In addition to this locality, the same filter entries and the same bias are reused at every output position. The learned parameter count is therefore

$$P_{\text{conv}} = k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{4.9.3}$$

It does not grow with the image height or width. For the $640 \times 480 \times 3$ input considered earlier, a layer with 16 filters of size 3×3, padding 1, and stride 1 has

$$16(3 \cdot 3 \cdot 3 + 1) = 448$$

learned parameters and output shape $640 \times 480 \times 16$. The many output entries do not require independent parameter vectors.

Parameter sharing also determines the backward pass. Write X^{pad} for the padded input and use stride 1. With $\delta_{u,v,j} = \partial \mathcal{L}/\partial z_{u,v,j}$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial K_{r,t,c,j}} = \sum_{u,v} \delta_{u,v,j} X^{\text{pad}}_{u+r-1,v+t-1,c}, \qquad \frac{\partial \mathcal{L}}{\partial b_j} = \sum_{u,v} \delta_{u,v,j}, \tag{4.9.4}$$

where $r = 1, \ldots, k_h$ and $t = 1, \ldots, k_w$. Every use of a kernel entry contributes to its derivative, and those contributions are added because they refer to the same parameter. This is the shared-parameter chain rule of Section 4.4 applied to a convolutional layer.

4.9.6 Pooling and a complete CNN classifier

Pooling summarizes a spatial window *within each channel*. Max pooling takes its largest value; average pooling takes its mean. Consider a 2×2 window with stride 2, no padding, and the input feature map

$$H = \begin{pmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 2 \\ 0 & 2 & 5 & 3 \\ 1 & 1 & 2 & 8 \end{pmatrix}.$$

The four nonoverlapping windows give

$$\text{MaxPool}(H) = \begin{pmatrix} 6 & 2 \\ 2 & 8 \end{pmatrix}, \qquad \text{AvgPool}(H) = \begin{pmatrix} 3.5 & 1.25 \\ 1 & 4.5 \end{pmatrix}.$$

For instance, the upper-left window has maximum $\max\{1, 3, 4, 6\} = 6$ and average $(1 + 3 + 4 + 6)/4 = 3.5$. Pooling reduces the spatial size, preserves the channel count, and has no learned parameters.

Global average pooling averages over all spatial positions instead of a small window:

$$\text{GAP}(h)_c = \frac{1}{HW} \sum_{u=1}^{H} \sum_{v=1}^{W} h_{u,v,c}. \tag{4.9.5}$$

It converts an $H \times W \times C$ feature map into a vector in $\mathbb{R}^C$, again without learned parameters. A final dense layer can turn this vector into class scores.

For ten-class classification of 32×32 RGB images, combine these operations as follows. The convolutional layers include biases, and their ReLU activations add no parameters.

Layer	Settings	Output shape	Parameters
Input	RGB image	$32 \times 32 \times 3$	0
Conv + ReLU	3×3, $p = 1$, $s = 1$, 16 filters	$32 \times 32 \times 16$	448
Max pool	2×2, $s = 2$	$16 \times 16 \times 16$	0
Conv + ReLU	3×3, $p = 1$, $s = 1$, 32 filters	$16 \times 16 \times 32$	4,640
Global average pool	Average each channel over space	32	0
Dense + softmax	32 features, 10 class scores	10	330

The second convolution has $3 \cdot 3 \cdot 16 \cdot 32 + 32 = 4{,}640$ parameters, since its input has 16 channels. The dense readout has $32 \cdot 10 + 10 = 330$ parameters. The total is $448 + 4{,}640 + 330 = 5{,}418$; the softmax only converts the ten scores to probabilities.

4.9.7 Receptive fields and dilation

The *receptive field* of a unit is the part of the original input that can affect it. Although each filter is local, stacking filters enlarges this region. Two width-three filters with stride 1 cover width $3 + (3 - 1) = 5$: the second-layer unit combines three first-layer units, whose input windows overlap. More generally, N layers of width k, stride 1, and no dilation have receptive-field width

$$r_N = 1 + N(k - 1). \tag{4.9.6}$$

Apply the same reasoning independently along the two image axes. Two 3×3 layers cover a 5×5 input region; three cover 7×7, away from the boundary.

Strides change the spacing between neighboring units measured in original input coordinates. Let j_ℓ be this spacing after layer ℓ, and let r_ℓ be the receptive-field width along one axis. Initialize $j_0 = r_0 = 1$. For a layer with filter width k_ℓ, stride s_ℓ, and no dilation,

$$j_\ell = s_\ell j_{\ell-1}, \qquad r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}. \tag{4.9.7}$$

The filter includes k_ℓ previous-layer positions, spanning $k_\ell - 1$ gaps of size $j_{\ell-1}$. This proves the second recurrence; moving to the next output advances s_ℓ such positions, proving the first. The recurrence also applies to local pooling windows. In the classifier above, the first convolution has $(r, j) = (3, 1)$, max pooling gives $(4, 2)$, and the second convolution gives $(8, 2)$.

Dilation spaces apart the input locations sampled *within* a filter. For dilation d in one dimension,

$$z_t = b + \sum_{r=0}^{k-1} K_{r+1} x_{t+dr}, \qquad k_{\text{effective}} = 1 + d(k - 1).$$

Unlike stride, dilation does not set the distance between successive output positions. A width-three filter samples offsets $0, 1, 2$ when $d = 1$, offsets $0, 2, 4$ when $d = 2$, and offsets $0, 4, 8$ when $d = 4$. These span widths $3, 5, 9$, respectively, while using the same number of learned weights [40].

For dilated layers, replace $k_\ell - 1$ in (4.9.7) by $d_\ell(k_\ell - 1)$. A stride-one stack with width 3 and dilations $1, 2, 4$ therefore has input span

$$r_3 = 1 + 2(1 + 2 + 4) = 15.$$

In general, a receptive-field span can include locations that are not sampled; dilation enlarges the span without adding filter weights. Output-size calculations likewise use the effective filter size: replace k_h, k_w in (4.9.2) by $1 + d_h(k_h - 1), 1 + d_w(k_w - 1)$, respectively.

4.9.8 Translation equivariance and invariant readout

Parameter sharing gives a precise relation between input and output translations. In one dimension, write $(C_K x)_t = b + \sum_{r=0}^{k-1} K_{r+1} x_{t+r}$ and define a translation by $(T_a x)_t = x_{t-a}$. For stride 1, on an infinite grid or a compatible periodic grid,

$$(C_K T_a x)_t = b + \sum_{r=0}^{k-1} K_{r+1} x_{t+r-a} = (T_a C_K x)_t. \tag{4.9.8}$$

Thus shifting the input shifts the feature map by the same amount. This property is called *translation equivariance*. The same calculation works in two dimensions. A coordinatewise activation preserves it, since $\rho(T_a z) = T_a \rho(z)$.

An *invariant* output instead stays unchanged after a shift. If a translation only permutes the spatial positions of an equivariant feature map, global average pooling is invariant:

$$\mathrm{GAP}(T_a h)_c = \frac{1}{HW} \sum_{u,v} (T_a h)_{u,v,c} = \frac{1}{HW} \sum_{u,v} h_{u,v,c} = \mathrm{GAP}(h)_c.$$

The middle equality is a change of summation indices. A dense classifier applied to this vector inherits its invariance.

These statements require their boundary and sampling assumptions. Zero padding at finite boundaries can break exact equivariance. With stride s, compatible input shifts by multiples of s can correspond to output shifts, while a one-pixel shift generally does not. Local max pooling may reduce sensitivity when a large feature moves within the same pooling window, but it is not exactly invariant to arbitrary translations. Image classification often benefits from an invariant readout; tasks that predict a label at each pixel usually need to retain the spatial feature map.

4.10 Modeling ordered context: recurrent networks

A catalog photograph can be classified with one label, but describing it as "a red coat" requires an ordered output. The probability of the caption, conditional on the photograph x, factors as

$$\Pr(y_1, y_2, y_3 \mid x) = \Pr(y_1 \mid x) \Pr(y_2 \mid y_1, x) \Pr(y_3 \mid y_1, y_2, x).$$

More generally, the probability of a length-T caption is $\prod_{t=1}^{T} \Pr(y_t \mid y_1, \ldots, y_{t-1}, x)$. A convolutional network can extract an image feature vector v, which initializes a recurrent state $h_0 = g(v)$. At successive steps, the recurrent network uses the previous state and an input word embedding to update its state and predict the next word. This connects spatial feature extraction to the problem of remembering ordered context.

4.10.1 Sequential prediction and a shared recurrent cell

For time-series prediction, suppose observations satisfy $x_s \in \mathbb{R}^d$. At time t, a lookback window of length p is available, and the target is τ steps ahead:

$$\widehat{y}_{t+\tau} = F_\theta(x_{t-p+1}, \ldots, x_t). \tag{4.10.1}$$

A recurrent model reads the observations in their time order rather than treating their order as interchangeable. Starting from an initial state at time $t-p$, it computes $h_s = F_{\mathrm{cell}}(h_{s-1}, x_s)$ and uses the final state to make the forecast. The same construction applies to word embeddings and other ordered input sequences.

For a general sequence $x_1, \ldots, x_T$, a plain recurrent neural network (RNN) has input dimension d, hidden dimension H, and output dimension K. Its cell is

$$u_t = A h_{t-1} + B x_t + b, \qquad h_t = \phi(u_t), \qquad a_t = C h_t + c, \tag{4.10.2}$$

where ϕ acts coordinatewise. Here

$$A \in \mathbb{R}^{H \times H}, \quad B \in \mathbb{R}^{H \times d}, \quad C \in \mathbb{R}^{K \times H}, \quad b \in \mathbb{R}^H, \quad c \in \mathbb{R}^K.$$

The vector a_t is an output score: use it directly for regression, or apply softmax to obtain class probabilities. Counting the entries of the five shared parameters gives

$$P_{\mathrm{RNN}} = H^2 + Hd + H + KH + K. \tag{4.10.3}$$

This count excludes a separately learned initial state or input encoder.

Unrolling (4.10.2) means writing one copy of the cell for each time step. The resulting computational graph has depth proportional to T, but all copies use the same A, B, C, b, c. Increasing the sequence length therefore increases the computation without increasing (4.10.3). A sequence-level prediction uses the final state; a sequence of predictions uses outputs at successive steps [9, Sections 10.1–10.2].

4.10.2 A linear RNN as a lag filter

The role of the recurrent weight can be seen without a nonlinearity. Take one scalar input and one hidden unit, use the identity activation, and set the hidden bias to zero. For each length-p window, initialize $h_{t-p} = 0$ and let

$$h_s = \gamma h_{s-1} + \beta x_s, \qquad \widehat{x}_{t+1} = \mu + h_t.$$

Substitution of the preceding state gives

$$h_t = \beta x_t + \gamma\beta x_{t-1} + \gamma^2 \beta x_{t-2} + \cdots + \gamma^{p-1}\beta x_{t-p+1}.$$

Consequently,

$$\widehat{x}_{t+1} = \mu + \sum_{j=1}^{p} \varphi_j x_{t+1-j}, \qquad \varphi_j = \beta\gamma^{j-1}. \tag{4.10.4}$$

This has the form of an AR(p) predictor, but its lag coefficients are geometrically constrained. A general AR(p) predictor chooses all p coefficients freely; this scalar RNN chooses only β and γ. The zero initialization is relevant: an arbitrary initial state would add $\gamma^p h_{t-p}$ to the forecast.

Example 4.12 (A three-step lag filter)**:** Let $p = 3$, $\beta = 1$, $\gamma = 1/2$, and suppose $(x_{t-2}, x_{t-1}, x_t) = (4, 2, 6)$. Starting from zero, the successive hidden states are

$$h_{t-2} = 4, \qquad h_{t-1} = \tfrac{1}{2}(4) + 2 = 4, \qquad h_t = \tfrac{1}{2}(4) + 6 = 8.$$

The lag-filter expression gives the same value:

$$\widehat{x}_{t+1} = \mu + 6 + \tfrac{1}{2}(2) + \tfrac{1}{4}(4) = \mu + 8.$$

The older an observation is, the more powers of γ its contribution passes through. ◇

4.10.3 Memory decay under the same inputs

To distinguish memory of an earlier state from the effect of new data, compare two trajectories h_t, h'_t driven by the *same* input sequence. Suppose ϕ is L_ϕ-Lipschitz in Euclidean norm. Then

$$\begin{aligned} \|h_t - h'_t\|_2 &= \left\|\phi(Ah_{t-1} + Bx_t + b) - \phi(Ah'_{t-1} + Bx_t + b)\right\|_2 \\ &\le L_\phi \left\|A(h_{t-1} - h'_{t-1})\right\|_2 \\ &\le L_\phi \left\|A\right\|_{\mathrm{op}} \left\|h_{t-1} - h'_{t-1}\right\|_2 . \end{aligned}$$

Writing $q = L_\phi \left\|A\right\|_{\mathrm{op}}$ and applying the inequality repeatedly proves

$$\|h_t - h'_t\|_2 \le q^t \left\|h_0 - h'_0\right\|_2 . \tag{4.10.5}$$

Thus $q < 1$ is a sufficient condition for uniform geometric forgetting of an initial-state perturbation. For coordinatewise tanh, one can take $L_\phi = 1$. The condition is sufficient, not necessary; if $q \ge 1$, this upper bound alone does not determine whether a particular trajectory forgets or amplifies a perturbation.

In the scalar linear recurrence, the difference is exactly $h_t - h'_t = \gamma^t(h_0 - h'_0)$. For $0 < \gamma < 1$, the smallest number of steps that reduces its magnitude by at least half is

$$k_{1/2} = \left\lceil \frac{\log(1/2)}{\log \gamma} \right\rceil .$$

For example, $\gamma = 0.8$ gives four steps, while $\gamma = 0.95$ gives fourteen. A stable contraction may therefore also make information from the distant past difficult to retain.

4.10.4 Backpropagation through time

Training uses ordinary backpropagation on the unrolled graph. For losses at every output, define

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t(a_t, y_t), \qquad e_t = \nabla_{a_t} \ell_t \in \mathbb{R}^K, \qquad \delta_t = \nabla_{u_t} \mathcal{L} \in \mathbb{R}^H.$$

For now, h_0 is fixed independently of the recurrent parameters. There are two outgoing routes from h_t: to the current output a_t, and to the next preactivation u_{t+1}. The chain rule therefore gives

$$\nabla_{h_t} \mathcal{L} = C^\top e_t + A^\top \delta_{t+1},$$

where $\delta_{T+1} = 0$. Since $\partial h_t / \partial u_t = \operatorname{diag}(\phi'(u_t))$, the backward recurrence is

$$\delta_t = \operatorname{diag}(\phi'(u_t))\big(C^\top e_t + A^\top \delta_{t+1}\big). \tag{4.10.6}$$

The first term is the error from the current output; the second is the error propagated from later time steps. For a loss only at the final output, take $e_t = 0$ for $t < T$.

Each appearance of a shared parameter contributes to its derivative. For example, A appears in every u_t, so its gradient is a sum of the corresponding outer products:

$$\begin{aligned} \nabla_A \mathcal{L} &= \sum_{t=1}^{T} \delta_t h_{t-1}^\top, & \nabla_B \mathcal{L} &= \sum_{t=1}^{T} \delta_t x_t^\top, \\ \nabla_C \mathcal{L} &= \sum_{t=1}^{T} e_t h_t^\top, & \nabla_b \mathcal{L} &= \sum_{t=1}^{T} \delta_t, \\ \nabla_c \mathcal{L} &= \sum_{t=1}^{T} e_t. \end{aligned} \tag{4.10.7}$$

If an image encoder or another learned map produces h_0, backpropagation continues into that map through $\nabla_{h_0} \mathcal{L} = A^\top \delta_1$.

For a loss ℓ_T depending only on the final state, let $J_s = \partial h_s / \partial h_{s-1} = \operatorname{diag}(\phi'(u_s))A$. Repeated application of the chain rule gives

$$\begin{aligned} \nabla_{h_t} \ell_T &= J_{t+1}^\top \cdots J_T^\top \nabla_{h_T} \ell_T, \\ \|\nabla_{h_t} \ell_T\|_2 &\le \left(\prod_{s=t+1}^{T} \|J_s\|_{\mathrm{op}} \right) \|\nabla_{h_T} \ell_T\|_2 . \end{aligned} \tag{4.10.8}$$

If every $\|J_s\|_{\mathrm{op}} \le q < 1$, the bound decreases as q^{T-t}. In a scalar example with constant derivative $1/2$, twenty steps multiply the gradient by $2^{-20} \approx 9.5 \times 10^{-7}$. With constant scalar derivative 1.5, the multiplier is instead $1.5^{20} \approx 3.3 \times 10^3$. These examples exhibit vanishing and exploding gradients. In multiple dimensions the actual behavior also depends on directions and on the order of the Jacobian products; a large product of norm upper bounds alone does not prove explosion. For losses at several times, their gradient contributions add [9, Sections 10.2 and 10.7].

Gradient clipping limits the norm of an update when its gradient is large. For example, a threshold $M > 0$ replaces a nonzero gradient g by $\min\{1, M/\|g\|_2\} g$. Gates address a different part of the problem by changing how the state is carried across time.

4.10.5 From exponential smoothing to a GRU

First compute the plain-RNN candidate $\widetilde{h}_t = \phi(Ah_{t-1} + Bx_t + b)$, but blend it with the old state using a fixed $\alpha \in [0, 1]$:

$$h_t = (1 - \alpha) h_{t-1} + \alpha \widetilde{h}_t. \tag{4.10.9}$$

Repeated substitution yields the exact identity

$$h_t = (1-\alpha)^t h_0 + \sum_{s=1}^{t} \alpha(1-\alpha)^{t-s}\widetilde{h}_s.$$

The weights on the initial state and all candidates sum to one. When $\alpha = 0$, the state is retained; when $\alpha = 1$, the candidate replaces it, recovering the plain RNN. An intermediate value spreads updates over time. The candidates themselves may depend on earlier states, so this expansion does not treat them as independent observations. Its full one-step state Jacobian is

$$\frac{\partial h_t}{\partial h_{t-1}} = (1-\alpha)I + \alpha\,\mathrm{diag}(\phi'(u_t))A.$$

The direct carry term $(1-\alpha)I$ supplies a route absent from the plain cell, but it does not by itself describe the full derivative.

An update gate learns a separate weight for each coordinate:

$$\begin{aligned} \alpha_t &= \sigma(A_\alpha h_{t-1} + B_\alpha x_t + b_\alpha) \in (0,1)^H, \\ h_t &= (1-\alpha_t)\odot h_{t-1} + \alpha_t \odot \widetilde{h}_t. \end{aligned} \tag{4.10.10}$$

Here $\sigma(v) = 1/(1+e^{-v})$ acts coordinatewise, and $\odot$ means coordinatewise multiplication. A gate near zero retains a coordinate; a gate near one replaces it with the candidate. With this convention, α_t is the weight on the *new* candidate. Because the gate is learned from the preceding state, the total Jacobian also contains derivatives through the gate.

A gated recurrent unit (GRU) additionally uses a reset gate to control how much old state enters the candidate:

$$\begin{aligned} \alpha_t &= \sigma(A_\alpha h_{t-1} + B_\alpha x_t + b_\alpha), \\ r_t &= \sigma(A_r h_{t-1} + B_r x_t + b_r), \\ \widetilde{h}_t &= \tanh\big(A_h(r_t \odot h_{t-1}) + B_h x_t + b_h\big), \\ h_t &= (1-\alpha_t)\odot h_{t-1} + \alpha_t \odot \widetilde{h}_t. \end{aligned} \tag{4.10.11}$$

All states and gates lie in $\mathbb{R}^H$; each A matrix is $H \times H$, each B matrix is $H \times d$, and each bias has H coordinates. If r_t is near zero, the candidate is formed mainly from the current input. This does not necessarily erase the final state: if α_t is also near zero, the update still retains h_{t-1}. If α_t is near one, it writes the candidate. In the notation of Cho et al. [39], their z_t weights the old state, so our convention is $\alpha_t = 1 - z_t$.

4.10.6 LSTM: a separate stored memory

Long short-term memory (LSTM) separates the stored cell state $c_t \in \mathbb{R}^H$ from the visible hidden state $h_t \in \mathbb{R}^H$. To avoid confusing the cell state with the output bias in (4.10.2), the symbol c_t below always has a time subscript. A standard cell computes

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & \widetilde{c}_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \end{aligned} \tag{4.10.12}$$

$$\begin{aligned} c_t &= f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \tag{4.10.13}$$

The matrices $W_\bullet$ have shape $H \times d$, the matrices $U_\bullet$ have shape $H \times H$, and the biases have H coordinates. The input gate i_t, forget gate f_t, and output gate o_t have coordinates in $(0,1)$. The candidate $\widetilde{c}_t$ has coordinates in $(-1,1)$. The original LSTM was introduced by Hochreiter and Schmidhuber [32]; the adaptive forget gate was added by Gers, Schmidhuber, and Cummins [31].

The forget and input gates are independent; their weights need not sum to one. In the limiting case $f_t = 1, i_t = 0$, the cell retains its memory. With $f_t = 0, i_t = 1$, it replaces memory by the candidate. With $f_t = i_t = 1$, it accumulates the candidate by addition. These statements apply coordinatewise when different

coordinates have different gates. An output gate near zero can hide a cell coordinate from the visible hidden state without erasing that coordinate of c_t. The exact values zero and one describe limiting gate behavior; a sigmoid at a finite input takes values strictly between them.

Repeated substitution in (4.10.13) gives

$$c_T = \left(\prod_{s=1}^{T} f_s\right) \odot c_0 + \sum_{t=1}^{T} \left(\prod_{s=t+1}^{T} f_s\right) \odot i_t \odot \widetilde{c}_t, \tag{4.10.14}$$

where every product is coordinatewise and an empty product is the all-ones vector. To see the identity, first expand

$$c_2 = f_2 \odot f_1 \odot c_0 + f_2 \odot i_1 \odot \widetilde{c}_1 + i_2 \odot \widetilde{c}_2.$$

At each subsequent step, the forget gate multiplies all existing terms and the input gate adds one new term; induction gives (4.10.14). Thus a contribution written at time t is carried forward with factor $\prod_{s=t+1}^{T} f_s$.

Holding the gates and candidate values fixed, the derivative along the direct cell-state route is

$$\left.\frac{\partial c_T}{\partial c_t}\right|_{\text{direct}} = \operatorname{diag}\left(\prod_{s=t+1}^{T} f_s\right). \tag{4.10.15}$$

Forget gates close to one can preserve this contribution over many steps; gates close to zero can erase it. This route avoids repeated application of the hidden-state weight matrix and activation derivative seen in (4.10.8). It is only one route through the graph: the gates and candidates depend on earlier hidden states, and the visible hidden state depends on the cell state. Their derivatives also enter the *total* gradient. Gated architectures supply ways to control memory and gradient propagation, but do not guarantee that all long-range dependencies will be learned.

4.11 Transformers and self-attention

4.11.1 From one context vector to weighted retrieval

A recurrent model can encode the caption "lightweight winter coat with a hood and pockets" into states $h_1, \ldots, h_T$. If an answering decoder receives only h_T, then answering "Does this coat have a hood?" requires the final state to preserve the earlier word *hood*. This is a fixed-context bottleneck: all information needed by later predictions must pass through one vector. Attention instead lets prediction step t read every encoder state through

$$c_t = \sum_{j=1}^{T} \alpha_{tj} h_j, \qquad \alpha_{tj} \geq 0, \quad \sum_{j=1}^{T} \alpha_{tj} = 1. \tag{4.11.1}$$

The weights depend on the current query and the available states, so different questions can retrieve different parts of the same caption. Reading encoder states from a decoder is *cross-attention*. In *self-attention*, the queries and the states being read come from the same sequence.

4.11.2 Queries, keys, values, and masks

Let $H \in \mathbb{R}^{T \times d_{\text{model}}}$ contain one row per token. A self-attention head uses shared linear maps

$$\begin{aligned} Q &= HW_Q \in \mathbb{R}^{T \times d_k}, & W_Q &\in \mathbb{R}^{d_{\text{model}} \times d_k}, \\ K &= HW_K \in \mathbb{R}^{T \times d_k}, & W_K &\in \mathbb{R}^{d_{\text{model}} \times d_k}, \\ V &= HW_V \in \mathbb{R}^{T \times d_v}, & W_V &\in \mathbb{R}^{d_{\text{model}} \times d_v}. \end{aligned}$$

Write q_i, k_j, v_j as column vectors obtained by transposing the corresponding rows of Q, K, V. A query q_i specifies what position i is looking for; a key k_j determines how position j matches that query; a value v_j

supplies the information retrieved from position j. Their roles are learned through the training objective. The scaled dot-product attention of Vaswani et al. [36] is

$$\mathrm{Attn}(Q, K, V; M) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V. \tag{4.11.2}$$

Here the score matrix has shape $T \times T$, the output has shape $T \times d_v$, and softmax normalizes *each row*, rather than the whole matrix. For a mask with entries 0 or $-\infty$, define the allowed keys in row i by $J_i = \{j : M_{ij} = 0\}$. Every J_i must be nonempty. Writing $s_{ij} = q_i^\top k_j/\sqrt{d_k}$, the output row $o_i^\top$ is determined by

$$\alpha_{ij} = \begin{cases} \dfrac{e^{s_{ij}}}{\sum_{r\in J_i} e^{s_{ir}}}, & j \in J_i, \\ 0, & j \notin J_i, \end{cases} \qquad o_i = \sum_{j\in J_i} \alpha_{ij} v_j. \tag{4.11.3}$$

Thus each output is a weighted combination of allowed value vectors. The query–key scores determine the weights; they are not themselves the output features.

Why divide by $\sqrt{d_k}$? For a simple scale calculation, suppose that all coordinates of q_i and k_j are mutually independent, centered, and have variance one. Then each product $q_{i\ell}k_{j\ell}$ has mean zero and variance one, and different coordinate products are independent. Consequently,

$$\mathrm{Var}(q_i^\top k_j) = \mathrm{Var}\left(\sum_{\ell=1}^{d_k} q_{i\ell}k_{j\ell}\right) = d_k, \qquad \mathrm{Var}\left(\frac{q_i^\top k_j}{\sqrt{d_k}}\right) = 1.$$

Without scaling, scores can grow in magnitude with the key dimension, making softmax strongly concentrated and its derivatives small. Scaling keeps the score variance of order one under this calculation. Learned queries and keys need not satisfy the independence assumptions; the argument motivates the scale rather than proving a distributional property of a trained network.

Blocking future positions. Unmasked encoder self-attention uses $M = 0$. For causal prediction, set $M_{ij} = 0$ for $j \le i$ and $M_{ij} = -\infty$ for $j > i$. For four positions,

$$M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Equation (4.11.3) then shows directly that the ith output uses only positions $j \le i$. The diagonal is allowed, so no row is empty. For next-token prediction, the representation at position i predicts token $i + 1$; it may read token i, but not the token it is trying to predict.

4.11.3 Position information and Transformer blocks

Shared projections and unmasked attention do not encode a token's position. If a permutation matrix P reorders the input rows, the output rows are reordered by the same P:

$$A(PQ, PK, PV) = PA(Q, K, V), \qquad A(Q, K, V) = \mathrm{Attn}(Q, K, V; 0).$$

This is *permutation equivariance*, proved in Proposition 4.15 in the optional proof section. It means that changing the order alone cannot change the output associated with a particular token when no order information is supplied. Shared position-wise maps, residual additions, and row-wise layer normalization preserve the same property.

To supply order, add a position vector p_t to each token embedding. Number feature coordinates from 0 to $d_{\mathrm{model}} - 1$ in the following formula. For even d_{model}, the original sinusoidal choice has, for $0 \le j < d_{\mathrm{model}}/2$,

$$p_{t,2j} = \sin\left(t/10000^{2j/d_{\mathrm{model}}}\right), \qquad p_{t,2j+1} = \cos\left(t/10000^{2j/d_{\mathrm{model}}}\right). \tag{4.11.4}$$

Learned position embeddings are another choice. When a token moves to a different position, the position vector added to it changes. A causal mask also supplies an order relation; the permutation statement above assumes no such mask.

Several heads and a position-wise feedforward map. Different heads learn separate projections and can attend to different relations. With r heads, each with value dimension d_v,

$$\mathrm{MHA}(H) = \mathrm{Concat}(\mathrm{head}_1, \dots, \mathrm{head}_r)W_O, \quad \mathrm{head}_s = \mathrm{Attn}(HW_Q^{(s)}, HW_K^{(s)}, HW_V^{(s)}; M), \tag{4.11.5}$$

where $W_O \in \mathbb{R}^{rd_v \times d_{\mathrm{model}}}$ makes the output dimension match the residual input. Omitting dropout from the notation, the original post-normalization encoder block is

$$\begin{aligned} U &= \mathrm{LN}(H + \mathrm{MHA}(H)), \\ H^+ &= \mathrm{LN}(U + \mathrm{FFN}(U)). \end{aligned} \tag{4.11.6}$$

Layer normalization acts separately on each row, normalizing its feature coordinates with a positive numerical stabilizer and then applying learned scale and offset parameters. The feedforward sublayer applies the same two-layer map $u \mapsto W_2\rho(W_1u + b_1) + b_2$ to each position, interpreting u as a column vector. Attention mixes positions; this map transforms features within each position. Residual paths add each sublayer's input to its output. The original decoder also contains masked self-attention and cross-attention, with decoder queries and encoder keys and values [36].

4.11.4 The product photo as a sequence of patches

Self-attention can process image patches as well as words. Resize a product photo to $224 \times 224 \times 3$ and split it into nonoverlapping 16×16 patches. The image has

$$T = (224/16)^2 = 196 \quad \text{patches}, \qquad 16 \cdot 16 \cdot 3 = 768 \quad \text{values per patch.} \tag{4.11.7}$$

Flatten patch j into $r_j \in \mathbb{R}^{768}$ and apply the same affine projection $e_j = Er_j + b \in \mathbb{R}^D$, with $D = d_{\mathrm{model}}$, to every patch. Adding position embeddings p_j preserves information about where the patch came from. The construction is therefore

$$(e_j + p_j)_{j=1}^{196} \longmapsto \text{Transformer encoder} \longmapsto (z_j)_{j=1}^{196} \longmapsto \text{pooled representation and class scores.}$$

For example, mean pooling gives $\bar{z} = 196^{-1}\sum_{j=1}^{196} z_j$, followed by a linear classification head. The original Vision Transformer (ViT) uses an additional learned classification token whose final representation feeds the head, giving 197 tokens in this example [41]; the slide's pooling diagram shows the same patch-sequence idea without that extra token.

The shared patch projection is also implementable as a convolution with D output channels, a 16×16 kernel, and stride 16, with no padding: each output location receives exactly one patch. Full self-attention then allows any patch to interact with any other patch in a single layer, so it can relate, for example, the coat's hood near the top of the image to pockets farther below.

4.11.5 Which architecture fits the needed relation?

Model	Parameters shared over	Main relation in one layer
Dense feedforward	No spatial or temporal weight tying by default	Learned feature composition
Convolutional	Image locations	Local neighborhoods; translation equivariance
RNN and LSTM	Time steps	Sequential state and ordered history
Transformer and ViT	Tokens or image patches	Content-dependent interactions among allowed positions

Dense attention forms a $T \times T$ score matrix, so materializing those scores uses quadratic memory in the sequence length. All known input positions can be processed in parallel within an attention layer, including with a causal mask, whereas recurrent states depend sequentially on preceding states. This parallelism does not eliminate the sequential dependence of autoregressive generation: producing the next token still requires the previously generated tokens.

Returning to catalog photos, a dense ReLU network supplies nonlinear class scores and backpropagation supplies their gradients. Augmentation and regularization influence how well the fitted predictor works on new photos. A CNN shares local visual detectors across image locations. An RNN or LSTM tracks ordered history in the product caption, while attention relates its words or the photo's patches directly. These are different structural assumptions about the task, rather than a ranking of architectures that is valid for every data set.

For the product-photo classifier, distinguish a feature map from a prediction. Under the translation and boundary conditions described in the convolution section, convolution moves a feature map when the image moves: this is equivariance. Global pooling over all spatial positions can remove that position dependence and make the resulting prediction invariant. Cropping, padding, and strides can affect these properties, so the ideal statement requires the corresponding conditions on the shift.

4.12 Optional mathematical arguments

These arguments expand the optional proof slides. They support the claims in the main text without changing its progression through the architectures.

4.12.1 ReLU parameter symmetries

Proposition 4.13 (Different parameters, the same ReLU feature)**.** *For any $c > 0$,*

$$a\,\rho(w^\top x + b) = \frac{a}{c}\,\rho((cw)^\top x + cb).$$

Permuting hidden units and their corresponding output weights also leaves the network function unchanged.

Proof. Positive homogeneity gives $\rho(cu) = c\rho(u)$. The sum over hidden units is unaffected by reordering its terms. □

The map from parameters to functions is therefore many-to-one even in a small network. Parameter distance need not measure prediction distance, and two runs can produce different weights for the same predictor. Penalties on the parameters can distinguish between such equivalent representations; this is one reason the chosen regularizer is part of the model specification.

4.12.2 Depth separation via the tent map

Depth can make some particular functions much cheaper to represent. Define the tent map on $[0, 1]$ by

$$\tau(x) = 2\rho(x) - 4\rho(x - \tfrac{1}{2}) + 2\rho(x - 1) = \begin{cases} 2x, & 0 \le x \le \frac{1}{2}, \\ 2 - 2x, & \frac{1}{2} \le x \le 1. \end{cases} \tag{4.12.1}$$

Its k-fold composition $\tau^{\circ k}$ has 2^k linear pieces on $[0, 1]$. Composing k copies of the fixed-width network in (4.12.1) represents it with depth growing linearly in k.

Proposition 4.14 (A one-dimensional depth separation)**.** *If a one-hidden-layer scalar ReLU network g with m units satisfies $\sup_{x\in[0,1]} |g(x) - \tau^{\circ k}(x)| < 1/2$, then $m \ge 2^{k-1}$. Exact representation requires $m \ge 2^k - 1$.*

Proof. The target has 2^{k-1} tent peaks with disjoint interiors. On each peak interval its endpoint values are 0, and its midpoint value is 1. If g were affine throughout one such interval, its midpoint value would be the mean of its endpoint values. Errors strictly below $1/2$ at the two endpoints and midpoint make that impossible. Thus g needs an interior breakpoint in every peak interval. A one-hidden-layer ReLU network has at most one breakpoint per unit. For exact representation, each of the $2^k - 1$ changes of slope also needs a breakpoint. □

This illustrates a specific depth separation, not a claim that every deep model is better than every shallow one; see also [35].

4.12.3 Why logistic loss keeps increasing the margin

For a fixed separating vector v, put $\gamma_i = \widetilde{y}_i v^\top x_i > 0$ and $\gamma_{\min} = \min_i \gamma_i$. Along the ray $w = rv$,

$$F(rv) = \frac{1}{n}\sum_{i=1}^{n} \log(1 + e^{-r\gamma_i}).$$

For $r \geq 0$, the bound $u/2 \leq \log(1+u) \leq u$ for $0 \leq u \leq 1$ gives

$$\frac{1}{2n} e^{-r\gamma_{\min}} \leq F(rv) \leq e^{-r\gamma_{\min}}.$$

The upper bound sums n terms bounded by $e^{-r\gamma_{\min}}$; the lower bound keeps one term attaining the smallest margin. Hence every separating ray can drive the loss to zero as its norm grows, and its smallest margin controls the exponential rate. This calculation motivates maximum-margin selection, but does not prove that gradient descent chooses the optimal direction. That dynamical statement is the result cited from [34].

4.12.4 Proof of minimum-norm selection by gradient descent

We prove Theorem 4.10.

Proof. Take a thin singular-value decomposition $X = U \operatorname{diag}(s_1, \ldots, s_n) V^\top$, with every $s_j > 0$. Since every update lies in $\operatorname{range}(X^\top) = \operatorname{range}(V)$, $\theta_t = V\alpha_t$. In coordinate j,

$$\alpha_{t+1,j} = \left(1 - \frac{\eta s_j^2}{n}\right)\alpha_{t,j} + \frac{\eta s_j}{n}(U^\top y)_j.$$

The multiplier lies in $[0, 1)$, so $\alpha_{t,j} \to (U^\top y)_j / s_j$. This is exactly (4.8.1). Every interpolant is $\theta^\dagger + v$ with $v \in \ker X$. The two summands are orthogonal, so $\|\theta^\dagger + v\|_2^2 = \|\theta^\dagger\|_2^2 + \|v\|_2^2$. □

4.12.5 The exact linear dropout calculation

Fix (x, y), let M_j be independent Bernoulli variables with retention probability $0 < q \leq 1$, and write $A = w^\top \widetilde{x}$, where $\widetilde{x}_j = M_j x_j / q$. Then

$$\mathbb{E}_M A = w^\top x, \qquad \operatorname{Var}_M(A) = \sum_j w_j^2 x_j^2 \operatorname{Var}(M_j/q) = \frac{1-q}{q}\sum_j w_j^2 x_j^2.$$

In the variance equality, cross-covariances vanish by independence. Write $y - A = (y - \mathbb{E}_M A) - (A - \mathbb{E}_M A)$. Squaring and taking expectation eliminates the centered cross term, so

$$\mathbb{E}_M \tfrac{1}{2}(y - A)^2 = \tfrac{1}{2}(y - \mathbb{E}_M A)^2 + \tfrac{1}{2}\operatorname{Var}_M(A).$$

Substitution proves (4.7.5). Averaging over training examples gives the quadratic penalty

$$\frac{1-q}{2q}\sum_{j=1}^{d} w_j^2 \left(\frac{1}{n}\sum_{i=1}^{n} X_{ij}^2\right).$$

Coordinates with larger empirical second moment receive a larger penalty in this particular linear square-loss model.

4.12.6 Permutation equivariance of attention

Proposition 4.15 (Permutation equivariance without positions)**.** *Let $Q, K \in \mathbb{R}^{T\times d_k}$, $V \in \mathbb{R}^{T\times d_v}$, and $A(Q,K,V) = \mathrm{Attn}(Q,K,V;0)$. For any $T \times T$ permutation matrix P,*

$$A(PQ, PK, PV) = PA(Q,K,V). \tag{4.12.2}$$

In particular, self-attention with shared projection matrices is permutation equivariant when it has no position information or mask.

Proof. Write $S = QK^\top/\sqrt{d_k}$ for the score matrix. Since $P^\top P = I$, permuting queries and keys gives

$$(PQ)(PK)^\top/\sqrt{d_k} = PSP^\top.$$

To check row softmax explicitly, let π be the permutation such that $(Pu)_i = u_{\pi(i)}$. Then

$$[\mathrm{softmax}_{\mathrm{row}}(PSP^\top)]_{ij} = \frac{\exp(S_{\pi(i),\pi(j)})}{\sum_{k=1}^{T}\exp(S_{\pi(i),\pi(k)})} = \frac{\exp(S_{\pi(i),\pi(j)})}{\sum_{r=1}^{T}\exp(S_{\pi(i),r})}.$$

The last equality uses the fact that π is a bijection. Hence

$$\mathrm{softmax}_{\mathrm{row}}(PSP^\top) = P\,\mathrm{softmax}_{\mathrm{row}}(S)P^\top.$$

Multiplying by PV gives

$$A(PQ,PK,PV) = P\,\mathrm{softmax}_{\mathrm{row}}(S)P^\top PV = PA(Q,K,V).$$

For self-attention, $(PH)W_Q = P(HW_Q)$, and likewise for keys and values, so the identity applies to permuting the input rows. The same argument applies head by head in multi-head attention. Shared position-wise maps and row-wise layer normalization commute with row permutations; residual addition does as well. A block built from these operations therefore retains equivariance when no position information or mask is present. Adding fixed position embeddings or holding a causal mask fixed when tokens are permuted falls outside these assumptions. □

Lecture 5

Reinforcement learning: MDPs, planning, and learning

5.1 The big picture: planning, learning, and policy optimization

Reinforcement learning studies decisions whose consequences extend into the future. An agent observes a state S_t, chooses an action A_t, and receives a reward R_{t+1} and a next state S_{t+1}. Its policy specifies how it chooses actions. With bounded rewards and an initial state distribution ρ, a common objective is the expected discounted return

$$J_\rho(\pi) = \mathbb{E}_{\rho,\pi}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right], \qquad \max_\pi J_\rho(\pi), \qquad 0 < \gamma < 1.$$

The policy affects both the rewards and the states encountered later. Write P for the transition probabilities and r for the expected one-step rewards. These are components of a Markov decision process (MDP), which we will define precisely below. The first question is whether these quantities are known. The second is what the algorithm learns or computes: a model, values, or a policy. Figure 5.1 organizes the main routes.

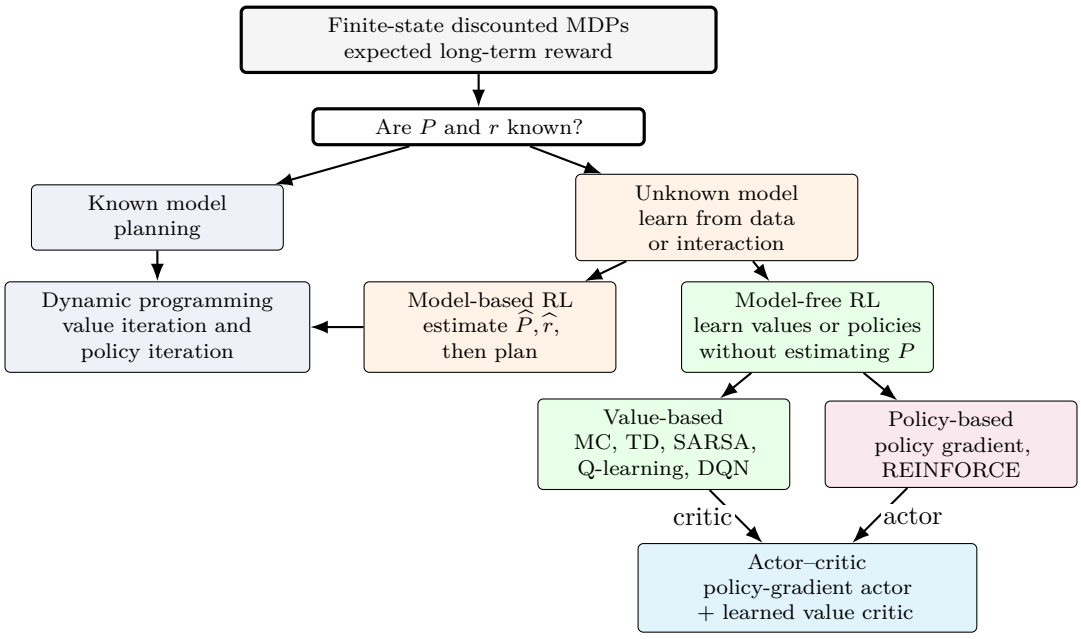


Figure 5.1: Roadmap for the RL module. A learned model feeds into the same planning machinery as a known model. Value learning supplies policy-evaluation and control tools. Actor–critic combines policy and value learning. The branches describe a teaching route; methods can combine these ideas.

What changes from supervised learning?

The applications in the slides illustrate the same interaction pattern. In a board game, a state describes the board, an action is a move, and the eventual outcome supplies a reward. In an Atari game, observations can be images and actions are controller inputs. In robot locomotion, motor commands influence subsequent motion and stability. In each case, a useful decision is judged by its future consequences as well as its immediate reward.

In supervised learning, the training examples are usually modeled as independent draws from a distribution that does not depend on the fitted predictor. In RL, actions influence which observations become available. An agent must learn about actions while also deciding which actions to take. Moreover, the reward for a useful decision may arrive several steps later.

Feature	Supervised learning	Reinforcement learning
Data law	Usually exogenous and i.i.d.	Dependent trajectories influenced by the behavior policy
Feedback	A label or loss for an example	Rewards that can be delayed across decisions
Objective	Expected prediction loss	Expected cumulative reward
Additional issues	Generalization from a sample	Exploration, coverage, and temporal credit assignment

Known model: planning

When both P and r are known, we can compute expected future rewards without collecting data to estimate the environment. Dynamic programming decomposes a long decision problem into one-step decisions and continuation values. We focus throughout on infinite-horizon discounted MDPs. Value iteration and policy iteration compute their values and optimal policies. This is the starting point of Puterman's treatment [23]: first understand the decision problem and the equations that characterize its solution.

Unknown model: model-based and model-free learning

A model-based learner estimates $\widehat{P}$ and $\widehat{r}$ from observations, then plans in the estimated model. Thus the planning algorithms remain useful when the model is learned. Their decisions depend on how accurately the data describe the relevant state–action pairs.

A model-free learner instead updates values or policies directly from experience, without first constructing a transition model. A state value $V^\pi(s)$ measures the expected return from s under policy π; an action value $Q^\pi(s, a)$ fixes the first action and then follows π. Monte Carlo (MC) and temporal-difference (TD) methods estimate a fixed policy's values. They are policy-evaluation tools that can support control when combined with policy improvement. Q-learning directly estimates optimal action values, from which a greedy policy can be extracted. The behavior policy controls which pairs are observed, so exploration and coverage are part of the learning problem [28, Chapters 5–6].

Policy optimization and actor–critic

Another route represents the policy itself by parameters $\pi_\theta(a \mid s)$ and adjusts θ to increase $J_\rho(\pi_\theta)$. Policy-gradient methods estimate the gradient of expected return from experience. REINFORCE uses sampled returns to weight changes in action probabilities. These methods connect directly to the stochastic-gradient methods studied earlier, although the policy also affects the distribution of future observations.

Actor–critic combines the two learned objects. The actor is the parameterized policy. The critic estimates values or advantages that guide the actor's update. A value-based method can also use a neural network, as in a deep Q-network (DQN), while an actor–critic method may use networks for both components. Function approximation is therefore a choice of representation across several routes, rather than a separate model-access category [28].

Where this lecture fits

We begin with the discounted MDP model, fixed-policy values and occupancies, and Bellman equations. Optimality then leads to value iteration and policy iteration. Generative-model access provides the first learning setting: estimate an MDP from samples and plan in that model. After proving a sample-complexity bound, we distinguish the other data-access settings and on-policy versus off-policy learning. MC and TD introduce sampled policy evaluation. Function approximation precedes control, including SARSA, Q-learning, and DQN. Importance sampling addresses off-policy evaluation, and policy gradients and actor–critic complete the lecture.

5.2 Sequential decisions and discounted Markov decision processes

A supervised learning problem asks for predictions on new inputs. In a sequential decision problem, an agent chooses an action, receives a reward, and observes a new state. Its action changes which states and rewards it can encounter later. Maximizing the immediate reward need not maximize the total reward: a useful action may first prepare for a better future opportunity. The objective therefore has to account for the consequences of a sequence of decisions.

We develop this idea using finite, infinite-horizon discounted Markov decision processes. The model and planning results follow Puterman [23, Chapters 2, 5–6], with the reward timing and value-function notation of Sutton and Barto [28, Chapters 3–4]. The progression from the model to policy evaluation, optimality, and planning also follows Nan Jiang's introductory MDP material [48, 46, 47]. Later sections ask how these calculations can be performed when the model is unknown.

5.2.1 States, actions, transitions, and rewards

Definition 5.1 (Finite discounted Markov decision process)**.** A finite discounted Markov decision process (MDP) consists of a finite state set $\mathcal{S}$, a finite nonempty action set $\mathcal{A}$, a transition law $P(s' \mid s, a)$, an expected one-step reward $r(s, a)$, and a discount factor $\gamma \in (0, 1)$. At time t, the agent observes S_t, chooses $A_t \in \mathcal{A}$, and then observes a reward R_{t+1} and the next state S_{t+1}. The conditional joint law of (R_{t+1}, S_{t+1}), given the history and the current action, depends only on (S_t, A_t), and does not depend on t. In particular,

$$\mathbb{P}(S_{t+1} = s' \mid H_t, A_t = a) = P(s' \mid S_t, a), \tag{5.2.1}$$

$$\mathbb{E}[R_{t+1} \mid H_t, A_t = a] = r(S_t, a), \tag{5.2.2}$$

where $H_t = (S_0, A_0, R_1, \ldots, A_{t-1}, R_t, S_t)$. We assume the rewards are uniformly bounded.

The state must retain the information needed to predict the next reward and state after an action. The Markov assumption does not assert that successive states are independent. For example, after an inventory order and a random demand, the next inventory depends on the current inventory. It need not depend on earlier inventories once the present state contains all relevant information. If demand also depends on the season, then the state may need to include the season.

The distinction between R_{t+1} and $r(s, a)$ is important. The former is one observed reward; the latter is its conditional mean. The index $t + 1$ records that the reward arrives after action A_t. The reward and next state may share randomness. Their conditional independence is not part of the MDP assumption and is not needed for the Bellman equations.

A transition law gives a probability distribution for each state–action pair. For example, in a three-state model with states s_0, s_1, s_2, the row

$$\big(P(s_0 \mid s_2, a_0), P(s_1 \mid s_2, a_0), P(s_2 \mid s_2, a_0)\big) = (0.4, 0, 0.6)$$

means that action a_0 at s_2 moves to s_0 with probability 0.4, remains at s_2 with probability 0.6, and cannot move directly to s_1. Different visits to the same state can therefore lead to different next states, even under the same action.

5.2.2 Decision rules and policies

A policy specifies how actions are chosen. A general randomized policy is a sequence $\pi = (\pi_t)_{t\geq 0}$, where $\pi_t(a \mid H_t)$ is a probability distribution on $\mathcal{A}$. It may use the observed history, but not future observations. A policy is *Markov* if its decision rule depends only on the current state and time, $\pi_t(a \mid S_t)$. It is *stationary* if the same rule $\pi(a \mid s)$ is used at every time. A deterministic stationary policy chooses a single action $\pi(s) \in \mathcal{A}$ at each state. We regard it as a stationary randomized policy whose action distribution is concentrated at $\pi(s)$.

These distinctions describe the decision rule rather than the environment. A stationary policy can produce random trajectories in a stochastic MDP. The Markov assumption on the environment also does not force the agent to ignore history. We will evaluate stationary randomized policies first, then show that one deterministic stationary policy is optimal even when all history-dependent randomized policies are allowed.

5.2.3 Why discount future rewards?

Without discounting, a system that produces reward 1 at every step would have an infinite total reward. Discounting assigns weight γ^t to the reward received after t transitions, so the same sequence has total reward $1 + \gamma + \gamma^2 + \cdots = 1/(1-\gamma)$. More generally, if $|R_{t+1}| \leq R_{\max}$ almost surely, then for every policy and initial state,

$$\mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t |R_{t+1}| \,\middle|\, S_0 = s\right] \leq \frac{R_{\max}}{1-\gamma}.$$

Thus the expected discounted objective is well defined, and its distant future can be approximated with a controlled error.

Discounting has two further mathematical consequences that we will prove: a deterministic stationary optimal policy exists, and the Bellman operators contract errors by a factor γ. A smaller γ therefore gives faster convergence of value iteration, while also changing which rewards the objective emphasizes. An economic interpretation is the *time value of money*. In this lecture, discounting primarily provides a bounded objective and a useful contraction structure.

For the remaining analysis, assume $0 \leq R_{t+1} \leq 1$ almost surely. A known bounded reward interval can be reduced to this one by an affine transformation. Because every trajectory has the same infinite sequence of discounted time weights, shifting all rewards by a constant changes every policy value by the same constant divided by $1-\gamma$, and positive scaling preserves the ordering of policies. An absorbing terminal state is represented by zero reward and a self-transition thereafter.

5.3 Discounted policy evaluation

5.3.1 Expected state values and action values

Fix a stationary randomized policy π. Its *state-value function* is

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, S_0 = s\right]. \tag{5.3.1}$$

Here and below, conditioning on an initial state means that we start the process at that state. This defines the value even if that state would have probability zero under some other initial distribution. Bounded rewards imply $0 \leq V^\pi(s) \leq 1/(1-\gamma)$.

The *action-value function* fixes the first action and uses π from the next decision onward:

$$Q^\pi(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, S_0 = s,\ A_0 = a,\ \pi \text{ thereafter}\right]. \tag{5.3.2}$$

This is a specified initial experiment, so it is meaningful even if $\pi(a \mid s) = 0$. Averaging that first action according to π gives

$$V^\pi(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s) Q^\pi(s,a). \tag{5.3.3}$$

The two functions answer different questions: $V^\pi(s)$ evaluates following the policy from s, whereas $Q^\pi(s,a)$ evaluates choosing a now and following the policy afterward.

Truncating the sum after H rewards produces an error bounded by

$$0 \le V^\pi(s) - \mathbb{E}_\pi\left[\sum_{t=0}^{H-1} \gamma^t R_{t+1} \,\middle|\, S_0 = s\right] \le \frac{\gamma^H}{1-\gamma}. \tag{5.3.4}$$

The inequality follows by bounding every omitted reward by 1 and summing a geometric series. If $(1-\gamma)\varepsilon < 1$, it is enough to take an integer

$$H \ge \frac{\log(1/((1-\gamma)\varepsilon))}{-\log\gamma}$$

to make the tail at most ε. Since $-\log\gamma \sim 1-\gamma$ as $\gamma \uparrow 1$, a discount factor close to 1 requires looking farther into the future. Here H is only an approximation parameter for the same infinite-horizon objective.

5.3.2 The Bellman equation for a fixed policy

Theorem 5.2 (Bellman expectation equation)**.** *For every stationary randomized policy π,*

$$V^\pi(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s)\left[r(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s' \mid s,a) V^\pi(s')\right]. \tag{5.3.5}$$

Proof. Separate the first reward from the remaining discounted sum:

$$V^\pi(s) = \mathbb{E}_\pi\left[R_1 + \gamma \sum_{k=0}^{\infty} \gamma^k R_{k+2} \,\middle|\, S_0 = s\right].$$

Let $H_1 = (S_0, A_0, R_1, S_1)$ be the information after the first transition. Conditional on H_1, the future evolves from S_1 according to the same transition and reward law as at time 0. Moreover, stationarity means that future actions use the same rule $\pi(\cdot \mid S_t)$. Consequently,

$$\mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{k+2} \,\middle|\, H_1\right] = V^\pi(S_1).$$

This is where both the Markov property and the stationary policy are used. Bounded discounted sums justify passing from finite partial sums to the infinite sum in these conditional expectations. The tower property now gives

$$V^\pi(s) = \mathbb{E}_\pi[R_1 + \gamma V^\pi(S_1) \mid S_0 = s].$$

Given $S_0 = s$, the first action has distribution $\pi(\cdot \mid s)$. Conditional on that action being a, the first reward has mean $r(s,a)$, and S_1 has distribution $P(\cdot \mid s,a)$. Linearity of expectation yields (5.3.5). There is no product of the reward and next-state value, so this argument does not require their conditional independence. □

5.3.3 A linear system and discounted state occupancy

Fix an ordering of the states and regard values as column vectors. Define the reward vector and transition matrix induced by π by

$$r^\pi(s) = \sum_a \pi(a \mid s) r(s,a), \qquad P^\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a). \tag{5.3.6}$$

Each row of P^π is a probability distribution. The Bellman equation therefore becomes

$$V^\pi = r^\pi + \gamma P^\pi V^\pi, \qquad (I - \gamma P^\pi) V^\pi = r^\pi. \tag{5.3.7}$$

Proposition 5.3 (Discounted resolvent)**.** *The matrix $I - \gamma P^\pi$ is invertible, and*

$$(I - \gamma P^\pi)^{-1} = \sum_{t=0}^{\infty} \gamma^t (P^\pi)^t, \qquad V^\pi = (I - \gamma P^\pi)^{-1} r^\pi. \tag{5.3.8}$$

Proof. For any vector z, row stochasticity gives $\|P^\pi z\|_\infty \le \|z\|_\infty$, so the induced matrix norm of $(P^\pi)^t$ is at most 1. The displayed series thus converges in matrix norm. If $M_H = \sum_{t=0}^{H} \gamma^t (P^\pi)^t$, then the finite geometric identity gives

$$(I - \gamma P^\pi) M_H = M_H (I - \gamma P^\pi) = I - \gamma^{H+1} (P^\pi)^{H+1}.$$

The last term tends to zero in norm. Taking the limit proves that the series is a two-sided inverse. Multiplying (5.3.7) by this inverse gives the value formula. □

The (s, s') entry of $(P^\pi)^t$ is the probability of being at s' at time t, starting at s. Thus the inverse matrix also records expected discounted visits. Define the normalized *discounted state occupancy* by

$$d_s^\pi(s') = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_\pi(S_t = s' \mid S_0 = s) = (1-\gamma)[(I - \gamma P^\pi)^{-1}]_{ss'}. \tag{5.3.9}$$

It includes the visit at time 0. Its entries are nonnegative and sum to $(1-\gamma)\sum_{t\ge 0}\gamma^t = 1$, so it is a probability distribution. The resolvent expression for the value gives

$$V^\pi(s) = \frac{1}{1-\gamma} \sum_{s' \in \mathcal{S}} d_s^\pi(s') r^\pi(s'). \tag{5.3.10}$$

The value is therefore the expected reward under discounted visitation, multiplied by the total discounted time $1/(1-\gamma)$.

Exercise 5.1 (Flow equation for state occupancy): Starting from (5.3.9), prove that for every s',

$$d_s^\pi(s') = (1-\gamma)\mathbf{1}\{s' = s\} + \gamma \sum_{x \in \mathcal{S}} P^\pi(x, s') d_s^\pi(x). \tag{5.3.11}$$

The first term accounts for the initial visit, and the second term for visits reached through a transition. Explain why the transition matrix acts on the occupancy distribution with this ordering of its indices.

Example 5.4 (A recurrent two-state calculation)**:** Suppose a fixed policy induces, in state order (s_1, s_2),

$$P^\pi = \begin{pmatrix} 1/2 & 1/2 \\ 0 & 1 \end{pmatrix}, \qquad r^\pi = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad \gamma = 0.9.$$

State s_2 produces reward 1 at every step, so its Bellman equation is $V^\pi(s_2) = 1 + 0.9 V^\pi(s_2)$, giving $V^\pi(s_2) = 10$. For s_1, the immediate reward is zero and the next state is equally likely to be s_1 or s_2. Hence

$$V^\pi(s_1) = 0.45 V^\pi(s_1) + 0.45 V^\pi(s_2), \qquad V^\pi(s_1) = \frac{90}{11} \approx 8.18.$$

The positive value at s_1 comes entirely from future rewards after reaching s_2, although the process may first remain at s_1 for several steps. ◇

5.3.4 Iterative evaluation of a fixed policy

Solving a linear system is one way to evaluate a policy. A second way is to update a proposed value function repeatedly. For a continuation value $v \in \mathbb{R}^{|\mathcal{S}|}$, define the policy Bellman operator

$$(\mathcal{T}^\pi v)(s) = r^\pi(s) + \gamma \sum_{s'} P^\pi(s, s') v(s'). \tag{5.3.12}$$

It assigns the true expected immediate reward to the first step, and assigns the proposed value v to the next state. The Bellman equation says that the correct value satisfies $\mathcal{T}^\pi V^\pi = V^\pi$.

Proposition 5.5 (Fixed-policy value iteration)**.** *For any initial vector v_0, let $v_{k+1} = \mathcal{T}^\pi v_k$. Then V^π is the unique fixed point of $\mathcal{T}^\pi$, and*

$$\|v_k - V^\pi\|_\infty \le \gamma^k \|v_0 - V^\pi\|_\infty . \tag{5.3.13}$$

In particular, with $v_0 = 0$, the error is at most $\gamma^k/(1-\gamma)$.

Proof. For any u, v, each row of P^π averages their coordinatewise differences. Consequently,

$$\|\mathcal{T}^\pi u - \mathcal{T}^\pi v\|_\infty = \gamma \max_s \left| \sum_{s'} P^\pi(s,s')(u(s') - v(s')) \right| \le \gamma \|u - v\|_\infty .$$

If u and v were two fixed points, their distance would be at most γ times itself. Since $\gamma < 1$, that distance must be zero. Existence is already given by the expected value V^π. Applying the same inequality with $u = v_k$ and $v = V^\pi$ bounds the error after one update by γ times the previous error. Repeating proves (5.3.13). The zero-initialized bound uses $\|V^\pi\|_\infty \le 1/(1-\gamma)$. □

All coordinates on the right-hand side use the old vector v_k. One such update is a Bellman step; it is not a full policy evaluation. The proposition explains why repeated Bellman steps converge to the solution of the linear system.

5.3.5 Bellman equations and iteration for action values

After a forced initial action a in state s, the expected first reward is $r(s,a)$, and the next state has law $P(\cdot \mid s,a)$. The continuation follows π. Therefore the same conditional expectation argument as above gives

$$\begin{aligned} Q^\pi(s,a) &= r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s') \\ &= r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s') Q^\pi(s',a'). \end{aligned} \tag{5.3.14}$$

The first equality uses a state value for the continuation. The second uses (5.3.3) to express the equation entirely in action values.

Define a Bellman operator on arrays indexed by state–action pairs:

$$(\mathcal{T}_Q^\pi q)(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s') q(s',a'). \tag{5.3.15}$$

For such arrays the norm $\|q\|_\infty$ means $\max_{s,a} |q(s,a)|$.

Proposition 5.6 (Fixed-policy action-value iteration)**.** *The operator $\mathcal{T}_Q^\pi$ is a γ-contraction in the supremum norm and has unique fixed point Q^π. For every initial array q_0, the updates $q_{k+1} = \mathcal{T}_Q^\pi q_k$ satisfy*

$$\|q_k - Q^\pi\|_\infty \le \gamma^k \|q_0 - Q^\pi\|_\infty . \tag{5.3.16}$$

For $q_0 = 0$, the right-hand side is at most $\gamma^k/(1-\gamma)$.

Proof. The nonnegative weights $P(s' \mid s,a)\pi(a' \mid s')$ sum to one over (s',a'). For any arrays $q, \widetilde{q}$,

$$\begin{aligned} |(\mathcal{T}_Q^\pi q)(s,a) - (\mathcal{T}_Q^\pi \widetilde{q})(s,a)| &\le \gamma \sum_{s',a'} P(s' \mid s,a)\pi(a' \mid s') |q(s',a') - \widetilde{q}(s',a')| \\ &\le \gamma \|q - \widetilde{q}\|_\infty . \end{aligned}$$

Taking the maximum proves contraction. Equation (5.3.14) gives the fixed point Q^π. The uniqueness and repeated error estimate then follow exactly as for fixed-policy value iteration. Finally, $0 \le Q^\pi \le 1/(1-\gamma)$. □

5.3.6 Discounted state–action occupancy

The state occupancy counts where a policy spends its discounted time. To also record its actions, define

$$\begin{aligned} d_s^\pi(s', a') &:= (1-\gamma)\sum_{t=0}^{\infty} \gamma^t \mathbb{P}_\pi(S_t = s', A_t = a' \mid S_0 = s) \\ &= d_s^\pi(s')\pi(a' \mid s'). \end{aligned} \tag{5.3.17}$$

We use the number of arguments to distinguish the joint occupancy from its state marginal. It is a probability distribution on $\mathcal{S} \times \mathcal{A}$, and averaging rewards under it gives

$$V^\pi(s) = \frac{1}{1-\gamma} \sum_{s',a'} d_s^\pi(s', a') r(s', a'). \tag{5.3.18}$$

Indeed, conditional on (S_t, A_t), the reward has mean $r(S_t, A_t)$; summing these expectations proves the identity.

Exercise 5.2 (Flow equation for state–action occupancy): Prove that, for every (s', a'),

$$d_s^\pi(s', a') = \pi(a' \mid s') \left[(1-\gamma)\mathbf{1}\{s' = s\} + \gamma \sum_{x,b} P(s' \mid x, b) d_s^\pi(x, b) \right]. \tag{5.3.19}$$

For action values, a different initial experiment is required: the first action is forced rather than drawn from π. Define its occupancy by

$$d_{s,a}^\pi(s', a') := (1-\gamma)\sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s', A_t = a' \mid S_0 = s, A_0 = a,\ \pi \text{ thereafter}). \tag{5.3.20}$$

It again has total mass one. Separating the initial visit from all later visits yields

$$d_{s,a}^\pi(s', a') = (1-\gamma)\mathbf{1}\{(s', a') = (s, a)\} + \gamma \sum_x P(x \mid s, a) d_x^\pi(s', a'). \tag{5.3.21}$$

At time 0 all mass is on the chosen pair (s, a); after one transition, the usual policy occupancy starts from the resulting state x. The reward representation is consequently

$$Q^\pi(s, a) = \frac{1}{1-\gamma} \sum_{s',a'} d_{s,a}^\pi(s', a') r(s', a'). \tag{5.3.22}$$

In general, $d_{s,a}^\pi(s', a')$ cannot be replaced by $d_s^\pi(s')\pi(a' \mid s')$, because the latter also draws the initial action from π. Averaging the forced-action occupancy over a with weights $\pi(a \mid s)$ recovers $d_s^\pi(s', a')$, consistently with $V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$.

5.4 Bellman optimality and stationary optimal policies

Policy evaluation fixes the action rule. Control seeks a rule with the largest value. For this purpose, allow every admissible policy, including randomized policies that use the entire observed history. For such a policy, $V^\pi(s)$ still denotes the expected discounted sum starting at time 0 from s. Define

$$V^*(s) = \sup_\pi \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, S_0 = s \right]. \tag{5.4.1}$$

A priori, this supremum need not be attained, nor need one policy attain it at all states. We will obtain both properties from the Bellman operator.

5.4.1 Monotonicity and contraction

The optimality operator replaces the policy average over actions by a maximum:

$$(\mathcal{T}v)(s) = \max_{a\in\mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s') \right]. \tag{5.4.2}$$

Every maximum is attained because the action set is finite. For vectors, $u \le v$ means that the inequality holds at every state.

Lemma 5.7 (Bellman operator properties)**.** *Both $\mathcal{T}^\pi$ and $\mathcal{T}$ are monotone. They are also γ-contractions:*

$$\|\mathcal{T}^\pi u - \mathcal{T}^\pi v\|_\infty \le \gamma \|u - v\|_\infty, \qquad \|\mathcal{T}u - \mathcal{T}v\|_\infty \le \gamma \|u - v\|_\infty. \tag{5.4.3}$$

For every policy π and vector v, $\mathcal{T}^\pi v \le \mathcal{T}v$.

Proof. Transition probabilities are nonnegative, so replacing u by a coordinatewise larger vector cannot decrease the expected continuation value of any action. Averaging or maximizing over actions preserves this ordering. The contraction for $\mathcal{T}^\pi$ was proved above. For finite families of real numbers,

$$\left| \max_a x_a - \max_a y_a \right| \le \max_a |x_a - y_a|.$$

To see this, choose an index attaining $\max_a x_a$; its excess over $\max_a y_a$ is at most its excess over its corresponding y_a. Interchanging the two families bounds the opposite difference. Applying this inequality to the action values in (5.4.2) gives

$$|(\mathcal{T}u)(s) - (\mathcal{T}v)(s)| \le \gamma \max_a \left| \sum_{s'} P(s' \mid s,a)(u(s') - v(s')) \right| \le \gamma \|u - v\|_\infty.$$

Finally, an average of the action values cannot exceed their maximum, which proves $\mathcal{T}^\pi v \le \mathcal{T}v$. □

Lemma 5.8 (A contraction has a unique fixed point)**.** *Let $\mathcal{F} : \mathbb{R}^m \to \mathbb{R}^m$ satisfy $\|\mathcal{F}u - \mathcal{F}v\|_\infty \le \gamma \|u - v\|_\infty$ with $0 < \gamma < 1$. Then it has a unique fixed point u^*, and $u_{k+1} = \mathcal{F}u_k$ converges to it from every initial vector, with*

$$\|u_k - u^*\|_\infty \le \gamma^k \|u_0 - u^*\|_\infty. \tag{5.4.4}$$

Proof. Successive differences satisfy $\|u_{k+1} - u_k\|_\infty \le \gamma^k \|u_1 - u_0\|_\infty$. For $\ell > k$, summing these differences gives

$$\|u_\ell - u_k\|_\infty \le \sum_{j=k}^{\ell-1} \gamma^j \|u_1 - u_0\|_\infty \le \frac{\gamma^k}{1-\gamma} \|u_1 - u_0\|_\infty.$$

The iterates are Cauchy in the complete space $\mathbb{R}^m$, so they converge to some u^*. The contraction inequality implies continuity of $\mathcal{F}$; passing to the limit in $u_{k+1} = \mathcal{F}u_k$ gives $u^* = \mathcal{F}u^*$. Two fixed points would have distance at most γ times their distance, forcing them to coincide. Comparing an iterate with this fixed point repeatedly proves the rate. □

5.4.2 Why one stationary policy attains the optimum

Theorem 5.9 (Bellman optimality and stationary attainment)**.** *The optimal value V^* is the unique fixed point of $\mathcal{T}$:*

$$V^*(s) = \max_{a\in\mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right]. \tag{5.4.5}$$

Any deterministic stationary policy choosing a maximizing action at each state in this equation attains V^* simultaneously at every state. In particular, history dependence and randomization cannot improve the optimal discounted value.

Proof. By the contraction lemmas, $\mathcal{T}$ has a unique fixed point, which we temporarily call v^*. Choose a deterministic policy π^* greedy with respect to v^*, so that

$$\mathcal{T}^{\pi^*} v^* = \mathcal{T} v^* = v^*.$$

Fixed-policy evaluation has a unique fixed point. Hence $v^* = V^{\pi^*}$, and in particular $0 \le v^* \le 1/(1-\gamma)$.

It remains to compare v^* with every admissible policy, rather than only stationary ones. Fix any history-dependent randomized policy π. Since the fixed-point equation takes a maximum over actions, for each state s and action a,

$$v^*(s) \ge r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v^*(s').$$

At time t, average this inequality using the action distribution $\pi_t(\cdot \mid H_t)$. The Markov property identifies its right-hand side with a conditional expected reward and next-state value, giving

$$v^*(S_t) \ge \mathbb{E}_\pi[R_{t+1} + \gamma v^*(S_{t+1}) \mid H_t].$$

Multiply by γ^t, take expectations conditional on $S_0 = s$, and sum from $t = 0$ to $H-1$. The intermediate value terms cancel, leaving the universal policy bound

$$v^*(s) \ge \mathbb{E}_\pi \left[\sum_{t=0}^{H-1} \gamma^t R_{t+1} + \gamma^H v^*(S_H) \,\middle|\, S_0 = s \right]. \tag{5.4.6}$$

The terminal term has absolute expectation at most $\gamma^H/(1-\gamma)$, which tends to zero. The reward partial sums converge to the discounted sum and are uniformly bounded by $1/(1-\gamma)$. Thus, letting $H \to \infty$, $v^*(s) \ge V^\pi(s)$. This comparison holds for every admissible policy. On the other hand, the stationary policy π^* attains v^*. Taking the supremum in (5.4.1) proves $v^* = V^*$. □

The maximizing action can depend on the current state but need not depend on time. This is the guarantee of a stationary optimal policy. It does not assert that the optimal policy is unique: several actions may attain the same maximum at one or more states.

5.4.3 Optimal action values

Let $Q^*(s,a)$ be the largest expected discounted value when the first action is forced to be a at s, with any admissible policy allowed afterward. The stationary policy π^* above is optimal at every possible next state. Therefore

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s'), \tag{5.4.7}$$

and

$$V^*(s) = \max_a Q^*(s,a). \tag{5.4.8}$$

Substituting the second identity into the first gives the Bellman optimality equation for action values:

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_b Q^*(s',b). \tag{5.4.9}$$

This suggests the optimal action-value operator

$$(\mathcal{T}_Q q)(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_b q(s',b). \tag{5.4.10}$$

Lemma 5.10 (Contraction of the optimal action-value operator)**.** *The operator $\mathcal{T}_Q$ is a γ-contraction:*

$$\|\mathcal{T}_Q q - \mathcal{T}_Q \widetilde{q}\|_\infty \le \gamma \, \|q - \widetilde{q}\|_\infty . \tag{5.4.11}$$

Its unique fixed point is Q^*. In particular, iterating $q_{k+1} = \mathcal{T}_Q q_k$ converges to Q^* from every initial array.

Proof. For any next state s', the maximum inequality used earlier gives $|\max_b q(s',b) - \max_b \widetilde{q}(s',b)| \le \|q - \widetilde{q}\|_\infty$. Averaging over s' and multiplying by γ proves the contraction. Equation (5.4.9) identifies Q^* as a fixed point, and Lemma 5.8 gives uniqueness and convergence. □

5.5 Planning algorithms

In planning, the transition law P and expected reward r are known. A Bellman update can therefore compute its expectation exactly. The two basic control algorithms differ in what they update: value iteration repeatedly improves a vector of value estimates, whereas policy iteration alternates full evaluation of a current policy with a change in its action choices.

5.5.1 Value iteration and a stopping certificate

Initialize any vector v_0 and update all states synchronously by

$$v_{k+1} = \mathcal{T} v_k, \qquad v_{k+1}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_k(s') \right]. \tag{5.5.1}$$

All updates use the previous vector v_k. The contraction theorem gives

$$\|v_k - V^*\|_\infty \le \gamma^k \|v_0 - V^*\|_\infty \le \frac{\gamma^k}{1-\gamma} \quad \text{if } v_0 = 0. \tag{5.5.2}$$

Thus each sweep reduces the worst-case value error by at least a factor γ. With dense transition arrays, one sweep takes $O(|\mathcal{S}|^2|\mathcal{A}|)$ arithmetic operations.

A greedy policy can be extracted from any proposed value vector v: choose $\pi_v(s) \in \arg\max_a [r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s')]$. Its quality can be certified without knowing V^*, using the computable Bellman residual $\|\mathcal{T} v - v\|_\infty$.

Proposition 5.11 (Residual and greedy-policy bounds)**.** *For every value vector v,*

$$\|v - V^*\|_\infty \le \frac{\|\mathcal{T} v - v\|_\infty}{1-\gamma}. \tag{5.5.3}$$

A deterministic policy π_v greedy with respect to v satisfies

$$\|V^* - V^{\pi_v}\|_\infty \le \frac{2\gamma}{1-\gamma} \|v - V^*\|_\infty \le \frac{2\gamma}{(1-\gamma)^2} \|\mathcal{T} v - v\|_\infty . \tag{5.5.4}$$

Proof. Using $\mathcal{T} V^* = V^*$ and contraction,

$$\|v - V^*\|_\infty \le \|v - \mathcal{T} v\|_\infty + \|\mathcal{T} v - \mathcal{T} V^*\|_\infty \le \|v - \mathcal{T} v\|_\infty + \gamma \|v - V^*\|_\infty .$$

Rearranging gives the residual bound. For the policy bound, write $e = \|v - V^*\|_\infty$ and $D = \|V^* - V^{\pi_v}\|_\infty$. Greediness gives $\mathcal{T} v = \mathcal{T}^{\pi_v} v$. Hence

$$\begin{aligned} D &= \|\mathcal{T} V^* - \mathcal{T}^{\pi_v} V^{\pi_v}\|_\infty \\ &\le \|\mathcal{T} V^* - \mathcal{T} v\|_\infty + \|\mathcal{T}^{\pi_v} v - \mathcal{T}^{\pi_v} V^{\pi_v}\|_\infty \\ &\le \gamma e + \gamma \|v - V^{\pi_v}\|_\infty \le \gamma e + \gamma(e + D). \end{aligned}$$

Thus $(1-\gamma)D \le 2\gamma e$, proving the first policy bound. Combining it with (5.5.3) proves the second. □

For value iteration, $\|\mathcal{T} v_k - v_k\|_\infty = \|v_{k+1} - v_k\|_\infty$. Therefore a sufficient stopping rule for a greedy policy with value error at most ε is

$$\|v_{k+1} - v_k\|_\infty \le \frac{(1-\gamma)^2}{2\gamma} \varepsilon. \tag{5.5.5}$$

This is an upper-bound certificate; a particular MDP may yield an accurate policy much earlier.

5.5.2 Policy improvement and exact policy iteration

Suppose the current policy π has been evaluated exactly. Improve its action choice at every state by selecting

$$\pi'(s) \in \arg\max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^{\pi}(s') \right]. \tag{5.5.6}$$

This rule compares one initial action followed by the old policy; it does not evaluate a different complete policy separately for each candidate action.

Theorem 5.12 (Policy improvement)**.** *If π' is greedy with respect to V^{π}, then $V^{\pi'} \geq V^{\pi}$ coordinatewise. More precisely,*

$$V^{\pi} \leq \mathcal{T}^{\pi'} V^{\pi} \leq (\mathcal{T}^{\pi'})^2 V^{\pi} \leq \cdots \longrightarrow V^{\pi'}. \tag{5.5.7}$$

Proof. Greediness and policy evaluation give

$$\mathcal{T}^{\pi'} V^{\pi} = \mathcal{T} V^{\pi} \geq \mathcal{T}^{\pi} V^{\pi} = V^{\pi}.$$

The operator $\mathcal{T}^{\pi'}$ is monotone, so applying it to this inequality repeatedly gives the nondecreasing sequence in (5.5.7). Fixed-policy value iteration converges to $V^{\pi'}$, proving the claim. □

Exact policy iteration starts with a deterministic stationary policy π_0. At iteration k, it first computes $V^{\pi_k} = (I - \gamma P^{\pi_k})^{-1} r^{\pi_k}$. It then selects a policy π_{k+1} greedy with respect to that value. All states use the same evaluated vector V^{π_k}. When the current action is among the maximizing actions at a state, retain it. Stop when the policy is unchanged. This tie rule prevents switching between equally good actions after optimality has already been reached.

Proposition 5.13 (Strict improvement before optimality)**.** *If $V^{\pi_k} \neq V^*$, then $V^{\pi_{k+1}} \geq V^{\pi_k}$ at every state, with a strict inequality at at least one state.*

Proof. The Bellman residual satisfies $\mathcal{T} V^{\pi_k} - V^{\pi_k} \geq 0$. If it were zero at every state, V^{π_k} would be a fixed point of $\mathcal{T}$, so uniqueness would imply $V^{\pi_k} = V^*$. Under the stated assumption, there is therefore a state s with $(\mathcal{T} V^{\pi_k})(s) > V^{\pi_k}(s)$. Policy improvement gives

$$V^{\pi_{k+1}} \geq \mathcal{T}^{\pi_{k+1}} V^{\pi_k} = \mathcal{T} V^{\pi_k} \geq V^{\pi_k}.$$

The first and last vectors consequently differ strictly at that state. □

Corollary 5.14 (No repeated policy before optimality)**.** *If $V^{\pi_k} \neq V^*$, then the policies $\pi_0, \pi_1, \ldots, \pi_k$ are pairwise distinct.*

Theorem 5.15 (Finite termination of exact policy iteration)**.** *With the tie rule above, exact policy iteration terminates after finitely many policy improvements and returns an optimal deterministic stationary policy.*

Proof. There are $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic stationary policies. By the preceding corollary, the algorithm cannot repeat a policy while its value is nonoptimal. It must therefore reach a policy π_k with $V^{\pi_k} = V^*$ after finitely many improvements. At such a policy,

$$\mathcal{T}^{\pi_k} V^* = V^* = \mathcal{T} V^*.$$

Thus the action already selected by π_k is maximizing at each state. The tie rule retains every action, giving $\pi_{k+1} = \pi_k$, so the algorithm stops. Conversely, if it stops at an unchanged policy, that policy's value satisfies $\mathcal{T} V^{\pi_k} = \mathcal{T}^{\pi_k} V^{\pi_k} = V^{\pi_k}$. The unique fixed point of $\mathcal{T}$ is V^*, so the returned policy is optimal. □

The strict-improvement condition concerns values, $V^{\pi_k} \neq V^*$, rather than inequality with one selected optimal policy. Different policies can have the same optimal value. Also, the last improvement from a nonoptimal policy to an optimal one is covered by the proposition: it is the old policy that must be nonoptimal.

5.5.3 A route example: value iteration versus policy iteration

Example 5.16 (Updating values or updating a route)**:** Consider four states A, B, C, G, where G is an absorbing goal, and let $\gamma = 0.9$. The transitions and rewards are deterministic. A common action set $\mathcal{A} = \{0, 1\}$ gives the model below; the two actions have the same outcome at C and at G.

	Action 0		Action 1	
State	Next state	Reward	Next state	Reward
A	B	0	C	0
B	C	0	G	0.95
C	G	1	G	1
G	G	0	G	0

Thus the distinct route choices are $A \to B$ or $A \to C$, $B \to C$ or $B \to G$, and $C \to G$. The reward is received on the indicated move, not for merely occupying the goal.

For value iteration, start with $v_0 = 0$. In state order (A, B, C, G), the first update gives

$$v_1 = (0, 0.95, 1, 0).$$

For example, $v_1(A) = 0$ because both immediate rewards at A are zero and every old continuation estimate is zero. The next update is

$$\begin{aligned} v_2(A) &= \max\{0.9 \cdot 0.95,\ 0.9 \cdot 1\} = 0.9, \\ v_2(B) &= \max\{0.9 \cdot 1,\ 0.95 + 0.9 \cdot 0\} = 0.95, \\ v_2(C) &= 1, \qquad v_2(G) = 0. \end{aligned}$$

The resulting vector $v_2 = (0.9, 0.95, 1, 0)$ is unchanged by another Bellman update, so it equals V^*. One update communicated the values of the final moves, and the next communicated them back to A.

For policy iteration, instead start with a policy π_0 selecting the route $A \to B \to C \to G$. A full evaluation of this policy gives

$$V^{\pi_0}(C) = 1, \qquad V^{\pi_0}(B) = 0.9, \qquad V^{\pi_0}(A) = 0.81, \qquad V^{\pi_0}(G) = 0.$$

These are the values of the entire current routes, including all their future steps. Greedy improvement compares at A $0.9V^{\pi_0}(B) = 0.81$ with $0.9V^{\pi_0}(C) = 0.9$, so it switches to $A \to C$. At B, it compares $0.9V^{\pi_0}(C) = 0.9$ with the immediate reward 0.95 from moving to G, so it switches to $B \to G$. Both decisions use the same old policy value. Reevaluating this improved policy gives $(0.9, 0.95, 1, 0) = V^*$.

Value iteration updated an estimated value table by one Bellman step at a time. Policy iteration evaluated a whole current policy, then changed its action choices and reevaluated. A greedy policy can be extracted from value iteration as well; the distinction is the sequence of computations, not whether an algorithm ever uses a policy. ◇

Both planning algorithms use the known transition probabilities and reward means. Learning replaces these exact expectations by data. The available sampling mechanism then matters: an oracle can provide observations at chosen state–action pairs, while an interacting learner must also decide which pairs to visit.

5.6 Learning from sampled experience

Planning uses the full reward and transition model to evaluate a backup $r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)v(s')$. In learning, these quantities are unknown. Instead, an interaction supplies a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}), \tag{5.6.1}$$

from which we can form a sampled target such as $R_{t+1} + \gamma v(S_{t+1})$. One transition reveals a reward and one next state, rather than the entire distribution $P(\cdot \mid S_t, A_t)$.

Throughout the learning sections, $\mathcal{S}$ and the common action set $\mathcal{A}$ are finite, $0 < \gamma < 1$, and $0 \le R_{t+1} \le 1$ almost surely. Write $V_{\max} = 1/(1 - \gamma)$. The reward and next state may be dependent: we use the joint conditional MDP law of (R_{t+1}, S_{t+1}) given (S_t, A_t), with mean reward $r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$. We first study sampling from a generative-model oracle and planning in an estimated MDP.

5.6.1 A generative-model oracle

A generative model is a sampling oracle. For any requested $(s, a) \in \mathcal{S} \times \mathcal{A}$, it returns a fresh pair (R, S') with the correct joint conditional law of reward and next state. In particular,

$$0 \le R \le 1, \qquad \mathbb{E}[R \mid s, a] = r(s, a), \qquad S' \sim P(\cdot \mid s, a).$$

The reward and next state in one pair need not be independent. Calls at a fixed pair provide independent samples. We use a balanced design with n calls at every pair, and write

$$S = |\mathcal{S}|, \qquad A = |\mathcal{A}|, \qquad N = SA\,n$$

for the numbers of states, actions, and total oracle calls. Thus n counts samples per pair, whereas N counts all observations. The state and action sets and the discount factor are known; r and P are estimated from data [49, 54].

5.6.2 The empirical MDP and its optimal policy

For a particular pair (s, a), denote its samples by $(R_i, S_i')_{i=1}^n$. Their empirical reward and transition probabilities are

$$\widehat{r}(s, a) = \frac{1}{n} \sum_{i=1}^{n} R_i, \qquad \widehat{P}(s' \mid s, a) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{S_i' = s'\}. \tag{5.6.2}$$

The samples in this display are local to the specified pair. These estimates define an MDP $\widehat{M} = (\mathcal{S}, \mathcal{A}, \widehat{P}, \widehat{r}, \gamma)$: its reward means lie in $[0, 1]$, and every transition row is a probability distribution.

Write $\widehat{V}^{\pi}$ for the value of π in $\widehat{M}$; V^{π} continues to denote its value in the true MDP M. Solve $\widehat{M}$ exactly, for example by policy iteration, and choose an empirical-optimal policy $\widehat{\pi}^*$. It satisfies

$$\widehat{V}^{\widehat{\pi}^*} = \widehat{V}^*, \qquad \widehat{\pi}^*(s) \in \arg\max_{a \in \mathcal{A}} \left[\widehat{r}(s, a) + \gamma \sum_{s'} \widehat{P}(s' \mid s, a) \widehat{V}^*(s') \right].$$

This procedure is called certainty-equivalence planning: apply the known-model planning algorithm to the estimated model. For $\varepsilon, \delta \in (0, 1)$, our statistical objective is

$$\mathbb{P}\Big(\left\| V^* - V^{\widehat{\pi}^*} \right\|_\infty \le \varepsilon \Big) \ge 1 - \delta. \tag{5.6.3}$$

The probability concerns the oracle samples, and the value difference is measured in the true MDP. Exact optimality in $\widehat{M}$ does not by itself imply exact optimality in M.

5.6.3 Coordinatewise estimation of rewards and transitions

Define the uniform model errors by

$$\varepsilon_r = \max_{s,a} |\widehat{r}(s, a) - r(s, a)|, \qquad \varepsilon_P = \max_{s,a} \left\| \widehat{P}(\cdot \mid s, a) - P(\cdot \mid s, a) \right\|_1.$$

For a fixed pair and $t > 0$, Hoeffding's inequality gives

$$\mathbb{P}\big(|\widehat{r}(s, a) - r(s, a)| > t \big) \le 2e^{-2nt^2}.$$

Taking a union bound over SA pairs shows that $\mathbb{P}(\varepsilon_r > t) \le 2SAe^{-2nt^2}$. Consequently, with probability at least $1 - \delta/2$,

$$\varepsilon_r \le \sqrt{\frac{\log(4SA/\delta)}{2n}}. \tag{5.6.4}$$

For a fixed triple (s, a, s'), the estimate $\widehat{P}(s' \mid s, a)$ is an average of independent indicators. Another application of Hoeffding gives

$$\mathbb{P}\big(|\widehat{P}(s' \mid s, a) - P(s' \mid s, a)| > t \big) \le 2e^{-2nt^2}.$$

There are S^2A such entries. A union bound implies that, with probability at least $1-\delta/2$,

$$\max_{s,a,s'} |\widehat{P}(s' \mid s,a) - P(s' \mid s,a)| \le \sqrt{\frac{\log(4S^2A/\delta)}{2n}}.$$

Summing the entrywise errors in each transition row yields

$$\varepsilon_P \le S\sqrt{\frac{\log(4S^2A/\delta)}{2n}}. \tag{5.6.5}$$

The reward and transition bounds hold together with probability at least $1-\delta$. Independence between these two events is unnecessary.

5.6.4 Simulation lemma: model error and value error

The next statement is deterministic: it compares two MDPs with the same state and action sets and discount factor, given their model errors. Applying it on the preceding concentration event gives a statistical guarantee. The argument follows [54, Lemma 1].

Lemma 5.17 (Simulation lemma)**.** *Suppose the true and empirical reward means lie in* $[0,1]$*, and their uniform model errors are* $\varepsilon_r, \varepsilon_P$*. Then every stationary policy* π*, including a policy chosen from the data, satisfies*

$$\left\| \widehat{V}^\pi - V^\pi \right\|_\infty \le \frac{\varepsilon_r}{1-\gamma} + \frac{\gamma\varepsilon_P}{2(1-\gamma)^2}. \tag{5.6.6}$$

Proof. First average the model errors under the policy. For every s,

$$|\widehat{r}^\pi(s) - r^\pi(s)| \le \sum_a \pi(a \mid s)|\widehat{r}(s,a) - r(s,a)| \le \varepsilon_r,$$

$$\sum_{s'} |\widehat{P}^\pi(s,s') - P^\pi(s,s')| \le \sum_a \pi(a \mid s) \sum_{s'} |\widehat{P}(s' \mid s,a) - P(s' \mid s,a)| \le \varepsilon_P.$$

Both P^π and $\widehat{P}^\pi$ have nonnegative entries and row sums one. Their two Bellman equations are

$$V^\pi = r^\pi + \gamma P^\pi V^\pi, \qquad \widehat{V}^\pi = \widehat{r}^\pi + \gamma \widehat{P}^\pi \widehat{V}^\pi.$$

Subtract the equations and add and subtract $\gamma\widehat{P}^\pi V^\pi$:

$$\begin{aligned} \widehat{V}^\pi - V^\pi &= \widehat{r}^\pi - r^\pi + \gamma(\widehat{P}^\pi \widehat{V}^\pi - P^\pi V^\pi) \\ &= \widehat{r}^\pi - r^\pi + \gamma\widehat{P}^\pi(\widehat{V}^\pi - V^\pi) + \gamma(\widehat{P}^\pi - P^\pi)V^\pi. \end{aligned}$$

For any vector z, stochasticity of $\widehat{P}^\pi$ gives

$$|(\widehat{P}^\pi z)(s)| \le \sum_{s'} \widehat{P}^\pi(s,s')|z(s')| \le \|z\|_\infty \sum_{s'} \widehat{P}^\pi(s,s') = \|z\|_\infty .$$

Taking the maximum over s proves $\left\|\widehat{P}^\pi z\right\|_\infty \le \|z\|_\infty$. Thus, if $D = \left\|\widehat{V}^\pi - V^\pi\right\|_\infty$, the triangle inequality applied to the Bellman difference implies

$$D \le \varepsilon_r + \gamma D + \gamma \left\|(\widehat{P}^\pi - P^\pi)V^\pi\right\|_\infty . \tag{5.6.7}$$

It remains to control the last term. Since both transition rows sum to one,

$$\sum_{s'} \left[\widehat{P}^\pi(s,s') - P^\pi(s,s')\right] = 0.$$

We may therefore subtract the same constant from every value without changing its product with the difference of these rows. The value bound $0 \le V^\pi(s') \le 1/(1-\gamma)$ implies

$$\left| V^\pi(s') - \frac{1}{2(1-\gamma)} \right| \le \frac{1}{2(1-\gamma)}.$$

For every s, it follows that

$$\begin{aligned} \left|[(\widehat{P}^\pi - P^\pi)V^\pi](s)\right| &= \left| \sum_{s'} [\widehat{P}^\pi(s,s') - P^\pi(s,s')] \left[V^\pi(s') - \frac{1}{2(1-\gamma)} \right] \right| \\ &\le \frac{1}{2(1-\gamma)} \sum_{s'} |\widehat{P}^\pi(s,s') - P^\pi(s,s')| \\ &\le \frac{\varepsilon_P}{2(1-\gamma)}. \end{aligned}$$

This explains the factor $1/2$: centering the interval of possible values halves its maximum absolute magnitude, and the constant part vanishes because the transition-row difference sums to zero. Take the maximum over s and substitute into (5.6.7). Rearranging yields

$$(1-\gamma)D \le \varepsilon_r + \frac{\gamma \varepsilon_P}{2(1-\gamma)}.$$

Divide by $1-\gamma > 0$ to obtain (5.6.6). All estimates used only model errors that hold simultaneously at every state–action pair. The conclusion is therefore uniform over stationary policies, including the data-dependent empirical optimum $\widehat{\pi}^*$. □

5.6.5 Evaluation error and the learned policy's loss

Let π^* be optimal in M, and let $\widehat{\pi}^*$ be optimal in $\widehat{M}$. For every state s, add and subtract their empirical-model values to obtain

$$\begin{aligned} V^*(s) - V^{\widehat{\pi}^*}(s) &= \left[V^{\pi^*}(s) - \widehat{V}^{\pi^*}(s)\right] + \left[\widehat{V}^{\pi^*}(s) - \widehat{V}^{\widehat{\pi}^*}(s)\right] \\ &\quad + \left[\widehat{V}^{\widehat{\pi}^*}(s) - V^{\widehat{\pi}^*}(s)\right] \\ &\le \left\| V^{\pi^*} - \widehat{V}^{\pi^*} \right\|_\infty + \left\| \widehat{V}^{\widehat{\pi}^*} - V^{\widehat{\pi}^*} \right\|_\infty. \end{aligned}$$

The middle bracket is nonpositive by optimality of $\widehat{\pi}^*$ in the empirical MDP. Bound the other two brackets separately by the simulation lemma. Since $V^* \ge V^{\widehat{\pi}^*}$ coordinatewise,

$$\left\| V^* - V^{\widehat{\pi}^*} \right\|_\infty \le \frac{2\varepsilon_r}{1-\gamma} + \frac{\gamma \varepsilon_P}{(1-\gamma)^2}. \tag{5.6.8}$$

This step uses a uniform model-error statement. It does not treat a function selected from the same samples as a fixed function when applying Hoeffding's inequality.

5.6.6 The naive sample-complexity bound

On the joint concentration event, substitute (5.6.4) and (5.6.5) into (5.6.8). Since $\log(4SA/\delta) \le \log(4S^2A/\delta)$, this gives

$$\begin{aligned} \left\| V^* - V^{\widehat{\pi}^*} \right\|_\infty &\le \frac{2(1-\gamma) + \gamma S}{(1-\gamma)^2} \sqrt{\frac{\log(4S^2A/\delta)}{2n}} \\ &\le \frac{2S}{(1-\gamma)^2} \sqrt{\frac{\log(4S^2A/\delta)}{2n}}. \end{aligned} \tag{5.6.9}$$

The last inequality uses $S \geq 1$ and $2(1-\gamma)+\gamma S \leq 2S$. Therefore the choice

$$n = \left\lceil \frac{2S^2 \log(4S^2A/\delta)}{\varepsilon^2(1-\gamma)^4} \right\rceil \tag{5.6.10}$$

makes the right-hand side at most ε, and proves (5.6.3). The sufficient total sample count is

$$N = SA\,n = \widetilde{O}\left(\frac{S^3A}{\varepsilon^2(1-\gamma)^4}\right).$$

Sharper analyses of empirical-MDP planning lead to the comparison in Table 5.1; only the first row is proved here. The Bernstein-based rate also uses the propagation of transition variances through the Bellman equations and control of the dependence of the learned policy on the data [59]. The other analyses are left for subsequent exercises.

Analysis	Total oracle calls: $N = \widetilde{O}(\cdot)$
Naive coordinate bounds	$\dfrac{S^3A}{\varepsilon^2(1-\gamma)^4}$
Direct ℓ_1 concentration	$\dfrac{S^2A}{\varepsilon^2(1-\gamma)^4}$
Fixed-Q^* comparison	$\dfrac{SA}{\varepsilon^2(1-\gamma)^6}$
Bernstein inequality	$\dfrac{SA}{\varepsilon^2(1-\gamma)^3}$

Table 5.1: Sample-complexity rates for empirical-MDP planning with n observations per state–action pair and $N = SA\,n$ total calls. The first three analyses follow [54]; the last rate is supported by [59, Theorem 1].

5.6.7 Data access, prediction, and control

The data-access assumption determines which observations the learner can obtain [49].

- In *online interaction*, the current action determines the next state and therefore affects which data can be collected later. A reset may provide a new initial state, but does not permit a query at every requested state.
- A *generative model*, as above, supplies a fresh transition at a chosen (s, a). This is query access to samples, rather than knowledge of the full transition matrix.
- An *offline dataset* contains previously collected transitions. The learner cannot acquire information about missing state–action pairs by merely reusing the same dataset.

In *prediction*, a target policy π is fixed and the objective is to estimate V^π or Q^π. In *control*, the objective is to improve the choice of actions and obtain a policy with large value. These are distinct from the data-access choices: either task can be studied with online, generative, or offline observations.

The *behavior policy* b generates the observed actions. The *target policy* π is the policy whose value we want to learn. Learning is *on-policy* when the observed actions follow that target policy, and *off-policy* when behavior and target differ. For example, online interaction can be off-policy, and an offline dataset can be on-policy for the fixed policy that generated it. Thus on/off-policy and online/offline describe different aspects of the setting [28, 50, 51].

Coverage is a further requirement. Positive probabilities for all actions at a visited state do not guarantee that other states can be reached. A convergence theorem requiring infinitely many visits must assume or establish those visits. Repeated computation on one observed transition can change an estimate, but does not create new independent information about the environment.

5.7 Monte Carlo policy evaluation

Fix a stationary policy π. For complete-return sampling, suppose that a rollout reaches a zero-reward absorbing state Δ at an almost-surely finite time τ. This is a condition on the collected data within the same infinite-horizon discounted MDP. The observed return following a visit at time $t < \tau$ is

$$G_t = \sum_{k=t}^{\tau-1} \gamma^{k-t} R_{k+1}. \tag{5.7.1}$$

Extending the episode by zero rewards after termination gives the same infinite discounted return. In particular, $0 \le G_t \le V_{\max}$. Monte Carlo (MC) evaluation uses realized returns as its targets. They can be computed backward along a completed rollout: set $G_\tau = 0$ and use $G_t = R_{t+1} + \gamma G_{t+1}$ [28, 50].

5.7.1 First-visit evaluation

Generate independent episodes using the same policy and initial state distribution. Fix a state s that an episode visits with positive probability. In each episode that visits s, retain the return following the *first* visit to s, and discard later visits to s in that episode. Denote the retained returns by $G^{(1)}(s), G^{(2)}(s), \ldots$. The first-visit estimate after n such episodes is

$$\widehat{V}_n(s) = \frac{1}{n} \sum_{j=1}^{n} G^{(j)}(s). \tag{5.7.2}$$

Proposition 5.18 (First-visit MC evaluation)**.** *Under the preceding assumptions, the retained returns for a fixed state are independent, identically distributed random variables with mean $V^\pi(s)$. Consequently,*

$$\mathbb{E}[\widehat{V}_n(s)] = V^\pi(s), \qquad \widehat{V}_n(s) \longrightarrow V^\pi(s) \quad \textit{almost surely.} \tag{5.7.3}$$

For every $\varepsilon > 0$,

$$\mathbb{P}\big(|\widehat{V}_n(s) - V^\pi(s)| \ge \varepsilon\big) \le 2\exp\big(-2n(1-\gamma)^2\varepsilon^2\big). \tag{5.7.4}$$

Proof. The first hitting time of s is a stopping time. The Markov property, together with the fixed stationary policy, implies that the future from this time has the law of a fresh trajectory started at s. Its expected return is therefore $V^\pi(s)$, independently of the preceding history. The retained returns come from separate independent episodes. Selecting only episodes that hit s leaves an independent sequence with this same conditional future law. Thus the sample mean is unbiased, and the strong law of large numbers gives almost-sure convergence. Finally, each retained return lies in an interval of length $V_{\max}$. Hoeffding's inequality for these independent returns gives $2\exp(-2n\varepsilon^2/V_{\max}^2)$, which is (5.7.4). □

In particular, error at most ε with probability at least $1-\delta$ is guaranteed by

$$n \ge \frac{\log(2/\delta)}{2\varepsilon^2(1-\gamma)^2}. \tag{5.7.5}$$

The sample size n in this statement is the number of retained returns for s, not the total number of episodes or transitions. Rarely visited states may require many episodes to provide that many returns.

An *every-visit* estimate retains returns after all visits to s. It can use more of an episode, but overlapping returns within one episode are dependent. The independent-sample Hoeffding argument above does not apply directly to all of these returns. Similarly, successive returns cut from one continuing trajectory are generally dependent.

5.7.2 The running-average update

The sample mean can be maintained without storing earlier returns:

$$\widehat{V}_n(s) = \widehat{V}_{n-1}(s) + \frac{1}{n}\big(G^{(n)}(s) - \widehat{V}_{n-1}(s)\big). \tag{5.7.6}$$

Indeed, multiply $\widehat{V}_{n-1}(s)$ by $(n-1)/n$ and add $G^{(n)}(s)/n$. More generally, a learning rate α_n gives

$$V(s) \leftarrow V(s) + \alpha_n\big(G(s) - V(s)\big). \tag{5.7.7}$$

The expression in parentheses is the prediction error. The update is also a stochastic gradient step for the scalar loss $\frac{1}{2}(V(s)-G(s))^2$. The sample-mean choice is $\alpha_n = 1/n$. A constant learning rate gives more weight to recent returns, but generally retains persistent fluctuations under noisy data.

5.7.3 Truncated rollouts for the same discounted objective

If rollouts do not reach a zero-reward absorbing state, a finite rollout only approximates the complete discounted return. For H observed transitions from s, use $G_0^{(H)} = \sum_{k=0}^{H-1} \gamma^k R_{k+1}$. Nonnegative bounded rewards imply the pathwise bound

$$0 \le G_0 - G_0^{(H)} \le \sum_{k=H}^{\infty} \gamma^k = \frac{\gamma^H}{1-\gamma}. \tag{5.7.8}$$

Thus independent truncated rollouts yield an estimate with sampling error and a separate deterministic truncation bias. For example, if $\widehat{V}_{n,H}(s)$ averages n independent rollouts started at s, Hoeffding and (5.7.8) give, with probability at least $1-\delta$,

$$|\widehat{V}_{n,H}(s) - V^\pi(s)| \le \frac{1}{1-\gamma}\sqrt{\frac{\log(2/\delta)}{2n}} + \frac{\gamma^H}{1-\gamma}. \tag{5.7.9}$$

This statement explicitly assumes access to independent rollouts from s. A single long trajectory needs a different dependence analysis.

5.8 Temporal-difference policy evaluation

Each update uses the actual next state in the observed transition. If this is the zero-reward absorbing state Δ, its continuation value is zero. A reset starts a new rollout and is not itself a sampled MDP transition used in the bootstrap target.

The Bellman expectation equation suggests using a one-step target. For a fixed stationary policy π, temporal-difference learning with one-step bootstrapping, denoted TD(0), updates

$$V_{t+1}(S_t) = V_t(S_t) + \alpha_t\big(R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)\big), \tag{5.8.1}$$

with $V_{t+1}(s) = V_t(s)$ for $s \neq S_t$. At a terminal next state, the bootstrap value is zero. The error $R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)$ is the *temporal-difference error*. TD can update after a single transition, without waiting for the episode to finish. Its target contains a current value estimate, which is why the method is called *bootstrapping*.

5.8.1 The conditional mean of a TD update

Let $\mathcal{F}_t$ contain the trajectory through S_t, the current value table V_t, and the chosen learning rate, before sampling $A_t \sim \pi(\cdot \mid S_t)$. The reward and next state are then drawn from the environment. Since V_t is fixed conditional on this history,

$$\begin{aligned}\mathbb{E}[R_{t+1} + \gamma V_t(S_{t+1}) \mid \mathcal{F}_t] &= \sum_{a\in\mathcal{A}} \pi(a \mid S_t)\left(r(S_t,a) + \gamma \sum_{s'} P(s' \mid S_t, a) V_t(s')\right) \\ &= (\mathcal{T}^\pi V_t)(S_t).\end{aligned} \tag{5.8.2}$$

Consequently,

$$\mathbb{E}[V_{t+1}(S_t) - V_t(S_t) \mid \mathcal{F}_t] = \alpha_t\big((\mathcal{T}^\pi V_t)(S_t) - V_t(S_t)\big). \tag{5.8.3}$$

TD therefore makes a noisy update toward a Bellman expectation. It does not require independent successive states: the conditional mean uses the Markov property at the current state.

There are two different unbiasedness questions. The TD target is an unbiased sample of $(\mathcal{T}^\pi V_t)(S_t)$, conditional on the current history. Unless $V_t = V^\pi$, its mean need not equal $V^\pi(S_t)$. In comparison, a complete on-policy MC return has conditional mean $V^\pi(S_t)$. This difference explains why a TD update can initially propagate an inaccurate estimate from a successor state. It does not prevent convergence to the Bellman fixed point. A claim that TD always has lower variance than MC would require additional conditions; here the essential distinction is the target being estimated.

Example 5.19 (Two updates on the same episode)**:** Consider states s, w, Δ, with Δ absorbing and all subsequent rewards zero. Take $\gamma = 0.9$. A fixed policy chooses wait at s, which gives reward zero and next state w. At w, every action gives reward one and next state Δ; call the action selected by the policy finish. The rollout is

$$s \xrightarrow[R_1=0]{\text{wait}} w \xrightarrow[R_2=1]{\text{finish}} \Delta.$$

Start from $V(s) = V(w) = 0$, and use learning rate $1/2$. After the episode, MC uses returns $G_0 = 0.9$ and $G_1 = 1$, giving $V(s) = 0.45$ and $V(w) = 0.5$. Online TD processes the first transition while $V(w) = 0$, so its first target is zero and it leaves $V(s) = 0$. It then updates $V(w) = 0.5$ from the second transition. On the next identical episode, the first TD target is $0 + 0.9(0.5) = 0.45$, so $V(s)$ becomes 0.225. The reward information moves backward through successive bootstrap updates. These numerical updates use a constant learning rate only to make the arithmetic visible. ◇

5.8.2 A stochastic-approximation convergence tool

Equation (5.8.3) describes a mean update; by itself it is not a proof of almost-sure convergence. We also need noise control and enough updates to every relevant coordinate. The following standard special case of asynchronous stochastic approximation makes those requirements explicit. We use it without proving the general theorem; see [53] for the underlying result.

Theorem 5.20 (Asynchronous contraction with sampled updates)**.** *Let $\mathcal{I}$ be a finite coordinate set and let $B : \mathbb{R}^{\mathcal{I}} \to \mathbb{R}^{\mathcal{I}}$ be a contraction in the supremum norm with factor $\beta < 1$, with fixed point x^*. Consider the updates*

$$x_{t+1}(i) = x_t(i) + \alpha_t(i)\big((Bx_t)(i) - x_t(i) + M_{t+1}(i)\big), \qquad i \in \mathcal{I}. \tag{5.8.4}$$

Relative to a filtration $(\mathcal{F}_t)$, suppose that x_t and $0 \le \alpha_t(i) \le 1$ are $\mathcal{F}_t$-measurable, that the noise is measurable at time $t+1$, and that, for a finite deterministic constant C,

$$\mathbb{E}[M_{t+1}(i) \mid \mathcal{F}_t] = 0, \qquad \mathbb{E}[M_{t+1}(i)^2 \mid \mathcal{F}_t] \le C. \tag{5.8.5}$$

Suppose the iterates stay in a deterministic bounded set. For each coordinate assume, almost surely,

$$\sum_{t=0}^{\infty} \alpha_t(i) = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t(i)^2 < \infty. \tag{5.8.6}$$

Then $x_t \to x^*$ almost surely. A zero learning rate leaves a coordinate unchanged, so the statement includes updates of just one coordinate at a time.

The requirements (5.8.6) concern each coordinate's *own updates*. A simple choice is learning rate $1/n$ on its n-th visit. If a coordinate is visited infinitely often, its learning rates sum to infinity while their squares have a finite sum. Using $\alpha_t = 1/(t+1)$ with the global clock can fail this requirement for a very rarely visited coordinate. A fixed positive learning rate fails the square-summability requirement under persistent noise.

Theorem 5.21 (Tabular TD(0) convergence)**.** *Fix a stationary policy π. Suppose every nonterminal state is visited infinitely often, the reward and next-state samples have the correct conditional MDP law, and step sizes use the history before drawing $A_t \sim \pi(\cdot \mid S_t)$. Suppose the learning rates of each state satisfy (5.8.6). For a bounded initial table and nonzero learning rates in $[0, 1]$, TD(0) satisfies $\|V_t - V^\pi\|_\infty \to 0$ almost surely. A terminal coordinate is kept at its known value zero and is excluded from the coverage requirement.*

Proof. First check boundedness rather than assuming it. For the common initialization $0 \le V_0 \le V_{\max}$, the sampled target stays in $[0, V_{\max}]$, because $0 \le R_{t+1} + \gamma V_t(S_{t+1}) \le 1 + \gamma V_{\max} = V_{\max}$. Convex-combination updates preserve this interval. More generally, put $B = \max\{\|V_0\|_\infty, V_{\max}\}$. If all entries lie in $[-B, B]$, the target lies in $[-\gamma B, 1 + \gamma B] \subseteq [-B, B]$; induction gives a uniform bound for any bounded initial table.

Put $\alpha_t(s) = \alpha_t \mathbf{1}_{\{S_t = s\}}$. On the visited state, subtract $(\mathcal{T}^\pi V_t)(S_t)$ from the sampled target to obtain the noise. For an unvisited coordinate set the noise to zero. The choice of visited state is known at time t, and (5.8.2) gives conditional mean zero. Both the sampled target and its conditional mean lie in $[-B, B]$, so the noise has absolute value at most $2B$ and conditional second moment at most $4B^2$. The operator $\mathcal{T}^\pi$ is a γ-contraction with fixed point V^π. Apply Theorem 5.20 to the nonterminal coordinates. □

The infinite-visit assumption can hold through repeated resets after absorption or through recurrence along a continuing trajectory. A policy that never reaches a state cannot learn that state's value from its own trajectory. The theorem does not assert a finite-sample convergence rate.

5.8.3 Exercises

Exercise 5.3 (Returns and one-step updates): For the rollout in Example 5.19, take two identical rollouts, initialize all values at zero, and use learning rate $1/2$. Compute the MC and online TD values after each complete rollout. Identify when each method first changes the value at s.

Exercise 5.4 (Sampling error and rollout length): For independent rollouts from a fixed state, choose n and H so that the right-hand side of (5.7.9) is at most ε. State separately the number of rollouts and the total number of sampled transitions. The target remains the infinite discounted value $V^\pi(s)$.

Exercise 5.5 (Coverage and local learning rates): Suppose a coordinate is visited at times $t_k = 2^k$. Compare the sum of its learning rates under the global rule $1/(t+1)$ and under the local rule $1/k$. Explain why infinitely many visits alone do not justify the global rule. Give an MDP in which some states are unreachable from the prescribed initial state, despite positive exploration of every action at every visited state.

5.9 Function approximation for policy evaluation

A table assigns a separate parameter to every state. This representation is useful when the state space is small, but a robot configuration or an image can have too many possible values for repeated visits to every state to be practical. We instead choose a family of functions

$$V_w : \mathcal{S} \longrightarrow \mathbb{R}, \qquad w \in \mathbb{R}^d.$$

Changing one parameter can change the predictions at many states. Thus the choice of representation determines both what can be represented and how information from one observed state is shared with others. The development below follows Chapters 9 and 11 of [28] and [55].

5.9.1 Linear features and Monte Carlo regression

For a feature map $\phi : \mathcal{S} \to \mathbb{R}^d$, a linear value function is

$$V_w(s) = w^\top \phi(s). \tag{5.9.1}$$

For a finite state space, choosing $\phi(s_i) = e_i$ recovers a table: $V_w(s_i) = w_i$. A smaller feature family restricts the possible value vectors. A neural network replaces the fixed features by a nonlinear parameterization, but still shares parameters across states.

Fix a policy π. Suppose $S \sim \nu$, and, conditional on $S = s$, we generate a complete on-policy return G with $\mathbb{E}[G \mid S = s] = V^\pi(s)$. The population regression loss is

$$\mathcal{L}(w) = \frac{1}{2}\mathbb{E}[(V_w(S) - G)^2]. \tag{5.9.2}$$

To identify its target, expand around the conditional mean:

$$\begin{aligned}\mathbb{E}[(V_w(S)-G)^2 \mid S] &= (V_w(S)-V^\pi(S))^2 + \mathbb{E}[(G-V^\pi(S))^2 \mid S] \\ &= (V_w(S)-V^\pi(S))^2 + \mathrm{Var}(G \mid S). \end{aligned} \tag{5.9.3}$$

The cross term vanishes because $\mathbb{E}[G-V^\pi(S) \mid S]=0$. The second term does not depend on w. Consequently, minimizing $\mathcal{L}$ fits V^π under the state weighting ν. Independent pairs (s_i, G_i) give the empirical loss $\frac{1}{2n}\sum_i (V_w(s_i)-G_i)^2$.

A stochastic gradient step for this loss is

$$w_{t+1} = w_t + \alpha_t (G_t - V_{w_t}(S_t)) \nabla_w V_{w_t}(S_t). \tag{5.9.4}$$

For the linear model, the gradient is $\phi(S_t)$. As a numerical illustration, take $w_0=(0,0)$, $\phi(s)=(1,1)$, and $\phi(z)=(1,0)$. An observed return $G=0.9$ at s, with $\alpha=1/2$, gives $w_1=(0.45,0.45)$. Hence $V_{w_1}(s)=0.9$ and $V_{w_1}(z)=0.45$. The observation at s has also changed the prediction at z, through their shared feature.

5.9.2 TD targets and semi-gradients

For an on-policy transition, define

$$\delta_t = R_{t+1} + \gamma V_{w_t}(S_{t+1}) - V_{w_t}(S_t).$$

The function-approximation version of TD(0) is

$$w_{t+1} = w_t + \alpha_t \delta_t \nabla_w V_{w_t}(S_t). \tag{5.9.5}$$

A zero-reward absorbing terminal state Δ has a fixed boundary value $V_w(\Delta)=0$ for every w; for linear features this can be enforced by $\phi(\Delta)=0$. Equivalently, one can represent only nonterminal states and use zero continuation at the boundary. Unlike a tabular update, this step can change predictions at states other than S_t.

The term *semi-gradient* specifies which quantities are differentiated. Set $Y_t = R_{t+1} + \gamma V_{w_t}(S_{t+1})$, hold this sampled target fixed, and differentiate only the current-state prediction:

$$\nabla_w \frac{1}{2}(V_w(S_t)-Y_t)^2\Big|_{w=w_t} = -\delta_t \nabla_w V_{w_t}(S_t).$$

The negative gradient gives (5.9.5). In contrast, differentiating through both predictions would give

$$\nabla_w \frac{1}{2}\big(R_{t+1}+\gamma V_w(S_{t+1}) - V_w(S_t)\big)^2\Big|_{w=w_t} = \delta_t\big(\gamma \nabla_w V_{w_t}(S_{t+1}) - \nabla_w V_{w_t}(S_t)\big).$$

These are different updates. The TD target is frozen during the current gradient calculation and is recomputed from the parameters at later updates.

Let $\mathcal{F}_t$ contain the history through S_t and the current parameters, before sampling the action from π. The conditional mean of the TD error is

$$\mathbb{E}[\delta_t \mid \mathcal{F}_t] = (\mathcal{T}^\pi V_{w_t})(S_t) - V_{w_t}(S_t). \tag{5.9.6}$$

Indeed, conditional on this history, V_{w_t} is a fixed function. Averaging $R_{t+1}+\gamma V_{w_t}(S_{t+1})$ over the action and environment gives $\mathcal{T}^\pi V_{w_t}$. Thus TD still estimates a Bellman discrepancy, but its effect on other states is now determined by the parameterization.

5.9.3 Linear TD and the projected Bellman equation

For linear features, the update is

$$w_{t+1} = w_t + \alpha_t\big(R_{t+1} + \gamma w_t^\top \phi(S_{t+1}) - w_t^\top \phi(S_t)\big)\phi(S_t).$$

To examine its mean direction, hold w fixed and draw $S \sim \nu$, $A \sim \pi(\cdot \mid S)$, and then (R, S') from the environment. Define

$$A_\nu = \mathbb{E}[\phi(S)(\phi(S) - \gamma\phi(S'))^\top], \qquad b_\nu = \mathbb{E}[R\phi(S)]. \tag{5.9.7}$$

Direct expansion gives

$$\mathbb{E}[(R + \gamma w^\top \phi(S') - w^\top \phi(S))\phi(S)] = b_\nu - A_\nu w. \tag{5.9.8}$$

This is the mean direction for a frozen parameter under the specified sampling law. For a single online trajectory, S_t and w_t can be dependent; the displayed expectation does not assert that conditioning only on w_t produces independent draws from ν.

For a finite state space, let Φ have rows $\phi(s)^\top$, and let $D_\nu = \mathrm{diag}(\nu(s))$. Then

$$A_\nu = \Phi^\top D_\nu (I - \gamma P^\pi)\Phi, \qquad b_\nu = \Phi^\top D_\nu r^\pi.$$

Suppose $\nu(s) > 0$ for every state and Φ has full column rank. Write Π_ν for the orthogonal projection onto $\mathcal{L} = \{\Phi w : w \in \mathbb{R}^d\}$ in the inner product $\langle u, v\rangle_\nu = u^\top D_\nu v$. The normal equations for projection show that

$$A_\nu w = b_\nu \quad \Longleftrightarrow \quad \Phi w = \Pi_\nu \mathcal{T}^\pi(\Phi w). \tag{5.9.9}$$

To check this equivalence, subtract Φw from $\mathcal{T}^\pi(\Phi w) = r^\pi + \gamma P^\pi \Phi w$. Its projection onto $\mathcal{L}$ is zero exactly when it is orthogonal to every column of Φ, namely when $\Phi^\top D_\nu(r^\pi + \gamma P^\pi \Phi w - \Phi w) = 0$.

A batch version of policy evaluation repeatedly fits a Bellman update:

$$w_{k+1} \in \arg\min_w \mathbb{E}_{S\sim\nu}\left[\left(V_w(S) - (\mathcal{T}^\pi V_{w_k})(S)\right)^2\right]. \tag{5.9.10}$$

In the linear case this means $V_{w_{k+1}} = \Pi_\nu \mathcal{T}^\pi V_{w_k}$. Observed one-step targets can be used for the regression, with the previous value function held fixed. This batch fit is distinct from an individual online semi-gradient TD step.

Proposition 5.22 (Stationary state weighting)**.** *Under the preceding finite-state, positive-weight, and full-rank assumptions, suppose additionally that $\nu^\top P^\pi = \nu^\top$. Then $\Pi_\nu \mathcal{T}^\pi$ is a γ-contraction in $\|z\|_\nu = (z^\top D_\nu z)^{1/2}$. It has a unique fixed point in $\mathcal{L}$. Moreover,*

$$x^\top A_\nu x \geq (1-\gamma)\mathbb{E}_{S\sim\nu}[(x^\top\phi(S))^2] > 0 \qquad (x \neq 0).$$

In particular, A_ν is nonsingular.

Proof. Jensen's inequality and stationarity give

$$\|P^\pi z\|_\nu^2 \leq \sum_s \nu(s) \sum_{s'} P^\pi(s,s') z(s')^2 = \sum_{s'} \nu(s') z(s')^2 = \|z\|_\nu^2 .$$

An orthogonal projection is nonexpansive in its defining norm: decompose any vector into its projection and its orthogonal residual, and use Pythagoras. Therefore

$$\|\Pi_\nu \mathcal{T}^\pi u - \Pi_\nu \mathcal{T}^\pi v\|_\nu \leq \gamma \|P^\pi(u-v)\|_\nu \leq \gamma \|u - v\|_\nu .$$

The finite-dimensional subspace $\mathcal{L}$ is complete, so contraction gives the unique fixed point. Finally put $z(s) = x^\top \phi(s)$. Under the stationary one-step law, S and S' have the same distribution. Cauchy–Schwarz implies $\mathbb{E}[z(S)z(S')] \leq \mathbb{E}[z(S)^2]$, and consequently

$$x^\top A_\nu x = \mathbb{E}[z(S)^2] - \gamma\mathbb{E}[z(S)z(S')] \geq (1-\gamma)\mathbb{E}[z(S)^2].$$

The last quantity is positive for $x \neq 0$ because the weighted feature Gram matrix $\Phi^\top D_\nu \Phi$ is positive definite. If $A_\nu x = 0$, its quadratic form is zero, so $x = 0$. □

This proposition explains why the weighting distribution matters. Convergence results for online on-policy linear TD also require appropriate visitation, noise, and learning-rate conditions. The proposition alone is a statement about the projected Bellman operator and the frozen mean direction. A nonlinear fit, or a fit under an arbitrary state distribution, need not preserve contraction. Even in a linear class, representability of the true value by itself does not ensure stability.

5.9.4 A fitted iteration that diverges

Example 5.23 (Two states with shared features)**:** There is one action at each of two states u, v. Both states move to v, all rewards are zero, and $\gamma = 0.9$. Use the one-dimensional feature map $\phi(u) = 1,\ \phi(v) = 2$, so $V_w(u) = w,\ V_w(v) = 2w$. The true value is zero and belongs to this class.

Choose the fitting distribution $\nu(u) = \nu(v) = 1/2$. At parameter w_k, both Bellman targets are $2\gamma w_k$. Thus the next parameter minimizes

$$\frac{1}{2}(w - 2\gamma w_k)^2 + \frac{1}{2}(2w - 2\gamma w_k)^2.$$

The derivative is

$$(w - 2\gamma w_k) + 2(2w - 2\gamma w_k) = 5w - 6\gamma w_k.$$

The objective is strictly convex, so its minimizer is

$$w_{k+1} = \frac{6\gamma}{5} w_k = 1.08 w_k.$$

Hence $w_k = (1.08)^k w_0$, which diverges for every $w_0 \neq 0$. The exact Bellman operator is a contraction and the true value is representable, but the fitting operation destabilizes the iteration. The chosen uniform distribution is not stationary for this chain: its stationary distribution puts all mass on v. Thus the example does not contradict Proposition 5.22, and does not assert divergence of all on-policy linear TD algorithms. ⋄

For control, shared parameters can similarly represent action values $Q_w(s, a)$. The next section develops the corresponding sampled optimality and policy-improvement methods.

5.10 Control from sampled transitions

Control combines policy evaluation with policy improvement. A value-based approach estimates Q^π and improves actions through $\pi_{\text{new}}(s) \in \arg\max_a Q^\pi(s, a)$. With a known model, policy iteration performs these operations exactly. With sampled experience, evaluation and improvement can proceed together; this is *generalized policy iteration* [28, 51].

Example 5.24 (A cash-or-wait improvement)**:** Let $\gamma = 0.9$ and use the common action set $\mathcal{A} = \{\text{cash}, \text{wait}\}$. At state s, cash gives reward 0.6 and next state Δ, whereas wait gives reward zero and next state w. At w, either action finishes: it gives reward one and next state Δ. State Δ is absorbing with zero reward under either action. All transitions are deterministic. For any continuation policy,

$$Q^\pi(s, \text{cash}) = 0.6, \qquad Q^\pi(s, \text{wait}) = 0 + 0.9 \cdot 1 = 0.9.$$

Replacing cash by wait at s improves its value. The action at w can remain unchanged because both choices have the same outcome. ⋄

5.10.1 SARSA: the next action actually taken

For a fixed policy π, observe $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, where the next action follows $\pi(\cdot \mid S_{t+1})$. The name *SARSA* records these five quantities: state, action, reward, state, action. Its update is

$$\begin{aligned} Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) \\ + \alpha_t\big[R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) - Q_t(S_t, A_t)\big]. \end{aligned} \tag{5.10.1}$$

Other entries remain unchanged. The continuation term is zero when the next state is the zero-reward absorbing state.

Let $\mathcal{G}_t$ contain the history, current table, and selected (S_t, A_t), before the next reward and state are observed. For a fixed stationary policy, the conditional target mean is

$$\begin{aligned} &\mathbb{E}\big[R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) \mid \mathcal{G}_t\big] \\ &\quad = r(S_t, A_t) + \gamma \sum_{s'} P(s' \mid S_t, A_t) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q_t(s', a') \\ &\quad = (\mathcal{T}_Q^\pi Q_t)(S_t, A_t). \end{aligned} \tag{5.10.2}$$

Thus fixed-policy SARSA samples the Bellman expectation equation for Q^π. For control, actions are often chosen using the current table, so the policy changes as learning proceeds. A fixed-policy contraction argument alone does not prove optimal-control convergence for this changing-policy recursion.

5.10.2 Expected SARSA

Instead of sampling A_{t+1}, average its value under a chosen target policy π:

$$Y_t^\pi = R_{t+1} + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid S_{t+1}) Q_t(S_{t+1}, a'). \tag{5.10.3}$$

Then use

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t \big(Y_t^\pi - Q_t(S_t, A_t)\big). \tag{5.10.4}$$

Its conditional mean is the same Bellman expectation backup in (5.10.2). The current pair can be supplied by a different behavior policy, since the target explicitly specifies the next-action distribution. If that target policy is greedy with respect to Q_t, the sum is a maximum and the update becomes Q-learning [51]. A zero continuation value is again used at an absorbing next state.

5.10.3 Q-learning: sampled optimality backups

The optimal action-value operator $\mathcal{T}_Q$ was defined in (5.4.10). Its unique fixed point is Q^*, and Lemma 5.10 gives its γ-contraction. Q-learning samples this optimality backup rather than averaging the next action under a specified policy.

After observing (5.6.1), Q-learning updates

$$\begin{aligned} Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) \\ + \alpha_t \left(R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right). \end{aligned} \tag{5.10.5}$$

Other entries remain unchanged. If $S_{t+1} = \Delta$, the maximum in the target is zero. The algorithm needs a sampled reward and next state, but no transition probabilities. It also needs no sampled next action in order to compute its target.

For this update, let $\mathcal{F}_t$ include the history, the current table, and the selected action A_t, just before the environment returns (R_{t+1}, S_{t+1}). The behavior policy may depend on the whole past. The conditional MDP law gives

$$\mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a') \,\Big|\, \mathcal{F}_t \Big] = (\mathcal{T}_Q Q_t)(S_t, A_t). \tag{5.10.6}$$

The behavior policy determines which coordinate is updated. It does not change this conditional mean once (S_t, A_t) has been selected. This is the key off-policy property of one-step tabular Q-learning. It does not permit arbitrary data selection after looking at the next state: the action and learning rate must be chosen before that transition noise is revealed.

Theorem 5.25 (Tabular Q-learning convergence)**.** *Suppose every nonterminal state–action pair is visited infinitely often, samples have the correct conditional MDP law, and step sizes are chosen before observing the next reward and state. Suppose the learning rates for each pair satisfy (5.8.6). For a bounded initial table and nonzero learning rates in $[0, 1]$, with terminal Q-values fixed at zero throughout, Q-learning satisfies*

$$\|Q_t - Q^*\|_\infty \longrightarrow 0 \quad \textit{almost surely.} \tag{5.10.7}$$

This is the discounted finite-table setting of [52, 53].

Proof. Put $B = \max\{\|Q_0\|_\infty, V_{\max}\}$. If all entries lie in $[-B, B]$, the target in (5.10.5) lies in $[-\gamma B, 1+\gamma B] \subseteq [-B, B]$. Convex-combination updates therefore keep all iterates bounded. In particular, an initial table in $[0, V_{\max}]$ stays in that interval.

Take coordinates $i = (s, a)$, with learning rate zero for unvisited pairs. Subtract the conditional mean $(\mathcal{T}_Q Q_t)(S_t, A_t)$ from the observed target to obtain a martingale-difference noise. Equation (5.10.6) verifies its conditional mean is zero, and its absolute value is at most $2B$, so its conditional second moment is bounded by $4B^2$. Lemma 5.10 supplies the contraction and its unique fixed point. All assumptions of Theorem 5.20 hold, which proves the claim. □

If π_t is greedy with respect to Q_t, its value also approaches optimality. To see this directly, write $\varepsilon_t = \|Q_t - Q^*\|_\infty$. For an optimal action a^* at state s, greediness gives

$$Q^*(s, \pi_t(s)) \geq Q_t(s, \pi_t(s)) - \varepsilon_t \geq Q_t(s, a^*) - \varepsilon_t \geq V^*(s) - 2\varepsilon_t.$$

Thus $V^* - \mathcal{T}^{\pi_t} V^* \leq 2\varepsilon_t \mathbf{1}$. Subtract the equation for V^{π_t} and use the resolvent $(I - \gamma P^{\pi_t})^{-1} = \sum_{k \geq 0} \gamma^k (P^{\pi_t})^k$ to obtain

$$0 \leq V^*(s) - V^{\pi_t}(s) \leq \frac{2\varepsilon_t}{1-\gamma} \quad \text{for every } s. \tag{5.10.8}$$

Hence $\|V^{\pi_t} - V^*\|_\infty \to 0$. The action selected in a state need not settle on a unique label if multiple actions are optimal.

5.10.4 Exploration and coverage

A common behavior rule is ε-greedy selection. Given a chosen greedy action $a_t^*(s) \in \arg\max_a Q_t(s, a)$, use

$$b_t(a \mid s) = (1 - \varepsilon_t)\mathbf{1}_{\{a = a_t^*(s)\}} + \frac{\varepsilon_t}{|\mathcal{A}|}. \tag{5.10.9}$$

A fixed positive exploration probability assigns every action a positive chance *when that state is visited*. It does not by itself guarantee that all states will be visited infinitely often. With suitable recurrence or exploratory resets, a fixed positive ε can supply the coverage required by Q-learning. The learned greedy policy can be optimal even though the exploratory behavior continues to take suboptimal actions.

Setting $\varepsilon_t \to 0$ is not a complete exploration argument. If exploration disappears too quickly, an initially underestimated action may receive only finitely many samples. Coverage is a separate hypothesis of Theorem 5.25, and each pair's local learning-rate sums must still satisfy (5.8.6).

A fixed exploratory behavior is compatible with Q-learning's optimality target, but SARSA under the same behavior evaluates the exploratory next actions that are actually taken. To obtain an optimal-control guarantee for SARSA, additional conditions on policy improvement are needed, such as becoming greedy in the limit while retaining sufficient exploration and suitable step sizes. The fixed-policy stochastic-approximation theorem must not be applied to a changing policy without checking those conditions.

5.10.5 Three targets for the same transition

Example 5.26 (Different choices of the future action)**:** Suppose the observed reward is zero and the observed next state is u. Take $\gamma = 0.9$ and

$$Q_t(u, a_1) = 0.8, \qquad Q_t(u, a_2) = 0.2.$$

The behavior policy selects a_1 with probability 0.75 and a_2 with probability 0.25. On this observation, the sampled next action happens to be a_2. The three targets are

Method	Target
SARSA	$0.9(0.2) = 0.18$
Expected SARSA under b	$0.9(0.75 \cdot 0.8 + 0.25 \cdot 0.2) = 0.585$
Q-learning	$0.9 \max\{0.8, 0.2\} = 0.72$

The differences come from the future-action rule. SARSA uses the sampled action, Expected SARSA averages over the stated policy, and Q-learning uses a greedy action. ◇

5.10.6 Control with an approximate action-value function

Represent action values by $Q_w(s, a)$ and fix a sampled target y_t.

The loss $\frac{1}{2}(Q_w(S_t, A_t) - y_t)^2$ gives the gradient step

$$w_{t+1} = w_t + \alpha_t \big(y_t - Q_{w_t}(S_t, A_t)\big) \nabla_w Q_{w_t}(S_t, A_t). \tag{5.10.10}$$

For example, the target may be

$$\begin{aligned} y_t^{\mathrm{SARSA}} &= R_{t+1} + \gamma Q_{w_t}(S_{t+1}, A_{t+1}), \\ y_t^{Q} &= R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_{w_t}(S_{t+1}, a'). \end{aligned} \tag{5.10.11}$$

The target is held fixed during the gradient step. Shared parameters can change values at many state–action pairs at once, so the finite-table convergence proof above no longer applies. Set the continuation term to zero at an absorbing next state [28, 51].

5.10.7 Fitted Q-iteration

Given a dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^N$ and an old estimate Q_k, first construct regression targets

$$y_i = r_i + \gamma \max_{a' \in \mathcal{A}} Q_k(s_i', a'). \tag{5.10.12}$$

Use $y_i = r_i$ when s_i' is the zero-reward absorbing state. Fit the next estimate in a function class $\mathcal{Q}$:

$$Q_{k+1} \in \arg\min_{f \in \mathcal{Q}} \frac{1}{N} \sum_{i=1}^{N} \big(f(s_i, a_i) - y_i\big)^2. \tag{5.10.13}$$

Each round freezes the targets and solves a regression problem. Repeat target construction and fitting. This approximates an optimal Bellman update, with additional error from finite data, the chosen function class, and incomplete optimization. Repeatedly reusing $\mathcal{D}$ does not add independent environment observations [51].

5.10.8 Deep Q-networks: replay and target parameters

A *deep Q-network* (DQN) represents Q_w using a neural network. Two components of the original method address correlated observations and rapidly changing targets [60].

Experience replay stores transitions in a buffer and samples minibatches from that collection. It lets a transition contribute to several optimization steps and breaks the temporal ordering of those steps; it does not turn reused observations into fresh environment data.

A *target network* has parameters $\bar{w}$ that are held fixed for several optimization steps. On a minibatch of size B, form

$$\begin{aligned} y_i &= r_i + \gamma(1 - d_i) \max_{a' \in \mathcal{A}} Q_{\bar{w}}(s_i', a'), \\ L(w) &= \frac{1}{2B} \sum_{i=1}^{B} \big(Q_w(s_i, a_i) - y_i\big)^2, \end{aligned} \tag{5.10.14}$$

where $d_i = 1$ for an absorbing next state and $d_i = 0$ otherwise. Differentiate only with respect to the current-network parameters in $Q_w(s_i, a_i)$. Periodically copy the current parameters to the target network, $\bar{w} \leftarrow w$. This gives a simplified DQN update; replay and target parameters do not by themselves supply the tabular Q-learning convergence guarantee for a neural-network approximation.

5.10.9 Exercises

Exercise 5.6 (The role of the next action): Fix a stationary policy π. Derive the conditional mean of the SARSA target. Compare it with Expected SARSA and Q-learning, and identify the corresponding Bellman operators. Explain why drawing the next action from a different policy changes SARSA's prediction objective.

Exercise 5.7 (Greedy control from an approximate table): Let π_Q be greedy with respect to a table Q. Prove that $\|V^* - V^{\pi_Q}\|_\infty \le 2\|Q - Q^*\|_\infty / (1 - \gamma)$. Explain why convergence of the value need not imply that a selected action label eventually stops changing when optimal actions are tied.

5.11 Off-policy learning and evaluation

A target policy π specifies the value or return to be estimated, whereas a behavior policy b specifies the actions that generate the data. These roles can differ even when the learner is interacting online with the environment. This section follows [28, Sections 5.5 and 7.3, Chapter 11] and [56].

5.11.1 Action coverage and a one-step change of measure

Importance sampling requires the support condition

$$\pi(a \mid s) > 0 \quad \Longrightarrow \quad b(a \mid s) > 0 \tag{5.11.1}$$

at states relevant to the target policy. Otherwise an action that matters under π may never be observed under b, so its contribution cannot be recovered by reweighting the observed actions. Condition (5.11.1) does not ensure that a particular state occurs in a finite dataset.

For an observed action drawn from b, define

$$\rho_t = \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}.$$

The denominator is positive on the event being sampled. A zero target probability gives ratio zero.

Proposition 5.27 (One-step importance sampling)**.** *For a bounded fixed function $f(a, r, s')$, under (5.11.1),*

$$\mathbb{E}_b[\rho_t f(A_t, R_{t+1}, S_{t+1}) \mid S_t = s] = \mathbb{E}_\pi[f(A_t, R_{t+1}, S_{t+1}) \mid S_t = s]. \tag{5.11.2}$$

In particular, $\mathbb{E}_b[\rho_t \mid S_t = s] = 1$.

Proof. The conditional environment law given the current state and action is the same under the two policies. Expanding the left-hand side gives

$$\begin{aligned}\sum_{a:\, b(a|s)>0} b(a \mid s)\frac{\pi(a \mid s)}{b(a \mid s)}\mathbb{E}[f(a, R, S') \mid s, a] \\ = \sum_a \pi(a \mid s)\mathbb{E}[f(a, R, S') \mid s, a].\end{aligned}$$

Actions excluded from the first sum have zero target probability by the support condition. The last sum is the right-hand side. Set $f = 1$ for the final assertion. □

The same argument can condition on the entire history before choosing the current action. A current value table is then fixed inside that conditional expectation. For a history-dependent behavior policy, the denominator must be its actual conditional action probability at that history; using an unrelated marginal probability would not give the identity.

5.11.2 An importance-corrected TD update

Let $\mathcal{F}_t$ contain the history through S_t and the current table V_t, before drawing A_t from b. Apply Proposition 5.27 to the one-step TD error. It gives

$$\mathbb{E}_b\big[\rho_t\big(R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)\big) \,\big|\, \mathcal{F}_t\big] = (\mathcal{T}^\pi V_t)(S_t) - V_t(S_t). \tag{5.11.3}$$

Indeed, the first two terms average to $\mathcal{T}^\pi V_t$, while the final term is unchanged because $\mathbb{E}_b[\rho_t \mid \mathcal{F}_t] = 1$. This suggests

$$V_{t+1}(S_t) = V_t(S_t) + \alpha_t \rho_t\big(R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)\big), \tag{5.11.4}$$

with all other entries unchanged.

The ratio corrects the action distribution at the visited state. It does not change how often the behavior visits each state. This distinction is especially important with shared features: the state weighting enters the projected Bellman equation and the mean parameter update. Equation (5.11.3) identifies an unbiased conditional direction; convergence additionally requires coverage, learning-rate, stability, and noise conditions. Large ratios can increase the noise substantially.

5.11.3 Complete trajectories and reward prefixes

To obtain a trajectory identity, consider a prefix of H decisions from the same initial distribution ρ and the same environment under both policies. This prefix truncates the discounted objective; it does not change the underlying MDP into a finite-horizon task. Write

$$\tau = (S_0, A_0, R_1, S_1, \ldots, A_{H-1}, R_H, S_H), \qquad G_H = \sum_{t=0}^{H-1} \gamma^t R_{t+1}.$$

In a trajectory probability or density, the initial-state factor and all joint reward–transition factors are the same under π and b. They cancel in the likelihood ratio, leaving

$$W_H(\tau) = \frac{p_\pi(\tau)}{p_b(\tau)} = \prod_{t=0}^{H-1} \rho_t. \tag{5.11.5}$$

This cancellation does not assume independence between the reward and next state. Under the action support condition along target-relevant paths, the target trajectory law is absolutely continuous with respect to the behavior law. Thus

$$J_{\rho,H}(\pi) := \mathbb{E}_\pi[G_H] = \mathbb{E}_b[W_H G_H]. \tag{5.11.6}$$

For a discrete trajectory space, this follows simply by substituting $p_\pi(\tau) = W_H(\tau) p_b(\tau)$ into the sum defining the expectation. The same argument applies to densities and integrals. Consequently, N independent behavior trajectories give the unbiased estimate $N^{-1} \sum_{i=1}^{N} W_H(\tau_i) G_H(\tau_i)$ of the truncated value $J_{\rho,H}(\pi)$. Its deterministic truncation error relative to the infinite discounted objective is at most $\gamma^H/(1-\gamma)$.

Only the actions up to a reward are needed to change that reward's distribution. For $t \geq 0$, let $W_{t+1} = \prod_{k=0}^{t} \rho_k$. The same likelihood-ratio argument applied to the prefix ending at R_{t+1}, S_{t+1} gives $\mathbb{E}_b[W_{t+1} R_{t+1}] = \mathbb{E}_\pi[R_{t+1}]$. Summing these identities yields the per-decision form

$$J_{\rho,H}(\pi) = \mathbb{E}_b\left[\sum_{t=0}^{H-1} \gamma^t W_{t+1} R_{t+1}\right]. \tag{5.11.7}$$

For another proof, condition the full product W_H on the prefix through R_{t+1}, S_{t+1}. Repeated use of $\mathbb{E}_b[\rho_k \mid \mathcal{F}_k] = 1$ makes the conditional expectation of all later ratios equal to one, leaving W_{t+1}. Per-decision weighting removes action ratios occurring after the reward being evaluated. It still can have substantial variance.

Example 5.28 (Exponential variance of trajectory weights)**:** Suppose there are two actions at each of H decisions. The target always selects a_1; the behavior selects each action with probability $1/2$, independently of its previous random choices. The trajectory weight is 2^H if all H actions equal a_1, and is zero otherwise. The first event has probability 2^{-H}. Hence

$$\mathbb{E}_b[W_H] = 2^{-H} 2^H = 1, \qquad \mathbb{E}_b[W_H^2] = 2^{-H} 2^{2H} = 2^H, \qquad \mathrm{Var}_b(W_H) = 2^H - 1.$$

At $H = 10$, only one trajectory in 1024 matches the target actions on average. An unbiased estimator can therefore remain highly variable with a modest sample. This example concerns the weight itself; the variance of a weighted-return estimate also depends on the rewards. ◇

5.11.4 Function approximation, offline data, and replay

Function approximation couples the predictions at different states, bootstrapping makes targets depend on current predictions, and off-policy data can give a state weighting different from the target policy's occupancy. These ingredients together can produce unstable value updates, even with linear features. This combination is often called the *deadly triad* [28, Chapter 11]. Correcting the current action by an importance ratio does not establish a contraction for the entire parameter update. The fitted-iteration example in Section 5.9 already shows why the state weighting and projection operation need their own analysis.

The distinction between on-policy and off-policy concerns the relation between behavior and target policies. The distinction between online and offline concerns whether further experience can be collected. Online

Q-learning can use off-policy data while actively interacting with the environment. Conversely, an offline dataset can be used to evaluate the same logging policy that produced it.

Experience replay samples stored transitions repeatedly. It can support optimization and reuse observations efficiently, but it does not create new information about unobserved pairs. Conditional on a finite replay dataset, uniformly sampling stored transitions for a particular pair averages its empirical rewards and transition frequencies. Thus the conditional mean target corresponds to an empirical Bellman operator. Whether this operator approximates the true environment depends on the original sampling and coverage, while convergence of a particular replay update also depends on its learning rates and representation.

5.12 Policy gradients

Value-based control learns action values and uses them to select actions. Policy-gradient methods instead choose a differentiable family of policies $\{\pi_\theta : \theta \in \mathbb{R}^d\}$ and optimize the return directly. Fix an initial distribution ρ, independent of θ, and define the unnormalized discounted objective

$$J_\rho(\theta) = \mathbb{E}_{\rho,\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]. \tag{5.12.1}$$

The optimization step is gradient ascent, $\theta \leftarrow \theta + \eta\,\widehat{\nabla J_\rho(\theta)}$, because larger return is preferred. The environment and its reward law do not depend on θ.

For the derivations below, use the finite state and action spaces of the MDP, bounded rewards, and policies with positive action probabilities. Assume that the probabilities are continuously differentiable and that, in a neighborhood of the parameter being considered, their scores $\nabla_\theta \log \pi_\theta(a \mid s)$ are uniformly bounded. These assumptions justify the discounted sums and differentiation steps. Analogous formulas apply to continuous action distributions under the corresponding support and integrability conditions. Our finite-space derivation follows [28, Chapter 13], [57], and [58].

5.12.1 Policy scores

The *score* of an action is

$$g_\theta(s,a) = \nabla_\theta \log \pi_\theta(a \mid s).$$

For linear state–action features $\phi(s,a) \in \mathbb{R}^d$, a softmax policy has the form

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{b\in\mathcal{A}} \exp(\theta^\top \phi(s,b))}.$$

Taking a logarithm and differentiating gives

$$g_\theta(s,a) = \phi(s,a) - \sum_b \pi_\theta(b \mid s)\phi(s,b). \tag{5.12.2}$$

In a neural policy, the same score is obtained by differentiating its log-probability output with respect to its parameters.

For any differentiable positive policy on a fixed finite action set, the score has conditional mean zero:

$$\sum_a \pi_\theta(a \mid s) g_\theta(s,a) = \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0. \tag{5.12.3}$$

This identity will remove rewards that precede an action and will allow state-dependent baselines.

5.12.2 A trajectory likelihood and the likelihood-ratio identity

First consider a prefix of H decisions,

$$\tau = (S_0, A_0, R_1, S_1, \ldots, A_{H-1}, R_H, S_H).$$

Let $K(r, s' \mid s, a)$ denote the joint mass or density of the reward and next state, when such a density is used. The trajectory law factors as

$$p_\theta(\tau) = \rho(S_0) \prod_{t=0}^{H-1} \pi_\theta(A_t \mid S_t) K(R_{t+1}, S_{t+1} \mid S_t, A_t). \tag{5.12.4}$$

The factor K is joint: no reward–next-state independence is being assumed. Only policy factors depend on the parameters. Thus, on the trajectory support,

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{H-1} g_\theta(S_t, A_t). \tag{5.12.5}$$

Write $Z_H = \sum_{k=0}^{H-1} \gamma^k R_{k+1}$ and $J_{\rho,H}(\theta) = \mathbb{E}_\theta[Z_H]$. The likelihood-ratio identity is

$$\begin{aligned} \nabla_\theta J_{\rho,H}(\theta) &= \int Z_H(\tau) \nabla_\theta p_\theta(\tau)\, d\tau \\ &= \mathbb{E}_\theta[Z_H(\tau) \nabla_\theta \log p_\theta(\tau)] \\ &= \mathbb{E}_\theta\left[Z_H \sum_{t=0}^{H-1} g_\theta(S_t, A_t) \right]. \end{aligned} \tag{5.12.6}$$

Here the integral includes sums for discrete components. For a finite prefix, the state–action sum is finite, and the reward integrals contain bounded random variables; the stated regularity permits differentiation. The identity estimates a gradient using sampled rewards and policy scores. It requires neither differentiating nor knowing the environment kernel.

5.12.3 Causality and reward-to-go

Let $\mathcal{H}_t$ contain the trajectory history through S_t, before A_t is chosen. Conditional on $\mathcal{H}_t$, the action law is $\pi_\theta(\cdot \mid S_t)$, so $\mathbb{E}[g_\theta(S_t, A_t) \mid \mathcal{H}_t] = 0$ by (5.12.3). For $k < t$, the reward R_{k+1} is already part of this history. Therefore

$$\mathbb{E}[g_\theta(S_t, A_t) R_{k+1}] = \mathbb{E}[R_{k+1} \mathbb{E}[g_\theta(S_t, A_t) \mid \mathcal{H}_t]] = 0.$$

An action cannot affect a reward received before that action. Define the return measured from time t to the prefix endpoint by

$$G_t^{(H)} = \sum_{k=t}^{H-1} \gamma^{k-t} R_{k+1}.$$

In (5.12.6), discard the zero-expectation past-reward terms for each score and factor out γ^t. This gives

$$\nabla_\theta J_{\rho,H}(\theta) = \mathbb{E}_\theta\left[\sum_{t=0}^{H-1} \gamma^t G_t^{(H)} g_\theta(S_t, A_t) \right]. \tag{5.12.7}$$

The factor γ^t records when the action occurs in the original objective. The return $G_t^{(H)}$ discounts rewards relative to time t, not relative to time zero.

5.12.4 The infinite discounted limit and occupancy form

The finite-prefix calculation is a proof device for the same discounted MDP. It does not require an infinite trajectory to have a differentiable probability density. To justify the limit, let C bound the score norm in a parameter neighborhood. Differentiating the expectation of a single reward using its prefix of $k+1$ decisions gives

$$\nabla_\theta \mathbb{E}_\theta[R_{k+1}] = \mathbb{E}_\theta\left[R_{k+1} \sum_{t=0}^{k} g_\theta(S_t, A_t) \right], \qquad \|\nabla_\theta \mathbb{E}_\theta[R_{k+1}]\| \le C(k+1).$$

Since $\sum_{k\geq 0}\gamma^k(k+1) = (1-\gamma)^{-2} < \infty$, the derivative series is locally uniformly convergent. We may differentiate the discounted objective term by term and rearrange the absolutely integrable sums. Equivalently, the reward-to-go expression has the uniform integrable bound $\sum_t \gamma^t C/(1-\gamma) = C/(1-\gamma)^2$.

Let $G_t = \sum_{k=t}^{\infty}\gamma^{k-t}R_{k+1}$. Conditioning on the current state and action gives

$$\mathbb{E}_\theta[G_t \mid S_t = s, A_t = a] = Q^{\pi_\theta}(s,a).$$

The score is a function of this same state–action pair. Consequently,

$$\nabla_\theta J_\rho(\theta) = \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_\theta[Q^{\pi_\theta}(S_t,A_t)g_\theta(S_t,A_t)]\,. \tag{5.12.8}$$

Define the normalized discounted state distribution from ρ by

$$d_\rho^{\pi_\theta}(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}_{\rho,\pi_\theta}(S_t = s).$$

It is a probability distribution: summing over states gives $(1-\gamma)\sum_t \gamma^t = 1$.

Theorem 5.29 (Policy-gradient theorem)**.** *Under the preceding assumptions,*

$$\nabla_\theta J_\rho(\theta) = \frac{1}{1-\gamma}\mathbb{E}_{\substack{S\sim d_\rho^{\pi_\theta}\\ A\sim\pi_\theta(\cdot|S)}}[Q^{\pi_\theta}(S,A)\nabla_\theta \log\pi_\theta(A\mid S)]\,. \tag{5.12.9}$$

Proof. Expand the expectation in (5.12.8) over s, a and collect the coefficients of the same state:

$$\begin{aligned}\nabla_\theta J_\rho(\theta) &= \sum_s \left[\sum_{t=0}^{\infty}\gamma^t \mathbb{P}_{\rho,\pi_\theta}(S_t = s)\right]\sum_a \pi_\theta(a\mid s)Q^{\pi_\theta}(s,a)g_\theta(s,a)\\ &= \frac{1}{1-\gamma}\sum_s d_\rho^{\pi_\theta}(s)\sum_a \pi_\theta(a\mid s)Q^{\pi_\theta}(s,a)g_\theta(s,a).\end{aligned}$$

This is the asserted expectation. □

The derivation already includes the effect of the policy on future state distributions. It does not differentiate the policy while pretending those distributions are fixed. The factor $1/(1-\gamma)$ is required because d_ρ^π is normalized whereas J_ρ is the unnormalized discounted return.

5.12.5 REINFORCE

Suppose an on-policy rollout reaches a zero-reward absorbing state Δ at a finite time T almost surely. Keep θ fixed while collecting that rollout. Compute returns backward, $G_T = 0$ and $G_t = R_{t+1} + \gamma G_{t+1}$, and form

$$\widehat{g} = \sum_{t=0}^{T-1}\gamma^t G_t \nabla_\theta \log\pi_\theta(A_t\mid S_t). \tag{5.12.10}$$

REINFORCE then performs $\theta \leftarrow \theta + \eta\widehat{g}$ and starts a new rollout. After absorption, all remaining rewards and returns are zero. Thus this is the infinite discounted reward-to-go estimator with its zero tail removed, and $\mathbb{E}[\widehat{g}] = \nabla_\theta J_\rho(\theta)$. Bounded scores and discounted returns give the integrability required for this equality even when T is random. A cutoff before actual absorption omits a nonzero tail and generally introduces bias.

Example 5.30 (A policy gradient for cash or wait)**:** There are states s, w, Δ and actions $\{\text{cash}, \text{wait}\}$. At s, cash gives reward 0.6 and moves to Δ; wait gives reward zero and moves to w. At w, every action gives reward one and moves to Δ. The state Δ is absorbing with zero rewards. Start at s, use $\gamma = 0.9$, and set

$$p_\theta = \pi_\theta(\text{wait}\mid s) = \frac{e^\theta}{1+e^\theta}.$$

The policies at w and Δ are fixed, positive, and independent of θ. The two complete returns are 0.6 and 0.9, so

$$J(\theta) = 0.6(1 - p_\theta) + 0.9p_\theta, \qquad J'(\theta) = 0.3p_\theta(1 - p_\theta).$$

At $\theta = 0$, the derivative is $0.075 > 0$.

The score at s is $1 - p_\theta$ after wait and $-p_\theta$ after cash. Later scores are zero. Thus the expectation of the REINFORCE estimate is

$$p_\theta(0.9)(1 - p_\theta) + (1 - p_\theta)(0.6)(-p_\theta) = 0.3p_\theta(1 - p_\theta),$$

agreeing with the direct derivative. Gradient ascent raises the probability of wait, using sampled returns rather than the model. ◇

5.12.6 Baselines and advantages

A state-dependent baseline can change the variability of the gradient estimate without changing its mean. For a fixed action-independent function $b(s)$, (5.12.3) gives

$$\mathbb{E}_{A\sim\pi_\theta(\cdot|s)}[b(s)g_\theta(s, A)] = b(s)\sum_a \nabla_\theta\pi_\theta(a \mid s) = 0.$$

More generally, a baseline known before the current action is sampled has the same property after conditioning on that information. It follows that the estimator

$$\widehat{g}_b = \sum_{t=0}^{T-1} \gamma^t(G_t - b(S_t))g_\theta(S_t, A_t) \tag{5.12.11}$$

has the same expectation as (5.12.10), under the same integrability conditions. An appropriate baseline can reduce variance. For example, it may be fitted from prior data and held fixed while the actor's batch is collected. The actor differentiates the log policy, not the baseline. Fitting a baseline from the very same random actions requires attention to the conditional independence used in this argument.

Taking $b(s) = V^\pi(s)$ defines the advantage

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Subtracting the zero baseline term from (5.12.9) gives

$$\nabla_\theta J_\rho(\theta) = \frac{1}{1-\gamma}\mathbb{E}_{\substack{S\sim d_\rho^{\pi_\theta} \\ A\sim\pi_\theta(\cdot|S)}}[A^{\pi_\theta}(S, A)g_\theta(S, A)]. \tag{5.12.12}$$

Since $V^\pi(s) = \sum_a \pi(a \mid s)Q^\pi(s, a)$, an action with positive advantage is better than the current policy's average action at that state. In Example 5.30, at $p_\theta = 1/2$, $V^\pi(s) = 0.75$; cash has advantage -0.15 and wait has advantage 0.15.

5.12.7 A critic and an actor–critic update

Let a critic V_w approximate the current policy's value, with $V_w(\Delta) = 0$ at a zero-reward absorbing terminal state. Define the one-step error

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

When $V_w = V^\pi$, the Bellman equation implies

$$\begin{aligned}\mathbb{E}[\delta_t \mid S_t = s, A_t = a] &= r(s, a) + \gamma\sum_{s'} P(s' \mid s, a)V^\pi(s') - V^\pi(s) \\ &= A^\pi(s, a).\end{aligned} \tag{5.12.13}$$

Thus an exact critic supplies a one-step unbiased conditional advantage estimate. With an approximate critic, its mean is instead $r(s, a) + \gamma(PV_w)(s, a) - V_w(s)$, which can differ from the true advantage. Bootstrapping can therefore introduce bias into an actor's update.

A basic on-policy actor–critic scheme uses separate parameters for π_θ and V_w. At time t in an episode, compute δ_t using the current parameters and update

$$\begin{aligned} w &\leftarrow w + \alpha\, \delta_t \nabla_w V_w(S_t), \\ \theta &\leftarrow \theta + \eta\, \gamma^t \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t). \end{aligned} \tag{5.12.14}$$

Both right-hand sides use the parameters before either update. At a terminal next state, its continuation value is zero. The critic takes a TD semi-gradient step, while the actor treats δ_t as a target and does not differentiate it. The weight γ^t matches the discounted objective used throughout this chapter.

With changing parameters and an approximate critic, (5.12.14) is an algorithmic update rather than an exact-gradient guarantee for a single fixed policy. Its convergence requires additional assumptions and analysis. Likewise, even an exact policy gradient does not imply that ascent finds a globally optimal policy in a restricted parameterization: the objective is generally nonconcave.

Value-based control learns Q and extracts actions from it. REINFORCE changes a policy using complete returns, and actor–critic changes a policy using estimated advantages supplied by a value learner. These methods connect sequential decision problems to stochastic optimization, while retaining the effects of sampling, approximation, and policy-dependent state distributions.

Appendix A

Mathematical background

A.1 Probability review

This section is a compact reference for the probability notation used in the course. It follows the sequence most useful for asymptotic statistics: convergence modes, the LLN and CLT, continuous mappings and Slutsky's theorem, tightness, stochastic orders, and the delta method. Unless stated otherwise, random vectors take values in $\mathbb{R}^d$, $\|\cdot\|$ is the Euclidean norm, and $X, X_1, X_2, \ldots$ are defined on a common probability space whenever the definition compares their sample paths.

Modes of convergence

Definition A.1 (Four modes of convergence)**.** Let $p > 0$.

(a) *Convergence in probability*, written $X_n \xrightarrow{\mathbb{P}} X$, means that, for every $\varepsilon > 0$,

$$\mathbb{P}(\|X_n - X\| > \varepsilon) \longrightarrow 0.$$

(b) *Convergence in distribution*, also called *convergence in law* or *weak convergence*, is written $X_n \xrightarrow{d} X$. For scalar random variables it means

$$F_{X_n}(x) \longrightarrow F_X(x)$$

at every continuity point x of F_X. For random vectors, the same definition uses the joint cdf and componentwise inequalities.

(c) *Convergence in p-th moment*, or *in L^p*, written $X_n \xrightarrow{L^p} X$, means

$$\mathbb{E}\,\|X_n - X\|^p \longrightarrow 0.$$

(d) *Almost-sure convergence*, written $X_n \xrightarrow{\text{a.s.}} X$, means

$$\mathbb{P}\Big(\lim_{n\to\infty} \|X_n - X\| = 0\Big) = 1.$$

Almost-sure convergence concerns entire sample paths. Convergence in probability and in moment also depend on the joint construction of X_n and X. Convergence in distribution is weaker: it compares only their laws, so the variables may even be defined on different probability spaces.

Proposition A.2 (Implications between modes)**.** *For $p > 0$,*

$$X_n \xrightarrow{\text{a.s.}} X \quad\Longrightarrow\quad X_n \xrightarrow{\mathbb{P}} X \quad\Longrightarrow\quad X_n \xrightarrow{d} X,$$

and

$$X_n \xrightarrow{L^p} X \quad\Longrightarrow\quad X_n \xrightarrow{\mathbb{P}} X.$$

If $p \geq q > 0$, then L^p-convergence also implies L^q-convergence.

Proof. Almost-sure convergence implies convergence in probability by applying bounded convergence to $\mathbf{1}\{\|X_n - X\| > \varepsilon\}$. Markov's inequality, $\mathbb{P}(\|X_n - X\| > \varepsilon) \leq \varepsilon^{-p}\mathbb{E}\,\|X_n - X\|^p$, proves the moment implication. The implication from probability to distribution follows from the subsequence characterization and bounded convergence. Finally, monotonicity of L^r norms on a probability space gives $\{\mathbb{E}\,\|X_n - X\|^q\}^{1/q} \leq \{\mathbb{E}\,\|X_n - X\|^p\}^{1/p}$. □

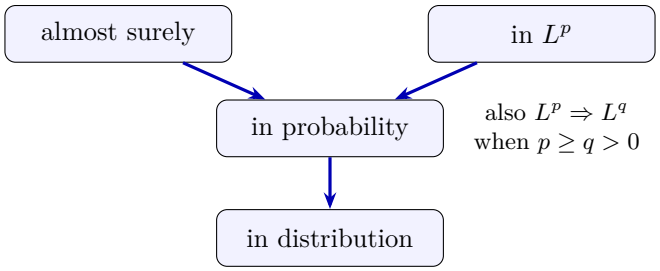


Figure A.1: The standard implications. In general, none of the displayed arrows can be reversed. Almost-sure and moment convergence are not ordered without additional assumptions.

The missing reverse arrows are genuine. The following examples are useful tests against overclaiming.

(i) Let X be Rademacher and set $X_n = -X$. Then $X_n \xrightarrow{d} X$, because X_n and X have the same law, but $\mathbb{P}(|X_n - X| > 1) = 1$. Thus distributional convergence need not imply convergence in probability.

(ii) Let A_n be independent with $\mathbb{P}(A_n) = 1/n$, and set $X_n = \mathbf{1}_{A_n}$. Then $X_n \to 0$ in every L^p, and hence in probability, but the second Borel–Cantelli lemma gives $X_n = 1$ infinitely often almost surely. Thus neither convergence in probability nor convergence in moment implies almost-sure convergence.

(iii) For any fixed $p > 0$, if $U \sim \mathrm{Unif}(0,1)$, set $X_n = n^{1/p}\mathbf{1}\{U \leq 1/n\}$. Then $X_n \to 0$ almost surely, but $\mathbb{E}|X_n|^p = 1$. Thus almost-sure convergence need not imply L^p-convergence.

Laws of large numbers and the central limit theorem

Let $X_1, X_2, \ldots$ be iid scalar random variables and $\overline{X}_n = n^{-1}\sum_{i=1}^n X_i$.

Theorem A.3 (Weak law of large numbers)**.** *If $\mathbb{E}|X_1| < \infty$ and $\mu := \mathbb{E}X_1$, then*

$$\overline{X}_n \xrightarrow{\mathbb{P}} \mu.$$

Theorem A.4 (Strong law of large numbers)**.** *Under the same iid and integrability assumptions,*

$$\overline{X}_n \xrightarrow{\text{a.s.}} \mu.$$

The SLLN therefore implies the WLLN, although the names refer to the modes of convergence rather than to different limits.

Theorem A.5 (Central limit theorem)**.** *Let $X_i \in \mathbb{R}^d$ be iid with mean μ and finite covariance matrix Σ. Then*

$$\sqrt{n}\,(\overline{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \Sigma). \tag{A.1.1}$$

For $d = 1$, this reads $\sqrt{n}(\overline{X}_n - \mu)/\sigma \xrightarrow{d} \mathcal{N}(0,1)$ when $0 < \sigma^2 := \mathrm{Var}(X_1) < \infty$.

The laws of large numbers identify the probability limit of the unscaled average. The CLT studies the remaining fluctuations on the finer $n^{-1/2}$ scale. It does not say that $\overline{X}_n - \mu$ converges to a nondegenerate normal random variable without the $\sqrt{n}$ rescaling.

Continuous mappings and Slutsky's theorem

Theorem A.6 (Continuous mapping theorem)**.** *Let $g : \mathbb{R}^d \to \mathbb{R}^k$ be measurable and continuous on a set C such that $\mathbb{P}(X \in C) = 1$. Then*

$$\begin{aligned} X_n \xrightarrow{\mathbb{P}} X \quad &\Longrightarrow \quad g(X_n) \xrightarrow{\mathbb{P}} g(X), \\ X_n \xrightarrow{\text{a.s.}} X \quad &\Longrightarrow \quad g(X_n) \xrightarrow{\text{a.s.}} g(X), \\ X_n \xrightarrow{d} X \quad &\Longrightarrow \quad g(X_n) \xrightarrow{d} g(X). \end{aligned}$$

The condition is continuity at the values that the limit can actually take; global continuity is sufficient but not necessary. For instance, squaring, matrix multiplication, and inversion away from singular matrices can all be handled by this theorem.

Theorem A.7 (Slutsky's theorem)**.** *The following three forms will be used repeatedly.*

(i) If $X_n \xrightarrow{d} c$*, where* c *is constant, then* $X_n \xrightarrow{\mathbb{P}} c$*.*

(ii) If $X_n \xrightarrow{d} X$ *and* $\|X_n - Y_n\| \xrightarrow{\mathbb{P}} 0$*, then* $Y_n \xrightarrow{d} X$*.*

(iii) If $X_n \xrightarrow{d} X$ *and* $Y_n \xrightarrow{\mathbb{P}} c$*, where* c *is constant, then*

$$(X_n, Y_n) \xrightarrow{d} (X, c).$$

Combining part (iii) with the continuous mapping theorem gives, whenever the operations are well defined,

$$X_n + Y_n \xrightarrow{d} X + c, \qquad X_n Y_n \xrightarrow{d} cX, \qquad \frac{X_n}{Y_n} \xrightarrow{d} \frac{X}{c} \quad (c \neq 0).$$

Vector–matrix versions follow by applying the relevant continuous map. The constant-limit condition matters: knowing only the two marginal limits does not identify the joint limit. For example, if $X_n \sim \mathcal{N}(0,1)$ and $Y_n = -X_n$, both marginals are standard normal, but (X_n, Y_n) is supported on the line $y = -x$, not distributed as two independent standard normals.

For example, if the scalar CLT holds and $\widehat{\sigma}_n \xrightarrow{\mathbb{P}} \sigma > 0$, then

$$\frac{\sqrt{n}(\overline{X}_n - \mu)}{\widehat{\sigma}_n} \xrightarrow{d} \mathcal{N}(0,1).$$

Replacing an unknown constant by a consistent estimator is one of the most common uses of Slutsky's theorem.

Tightness and escaping mass

Definition A.8 (Uniform tightness)**.** A collection $\{X_\alpha : \alpha \in A\}$ in $\mathbb{R}^d$ is *uniformly tight* if, for every $\varepsilon > 0$, there is a compact set $K_\varepsilon \subset \mathbb{R}^d$ such that

$$\sup_{\alpha \in A} \mathbb{P}(X_\alpha \notin K_\varepsilon) < \varepsilon.$$

In $\mathbb{R}^d$, this is equivalent to requiring some finite M such that $\sup_{\alpha \in A} \mathbb{P}(\|X_\alpha\| > M) < \varepsilon$. A sequence is called *tight* when its associated collection is uniformly tight.

Proposition A.9 (Weak convergence implies tightness)**.** *If* $X_n \xrightarrow{d} X$*, then* (X_n) *is tight.*

Proof. Given $\varepsilon > 0$, choose a continuity point M_0 of the cdf of $\|X\|$ such that $\mathbb{P}(\|X\| > M_0) < \varepsilon/2$. The continuous mapping theorem and the scalar definition of weak convergence give $\mathbb{P}(\|X_n\| > M_0) \to \mathbb{P}(\|X\| > M_0)$, so the desired bound holds for all sufficiently large n. Enlarging M_0 to control the finitely many remaining random variables proves the claim. □

Theorem A.10 (Prokhorov's theorem in $\mathbb{R}^d$)**.** *If* (X_n) *is tight, then every subsequence contains a further subsequence that converges in distribution to some* $\mathbb{R}^d$*-valued random vector.*

Thus tightness prevents mass from escaping and supplies candidate subsequential limits; a separate argument must still identify the limit and prove its uniqueness.

Example A.11 (Why tightness matters: mass escaping to infinity)**:** Let $X_n = n$ deterministically. For any compact $K \subset \mathbb{R}$, all sufficiently large n satisfy $\mathbb{P}(X_n \notin K) = 1$, so the sequence is not tight and has no weakly convergent subsequence on $\mathbb{R}$. Its cdf is

$$F_n(x) = \mathbf{1}\{x \geq n\}.$$

For every fixed x, $F_n(x) \to 0$. The pointwise limit is not a cdf, because it does not approach one as $x \to \infty$. Thus pointwise-looking limits can lose all probability mass when tightness is absent.

Proposition A.12 (A convenient moment criterion)**.** *If, for some $p > 0$, $\sup_{\alpha \in A} \mathbb{E}\,\|X_\alpha\|^p \le C < \infty$, then $\{X_\alpha : \alpha \in A\}$ is uniformly tight.*

Proof. Markov's inequality gives

$$\sup_{\alpha \in A} \mathbb{P}(\|X_\alpha\| > M) \le \frac{C}{M^p} \longrightarrow 0 \qquad (M \to \infty).$$

□

A standard weak-convergence proof consequently has three steps:

1. prove tightness;
2. take an arbitrary convergent subsequence;
3. identify its limit and show that every subsequence has the same one.

Stochastic orders $o_{\mathbb{P}}$ and $O_{\mathbb{P}}$

Definition A.13 (Stochastic order)**.** Let $a_n > 0$ be deterministic.

(a) $X_n = O_{\mathbb{P}}(a_n)$ means that X_n/a_n is tight. Equivalently, for every $\varepsilon > 0$, there are finite M and N such that

$$\mathbb{P}(\|X_n\| > M a_n) < \varepsilon, \qquad n \ge N.$$

(b) $X_n = o_{\mathbb{P}}(a_n)$ means $X_n/a_n \xrightarrow{\mathbb{P}} 0$.

Thus $O_{\mathbb{P}}(1)$ means bounded in probability and $o_{\mathbb{P}}(1)$ means convergence to zero in probability.

Proposition A.14 (Calculation rules)**.** *Let $a_n, b_n > 0$, and let products below be scalar or dimensionally compatible matrix–vector products.*

$$\begin{aligned}
X_n = O_{\mathbb{P}}(a_n),\ Y_n = O_{\mathbb{P}}(b_n) \quad &\Longrightarrow \quad X_n + Y_n = O_{\mathbb{P}}(a_n + b_n),\\
X_n = o_{\mathbb{P}}(a_n),\ Y_n = o_{\mathbb{P}}(b_n) \quad &\Longrightarrow \quad X_n + Y_n = o_{\mathbb{P}}(a_n + b_n),\\
X_n = O_{\mathbb{P}}(a_n),\ Y_n = O_{\mathbb{P}}(b_n) \quad &\Longrightarrow \quad X_n Y_n = O_{\mathbb{P}}(a_n b_n),\\
X_n = o_{\mathbb{P}}(a_n),\ Y_n = O_{\mathbb{P}}(b_n) \quad &\Longrightarrow \quad X_n Y_n = o_{\mathbb{P}}(a_n b_n).
\end{aligned}$$

Also, $X_n = o_{\mathbb{P}}(a_n)$ implies $X_n = O_{\mathbb{P}}(a_n)$. If $a_n = O(b_n)$, then $O_{\mathbb{P}}(a_n)$ is also $O_{\mathbb{P}}(b_n)$; if $a_n = o(b_n)$, then $O_{\mathbb{P}}(a_n)$ is $o_{\mathbb{P}}(b_n)$. Finally, if $X_n/a_n \xrightarrow{\mathbb{P}} c \neq 0$, then

$$X_n^{-1} = a_n^{-1}\{c^{-1} + o_{\mathbb{P}}(1)\}.$$

The last rule requires a nonzero probability limit; stochastic order alone does not justify taking reciprocals.

The sum rules follow from the triangle inequality and union bounds. The product rules follow by first restricting the $O_{\mathbb{P}}(1)$ factor to a high-probability bounded set. The inverse rule is the continuous mapping theorem applied away from zero.

Lemma A.15 (Continuous transformations of small remainders)**.** *Let $R : \mathbb{R}^d \to \mathbb{R}^k$ be measurable, let $q > 0$, and suppose $X_n \xrightarrow{\mathbb{P}} 0$. Define the ratio below to be zero when $X_n = 0$.*

(i) If $\|R(h)\| = o(\|h\|^q)$ as $h \to 0$, then

$$\frac{\|R(X_n)\|}{\|X_n\|^q} \xrightarrow{\mathbb{P}} 0.$$

(ii) If $\|R(h)\| = O(\|h\|^q)$ as $h \to 0$, then

$$\frac{\|R(X_n)\|}{\|X_n\|^q} = O_{\mathbb{P}}(1).$$

Proof. For $h \neq 0$, set $G(h) = \|R(h)\| \,/\, \|h\|^q$, with $G(0) = 0$. In case (i), G is continuous at zero and the continuous mapping theorem gives $G(X_n) \xrightarrow{\mathbb{P}} 0$. In case (ii), G is bounded in a neighborhood of zero; since $X_n \xrightarrow{\mathbb{P}} 0$, this gives $G(X_n) = O_{\mathbb{P}}(1)$. □

Example A.16 (Reading a CLT as a stochastic rate)**:** If $\sqrt{n}(\widehat{\theta}_n - \theta) \xrightarrow{d} Z$, then the scaled sequence is tight, so

$$\widehat{\theta}_n - \theta = O_{\mathbb{P}}(n^{-1/2}).$$

If an expansion additionally contains $r_n = o_{\mathbb{P}}(n^{-1/2})$, then

$$\sqrt{n}\{\widehat{\theta}_n - \theta + r_n\} = \sqrt{n}(\widehat{\theta}_n - \theta) + o_{\mathbb{P}}(1),$$

and Slutsky's theorem shows that the limiting distribution is unchanged.

The delta method

Suppose T_n estimates θ and $r_n(T_n - \theta) \xrightarrow{d} Z$. The delta method answers the natural question: what happens to the same asymptotic approximation after applying a smooth transformation to T_n?

Theorem A.17 (Delta method)**.** *Suppose $r_n \to \infty$ and*

$$r_n(T_n - \theta) \xrightarrow{d} Z, \qquad T_n, \theta \in \mathbb{R}^d.$$

If $g : \mathbb{R}^d \to \mathbb{R}^k$ is differentiable at θ, with Jacobian $G = Dg(\theta)$, then

$$r_n\{g(T_n) - g(\theta)\} \xrightarrow{d} GZ. \tag{A.1.2}$$

Moreover, the delta method gives the asymptotic linearization

$$r_n\{g(T_n) - g(\theta)\} - G\, r_n(T_n - \theta) \xrightarrow{\mathbb{P}} 0. \tag{A.1.3}$$

In particular, if $\sqrt{n}(T_n - \theta) \xrightarrow{d} \mathcal{N}(0, \Sigma)$, then

$$\sqrt{n}\{g(T_n) - g(\theta)\} \xrightarrow{d} \mathcal{N}(0, G\Sigma G^\top). \tag{A.1.4}$$

Proof. Convergence in distribution implies $r_n(T_n - \theta) = O_{\mathbb{P}}(1)$, hence $T_n - \theta = o_{\mathbb{P}}(1)$. Differentiability gives

$$g(T_n) - g(\theta) = G(T_n - \theta) + R_n, \qquad \frac{\|R_n\|}{\|T_n - \theta\|} \xrightarrow{\mathbb{P}} 0.$$

The stochastic-order product rule yields $r_n R_n = o_{\mathbb{P}}(1)$, which is (A.1.3). Equation (A.1.2) now follows from Slutsky's theorem; the normal case is the linear transformation of a Gaussian vector. □

Example A.18 (Logarithm of a sample mean)**:** Suppose $\mu > 0$ and $\sqrt{n}(\overline{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$. Since $g(x) = \log x$ has $g'(\mu) = 1/\mu$,

$$\sqrt{n}\{\log \overline{X}_n - \log \mu\} \xrightarrow{d} \mathcal{N}\left(0, \frac{\sigma^2}{\mu^2}\right).$$

Because $\overline{X}_n \xrightarrow{\mathbb{P}} \mu > 0$, the logarithm is defined with probability tending to one.

If $Dg(\theta) = 0$, the first-order delta method gives a degenerate limit and a second-order scaling may be needed. For example, if $\sqrt{n}T_n \xrightarrow{d} \mathcal{N}(0, \sigma^2)$, then the continuous mapping theorem gives

$$nT_n^2 \xrightarrow{d} \sigma^2 \chi_1^2.$$

This is the simplest second-order delta-method calculation.

Bibliographical notes

The population-risk and empirical-risk framework is standard; see Mohri, Rostamizadeh, and Talwalkar [19] and Shalev-Shwartz and Ben-David [26]. The model-class examples, capacity discussion, validation protocol, and estimator bias–variance material draw on Goodfellow, Bengio, and Courville [9], with notation adjusted to emphasize the repeated-sampling interpretation. The double-descent terminology and unified curve are due to Belkin et al. [1]; the ridgeless linear calculation, random-feature theory, and deep-network experiments are represented by [10, 17, 20]. Hoeffding's inequality originates in [11]; a modern treatment of concentration, including Bernstein's inequality, appears in [3]. The expected-error exercise, sub-Gaussian and sub-exponential characterizations, product rules, and expected-maximum bound follow [29]. The finite-even-moment tail bound is adapted from [30, Exercise 2.20]; the even-moment Rosenthal inequality used in its proof is proved above. The convex-optimization proofs follow the standard first-order analysis in Bubeck [4].

Stochastic approximation begins with Robbins and Monro [25]. Classical trajectory averaging and its covariance-efficient asymptotic normality, often called Polyak–Ruppert averaging, are treated by Polyak and Juditsky in [22]. Bottou, Curtis, and Nocedal [2] connect these ideas to large-scale machine learning. Online plug-in and batch-means covariance estimators for averaged SGD are developed by Chen et al. [5]; the perturbed-trajectory bootstrap is due to Fang, Xu, and Yang [6]; and the random-scaling construction follows Lee et al. [14]. Last-iterate and averaging guarantees for general nonsmooth convex SGD are developed by Shamir and Zhang [27]. Momentum originates with Polyak [21]. AdaGrad and RMSprop are discussed alongside other practical optimization methods by Goodfellow, Bengio, and Courville [9].

The MDP terminology and known-model dynamic-programming route follow Puterman [23]. The sampling-based reinforcement-learning algorithms and policy-gradient roadmap follow Sutton and Barto [28]. Neural-network optimization and the practical role of momentum are developed in Goodfellow, Bengio, and Courville [9]; the Adam update is due to Kingma and Ba [12]. Reddi, Kale, and Kumar [24] give nonconvergence examples for Adam and introduce AMSGrad. Decoupled weight decay and AdamW are due to Loshchilov and Hutter [16]. Muon was introduced by Jordan et al. [13]; Liu et al. develop and evaluate a scaled configuration for large language-model training [15].

Lecture 5 develops finite-state dynamic programming from [23, Chapters 2–6]. The sampling algorithms use [28, Chapters 5–6], with the separation of planning, model access, and learning informed by Nan Jiang's course materials [46, 47, 48, 49, 50, 51]. The tabular convergence arguments apply the asynchronous stochastic-approximation theorem in [53]; Q-learning also follows [52].

Bibliography

[1] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

[2] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

[3] S. Boucheron, G. Lugosi, and P. Massart. *Concentration Inequalities: A Nonasymptotic Theory of Randomness*. Oxford University Press, 2013.

[4] S. Bubeck. Convex optimization: Algorithms and complexity. *Foundations and Trends in Machine Learning*, 8(3–4):231–358, 2015.

[5] X. Chen, J. D. Lee, X. T. Tong, and Y. Zhang. Statistical inference for model parameters in stochastic gradient descent. *The Annals of Statistics*, 48(1):251–273, 2020.

[6] Y. Fang, J. Xu, and L. Yang. Online bootstrap confidence intervals for the stochastic gradient descent estimator. *Journal of Machine Learning Research*, 19(78):1–21, 2018.

[7] G. Garrigos and R. M. Gower. Handbook of convergence theorems for (stochastic) gradient methods. *arXiv preprint arXiv:2301.11235*, 2023.

[8] G. Goh. Why momentum really works. *Distill*, 2017. doi:10.23915/distill.00006.

[9] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[10] T. Hastie, A. Montanari, S. Rosset, and R. J. Tibshirani. Surprises in high-dimensional ridgeless least squares interpolation. *The Annals of Statistics*, 50(2):949–986, 2022.

[11] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[12] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[13] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Technical note and open implementation, 2024. https://kellerjordan.github.io/posts/muon/.

[14] S. Lee, Y. Liao, M. H. Seo, and Y. Shin. Fast and robust online inference with stochastic gradient descent via random scaling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(7):7381–7389, 2022.

[15] J. Liu, J. Su, X. Yao, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.

[16] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[17] S. Mei and A. Montanari. The generalization error of random features regression: Precise asymptotics and the double descent curve. *Communications on Pure and Applied Mathematics*, 75(4):667–766, 2022.

[18] D. Merkulov and I. Oseledets. Another approach to build Lyapunov functions for first-order methods in the quadratic case. *arXiv preprint arXiv:2310.15880*, 2024.

[19] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of Machine Learning*. MIT Press, second edition, 2018.

[20] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations*, 2020.

[21] B. T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi:10.1016/0041-5553(64)90137-5.

[22] B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

[23] M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[24] S. J. Reddi, S. Kale, and S. Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations*, 2018.

[25] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[26] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

[27] O. Shamir and T. Zhang. Stochastic gradient descent for non-smooth optimization: Convergence results and optimal averaging schemes. In *Proceedings of the 30th International Conference on Machine Learning*, 2013.

[28] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.

[29] R. Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.

[30] M. J. Wainwright. *High-Dimensional Statistics: A Non-Asymptotic Viewpoint*. Cambridge University Press, 2019.

[31] F. A. Gers, J. Schmidhuber, and F. Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

[32] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[33] M. Leshno, V. Ya. Lin, A. Pinkus, and S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[34] D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar, and N. Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70):1–57, 2018.

[35] M. Telgarsky. Benefits of depth in neural networks. In *Proceedings of the 29th Conference on Learning Theory*, volume 49 of PMLR, pages 1517–1539, 2016.

[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[37] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). arXiv:1606.08415, 2016.

[38] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[39] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014.

[40] F. Yu and V. Koltun. Multi-scale context aggregation by dilated convolutions. In *International Conference on Learning Representations*, 2016.

[41] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[42] G. Raskutti, M. J. Wainwright, and B. Yu. Early stopping and non-parametric regression: An optimal data-dependent stopping rule. *Journal of Machine Learning Research*, 15(11):335–366, 2014.

[43] D. Zou, J. Wu, V. Braverman, Q. Gu, D. P. Foster, and S. M. Kakade. The benefits of implicit regularization from SGD in least squares problems. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[44] S. Gunasekar, J. D. Lee, D. Soudry, and N. Srebro. Characterizing implicit bias in terms of optimization geometry. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of PMLR, pages 1832–1841, 2018.

[45] B. Woodworth, S. Gunasekar, J. D. Lee, E. Moroshko, P. Savarese, I. Golan, D. Soudry, and N. Srebro. Kernel and rich regimes in overparametrized models. In *Proceedings of the 33rd Conference on Learning Theory*, volume 125 of PMLR, pages 3635–3673, 2020.

[46] N. Jiang. CS 443: Reinforcement Learning. University of Illinois Urbana–Champaign, course materials. https://nanjiang.cs.illinois.edu/cs443/.

[47] N. Jiang. CS 542: Statistical Reinforcement Learning. University of Illinois Urbana–Champaign, course materials. https://nanjiang.cs.illinois.edu/cs542/.

[48] N. Jiang. MDP Preliminaries. Lecture notes, updated September 27, 2024. https://nanjiang.cs.illinois.edu/files/cs542f22/note1.pdf.

[49] N. Jiang. Learning settings, CS 443. Lecture materials linked from https://nanjiang.cs.illinois.edu/cs443/.

[50] N. Jiang. Tabular RL for Value Prediction. CS 443 lecture slides. https://nanjiang.cs.illinois.edu/files/cs443s23/6_mc_td.pdf.

[51] N. Jiang. Algorithms for control. CS 443 lecture slides. https://nanjiang.cs.illinois.edu/files/cs443s23/8_control.pdf.

[52] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992. https://doi.org/10.1007/BF00992698.

[53] J. N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16:185–202, 1994. https://www.mit.edu/~jnt/Papers/J052-94-jnt-q.pdf.

[54] N. Jiang. Notes on Tabular Methods. CS 542 lecture notes. https://nanjiang.cs.illinois.edu/files/cs542f22/note3.pdf.

[55] N. Jiang. Value Prediction with Function Approximation. CS 443 lecture slides. https://nanjiang.cs.illinois.edu/files/cs443s23/7_td_fa.pdf.

[56] N. Jiang. Importance Sampling. CS 443 lecture slides. https://nanjiang.cs.illinois.edu/files/cs443s23/9_is.pdf.

[57] N. Jiang. Policy Gradient. CS 443 lecture slides. https://nanjiang.cs.illinois.edu/files/cs443s24/10_pg.pdf.

[58] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999. Official proceedings.

[59] A. Agarwal, S. Kakade, and L. F. Yang. Model-based reinforcement learning with a generative model is minimax optimal. *Proceedings of the 33rd Conference on Learning Theory*, Proceedings of Machine Learning Research, 125:67–83, 2020. https://proceedings.mlr.press/v125/agarwal20b.html.

[60] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. https://doi.org/10.1038/nature14236.